STUDY TEXTBOOK

BOOK TITLE

Subtitle

First edition · 10 October 2026

Contents

Start here: from zero **ix**

1 Linear systems and Gaussian elimination **1**

1.1 Two ways to see $A\mathbf{x} = \mathbf{b}$ . . . 2
1.2 Elimination to echelon form . . . 3
1.3 Pivots, free variables, and the shape of a solution set . . . 5
1.4 LU factorization (and $PA = LU$ with row exchanges) . . . 6
1.5 Cost and stability: why elimination is $O(n^3)$ and why we pivot . . . 9
1.6 Hand derivation . . . 11
1.7 Common mistakes . . . 12
1.8 Summary . . . 13
Notes . . . 13
Further reading . . . 14
Additional exercises . . . 14

2 Matrices and matrix algebra **16**

2.1 A matrix is a box of coefficients — and a recipe . . . 17
2.2 $A\mathbf{x}$ as a column combination . . . 17
2.3 Matrix multiplication as composition . . . 18
2.4 Transpose, inverses, and when they exist . . . 20
2.5 Special matrices: diagonal, triangular, permutation, block . . . 22
2.6 Hand derivation . . . 24
2.7 Common mistakes . . . 25
2.8 Summary . . . 25
Notes . . . 26
Further reading . . . 26
Additional exercises . . . 27

3 Vector spaces, subspaces, and the four fundamental subspaces **29**

3.1 Vector spaces: axioms with motivation, not ceremony . . . 30
3.2 Subspaces . . . 30
3.3 Span and linear independence . . . 31
3.4 Basis and dimension . . . 31
3.5 The four fundamental subspaces of A . . . 32
3.6 Rank and the rank–nullity theorem . . . 36
3.7 Coordinates . . . 38
3.8 Hand derivation . . . 38
3.9 Common mistakes . . . 39
3.10 Summary . . . 40
Notes . . . 40
Further reading . . . 41
Additional exercises . . . 41

4 Linear maps and change of basis **43**

4.1 Linear maps: definition and examples . . . 44
4.2 Kernel and image . . . 45
4.3 Every linear map is a matrix . . . 45
4.4 Change of basis and similarity . . . 46
4.5 A geometric catalog: rotations, reflections, projections, shears . . . 48
4.6 Hand derivation . . . 51
4.7 Common mistakes . . . 51
4.8 Summary . . . 52
Notes . . . 53
Further reading . . . 53
Additional exercises . . . 53

5 Orthogonality, projections, and QR **55**

5.1 Inner product, norm, angle . . . 56
5.2 Orthogonal and orthonormal sets . . . 57
5.3 Projection onto a line and onto a subspace . . . 58
5.4 Gram–Schmidt and the QR factorization . . . 60
5.5 Orthogonal complements: the four subspaces pair up . . . 64
5.6 Hand derivation . . . 65

5.7 Common mistakes . . . 65
5.8 Summary . . . 66
Further reading . . . 67
Notes . . . 68

6 Determinants **69**
6.1 Three definitions that agree . . . 70
6.2 Properties: product, transpose, row operations . . . 72
6.3 Determinant as signed volume . . . 72
6.4 det $A = 0$ means collapse . . . 75
6.5 Cramer's rule, a curiosity . . . 77
6.6 Hand derivation . . . 78
6.7 Common mistakes . . . 78
6.8 Summary . . . 79
Further reading . . . 79
Additional exercises . . . 80
Notes . . . 81

7 Eigenvalues and diagonalisation **82**
7.1 The eigen equation . . . 83
7.2 Characteristic polynomial and finding them by hand . . . 83
7.3 Eigenspaces and multiplicities . . . 85
7.4 Diagonalisation and powers of A . . . 86
7.5 Defective matrices . . . 87
7.6 Applications: recurrences and Markov chains . . . 88
7.7 Hand derivation . . . 92
7.8 Common mistakes . . . 92
7.9 Summary . . . 93
Further reading . . . 93
Additional exercises . . . 94
Notes . . . 95

8 Least squares and linear regression **96**
8.1 Overdetermined systems: the best we can do . . . 97
8.2 The normal equations via projection . . . 97

8.3 Linear regression: features, intercept, many columns . . . 99
8.4 Feature maps and the overfitting preview . . . 99
8.5 Solving in practice: QR, not A^TA . . . 101
8.6 Residuals, R^2, and honest evaluation . . . 102
8.7 Hand derivation . . . 104
8.8 Common mistakes . . . 105
8.9 Summary . . . 106
Further reading . . . 106
Additional exercises . . . 107
Notes . . . 108

9 Symmetric matrices and the spectral theorem 109
9.1 Symmetric matrices and why they are special . . . 110
9.2 The spectral theorem . . . 110
9.3 Quadratic forms and definiteness . . . 112
9.4 Rayleigh quotient . . . 114
9.5 Covariance matrices: PSD by data . . . 115
9.6 Hand derivation . . . 117
9.7 Common mistakes . . . 117
9.8 Summary . . . 118
Further reading . . . 118
Additional exercises . . . 119
Notes . . . 120

10 The SVD and PCA 121
10.1 Statement: $A = U\Sigma V^T$ . . . 122
10.2 Geometry: sphere to ellipsoid . . . 123
10.3 Rank, condition number, pseudoinverse . . . 124
10.4 Low-rank approximation and Eckart–Young . . . 125
10.5 PCA: covariance eig = SVD of centered data . . . 127
10.6 Hand derivation . . . 130
10.7 Common mistakes . . . 130
10.8 Summary . . . 131
Further reading . . . 131
Additional exercises . . . 132
Notes . . . 133

11 Multivariable calculus: gradients, Jacobians, Hessians **134**

11.1 Partial derivatives . . . 135
11.2 The gradient and the steepest direction . . . 135
11.3 The Jacobian: local linear approximation . . . 135
11.4 The Hessian and second-order information . . . 138
11.5 The chain rule in matrix form (the backprop engine) . . . 139
11.6 Worked library . . . 139
11.7 Hand derivation . . . 142
11.8 Common mistakes . . . 143
11.9 Summary . . . 144
Further reading . . . 144
Additional exercises . . . 145
Notes . . . 146

12 Probability and random variables **147**

12.1 Sample spaces and the axioms . . . 148
12.2 Conditional probability and Bayes . . . 148
12.3 Independence . . . 149
12.4 Discrete random variables . . . 150
12.5 Continuous random variables . . . 150
12.6 Expectation, variance, moments . . . 151
12.7 Joint, marginal, conditional; covariance . . . 151
12.8 The multivariate Gaussian . . . 152
12.9 LLN and CLT, stated and simulated . . . 153
12.10 Hand derivation . . . 155
12.11 Common mistakes . . . 155
12.12 Summary . . . 156
Further reading . . . 156
Additional exercises . . . 157
Notes . . . 158

13 Statistics, estimation, and Bayesian inference **159**

13.1 Samples, statistics, estimators . . . 160
13.2 Bias, variance, and the trade-off . . . 160
13.3 Maximum likelihood . . . 161

13.4 Bayesian inference: prior, likelihood, posterior . . . 163
13.5 MAP vs. MLE . . . 164
13.6 Regression revisited: least squares = MLE . . . 164
13.7 Hand derivation . . . 166
13.8 Common mistakes . . . 166
13.9 Summary . . . 167
Further reading . . . 167
Additional exercises . . . 168
Notes . . . 169

14 Convexity, optimization, and Lagrange multipliers 170
14.1 Optimization problems in ML . . . 171
14.2 Convex sets and functions . . . 171
14.3 Gradient descent: the update, the step size, the failures . . . 171
14.4 Momentum and friends, briefly . . . 174
14.5 Constrained optimization and Lagrange multipliers . . . 174
14.6 Normal equations, derived by calculus . . . 175
14.7 Newton's method in one section . . . 175
14.8 Backpropagation is the chain rule . . . 176
14.9 Hand derivation . . . 180
14.10 Common mistakes . . . 180
14.11 Summary . . . 181
Further reading . . . 182
Additional exercises . . . 182
Notes . . . 183

15 Information theory: bits, entropy, and why the loss is a log 184
15.1 Surprise and information . . . 185
15.2 Entropy and its properties . . . 185
15.3 Codes and the price of a wrong model . . . 186
15.4 Cross-entropy and KL divergence . . . 187
15.5 Mutual information . . . 188
15.6 Entropy of the Gaussian . . . 188
15.7 Why cross-entropy loss is MLE . . . 188
15.8 Hand derivation . . . 189

15.9 Common mistakes 190
15.10Summary 191
Further reading 191
Notes 193

16 Putting it together: a machine learning toolkit **196**
16.1 The shape of a learning problem 197
16.2 One contract: `fit`, `predict`, `transform` 197
16.3 The library 199
16.4 Checking without a framework 201
16.5 A real pipeline: PCA then logistic 201
16.6 The bias–variance dial 202
16.7 Hand derivation: the ridge normal equations 203
16.8 Common mistakes 204
16.9 Summary 205
Further reading 206
Notes 206
Additional exercises 207

Start here: from zero

This book starts at the point where school algebra ends, and ends at the mathematics inside a working machine-learning library. If you can solve two equations in two unknowns, you can begin. This chapter explains what "linear algebra" is for, fixes the notation, and gives the single-variable calculus needed for Chapter 11, so that nothing later arrives unannounced.

Why this subject exists

A spreadsheet row of numbers is a *vector*. A spreadsheet is a *matrix*. A rule that turns one row into another by multiplying and adding — a photo filter, a currency conversion, one layer of a neural network — is a *linear map*. Linear algebra is the study of these objects, and it is useful because the same few ideas (solve a system, find the best fit, find the important directions) show up in every field that handles data.

A system of equations, from school

Solve $2x - y = 0$ and $-x + 2y = 3$ by substitution. The first equation gives $y = 2x$. Putting that into the second: $-x + 2(2x) = 3$, so $3x = 3$, so $x = 1$ and $y = 2$. Check: $2(1) - 2 = 0$ and $-1 + 2(2) = 3$. Chapter 1 solves this same system two other ways and then solves systems with hundreds of unknowns, where substitution would be hopeless.

The notation, in one page

Symbol	Meaning
$\mathbb{R}$	the real numbers, i.e. the number line
$\mathbb{R}^n$	lists of n real numbers; $\mathbb{R}^2$ is the plane, $\mathbb{R}^3$ is space
$\mathbf{x}$	a vector, written as a column, such as $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$
A	a matrix, a rectangular table; A is $m \times n$ if it has m rows and n columns
a_{ij}	the entry of A in row i, column j
A^{T}	the transpose: swap rows and columns
$\sum_{i=1}^{n} a_i$	$a_1 + a_2 + \cdots + a_n$
$\in$	"is an element of"; $\mathbf{x} \in \mathbb{R}^3$ says $\mathbf{x}$ is a list of three numbers
$\forall$, $\exists$	"for every", "there exists"

A calculus primer for Chapter 11

You will not need calculus until Chapter 11, but it is better to meet it here. The *derivative* $f'(x)$ of a function f is the slope of its graph at x: how fast f changes when x is nudged. For example $f(x) = x^2$ has $f'(x) = 2x$; at $x = 3$ the slope is 6, so a nudge of 0.01 raises f by about 0.06 (indeed $3.01^2 - 3^2 = 0.0601$). The *chain rule* handles a function inside a function: if $f(x) = (3x + 1)^2$ then $f'(x) = 2(3x + 1) \cdot 3 = 18x + 6$, the derivative of the outside evaluated at the inside, times the derivative of the inside. You can confirm by expanding $(3x + 1)^2 = 9x^2 + 6x + 1$ and differentiating directly. Chapter 11 extends exactly these two ideas to functions of many variables.

How the books of this series fit together

This is Book 3 of eight, and it is the mathematical spine of the data half of the series.

Book	Connection
2 Discrete maths	Teaches how to read and write proofs; this book states each theorem and then proves it in that style. Chapter 12 here builds on its finite probability.
4 Machine learning	Cites this book by chapter: least squares (Chapter 8), PCA (Chapter 10), gradients and backpropagation (Chapters 11 and 14), losses (Chapter 15).
6 Quant research	Uses the statistics of Chapter 13 and the covariance ideas of Chapters 9–10.

How to use the boxes in each chapter

Every chapter opens with *After this chapter you can*, listing concrete skills. From Chapter 2, a *Before you start* box names the earlier chapters it relies on and gives a one-sentence recap, so you can tell whether to reread something. Each chapter ends with a *Hand derivation* section: do it on paper before you trust the code.

CHAPTER 1

Linear systems and Gaussian elimination

Nearly every computation in this book — least squares, eigenvalue computations, and the neural-network training of part IV — ends with a system of linear equations. This chapter builds the one algorithm that solves them honestly: Gaussian elimination. After it you will be able to (a) read a linear system two ways, as intersecting hyperplanes and as a combination of columns; (b) reduce any system to echelon form and read off whether it has zero, one, or infinitely many solutions; (c) package the work of elimination as the factorization $PA = LU$; and (d) say why elimination costs about n^3 operations and why a naive computer must swap rows to stay accurate.

Why this exists. Whenever several unknown quantities are tied together by several equations — the prices of three goods from three receipts, the weights in a model from many examples — you have a linear system. Solving one with 2 unknowns is school algebra; solving one with 2000 is a computation that needs a method that is systematic, fast and numerically safe. That method is Gaussian elimination, and almost everything else in this book is built on understanding what it does.

BEFORE YOU START

School algebra only: you can solve two equations in two unknowns by substitution. The *Start here* chapter reviews the symbols used in this book (sums, sets, vectors) in a few pages.

AFTER THIS CHAPTER YOU CAN

- write a system of equations as $A\mathbf{x} = \mathbf{b}$ and read it as lines meeting, and as a recipe of columns;
- reduce any system to echelon form by hand and say whether it has no, one, or infinitely many solutions;
- factor a matrix as $PA = LU$ and use it to solve many right-hand sides cheaply;
- explain why elimination takes about n^3 operations and why a computer swaps rows.

HOW TO THINK ABOUT THIS

Elimination is just the school method of subtracting one equation from another, done in an organised order so that it never gets lost. Each step makes the system simpler while keeping the same solutions. Hold two pictures in your head: lines crossing at a point, and a recipe that mixes columns to hit a target. Switching between them is half of linear algebra.

1.1 Two ways to see $A\mathbf{x} = \mathbf{b}$

A *linear system* is a set of equations

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1, \quad \ldots \quad a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m,$$

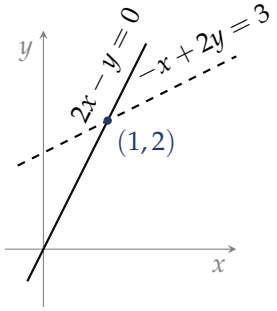
which we compress into $A\mathbf{x} = \mathbf{b}$: the *coefficient matrix* A is $m \times n$, $\mathbf{x}$ is the unknown column vector in $\mathbb{R}^n$, and $\mathbf{b} \in \mathbb{R}^m$ is given. Take the smallest non-trivial case, $m = n = 2$:

$$2x - y = 0, \qquad -x + 2y = 3.$$

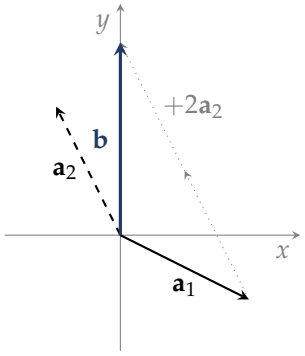
There are two honest ways to look at it, and you need both.

Row picture. Each equation is a line in the plane; solving means finding where the lines meet. Here they meet at the single point $(x, y) = (1, 2)$.

Column picture. Write the system as $x\,\mathbf{a}_1 + y\,\mathbf{a}_2 = \mathbf{b}$, where $\mathbf{a}_1 = (2, -1)$ and $\mathbf{a}_2 = (-1, 2)$ are the *columns* of A. Solving means finding the right weights: how many copies of each column add up to $\mathbf{b} = (0, 3)$. The answer $x = 1,\ y = 2$ says $\mathbf{a}_1 + 2\mathbf{a}_2 = \mathbf{b}$.



(a) row picture: intersect



(b) column picture: $\mathbf{a}_1 + 2\mathbf{a}_2 = \mathbf{b}$

Figure 1.1: Two pictures of the same 2×2 system. Notice that the *weights* in the column picture (b) are exactly the coordinates of the meeting point in the row picture (a). The row picture shows what a solution *is*; the column picture shows why one *exists* — it exists precisely when $\mathbf{b}$ can be built from the columns.

The column picture is the one that scales. Three equations in three unknowns ask: is $\mathbf{b}$ a combination of three vectors in $\mathbb{R}^3$? If those columns fill out all of $\mathbb{R}^3$, every $\mathbf{b}$ is reachable and the system always has a solution. If the three columns happen to lie in one plane, only $\mathbf{b}$'s in that plane work — most systems are then unsolvable. The vector-spaces chapter turns

this into the four fundamental subspaces; for now keep the image: $A\mathbf{x}$ *is a combination of the columns of* A *with weights* $\mathbf{x}$.

Remark. The row picture and column picture are the same arithmetic seen from two sides. Experienced readers switch between them without noticing. Practise the switch now, on paper, until it is automatic.

1.2 Elimination to echelon form

The row picture tells us what we want; elimination is how we get it. The idea is mechanical: use one equation to remove a variable from the equations below it, then repeat. Three moves — the *elementary row operations* — are legal because each leaves the solution set unchanged:

1. add a multiple of one equation to another;
2. multiply an equation by a non-zero number;
3. swap two equations.

Writing only the coefficients, in the *augmented matrix* $[\,A \mid \mathbf{b}\,]$, we eliminate column by column until the system is *upper triangular* — in *echelon form* — and then solve by *back substitution*, bottom row upward.

Example 1.1. Solve the system

$$x + 2y + z = 7, \qquad 3x + 8y + z = 17, \qquad 2y + 3z = 11.$$

Strategy. Use the 1 in position $(1,1)$ — the first *pivot* — to clear column 1: subtract $3\times$ row 1 from row 2. Row 3 already has a zero in column 1, so it is left alone — a step may be a no-op. Then subtract row 2 from row 3 to clear column 2. Finally read z, then y, then x from the bottom up.

Solution. The augmented matrix and its two elimination steps appear in figure 1.2:

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 7 \\ 3 & 8 & 1 & 17 \\ 0 & 2 & 3 & 11 \end{array}\right] \xrightarrow{R_2 - 3R_1} \left[\begin{array}{ccc|c} 1 & 2 & 1 & 7 \\ 0 & 2 & -2 & -4 \\ 0 & 2 & 3 & 11 \end{array}\right] \xrightarrow{R_3 - R_2} \left[\begin{array}{ccc|c} 1 & 2 & 1 & 7 \\ 0 & 2 & -2 & -4 \\ 0 & 0 & 5 & 15 \end{array}\right].$$

Back substitution: $5z = 15$ gives $z = 3$; then $2y - 2(3) = -4$ gives $y = 1$; then $x + 2(1) + 3 = 7$ gives $x = 2$. The solution is $(x, y, z) = (2, 1, 3)$. Check: $2 + 2 + 3 = 7$; $6 + 8 + 3 = 17$; $2 + 9 = 11$. ✓

$$\underset{\text{start}}{\left[\begin{array}{ccc|c} 1 & 2 & 1 & 7 \\ 3 & 8 & 1 & 17 \\ 0 & 2 & 3 & 11 \end{array}\right]} \xrightarrow[(R_3 \text{ already } 0)]{R_2 - 3R_1} \underset{\text{after column 1}}{\left[\begin{array}{ccc|c} 1 & 2 & 1 & 7 \\ 0 & 2 & -2 & -4 \\ 0 & 2 & 3 & 11 \end{array}\right]} \xrightarrow{R_3 - R_2} \underset{\text{echelon form}}{\left[\begin{array}{ccc|c} 1 & 2 & 1 & 7 \\ 0 & 2 & -2 & -4 \\ 0 & 0 & 5 & 15 \end{array}\right]}$$

Figure 1.2: Elimination on example 1.1, one state per step. Notice the shaded *pivots* marching down the diagonal, and that only the entries below the current pivot are touched. The third state is upper triangular: back substitution can start.

Two bookkeeping rules make this reliable on paper. First, *one column at a time, left to right* — never clear a column above the current pivot. Second, *record each multiplier*: the number ℓ_{ik} that makes $R_i \leftarrow R_i - \ell_{ik} R_k$ cancel the (i,k) entry is $\ell_{ik} = a_{ik}/a_{kk}$. Those multipliers are not scratch work; section 1.4 shows they *are* the matrix L.

```python
import numpy as np

A = np.array([[1., 2., 1.],
              [3., 8., 1.],
              [0., 2., 3.]])
b = np.array([7., 17., 11.])
n = len(b)

Ab = np.column_stack([A, b]) # the augmented matrix [A | b]
for col in range(n - 1): # eliminate below each pivot, left to right
    for row in range(col + 1, n):
        m = Ab[row, col] / Ab[col, col] # this multiplier kills Ab[row, col]
        Ab[row] -= m * Ab[col]
print("echelon form [U | c]:")
print(Ab)

x = np.zeros(n) # back substitution reads U bottom to top
for row in range(n - 1, -1, -1):
    x[row] = (Ab[row, n] - Ab[row, row + 1:n] @ x[row + 1:n]) / Ab[row, row]
print("x =", x)
print("np.linalg.solve agrees:", np.linalg.solve(A, b))
print("residual ||Ax-b|| =", np.linalg.norm(A @ x - b))
```

Listing 1.1: Forward elimination in plain Python on example 1.1, checked against `numpy`. File: `code/ch01_forward_elimination.py`.

```
echelon form [U | c]:
[[ 1. 2. 1. 7.]
 [ 0. 2. -2. -4.]
 [ 0. 0. 5. 15.]]
x = [2. 1. 3.]
np.linalg.solve agrees: [2. 1. 3.]
residual ||Ax-b|| = 0.0
```

Listing 1.2: Output of listing 1.1. The echelon form matches figure 1.2 exactly; pivots $1, 2, 5$.

Exercise 1.1 (easy)**.** By hand, eliminate

$$x + 2y + z = 6, \qquad 2x + y + 2z = 6, \qquad x - y + z = 2,$$

to echelon form, then back-substitute. Write down the three pivots and every multiplier you used.

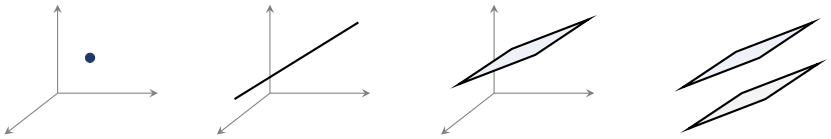
1.3 Pivots, free variables, and the shape of a solution set

In example 1.1 every column had a pivot. That need not happen — and what happens instead is the whole story of a linear system.

Definition 1.2. A *pivot* is the first non-zero entry in its row once the matrix is in echelon form. The columns containing pivots are *pivot columns*; the remaining variables are *free variables* — they may take any value, and the pivot variables are then determined by back substitution. The number of pivots is the *rank* of A.

Reading the echelon form $[\,U \mid \mathbf{c}\,]$ settles existence and uniqueness at a glance:

- a row $[\,0 \;\cdots\; 0 \mid c\,]$ with $c \neq 0$ says $0 = c$: **no solution**;
- otherwise, if every variable column is a pivot column: **exactly one solution**;
- otherwise, free variables exist: **infinitely many solutions**, and each free variable adds one degree of freedom to the solution set.



(a) point (b) line (c) plane (d) empty set

Figure 1.3: The four shapes a solution set can take, drawn in $\mathbb{R}^3$. Notice there is no "two solutions" case: the moment two distinct points solve the system, the whole line through them does. A solution set is always empty, a point, a line, a plane, ... — never a finite blob.

Example 1.3. Find all solutions of $A\mathbf{x} = \mathbf{b}$ for

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 1 & 0 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 4 \\ 8 \\ 1 \end{bmatrix}.$$

Strategy. Eliminate as usual and watch for a row of zeros: it signals a redundant (or contradictory) equation. Then separate pivot and free variables per definition 1.2 and parametrize.

Solution. $R_2 - 2R_1$ produces the zero row $[\,0\;0\;0 \mid 0\,]$ — consistent, since $0 = 0$. Then $R_3 - R_1 = [\,0 \;-1\; -1 \mid -3\,]$, and the echelon form is

$$\left[\begin{array}{ccc|c} 1 & 2 & 1 & 4 \\ 0 & -1 & -1 & -3 \\ 0 & 0 & 0 & 0 \end{array}\right].$$

Pivots sit in columns 1 and 2, so $x_3 = t$ is free. Back substitution: $x_2 = 3 - t$ and $x_1 = 4 - 2x_2 - x_3 = -2 + t$. All solutions:

$$\mathbf{x} = \begin{bmatrix} -2 \\ 3 \\ 0 \end{bmatrix} + t \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}, \qquad t \in \mathbb{R},$$

a line in $\mathbb{R}^3$ — case (b) of figure 1.3. The vector $\mathbf{x}_p = (-2, 3, 0)$ is a *particular solution*; the vector $(1, -1, 1)$ spans the *null space* of A (check: $A(1,-1,1)^T = \mathbf{0}$).

That decomposition in the last line of example 1.3 is not a coincidence — it is the structural theorem of the chapter.

Theorem 1.4. *If $\mathbf{x}_p$ is one solution of $A\mathbf{x} = \mathbf{b}$, then the complete solution set is*

$$\{\, \mathbf{x}_p + \mathbf{x}_h : \mathbf{x}_h \in N(A) \,\},$$

where $N(A) = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}$ is the null space of A.

Proof. Take any $\mathbf{x}_h \in N(A)$: then $A(\mathbf{x}_p + \mathbf{x}_h) = A\mathbf{x}_p + A\mathbf{x}_h = \mathbf{b} + \mathbf{0} = \mathbf{b}$, so every such vector is a solution. Conversely, if $\mathbf{x}$ solves the system, write $\mathbf{x} = \mathbf{x}_p + (\mathbf{x} - \mathbf{x}_p)$ and note $A(\mathbf{x} - \mathbf{x}_p) = \mathbf{b} - \mathbf{b} = \mathbf{0}$, so $\mathbf{x} - \mathbf{x}_p \in N(A)$. ∎

This is why figure 1.3 shows only flats: the solution set is the null space — always a flat through the origin — shifted to pass through $\mathbf{x}_p$.

Exercise 1.2 (medium)**.** For the same matrix A as example 1.3, try $\mathbf{b} = (4, 9, 1)^T$. What row appears in elimination, and what does it say about the solution set? Then pick any $\mathbf{b}$ in the plane $2b_1 - b_2 = 0$ and confirm the system is solvable.

Example 1.5. Balance the reaction $\mathrm{Fe_2O_3 + CO \to Fe + CO_2}$: find whole numbers a, b, c, d with $a\,\mathrm{Fe_2O_3} + b\,\mathrm{CO} \to c\,\mathrm{Fe} + d\,\mathrm{CO_2}$.

Strategy. Each element gives one linear equation in the four coefficients. Four unknowns, three equations, zero right-hand sides — this is the homogeneous case of section 16.1, so the answer will be a free-variable family, not a single vector.

Solution. Fe: $2a = c$; C: $b = d$; O: $3a + b = 2d$. The C and Fe equations hand $b = d$ and $c = 2a$; the O equation becomes $3a + d = 2d$, i.e. $d = 3a$, so $b = 3a$ too. Every solution is $(a, b, c, d) = a\,(1, 3, 2, 3)$ — the solution set is a *ray*, exactly the null space of the 3×4 atom matrix (figure 1.4). There is no "the" answer until the physical rule arrives: chemists want the smallest whole numbers, so $a = 1$ and

$$\mathrm{Fe_2O_3} + 3\,\mathrm{CO} \longrightarrow 2\,\mathrm{Fe} + 3\,\mathrm{CO_2}.$$

Check the atoms: Fe $2 = 2$, C $3 = 3$, O $3 + 3 = 6$. ✓

1.4 LU factorization (and $PA = LU$ with row exchanges)

Here is the observation that turns elimination from a procedure into a theory. Each step "subtract ℓ_{ik} times row k from row i" can itself be undone by a matrix. Suppose elimination needs no row swaps. Then the multipliers ℓ_{ik}, placed in a lower-triangular matrix L with 1s on the diagonal, satisfy

$$A = LU, \qquad L \text{ lower triangular, } U \text{ upper triangular.}$$

$a\,\mathrm{Fe_2O_3} + b\,\mathrm{CO} \longrightarrow c\,\mathrm{Fe} + d\,\mathrm{CO_2}$

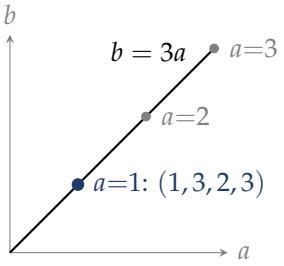
Fe: $2a = c$

C: $b = d$

O: $3a + b = 2d$

one free variable a: $b = d = 3a,\ c = 2a$

atoms → equations



the ray $t\,(1,3,2,3)$
integer points = valid balances

Figure 1.4: Chemical balancing is elimination on a homogeneous system. Notice the pattern of figure 1.3(b) reappearing: one free variable means a ray of answers, and the "smallest whole-number" rule picks the first lattice point on it.

This is the *LU factorization*: U is exactly the echelon form elimination produced, and L is nothing but the multiplier record kept below the diagonal. Solving $A\mathbf{x} = \mathbf{b}$ becomes two triangular solves — forward substitution $L\mathbf{c} = \mathbf{b}$, then back substitution $U\mathbf{x} = \mathbf{c}$ — and the factorization is reusable: a new $\mathbf{b}$ costs only $O(n^2)$, not another elimination.

If a pivot position is zero while nonzero entries sit below it, swap rows first. Collecting all swaps into a *permutation matrix* P gives the complete statement: *every* square matrix admits

$$PA = LU.$$

Example 1.6. Factor $A = \begin{bmatrix} 1 & 1 & 2 \\ 2 & 2 & 3 \\ 1 & 0 & 1 \end{bmatrix}$ as $PA = LU$. **Strategy.** Column 1 is fine (pivot 1, eliminate with multipliers 2 and 1). Column 2 then has a 0 in the pivot position — so swap rows 2 and 3 *before* eliminating, and keep the swap in P.

Solution. After $R_2 - 2R_1$ and $R_3 - R_1$:

$$\begin{bmatrix} 1 & 1 & 2 \\ 0 & 0 & -1 \\ 0 & -1 & -1 \end{bmatrix} \xrightarrow{\text{swap } R_2 \leftrightarrow R_3} \begin{bmatrix} 1 & 1 & 2 \\ 0 & -1 & -1 \\ 0 & 0 & -1 \end{bmatrix} = U.$$

It is cleaner to apply the swap to A up front — eliminate PA where P exchanges rows 2 and 3:

$$PA = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & 2 & 3 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 1 & 1 & 2 \\ 0 & -1 & -1 \\ 0 & 0 & -1 \end{bmatrix}.$$

Multipliers went straight into L (below-diagonal, in the order the rows ended up). Check $LU = PA$: row 2 of LU is $1 \cdot (1,1,2) + 1 \cdot (0,-1,-1) = (1,0,1)$ — the second row of PA. ✓

```
import numpy as np
from scipy.linalg import lu

```

```
A = np.array([[1., 1., 2.], [2., 2., 3.], [1., 0., 1.]])
P = np.array([[1., 0., 0.], [0., 0., 1.], [0., 1., 0.]]) # swaps rows
    2,3
L = np.array([[1., 0., 0.], [1., 1., 0.], [2., 0., 1.]])
U = np.array([[1., 1., 2.], [0., -1., -1.], [0., 0., -1.]])

print("PA =\n", P @ A)
print("L @ U =\n", L @ U)
print("||PA - LU|| =", np.linalg.norm(P @ A - L @ U))

P_sp, L_sp, U_sp = lu(A) # scipy picks its own pivots
print("scipy P =\n", P_sp)
print("||P_sp A - L_sp U_sp|| =", np.linalg.norm(P_sp @ A - L_sp @
    U_sp))
```

Listing 1.3: Verifying example 1.6 and comparing with `scipy.linalg.lu`. File: `code/ch01_lu_factorization.py`.

```
PA =
 [[1. 1. 2.]
 [1. 0. 1.]
 [2. 2. 3.]]
L @ U =
 [[1. 1. 2.]
 [1. 0. 1.]
 [2. 2. 3.]]
||PA - LU|| = 0.0
scipy P =
 [[0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]]
||P_sp A - L_sp U_sp|| = 0.0
```

Listing 1.4: Output of listing 1.3. Our hand factorization is exact; scipy chooses different pivots — a different, equally valid $PA = LU$.

Remark. The scipy output makes an important point: $PA = LU$ is not unique. Any order you pull the rows down in gives a valid factorization; `scipy` picks pivots by the rule of section 1.5, not by the order we happened to choose.

Exercise 1.3 (medium). Factor $A = \begin{bmatrix} 2 & 1 & 0 \\ 4 & 3 & 1 \\ 0 & 2 & 1 \end{bmatrix}$ as $A = LU$ (no swaps needed). Then solve $A\mathbf{x} = (4, 10, 3)^T$ by solving $L\mathbf{c} = \mathbf{b}$ then $U\mathbf{x} = \mathbf{c}$.

Example 1.7. You factored $A = \begin{bmatrix} 2 & 1 & 0 \\ 4 & 3 & 1 \\ 0 & 2 & 1 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix}}_{L} \underbrace{\begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & -1 \end{bmatrix}}_{U}$ once. Now two queries arrive: solve $A\mathbf{x} = \mathbf{b}_1$ for $\mathbf{b}_1 = (4, 10, 3)^T$ and $A\mathbf{x} = \mathbf{b}_2$ for $\mathbf{b}_2 = (1, 3, 2)^T$. What does the second one cost?

Strategy. Do not re-eliminate. Each solve is two triangular systems: $L\mathbf{c} = \mathbf{b}$ forward, $U\mathbf{x} = \mathbf{c}$ back (figure 1.5).

Solution. $\mathbf{b}_1$: $c_1 = 4$, $c_2 = 10 - 2 \cdot 4 = 2$, $c_3 = 3 - 2 \cdot 2 = -1$; then $-x_3 = -1 \Rightarrow x_3 = 1$, $x_2 = 2 - 1 = 1$, $2x_1 = 4 - 1 \Rightarrow x_1 = \frac{3}{2}$. So $\mathbf{x}_1 = (\frac{3}{2}, 1, 1)^T$. $\mathbf{b}_2$: $c_1 = 1$, $c_2 = 3 - 2 = 1$, $c_3 = 2 - 2 = 0$; then $x_3 = 0$, $x_2 = 1$, $2x_1 = 1 - 1 \Rightarrow x_1 = 0$. So $\mathbf{x}_2 = (0, 1, 0)^T$ — and indeed $A(0, 1, 0)^T = (1, 3, 2)^T$. ✓The second solve cost two $O(n^2)$ sweeps — nothing like the $\frac{2}{3}n^3$ of elimination. This is why libraries return a factorization object, not an answer.

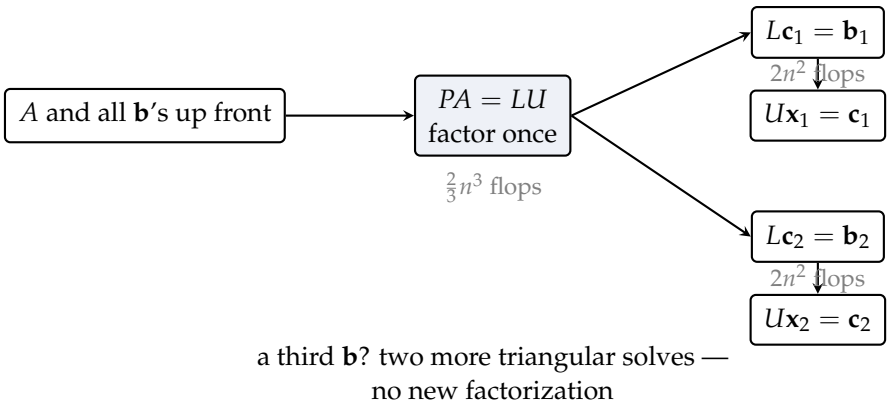


Figure 1.5: Factor once, solve per right-hand side. Notice where the $O(n^3)$ sits: entirely in the factorization. Every later $\mathbf{b}$ pays only triangular-solve prices.

1.5 Cost and stability: why elimination is $O(n^3)$ and why we pivot

The count. To clear column k of an $n \times n$ system you touch the lower-right $(n-k) \times (n-k)$ block: about $(n-k)^2$ multiply-add pairs. Summing over columns,

$$\sum_{k=1}^{n-1} (n-k)^2 = \frac{n(n-1)(2n-1)}{6} \approx \frac{n^3}{3}.$$

So elimination costs about $n^3/3$ multiply-add steps — cubic. That scaling decides feasibility: doubling n multiplies the work by eight, so $n = 10^4$ is fine on a laptop while $n = 10^6$ is not (structured matrices excepted — see the exercises).

```
import time
import numpy as np

def eliminate(A, b):
    n = len(b)
    Ab = np.column_stack([A, b])
    for col in range(n - 1):
        for row in range(col + 1, n):
            Ab[row] -= (Ab[row, col] / Ab[col, col]) * Ab[col]
    return Ab

np.linalg.solve(np.eye(3), np.ones(3)) # BLAS warm-up
print(f"{'n':>5} {'ours (s)':>10} {'solve (s)':>10}")
```

```
for n in [50, 100, 200, 400]:
    rng = np.random.default_rng(0)
    A, b = rng.standard_normal((n, n)), rng.standard_normal(n)
    t0 = time.perf_counter(); eliminate(A, b); t1 = time.perf_counter()
    t2 = time.perf_counter(); np.linalg.solve(A, b); t3 = time.
        perf_counter()
    print(f"{n:>5} {t1-t0:>10.4f} {t3-t2:>10.4f}")
```

Listing 1.5: Timing our loop-based elimination against `np.linalg.solve`. File: `code/ch01_cost_scaling.py`.

```
    n ours (s) solve (s)
   50 0.0008 0.0035
  100 0.0031 0.0002
  200 0.0141 0.0002
  400 0.0648 0.0007
```

Listing 1.6: Output of listing 1.5. Each doubling of n multiplies our time by roughly 4–5 in this range (the pure-Python per-operation overhead masks the asymptotic factor of 8; the fitted log-log slope is ≈ 2.2), while LAPACK behind `np.linalg.solve` is two orders of magnitude faster and its measured slope at larger n is ≈ 3.1.

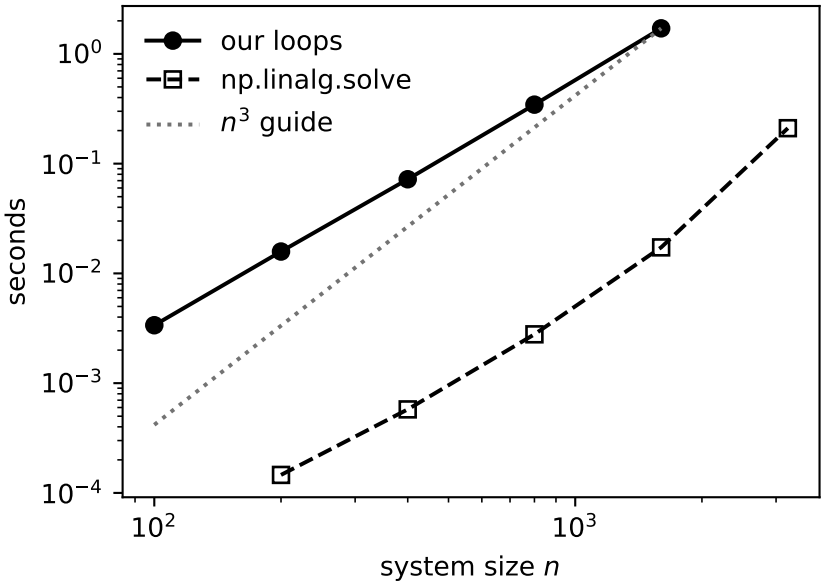


Figure 1.6: Measured cost of elimination vs n on log-log axes, regenerated by `code/ch01_cost_figure.py`. Notice both curves trending toward the dotted n^3 guide: the algorithms differ only in their constant, not their shape. A slope of 3 on these axes is what "cubic" looks like.

The stability problem. The count says nothing about whether the answer is *right*. Floating-point arithmetic rounds every operation, and elimination can amplify rounding catastrophi-

cally when a pivot is tiny. Consider

$$\begin{bmatrix} 10^{-20} & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad \text{true answer } x \approx 1,\ y \approx 1.$$

Eliminating without a swap uses multiplier 10^{20}: $R_2 - 10^{20}R_1$ gives $(1 - 10^{20})y = 2 - 10^{20}$. In double precision $1 - 10^{20} = -10^{20}$, so $y = 1$ exactly — and then the first equation reports $x = (1 - y)/10^{-20} = 0$. Plausible-looking output, confidently wrong. Swap the rows first and both multipliers stay below 1, giving $x \approx 1$, $y \approx 1$.

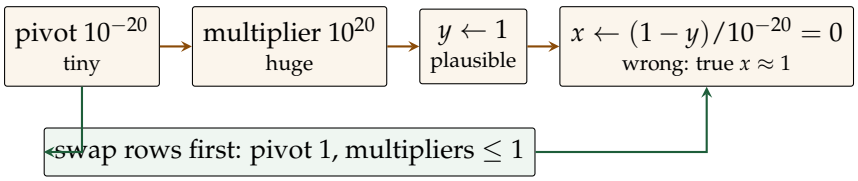


Figure 1.7: How a tiny pivot destroys an answer, and the one-line fix. Notice the failure is silent — every intermediate number looks reasonable. This is why production solvers never take the diagonal entry as pivot blindly.

The fix is *partial pivoting*: at each column, swap the largest-magnitude available entry into the pivot position. This keeps every multiplier $|\ell_{ik}| \leq 1$, which bounds how much rounding can grow (the *growth factor* stays small for essentially all real problems). It costs a comparison per row — negligible against n^3 — and it is why `np.linalg.solve` computes $PA = LU$ with pivoting rather than the naive LU of section 1.4.

Exercise 1.4 (hard). Run the 10^{-20} system in Python three ways: no pivoting, with the row swap, and `np.linalg.solve`. Report the three x-values. Then replace 10^{-20} by 10^{-8}: at what pivot size does the no-swap answer recover, and why?

1.6 Hand derivation

Close the interpreter. Prove theorem 1.4 from scratch: the solution set of $A\mathbf{x} = \mathbf{b}$ is a particular solution plus the whole null space.

1. Show that the set $S = \{\mathbf{x}_p + \mathbf{x}_h : \mathbf{x}_h \in N(A)\}$ contains only solutions (the $S \subseteq \{\text{solutions}\}$ direction).
2. Show that every solution lies in S.
3. Apply it: for example 1.3, verify directly that the difference of any two solutions you can name is a multiple of $(1, -1, 1)^T$.
4. Conclude the count: if A is $m \times n$ of rank r, the solution set (when non-empty) is a flat of dimension $n - r$. Why does that make "exactly two solutions" impossible?

1.7 Common mistakes

SIGN ERRORS IN ELIMINATION BOOKKEEPING

Subtracting a multiple of a row the wrong way — $R_3 + R_1$ when you meant $R_3 - R_1$ — is the single most common error by hand. Defence: after each step, *recheck one untouched entry* and recompute one new zero explicitly. In code, always compare against `np.linalg.solve` before trusting a hand-rolled elimination.

ASSUMING EVERY SQUARE SYSTEM HAS A UNIQUE SOLUTION

Square says nothing: 2×2 can be inconsistent or underdetermined. The pivot structure of the echelon form — not the shape of A — decides between the four cases of figure 1.3.

CONFUSING PIVOT AND FREE VARIABLES

A variable is free iff its *column* lacks a pivot — the test is on the matrix, not on which variables "look" free. Mislabeling here corrupts the whole parametrized solution in examples like 1.3.

SOLVING WITHOUT PIVOTING AND TRUSTING GARBAGE

A zero or tiny pivot does not mean "no solution" — it means "swap a row". And a solver that ran to completion still may have produced a wrong $\mathbf{x}$; the residual $\|A\mathbf{x} - \mathbf{b}\|$ is cheap, so compute it.

1.8 Summary

Concept	Statement	Where used next
Two pictures	$A\mathbf{x} = \mathbf{b}$ = intersecting hyperplanes (rows) = combination of columns	drives the vector-spaces chapter
Row operations	add multiple, scale, swap — all preserve the solution set	everywhere
Echelon form	stair-step zeros; pivots first-nonzero in row	definition 1.2
Rank r	number of pivots	vector-spaces chapter
Solution count	$0 = c$ row: none; $r = n$: one; $r < n$: ∞ (dim $n - r$)	figure 1.3
Solution shape	$\mathbf{x}_p + N(A)$	theorem 1.4; least squares
$PA = LU$	multipliers into L; swaps into P; U = echelon form	every solve later in book
Cost	$\sim n^3/3$ multiply-adds to factor; $O(n^2)$ per new $\mathbf{b}$	optimisation chapters
Partial pivoting	largest $\lvert a_{ik}\rvert$ into pivot; keeps multipliers ≤ 1	all numerical work

DEFINITION OF DONE

You can now solve any small linear system by hand and read off the shape of its solution set without solving at all — the skill every later chapter assumes.

Notes

The structure follows MIT 18.06SC Unit I (*Ax = b and the four subspaces*) [2]: row/column pictures, elimination, LU, then the geometry of solution sets. The interactive Immersive Linear Algebra chapter on Gaussian elimination [6] is a good second pass for the geometry. VMLS chapter 8 [4] treats the same material with a data-first slant and is the source of the exercise style used here. MML §2.3 [5] covers elimination in the compact "for ML" register this book aims at. The pivoting example of section 1.5 is the standard one from Strang's textbook treatment [3]. Timing methodology is best-of-several-repeats after a warm-up call. For the competitive-programming reader this book is aimed at, Sannemo's *Principles of Algorithmic Problem Solving* [1] is the sibling text — its systems material overlaps this chapter's.

WATCH INSTEAD

The first two lectures of MIT's 18.06 match this chapter's first half almost section for section:

1. *The Geometry of Linear Equations* — MIT OpenCourseWare, 18.06 Lecture 1, 39:49, https://youtu.be/J7DzL2_Na80 — the row and column pictures of §1.1 (accessed 2026-10-11).
2. *Elimination with Matrices* — MIT OpenCourseWare, 18.06 Lecture 2, 47:41, https://youtu.be/QVKj3LADCnA — the elimination-to-echelon mechanics of §1.2–§1.4 (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., chs. 1–2 (textbook level) — the canonical treatment of row/column pictures and elimination this chapter follows. Companion to the lectures above.
- MIT OpenCourseWare, *18.06SC Linear Algebra, Fall 2011*, Unit I (self-study level) — worked recitations on the same material. Free at OCW (course "18.06SC", Unit I page), at https://ocw.mit.edu. Verified 2026-10-11.
- Ström, Åström, and Akenine-Möller, *Immersive Linear Algebra* (interactive level) — free at https://immersivemath.com, chapter "Gaussian elimination" — a second, visual pass at the solution-set geometry. Verified 2026-10-11.
- S. Boyd and L. Vandenberghe, *Introduction to Applied Linear Algebra* (VMLS), ch. 8 (free PDF), https://web.stanford.edu/~boyd/vmls/vmls.pdf — the data-first version of our exercise style. Verified 2026-10-11.
- Margalit and Rabinoff, *Interactive Linear Algebra* (Georgia Tech), "Systems of Linear Equations" — free at https://textbooks.math.gatech.edu/ila/ — hyperlinked remediation for any step you found fast here. Verified 2026-10-11.
- J. Hefferon, *Linear Algebra* (free textbook) — rigorous alternative take, with many more proofs than we give. https://hefferon.net/linearalgebra/index.html Verified 2026-10-11.

Additional exercises

Exercise 1.5 (easy)**.** Solve by elimination: $x + y = 7$, $2x - y = 2$. Sketch the row picture and the column picture.

Exercise 1.6 (easy)**.** Write down the augmented matrix of $x_1 - 2x_2 = 1$, $3x_1 + x_2 = 9$, $-x_1 + x_2 = 0$, eliminate it, and state which of the four cases of figure 1.3 this is.

Exercise 1.7 (easy)**.** How many pivots does $\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix}$ have? What is its rank? How many free variables does $A\mathbf{x} = \mathbf{b}$ have?

Exercise 1.8 (easy)**.** Is $(1, 1, 0)^T$ in the null space of $A = \begin{bmatrix} 1 & -1 & 2 \\ 0 & 1 & 1 \end{bmatrix}$? Is $(2, 2, -1)^T$?

Exercise 1.9 (medium). Find all solutions: $x + 2y - z = 3$, $2x + 4y - 2z = 6$, $x + y + z = 3$. Write the answer as $\mathbf{x}_p + t\mathbf{x}_h$.

Exercise 1.10 (medium). Give a 3×3 example of each: (a) unique solution; (b) inconsistent; (c) a line of solutions; (d) a plane of solutions. Verify each by elimination.

Exercise 1.11 (medium). Factor $A = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 0 \\ 2 & 0 & 3 \end{bmatrix}$ as $PA = LU$, writing P, L, U explicitly.

Exercise 1.12 (medium). Show that if $A = LU$ with L unit lower triangular and U invertible upper triangular, then solving $A\mathbf{x} = \mathbf{b}$ costs $O(n^2)$ once L, U are known. Count the multiply-adds of both triangular solves exactly.

Exercise 1.13 (medium). Prove that the set of solutions of $A\mathbf{x} = \mathbf{0}$ is closed under addition and scalar multiplication (so $N(A)$ is a subspace). Where does the proof fail for $A\mathbf{x} = \mathbf{b}$, $\mathbf{b} \neq \mathbf{0}$?

Exercise 1.14 (hard). Implement Gaussian elimination with *partial pivoting* in Python: at column k, swap the row with largest $|a_{ik}|$ (among $i \geq k$) into position before eliminating. Test on the 10^{-20} example of section 1.5 and on 50 random 20×20 systems against `np.linalg.solve`. File: `code/ch01_partial_pivoting.py`.

Exercise 1.15 (hard). A *tridiagonal* matrix has nonzeros only on the diagonal and its two neighbours. Show that elimination on an $n \times n$ tridiagonal matrix costs $O(n)$, not $O(n^3)$, by counting operations — then implement and time it for n up to 10^6.

Exercise 1.16 (hard). Prove that $A = LU$ (no swaps needed) exists iff every leading $k \times k$ submatrix of A is invertible. (Hint: what makes each pivot nonzero?) This is why pivoting is sometimes a mathematical necessity, not just a numerical one.

Exercise 1.17 (medium). Balance butane combustion, $a\,\mathrm{C_4H_{10}} + b\,\mathrm{O_2} \to c\,\mathrm{CO_2} + d\,\mathrm{H_2O}$, by writing the three atom equations, finding the free-variable family, and taking the smallest whole numbers.

Exercise 1.18 (medium). Using the LU of example 1.7, solve $A\mathbf{x} = (2, 5, 4)^T$ without refactoring — then verify by multiplying A times your answer.

Exercise 1.19 (hard). Show a homogeneous system $A\mathbf{x} = \mathbf{0}$ is never inconsistent. Deduce that a chemical-balance system always has infinitely many solutions whenever there are more unknown coefficients than elements, and explain why "take the smallest positive integers" is a convention, not a theorem.

CHAPTER 2

Matrices and matrix algebra

Chapter 1 used a matrix as a box of coefficients feeding an algorithm. This chapter treats the matrix as an object in its own right — something you can multiply, transpose, invert and factor — and builds the small algebra that every later chapter uses without comment. After it you will be able to (a) read $A\mathbf{x}$ as a weighted combination of A's columns; (b) read AB both as composition of two actions and as a grid of inner products; (c) decide when A^{-1} exists and compute it by Gauss–Jordan; and (d) recognize the four special matrix shapes that make algebra cheap: diagonal, triangular, permutation, block.

BEFORE YOU START

Chapter 1: elimination and the picture of $A\mathbf{x} = \mathbf{b}$. *Recap:* a matrix is a rectangular table of numbers; multiplying it by a column vector $\mathbf{x}$ forms the combination of the matrix's columns with weights taken from $\mathbf{x}$.

AFTER THIS CHAPTER YOU CAN

- multiply matrices four different ways and know which one a given question wants;
- read a matrix as a function that moves vectors around, and a product as doing one move after another;
- decide whether a matrix has an inverse and compute it by elimination;
- use triangular, diagonal, permutation and block matrices to simplify a computation.

HOW TO THINK ABOUT THIS

A matrix is not a table to be memorised but a machine: feed it a vector, it returns another. Multiplying two matrices means running one machine after another, which is why the order matters (putting on socks then shoes is not shoes then socks). When a product confuses you, ask what each machine does to a single vector.

2.1 A matrix is a box of coefficients — and a recipe

An $m \times n$ *matrix* A is a rectangular array of numbers with m rows and n columns; we write a_{ij} or A_{ij} for the entry in row i, column j. That is the storage view. The operational view — the one worth tattooing — is that A is a *recipe*: feed it a vector $\mathbf{x} \in \mathbb{R}^n$ and it produces a vector $A\mathbf{x} \in \mathbb{R}^m$. A 3×3 matrix is a machine $\mathbb{R}^3 \to \mathbb{R}^3$; a 2×3 matrix maps $\mathbb{R}^3 \to \mathbb{R}^2$. Nothing about a matrix is mysterious once you hold both views: the box *is* the recipe, written out entry by entry.

Two special members of the family show up constantly. The *zero matrix* 0 sends everything to $\mathbf{0}$. The *identity matrix* I_n — ones on the diagonal, zeros elsewhere — is the recipe "change nothing": $I_n\mathbf{x} = \mathbf{x}$. And matrices of the same shape add entrywise: $(A + B)_{ij} = a_{ij} + b_{ij}$, which is exactly "do both recipes and add the results"; likewise cA scales every entry. So the set of $m \times n$ matrices is itself a vector space — a fact the vector-spaces chapter cashes in.

2.2 $A\mathbf{x}$ as a column combination

Chapter 1 introduced the column picture for solving; here it becomes the definition. The product $A\mathbf{x}$ is the combination of A's *columns* weighted by the entries of $\mathbf{x}$:

$$A\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n, \qquad \mathbf{a}_j = \text{column } j \text{ of } A.$$

Example 2.1. Compute $A\mathbf{x}$ for $A = \begin{bmatrix} 1 & 0 & 2 \\ 2 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$, $\mathbf{x} = (3, -1, 2)^T$. **Strategy.** Read $A\mathbf{x}$ column-wise: take 3 of column 1, subtract one of column 2, add two of column 3. Then sanity-check one entry the row way.

Solution. $3\mathbf{a}_1 - 1\mathbf{a}_2 + 2\mathbf{a}_3 = 3(1,2,0)^T - (0,1,1)^T + 2(2,0,1)^T = (7,5,1)^T$. Row check: row 1 gives $1 \cdot 3 + 0 \cdot (-1) + 2 \cdot 2 = 7$; row 2 gives $6 - 1 + 0 = 5$; row 3 gives $0 - 1 + 2 = 1$. ✓

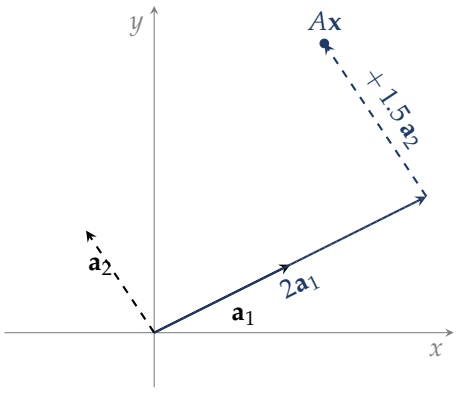


Figure 2.1: $A\mathbf{x}$ as a weighted column combination. Notice the recipe nature of the picture: the columns $\mathbf{a}_1, \mathbf{a}_2$ are fixed by A; the weights come from $\mathbf{x}$; the output is where the chain ends. Change $\mathbf{x}$ and the endpoint moves — that motion *is* the matrix acting.

Two consequences fall out immediately and are worth memorizing rather than re-deriving. First, $A\mathbf{e}_j = \mathbf{a}_j$: multiplying by the j-th *standard basis vector* $\mathbf{e}_j$ (1 in position j, 0 elsewhere)

picks out column j — so you can probe a matrix column by column. Second, $A\mathbf{x} = \mathbf{0}$ has a nontrivial solution exactly when the columns of A are *dependent* (some column is a combination of the others). That single sentence is the seed of rank, null space, and invertibility all at once.

Exercise 2.1 (easy)**.** Let $A = \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix}$, $\mathbf{x} = (4, 1)^T$. Compute $A\mathbf{x}$ by the column rule, then verify entry 2 by the row rule. Then find a nonzero $\mathbf{x}$ with $A\mathbf{x} = (2, -1)^T$ — reading $A\mathbf{x}$ as a column combination, which column is being singled out?

2.3 Matrix multiplication as composition

If B is a recipe and A is a recipe, the obvious thing to build is "do B first, then A" — the *composition* $A \circ B$. Matrix multiplication is exactly that, packaged as one matrix:

$$(AB)\mathbf{x} = A(B\mathbf{x}), \qquad \text{for every } \mathbf{x}.$$

Working out what entry of AB must be to make this true gives the familiar rule: if A is $m \times k$ and B is $k \times n$,

$$(AB)_{ij} = \sum_{l=1}^{k} a_{il}\, b_{lj} = (\text{row } i \text{ of } A) \cdot (\text{column } j \text{ of } B).$$

The product exists only when the inner dimensions match — columns of A = rows of B — and the result is $m \times n$. Figure 2.2 is the picture to hold: entry (i, j) lives where row i of A meets column j of B.

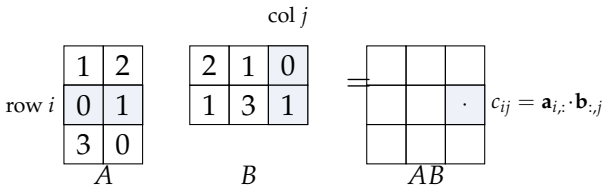


Figure 2.2: AB as a grid of inner products. Notice the highlighted cell: its value is row i of A dotted with column j of B, and nothing else in either matrix touches it. Each of the mn cells is an independent dot product — which is also why matrix multiplication parallelizes so well.

The algebra that follows is best learned by contrast with numbers:

- *Associative*: $A(BC) = (AB)C$, always — because both sides mean "C, then B, then A" and composition is unambiguous. (You will prove this properly in the hand derivation.)
- *Distributive*: $A(B + C) = AB + AC$ and $(A + B)C = AC + BC$.
- *Not commutative*: $AB \neq BA$ in general. Order matters because the recipes do different things in different orders — and because often only one of the two products is even defined.

```
import time
import numpy as np

```

```
def matmul(A, B):
    n, k, m = A.shape[0], B.shape[0], B.shape[1]
    C = np.zeros((n, m))
    for i in range(n):
        for j in range(m):
            for l in range(k):
                C[i, j] += A[i, l] * B[l, j]
    return C

rng = np.random.default_rng(1)
np.seterr(all="ignore") # Accelerate BLAS prints spurious fp warnings
np.matmul(np.eye(4), np.eye(4)) # warm-up

print(f"{'n':>5} {'ours (s)':>10} {'np (s)':>10} {'agree?':>8}")
for n in [128, 256]:
    A, B = rng.standard_normal((n, n)), rng.standard_normal((n, n))
    t0 = time.perf_counter(); C_ours = matmul(A, B); t1 = time.
        perf_counter()
    t2 = time.perf_counter(); C_np = np.matmul(A, B); t3 = time.
        perf_counter()
    print(f"{n:>5} {t1-t0:>10.4f} {t3-t2:>10.4f} {str(np.allclose(
        C_ours, C_np)):>8}")
for n in [512, 1024]:
    A, B = rng.standard_normal((n, n)), rng.standard_normal((n, n))
    t2 = time.perf_counter(); np.matmul(A, B); t3 = time.perf_counter()
    print(f"{n:>5} {'--':>10} {t3-t2:>10.4f} {'--':>8}")
```

Listing 2.1: Naive triple-loop `matmul` against `np.matmul`: correct, and painfully slow. File: `code/ch02_matmul_triple.py`.

```
    n ours (s) np (s) agree?
  128 0.4196 0.0001 True
  256 3.3752 0.0002 True
  512 -- 0.0007 --
 1024 -- 0.0062 --
```

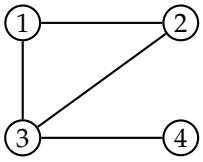
Listing 2.2: Output of listing 2.1. Our loop agrees exactly with the library — the rule is the same; doubling n multiplies our time by ~ 8 ($2n^3$ multiply-adds), while LAPACK stays four orders of magnitude faster by blocking for cache.

Exercise 2.2 (easy)**.** For $A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$ and $B = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, compute AB and BA by hand. Are they equal? Then explain in one sentence what each does to the vector $\mathbf{e}_1$.

Example 2.2. The small graph of figure 2.3 has edges $1{-}2$, $1{-}3$, $2{-}3$, $3{-}4$. How many two-step walks go from vertex 1 to vertex 3?

Strategy. Write the adjacency matrix A with $a_{ij} = 1$ when $i{-}j$ is an edge. Then $(A^2)_{ij} = \sum_k a_{ik}a_{kj}$ counts walks $i \to k \to j$ — a matrix product that is literally a path counter.

Solution. $A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$, and $(A^2)_{13} = a_{11}a_{13} + a_{12}a_{23} + a_{13}a_{33} + a_{14}a_{43} = 0 + 1 \cdot 1 + 0 + 0 = 1$. The single two-step walk is $1 \to 2 \to 3$ — $1 \to 3 \to 3$ is not a walk because there is no loop edge $3{-}3$. The same square gives $(A^2)_{33} = 3$ (three two-step loops back to 3). ✓



walks $1 \to 3$ in two steps:
$1 \to 2 \to 3$ and $1 \to 3 \to \cdots$?

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \qquad A^2 = \begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 1 \\ 1 & 1 & 3 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}$$

$(A^2)_{13} = 1$: exactly one two-step walk from 1 to 3, namely $1 \to 2 \to 3$

Figure 2.3: The adjacency matrix squared counts two-step walks. Notice why matrix multiplication is the right tool: entry (i, j) of A^2 sums over *every possible middle stop* k — the dot-product grid of figure 2.2 is a walk counter.

2.4 Transpose, inverses, and when they exist

The *transpose* A^T flips a matrix across its diagonal: $(A^T)_{ij} = a_{ji}$ — rows become columns. Two rules carry the whole algebra of the transpose:

$$(A^T)^T = A, \qquad (AB)^T = B^T A^T.$$

The second is the *reverse-order* rule and it surprises everyone once. Why reversed? Because $(AB)\mathbf{x} = A(B\mathbf{x})$, and an inner product reads symmetrically: row i of AB dotted with $\mathbf{x}$ equals column i of $B^T A^T$ dotted with $\mathbf{x}$ — the next example makes it concrete. A *symmetric* matrix satisfies $A^T = A$; they own the spectral theorem later, and you build one for free whenever you form $A^T A$.

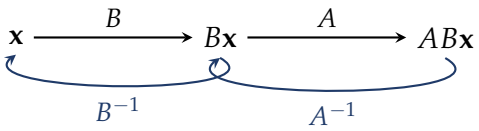
Example 2.3. Verify $(AB)^T = B^T A^T$ on paper for $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, then prove it in general. **Strategy.** Compute both sides on the 2×2 case first — one matrix each, ten seconds — and only then do the index bookkeeping for the proof. The check is not the proof; it is what keeps the proof honest.

Solution. $AB = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}$, so $(AB)^T = \begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}$. And $B^T A^T = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}$. Equal. ✓*Proof.* The (i, j) entry of $(AB)^T$ is the (j, i) entry of AB: $\sum_l a_{jl} b_{li}$. The (i, j) entry of $B^T A^T$ is $\sum_l (B^T)_{il} (A^T)_{lj} = \sum_l b_{li} a_{jl}$ — the same sum, terms commuted. □

Now the inverse. A square matrix A is *invertible* when a matrix A^{-1} exists with

$$A^{-1}A = I = AA^{-1}.$$

Invertible means exactly: the recipe can be undone — A sends distinct inputs to distinct outputs and hits everything in range. Chapter-1 language translates that into a test you already own: *A is invertible iff elimination produces n pivots* (iff the only solution of $A\mathbf{x} = \mathbf{0}$ is $\mathbf{x} = \mathbf{0}$, iff the columns are independent). To actually compute A^{-1}, run elimination on the $2n$-column matrix $[\,A \mid I\,]$ and continue past echelon form until the left half is I; the right half is then A^{-1}. This is *Gauss–Jordan elimination*.



undo AB by undoing A *then* B:
$(AB)^{-1} = B^{-1}A^{-1}$

Figure 2.4: The inverse as undo, and why $(AB)^{-1} = B^{-1}A^{-1}$. Notice the order: to walk the chain back you must undo the *last* step first. Same reason you take off shoes before socks.

Example 2.4. Invert $A = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 5 & 1 \\ 0 & 1 & 3 \end{bmatrix}$ by Gauss–Jordan. **Strategy.** Augment $[\,A \mid I\,]$, eliminate down to echelon form exactly as in Chapter 1, then keep going: scale each pivot to 1 and clear *upward* so the left half becomes I. Whatever happened to the right half is the answer.

Solution.

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 0 & 1 & 0 & 0 \\ 2 & 5 & 1 & 0 & 1 & 0 \\ 0 & 1 & 3 & 0 & 0 & 1 \end{array}\right] \to \left[\begin{array}{ccc|ccc} 1 & 2 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & -2 & 1 & 0 \\ 0 & 0 & 2 & 2 & -1 & 1 \end{array}\right] \to \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 7 & -3 & 1 \\ 0 & 1 & 0 & -3 & 3/2 & -1/2 \\ 0 & 0 & 1 & 1 & -1/2 & 1/2 \end{array}\right]$$

($R_2 - 2R_1$, then $R_3 - R_2$, then scale and clear upward.) So $A^{-1} = \begin{bmatrix} 7 & -3 & 1 \\ -3 & 3/2 & -1/2 \\ 1 & -1/2 & 1/2 \end{bmatrix}$, and the code below confirms $AA^{-1} = I$ exactly.

```
import numpy as np

A = np.array([[1., 2., 0.], [2., 5., 1.], [0., 1., 3.]])
n = A.shape[0]
AI = np.column_stack([A, np.eye(n)])

for col in range(n):
    AI[col] = AI[col] / AI[col, col] # normalize pivot row
    for row in range(n):
        if row != col:
```

```
            AI[row] -= AI[row, col] * AI[col] # kill the column elsewhere

A_inv = AI[:, n:]
print("A_inv by Gauss-Jordan:")
print(A_inv)
print("np.linalg.inv agrees:", np.allclose(A_inv, np.linalg.inv(A)))
print("A @ A_inv:")
print(A @ A_inv)
```

Listing 2.3: Gauss–Jordan inverse on example 2.4, verified against `np.linalg.inv`. File: `code/ch02_gauss_jordan_inverse.py`.

```
A_inv by Gauss-Jordan:
[[ 7. -3. 1. ]
 [-3. 1.5 -0.5]
 [ 1. -0.5 0.5]]
np.linalg.inv agrees: True
A @ A_inv:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Listing 2.4: Output of listing 2.3. The hand computation and the library agree to the last digit; $AA^{-1} = I$ on the nose.

The practical rule deserves its own line, because violating it is a rite of passage everyone fails once: **never compute A^{-1} to solve $A\mathbf{x} = \mathbf{b}$.** Chapter 1's LU solve is faster, more stable, and gives the same answer. The inverse exists in this book as an algebraic object — for proofs, small closed forms, and understanding — not as a solver.

Exercise 2.3 (medium)**.** Two claims, prove or disprove each: (a) if A and B are invertible, so is AB, and $(AB)^{-1} = B^{-1}A^{-1}$; (b) if $AB = I$ for square A, B, then also $BA = I$. (For (b), think of A's columns: what does $AB = I$ say about them?)

2.5 Special matrices: diagonal, triangular, permutation, block

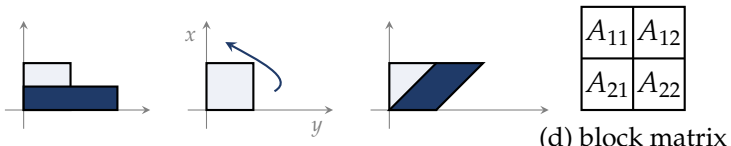
Four shapes recur so often that each gets a name and a shortcut; figure 4.5 shows three of them acting on the plane plus the fourth in box form.

Diagonal $D = \mathrm{diag}(d_1, \ldots, d_n)$: scales axis i by d_i and does nothing else. $D\mathbf{x}$ is entrywise scaling; $D_1 D_2$ is diagonal again $(d_i d_i')$; $D^{-1} = \mathrm{diag}(1/d_i)$ when no $d_i = 0$. The cheapest matrix there is — eigenvalue theory later asks "how close to diagonal can we make A?"

Triangular upper U or lower L: the output of Chapter 1. Triangular systems solve by substitution in $O(n^2)$; products stay triangular; diagonal entries are the pivots (Chapter 6: the eigenvalues).

Permutation P: I with rows reordered — one 1 per row and column. $P\mathbf{x}$ relabels coordinates; $P^T = P^{-1}$ undoes it. They are why 'shuffle' and 'reorder' are free in linear algebra.

Block $\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$: matrices whose entries are matrices. The multiplication rule is unchanged — block row times block column, summed — provided every inner product is dimensionally legal. Proofs and parallel code both lean on this constantly.



(a) diag$(2, \frac{1}{2})$ (b) permutation (c) $\left[\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right]$ (shear) (d) block matrix

Figure 2.5: Three special matrices acting on the unit square, and the fourth in box form. Notice that diagonal only stretches axes, permutation only relabels them, and the shear (upper triangular) leaves the bottom edge alone — the special structures are exactly the restrictions that make their algebra cheap.

```python
import numpy as np

# a tiny "dataset": each row tagged with its original index
X = np.array([[0., 10.], [1., 11.], [2., 12.], [3., 13.]])
sigma = [2, 0, 3, 1] # new order of rows

n = len(sigma)
P = np.zeros((n, n))
P[np.arange(n), sigma] = 1 # row i of PX is row sigma[i] of X

print("P =\n", P)
print("P @ X =\n", P @ X)
print("X[sigma] =\n", X[sigma])
print("same thing:", np.allclose(P @ X, X[sigma]))
print("P.T @ P = I:", np.allclose(P.T @ P, np.eye(n)))
print("undo with P.T:", np.allclose(P.T @ (P @ X), X))
```

Listing 2.5: A permutation matrix is a reorder, and P^T undoes it — the same trick a data loader uses when it shuffles sample order. File: `code/ch02_permutation_dataset.py`.

```
P =
 [[0. 0. 1. 0.]
 [1. 0. 0. 0.]
 [0. 0. 0. 1.]
 [0. 1. 0. 0.]]
P @ X =
 [[ 2. 12.]
 [ 0. 10.]
 [ 3. 13.]
 [ 1. 11.]]
X[sigma] =
 [[ 2. 12.]
 [ 0. 10.]
 [ 3. 13.]
 [ 1. 11.]]
```

```
same thing: True
P.T @ P = I: True
undo with P.T: True
```

Listing 2.6: Output of listing 2.5. $P@X$ is exactly fancy-indexing `X[sigma]`; $P^TP = I$ because a permutation never duplicates or drops a row.

Example 2.5. Shuffle a three-row dataset A so that row 3 comes first, then rows 1 and 2 in order. Write the matrix that does it — and check it really is a *permutation matrix*.

Strategy. A row shuffle is a left multiplication (row operations live on the left). Build P row by row: row i of PA should be the picked row of A — put a single 1 in the right column.

Solution. We want $(PA)_1 = A_3$, $(PA)_2 = A_1$, $(PA)_3 = A_2$:

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \qquad PA = \begin{bmatrix} a_{31} & a_{32} \\ a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}.$$

Each row of P is a standard basis row $\mathbf{e}_j^T$, which *picks* row j of A (figure 2.6). Every row shuffle of a training set is exactly this: one left multiplication, no data touched twice, invertible by P^T. ✓

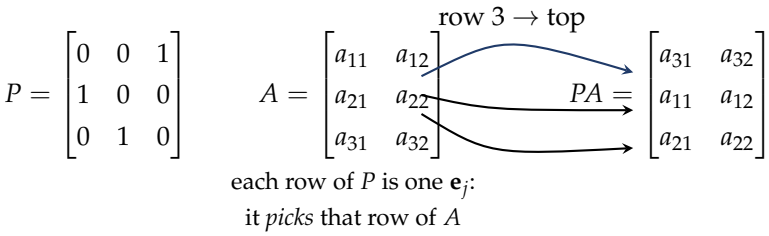


Figure 2.6: A permutation matrix is a shuffle. Notice the mechanism: each row of P is one $\mathbf{e}_j^T$, and $\mathbf{e}_j^T A$ is row j of A — a product of picks.

2.6 Hand derivation

Two proofs, both short, both worth doing with the interpreter closed.

1. Prove $(AB)^{-1} = B^{-1}A^{-1}$ directly from the definition: a matrix X is *the* inverse of M iff $MX = I$ and $XM = I$. Compute $(B^{-1}A^{-1})(AB)$ and $(AB)(B^{-1}A^{-1})$ using associativity only — no entries allowed. Conclude uniqueness of the inverse: if X and Y both satisfy the definition for A, then $X = Y$ (multiply $AX = I$ by Y on the left).
2. Prove associativity $(AB)C = A(BC)$ via composition: define $f_B(\mathbf{x}) = B\mathbf{x}$ and show both sides name the same function, $f_A \circ f_B \circ f_C$, applied to an arbitrary $\mathbf{x}$. This is the "functions compose associatively" argument — write it out fully, including the step where you use the definition $(MN)\mathbf{x} = M(N\mathbf{x})$.

2.7 Common mistakes

ASSUMING $AB = BA$

Matrix multiplication is not commutative, full stop. When a manipulation needs you to swap the order — expanding $(A+B)^2$, moving a term past another — write out both orders first and check which you actually have. $(A+B)^2 = A^2 + AB + BA + B^2$, and that middle pair usually cannot merge.

WRONG ORDER IN $(AB)^T$

The transpose of a product is the product of the transposes *reversed*: $B^T A^T$, not $A^T B^T$. Same reversal for inverses. Mnemonic: to undo AB you meet B last, so B's undo comes first.

DIMENSION MISMATCHES READ AS "ALMOST RIGHT"

A 3×2 times a 3×2 is not "nearly" a product — it is undefined, and no reshaping the answer rescues it. Before every multiplication, name the shapes: $(m \times k)(k \times n) \to (m \times n)$. If the inner numbers disagree, stop.

BELIEVING EVERY NONZERO MATRIX IS INVERTIBLE

$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ is about as nonzero as matrices get, and it crushes the whole plane onto a line — no undo exists. Invertibility is about the columns being independent, which Chapter 1 already showed you how to test: n pivots, or nothing.

2.8 Summary

Concept	Statement	Where used next
$A\mathbf{x}$	column combination, weights from $\mathbf{x}$	everything
AB	composition; $(AB)_{ij} = \text{row}_i \cdot \text{col}_j$; needs inner dims equal	every model
Associativity	$A(BC) = (AB)C$ via composition	proofs, chaining layers
$AB \neq BA$	order is part of the meaning	commutator exercises
Transpose	$(A^T)_{ij} = a_{ji}$; $(AB)^T = B^T A^T$	$A^T A$, symmetric ch.
Inverse	$AA^{-1} = I$; exists iff n pivots; Gauss–Jordan on $[A\|I]$	least squares (formally)
$(AB)^{-1}$	$B^{-1}A^{-1}$ — undo last step first	hand derivation
Never invert to solve	LU-solve is cheaper and stabler	all numerical work
Special shapes	diagonal (scale), triangular (substitute), permutation ($P^T = P^{-1}$), block (same rules)	rest of book

DEFINITION OF DONE

You can now multiply matrices both ways — as dot-product grids and as composed recipes — invert a small one by hand, and spot the four shapes that make later algebra cheap.

Notes

The composition-first treatment of matrix multiplication follows MIT 18.06SC Unit I [2]; the column-combination mantra for $A\mathbf{x}$ is Strang's, restated. VMLS chapters 6 and 10 [4] supply the data-first framing (the permutation-as-shuffle example is theirs in spirit) and the gallery treatment of special matrices. MML §2.2 [5] is the compact reference for the same algebra. The Immersive Linear Algebra chapter on the matrix [6] has the interactive versions of figures 2.1 and 2.2. The never-invert-to-solve rule is standard numerical practice [3].

WATCH INSTEAD

1. *Matrix multiplication as composition* — 3Blue1Brown, Essence of Linear Algebra ch. 4, 10:03, https://youtu.be/XkY2DOUCWMU — the recipe-composition view of §2.3 (accessed 2026-10-11).
2. *Multiplication and Inverse Matrices* — MIT OpenCourseWare, 18.06 Lecture 3, 46:49, https://youtu.be/hMmXCdVym3A — the five-product viewpoints plus Gauss–Jordan inversion of §2.3–§2.4 (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., ch. 2 (textbook level) — the matrix-multiplication and inverse material at full length.
- MIT OpenCourseWare, *18.06SC Linear Algebra* — the OCW course pages behind the watch picks; worked recitations on products and inverses.
  OCW (course "18.06SC"), at https://ocw.mit.edu Verified 2026-10-11.
- S. Boyd and L. Vandenberghe, *VMLS*, chs. 6, 10 (free PDF) — matrix algebra and inverses with data-first examples (the row-shuffle view is theirs).
  https://web.stanford.edu/~boyd/vmls/vmls.pdf Verified 2026-10-11.
- Margalit and Rabinoff, *Interactive Linear Algebra* (Georgia Tech), "Matrix Algebra" — the same algebra hyperlinked.
  https://textbooks.math.gatech.edu/ila/ Verified 2026-10-11.
- M. Deisenroth et al., *Mathematics for Machine Learning*, §2.2 (free PDF) — the compact ML-register reference.
  https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.
- Ström, Åström, and Akenine-Möller, *Immersive Linear Algebra*, chapter "The matrix" — interactive versions of our figures 2.1 and 2.2. Free at https://immersivemath.com. Verified 2026-10-11.

Additional exercises

Exercise 2.4 (easy). Let $A = \begin{bmatrix} 1 & -2 \\ 3 & 0 \end{bmatrix}$, $\mathbf{x} = (5, 1)^T$, $\mathbf{y} = (0, 4)^T$. Compute $A\mathbf{x}$, $A\mathbf{y}$, and $A(\mathbf{x} + \mathbf{y})$; confirm the third equals the sum of the first two. Which property of the recipe is that?

Exercise 2.5 (easy). For $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, compute A^T, AA^T, and A^TA. Which two are equal here, and is that an accident?

Exercise 2.6 (easy). Write the 3×3 permutation matrix that sends (x_1, x_2, x_3) to (x_3, x_1, x_2). Apply it to $\mathbf{x} = (1, 2, 3)^T$ twice: what do you get and why?

Exercise 2.7 (easy). $D = \mathrm{diag}(2, -1, 3)$. Compute D^2, D^3, and D^{-1}. What is D^k in general?

Exercise 2.8 (medium). For $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ and $B = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, compute the *commutator* $[A, B] = AB - BA$. Then compute $[A, [A, B]]$. When is $[A, B] = 0$ in general — what does it say about the two recipes?

Exercise 2.9 (medium). Show that $(A^TA)^T = A^TA$ for any matrix A — every Gram matrix is symmetric. Where does this fact get used when we reach least squares?

Exercise 2.10 (medium). Let E_{ij} be the matrix that is 1 in position (i, j) and 0 elsewhere. Compute $E_{12}A$ and AE_{12} for a general 3×3 matrix A — describe the action in words. (These are the atoms of the "row operation as matrix" story.)

Exercise 2.11 (medium — derive). A is 2×3, B is 3×2. Show that $\mathrm{tr}(AB) = \mathrm{tr}(BA)$, where tr sums the diagonal. Does your argument care that the matrices are not square in the same direction?

Exercise 2.12 (hard). Prove the block multiplication rule: if $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ and B similarly partitioned with all inner products legal, then the (i, j) block of AB is $\sum_k A_{ik}B_{kj}$. (Hint: expand one entry of the left side and one of the right side; the sums are the same.)

Exercise 2.13 (hard). Show that a square triangular matrix is invertible iff every diagonal entry is nonzero, and that its inverse is triangular of the same kind. (Use Chapter 1's pivot test for the first half; substitution for the second.)

Exercise 2.14 (hard). Implement blocked matrix multiplication in Python: cut two $n \times n$ matrices into $b \times b$ blocks, multiply block-wise (one `np.matmul` per block pair), and reassemble. Verify against `np.matmul` for $n = 512$, $b = 64$. File: `code/ch02_blocked_matmul.py`.

Exercise 2.15 (easy). For the graph of example 2.2, compute $(A^2)_{44}$ and $(A^2)_{14}$ and name the walks each counts. What does $(A^3)_{14}$ count? Compute it.

Exercise 2.16 (medium). Invert $\begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ by Gauss–Jordan on $[\,A \mid I\,]$. Verify by multiplying back in both orders.

Exercise 2.17 (medium)**.** Let P be the 3×3 shift permutation of example 2.5. Compute P^T, then PP^T — and conclude why $P^{-1} = P^T$ for every permutation matrix.

Exercise 2.18 (hard)**.** Show $(A^2)_{ii} = \deg(i)$ for a graph with no loops, and that $\mathrm{tr}(A^2)$ equals twice the edge count. What does $\mathrm{tr}(A^3)$ count on a triangle-rich graph?

CHAPTER 3

Vector spaces, subspaces, and the four fundamental subspaces

Chapters 1–2 worked inside $\mathbb{R}^n$ without ever saying what $\mathbb{R}^n$ *is*. This chapter names the game: a vector space is a place where linear algebra works, a subspace is a smaller place inside it that still plays by the same rules, and every matrix quietly cuts its input space and output space into four pieces that explain exactly what the matrix can and cannot do. After this chapter you will be able to test whether a set is a subspace, extract a basis by elimination, count dimensions without computing anything, and draw the four fundamental subspaces of any matrix — the single most reused picture in applied linear algebra.

BEFORE YOU START

Chapters 1–2. *Recap:* the columns of A are vectors; $A\mathbf{x}$ is a combination of them. The question "which right-hand sides $\mathbf{b}$ can we reach?" is the question this chapter answers in full.

AFTER THIS CHAPTER YOU CAN

- decide whether a set of vectors is a subspace and whether vectors are linearly independent;
- find a basis and the dimension of a subspace;
- compute bases for the column space, row space, null space and left null space of a matrix;
- use rank–nullity to count solutions without solving.

HOW TO THINK ABOUT THIS

A subspace is a flat sheet through the origin that is closed under adding and scaling, the natural home of solutions. The four fundamental subspaces answer four questions about a matrix: what can it reach, what does it destroy, and the same for its transpose. Try to see them as the answers to "which $\mathbf{b}$ are reachable?" and "which $\mathbf{x}$ vanish?"

3.1 Vector spaces: axioms with motivation, not ceremony

A *vector space* over $\mathbb{R}$ is a set V together with two operations — addition $\mathbf{u}+\mathbf{v}$ and scalar multiplication $c\mathbf{v}$ — that stay inside V and obey the rules you already use on arrows: commutativity and associativity of $+$, a zero vector, additive inverses, $1\mathbf{v}=\mathbf{v}$, and the two distributive laws. That is the complete list, and the honest reason for the list is parsimony, not piety: everything we prove about $\mathbb{R}^n$ uses only these properties, so it automatically holds anywhere they hold. Write them once, then forget them — you will never again verify an axiom in this book, but every theorem will quietly depend on them.

The examples that matter here: $\mathbb{R}^n$ itself; the set of $m\times n$ matrices (Chapter 2 noted it is a vector space); the polynomials of degree at most n, written $\mathcal{P}_n$; and the set of solutions $\{\mathbf{x} : A\mathbf{x}=\mathbf{0}\}$, which is a vector space *because* the equations are linear — add two solutions or scale one and you still have a solution. That last example is the point of the whole chapter: solution sets of linear systems are exactly the subspaces we are about to name.

Exercise 3.1 (easy)**.** For each set, decide if it is a vector space under the usual operations: (a) vectors in $\mathbb{R}^3$ with integer coordinates; (b) solutions of $A\mathbf{x}=\mathbf{b}$ with $\mathbf{b}\neq\mathbf{0}$; (c) polynomials of degree *exactly* 3.

3.2 Subspaces

A *subspace* of V is a subset that is itself a vector space. You do not need to check ten axioms — almost all are inherited. Only three things can go wrong:

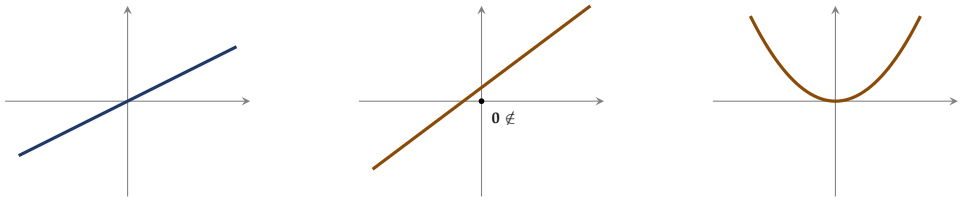
1. $\mathbf{0}\in S$ (the set must contain the zero vector),
2. closed under $+$ ($\mathbf{u},\mathbf{v}\in S\Rightarrow\mathbf{u}+\mathbf{v}\in S$),
3. closed under scaling ($\mathbf{v}\in S\Rightarrow c\mathbf{v}\in S$ for all $c\in\mathbb{R}$).

Geometrically, subspaces of $\mathbb{R}^3$ are exactly: $\{\mathbf{0}\}$, lines through the origin, planes through the origin, and $\mathbb{R}^3$ itself. *Through the origin* is the tell — a subspace must contain $\mathbf{0}$, and it must be flat.

Example 3.1. Which of these are subspaces of $\mathbb{R}^3$? $S_1=\{(x,y,z) : x+2y+z=0\}$ and $S_2=\{(x,y,z) : x+2y+z=1\}$. **Strategy.** Same equation, different right-hand side — run the three-check on each. Expect exactly one to pass, and the fastest kill is property (1).

Solution. S_1: $\mathbf{0}$ satisfies $0+0+0=0$ ✓. If (x,y,z) and (x',y',z') are in S_1, the sum has $x+x'+2(y+y')+(z+z')=0+0=0$ ✓. Scaling works identically ✓. So S_1 is a subspace — the plane through the origin with normal $(1,2,1)$. S_2: $\mathbf{0}$ fails $0+0+0=1$ immediately — also, doubling $(0,0,1)\in S_2$ leaves the set. Not a subspace: it is the same plane shifted off the origin.

The two subspaces attached to a matrix are worth naming now. The *null space* (or kernel) $\ker(A)=\{\mathbf{x} : A\mathbf{x}=\mathbf{0}\}$ is a subspace of $\mathbb{R}^n$ — the solution set from Chapter 1, finally given an address. The *column space* $\mathrm{col}(A)=\{A\mathbf{x} : \mathbf{x}\in\mathbb{R}^n\}$ — every vector you can *build* from the columns — is a subspace of $\mathbb{R}^m$.



(a) line through **0**: subspace (b) same slope, misses **0**: not (c) contains **0** but bends: not

Figure 3.1: The subspace test as a picture. (a) passes all three checks; (b) fails the first because it misses **0** — no amount of flatness rescues a shifted line; (c) contains **0** but fails closure: a scaled point on the curve leaves the curve.

3.3 Span and linear independence

The *span* of vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ is the set of all their linear combinations — everything you can reach by scaling and adding. It is always a subspace (check the three properties; they fall out in one line each). Vectors are *linearly independent* if the only way to combine them into **0** is the trivial way $c_1 = \cdots = c_k = 0$; otherwise some vector is redundant — a combination of the others. In equation form: the columns of A are independent exactly when $\ker(A) = \{\mathbf{0}\}$ — the connection between this section and Chapter 1's free variables is that free variables are precisely what produces nonzero null-space vectors.

Spanning and independence pull in opposite directions: spanning says the set is *big enough* (you can reach everything); independence says it is *tight enough* (nothing wasted). Figure 3.2 keeps the two apart — the confusion between them is mistake number one on every exam.

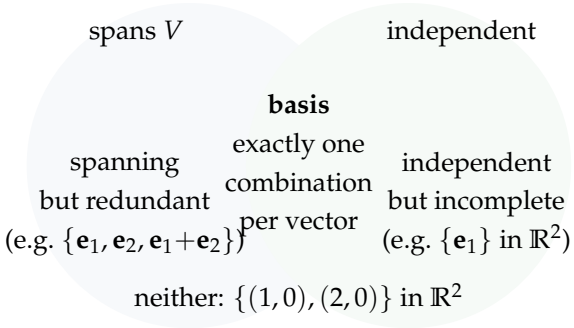


Figure 3.2: Spanning ∩ independent = basis. Notice the two failure modes are genuinely different diseases: the left set can reach everything but has a member it does not need; the right set has no redundancy but cannot reach most of $\mathbb{R}^2$.

3.4 Basis and dimension

A *basis* of V is a set that is both spanning and independent — a minimal generating set, equivalently a maximal independent set. The payoff: every $\mathbf{v} \in V$ is a combination of basis vectors in *exactly one* way (uniqueness is what independence buys; existence is what spanning buys). Any two bases of V have the same size — a nontrivial theorem whose proof is a careful exchange argument we take on faith here — and that common size is the *dimension* $\dim V$.

Example 3.2. What is $\dim \mathbb{R}^{m\times n}$ (all $m \times n$ matrices)? And $\dim \mathcal{P}_n$ (polynomials $\leq n$)? **Strategy.** Don't count the space — exhibit a basis and count *it*. For matrices the atoms are obvious: one nonzero entry at a time.

Solution. Let E_{ij} be the matrix that is 1 in position (i, j) and 0 elsewhere. Every A is $\sum_{i,j} a_{ij}E_{ij}$, and the only combination giving the zero matrix has all $a_{ij} = 0$ — spanning and independent. There are mn of them: $\dim \mathbb{R}^{m\times n} = mn$. For polynomials, $\{1, t, t^2, \ldots, t^n\}$ spans by definition and is independent (a polynomial that is identically zero has all coefficients zero): $\dim \mathcal{P}_n = n + 1$.

Exercise 3.2 (easy)**.** Find a basis and the dimension of the subspace of symmetric 2×2 matrices $\{A : A^T = A\}$. Then of the skew-symmetric ones $\{A : A^T = -A\}$.

3.5 The four fundamental subspaces of A

Every $m \times n$ matrix comes with four subspaces, two in the domain $\mathbb{R}^n$ and two in the codomain $\mathbb{R}^m$:

- $\mathrm{col}(A) \subseteq \mathbb{R}^m$ — what A can output: the span of its columns.
- $\ker(A) \subseteq \mathbb{R}^n$ — what A kills: the solutions of $A\mathbf{x} = \mathbf{0}$.
- $\mathrm{row}(A) = \mathrm{col}(A^T) \subseteq \mathbb{R}^n$ — the span of its rows.
- $\ker(A^T) \subseteq \mathbb{R}^m$ — the *left null space*: the $\mathbf{y}$ with $\mathbf{y}^T A = \mathbf{0}^T$, i.e. the codomain vectors dotted to zero by every column of A (the perpendicularity is made precise in Chapter 7).

Example 3.3. Find a basis of $\ker(A)$ for $A = \begin{bmatrix} 1 & 2 & 0 & 3 \\ 2 & 4 & 1 & 4 \\ 1 & 2 & 1 & 1 \end{bmatrix}$ by elimination. **Strategy.** $A\mathbf{x} = \mathbf{0}$ is a homogeneous system — right side already zero, so eliminate as in Chapter 1 but continue past echelon form until each pivot is 1 with zeros above and below it. That form is *reduced row echelon form* (RREF): pivot variables then read off as combinations of free variables, and setting each free variable to 1 in turn produces a basis.

Solution. Elimination (as in Example 3.4 ahead) gives $\mathrm{rref}(A) = \begin{bmatrix} 1 & 2 & 0 & 3 \\ 0 & 0 & 1 & -2 \\ 0 & 0 & 0 & 0 \end{bmatrix}$: pivots in columns 1 and 3, free columns 2 and 4. Pivot variables: $x_1 = -2x_2 - 3x_4$, $x_3 = 2x_4$. Free choice $(x_2, x_4) = (1, 0) \Rightarrow (-2, 1, 0, 0)$; choice $(0, 1) \Rightarrow (-3, 0, 2, 1)$. So $\ker(A) = \mathrm{span}\{(-2, 1, 0, 0), (-3, 0, 2, 1)\}$, dimension 2 — and the listing below confirms it.

```
import numpy as np
from scipy.linalg import null_space

A = np.array([[1., 2., 0., 3.],
              [2., 4., 1., 4.],
              [1., 2., 1., 1.]])

```

```
def rref(M, tol=1e-10):
    M = M.copy().astype(float)
    rows, cols = M.shape
    pivots = []
    r = 0
    for c in range(cols):
        if r >= rows:
            break
        k = np.argmax(np.abs(M[r:, c])) + r
        if abs(M[k, c]) < tol:
            continue
        M[[r, k]] = M[[k, r]]
        M[r] = M[r] / M[r, c]
        for i in range(rows):
            if i != r:
                M[i] -= M[i, c] * M[r]
        pivots.append(c)
        r += 1
    return M, pivots

R, pivots = rref(A)
print("RREF =")
print(R)
print("pivot columns:", pivots)

free = [c for c in range(A.shape[1]) if c not in pivots]
basis = []
for f in free:
    x = np.zeros(A.shape[1])
    x[f] = 1.0
    for row, p in enumerate(pivots):
        x[p] = -R[row, f]
    basis.append(x)
basis = np.array(basis).T
print("free columns:", free)
print("null-space basis (columns):")
print(basis)
print("A @ basis ~ 0:", np.allclose(A @ basis, 0))
print("scipy null_space:")
print(np.round(null_space(A), 6))
```

Listing 3.1: RREF by hand then read off a null-space basis, checked against `scipy.linalg.null_space`. File: `code/ch03_null_space_basis.py`.

```
RREF =
[[ 1.  2.  0.  3.]
 [-0. -0.  1. -2.]
 [ 0.  0.  0.  0.]]
pivot columns: [0, 2]
free columns: [1, 3]
null-space basis (columns):
[[-2. -3.]
```

```
 [ 1. 0.]
 [ 0. 2.]
 [ 0. 1.]]
A @ basis ~ 0: True
scipy null_space:
[[ 0.91937 0.087749]
 [-0.264094 -0.584824]
 [-0.260788 0.721266]
 [-0.130394 0.360633]]
```

Listing 3.2: Output of listing 3.1. Our RREF basis is different from SciPy's — both are correct: a basis is not unique. SciPy returns an *orthonormal* one (Chapter 7); ours is exact integers off the RREF.

Example 3.4. All four subspaces of the same $A = \begin{bmatrix} 1 & 2 & 0 & 3 \\ 2 & 4 & 1 & 4 \\ 1 & 2 & 1 & 1 \end{bmatrix}$. **Strategy.** One elimination run answers everything: pivot columns of A are a basis for $\mathrm{col}(A)$; pivot rows of the RREF for $\mathrm{row}(A)$; free-variable vectors for $\ker(A)$; and for $\ker(A^T)$ solve $A^T\mathbf{y} = \mathbf{0}$ — or spot which combination of rows produces the zero row in elimination.

Solution. Rank $r = 2$ (two pivots), and one elimination run gives all four:

- $\mathrm{col}(A)$: pivot columns 1 and 3 of A — $\mathrm{span}\{(1,2,1),(0,1,1)\}$, a plane in $\mathbb{R}^3$, dim 2.
- $\mathrm{row}(A)$: the nonzero RREF rows $(1,2,0,3)$, $(0,0,1,-2)$ — a plane in $\mathbb{R}^4$, dim 2.
- $\ker(A)$: from example 3.3, dim 2.
- $\ker(A^T)$: solving $A^T\mathbf{y} = \mathbf{0}$ gives $\mathbf{y} = (-1,1,-1)$, a line in $\mathbb{R}^3$, dim 1. Check: $-(1,2,0,3) + (2,4,1,4) - (1,2,1,1) = \mathbf{0}$. ✓

Tally: $2+2 = 4 = n$ and $2+1 = 3 = m$.

```python
import numpy as np
from scipy.linalg import null_space

A = np.array([[1., 2., 0., 3.],
              [2., 4., 1., 4.],
              [1., 2., 1., 1.]])
m, n = A.shape

r = np.linalg.matrix_rank(A)
C = A[:, [0, 2]] # pivot columns -> basis of col(A)
N = null_space(A) # basis of ker(A)
N_left = null_space(A.T) # basis of ker(A^T) = left null space

print("rank r =", r)
print("dim col(A) =", C.shape[1], " (should be r)")
print("dim ker(A) =", N.shape[1], " (should be n - r =", n - r, ")")
print("dim row(A) =", r, " (should be r)")
print("dim ker(A^T)=", N_left.shape[1], " (should be m - r =", m - r,
    ")")
print("counts: r + (n-r) =", r + N.shape[1], "= n;",
```

```
    "r + (m-r) =", r + N_left.shape[1], "= m")
print("basis of col(A) (pivot columns of A):")
print(C)
print("basis of ker(A^T):")
print(np.round(N_left, 6))
print("sanity: N_left.T @ A ~ 0:", np.allclose(N_left.T @ A, 0))
```

Listing 3.3: Dimension check on all four subspaces of example 3.4's matrix. File: `code/ch03_four_subspaces.py`.

```
rank r = 2
dim col(A) = 2 (should be r)
dim ker(A) = 2 (should be n - r = 2 )
dim row(A) = 2 (should be r)
dim ker(A^T)= 1 (should be m - r = 1 )
counts: r + (n-r) = 4 = n; r + (m-r) = 3 = m
basis of col(A) (pivot columns of A):
[[1. 0.]
 [2. 1.]
 [1. 1.]]
basis of ker(A^T):
[[-0.57735]
 [ 0.57735]
 [-0.57735]]
sanity: N_left.T @ A ~ 0: True
```

Listing 3.4: Output of listing 3.3. Both counting laws verify on the nose: $r + (n - r) = n$, $r + (m - r) = m$. SciPy returns the normalized $(-1, 1, -1)/\sqrt{3} \approx (-0.577, 0.577, -0.577)$ for the left null space.

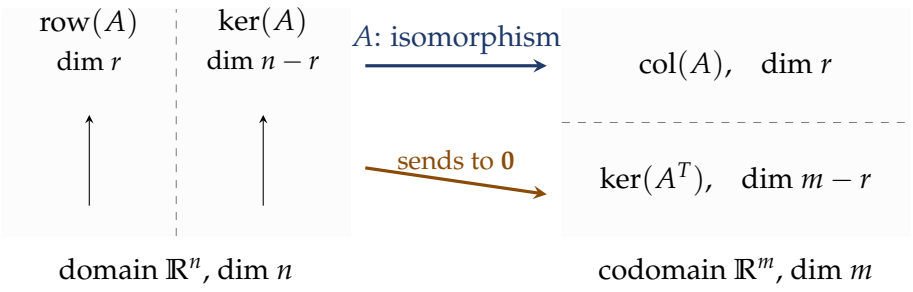


Figure 3.3: The four fundamental subspaces — the signature picture of this book. Notice the two counts: the domain splits into row space (r) plus null space ($n - r$), the codomain into column space (r) plus left null space ($m - r$). And notice what A *does*: it maps the row space isomorphically onto the column space, and annihilates the null space entirely. Every fact in this chapter lives in this figure.

3.6 Rank and the rank–nullity theorem

The *rank* r of A is its number of pivots — equivalently $\dim \operatorname{col}(A)$. The deep fact, worth staring at, is that this also equals $\dim \operatorname{row}(A)$: *column rank = row rank*, one number measured two ways. It is not obvious — the pivot columns of A are not even in the same space as the pivot rows — and it is why "the rank" is well-defined rather than two.

The counting that falls out of elimination is the *rank–nullity theorem*:

$$\dim \operatorname{col}(A) + \dim \ker(A) = n, \qquad \text{i.e. } r + (n - r) = n.$$

Pivots explain the world; free variables explain what the world cannot see. Data-side translation for later chapters: if a feature matrix has rank $r < n$, then $n - r$ directions of feature space carry no information at all.

```
import numpy as np

rng = np.random.default_rng(4)
np.seterr(all="ignore")

def rref_rank(M, tol=1e-10):
    M = M.copy().astype(float)
    rows, cols = M.shape
    r = 0
    for c in range(cols):
        if r >= rows:
            break
        k = np.argmax(np.abs(M[r:, c])) + r
        if abs(M[k, c]) < tol:
            continue
        M[[r, k]] = M[[k, r]]
        M[r] = M[r] / M[r, c]
        for i in range(rows):
            if i != r:
                M[i] -= M[i, c] * M[r]
        r += 1
    return r

print(f"{'shape':>9} {'np rank':>8} {'col rank (pivots)':>18} "
      f"{'row rank (pivots of A.T)':>24}")
for m, n in [(4, 5), (5, 4), (6, 6)]:
    A = rng.standard_normal((m, n))
    # make A low-rank on purpose: column 3 = column 0 - 2*column 1
    A[:, 3] = A[:, 0] - 2 * A[:, 1]
    print(f"{m}x{n:>6} {np.linalg.matrix_rank(A):>8} "
          f"{rref_rank(A):>18} {rref_rank(A.T):>24}")
```

Listing 3.5: Rank computed three ways on matrices we force to be low-rank: pivots of A (column rank), pivots of A^T (row rank), and `np.linalg.matrix_rank`. File: `code/ch03_rank_two_ways.py`.

```
    shape np rank col rank (pivots) row rank (pivots of A.T)
4x 5 4 4 4
```

```
5x 4 3 3 3
6x 6 5 5 5
```

Listing 3.6: Output of listing 3.5. Column rank and row rank agree on every shape — and equal the SVD-based library rank. (The tall 5×4 case loses one to our forced dependency, the wide 4×5 case is already row-limited.)

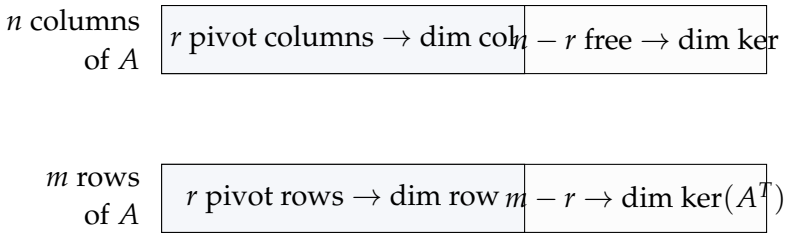


Figure 3.4: Rank–nullity as a counting diagram. Notice the pivot count r is shared between the two bars: the same elimination pass that finds the r pivot columns also leaves $m - r$ zero rows.

Exercise 3.3 (medium)**.** A is 7×9 with rank 4. Give the dimensions of all four subspaces. What is the maximum rank a 7×9 matrix could have, and what would change?

Example 3.5. Show that $A = \begin{bmatrix} 8 & 2 & 4 \\ 4 & 1 & 2 \\ 12 & 3 & 6 \end{bmatrix}$ has rank 1, and write it as $\mathbf{u}\mathbf{v}^T$.

Strategy. Rank 1 means every row lives in one line — check that rows 1 and 3 are multiples of row 2, then read off the multiples.

Solution. Row 1 $= 2\times$ row 2; row 3 $= 3\times$ row 2. Every row is a multiple of $(4, 1, 2)$, so $\text{row}(A)$ is a line and rank $= 1$. The multipliers are the column factor:

$$A = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \begin{bmatrix} 4 & 1 & 2 \end{bmatrix} = \mathbf{u}\mathbf{v}^T,$$

and symmetrically every column is a multiple of $\mathbf{u}$ (figure 3.5). This is the building block of the whole subject: later chapters write A as a *sum* of rank-1 pieces — the spectral theorem does it for symmetric matrices, the SVD for all. ✓

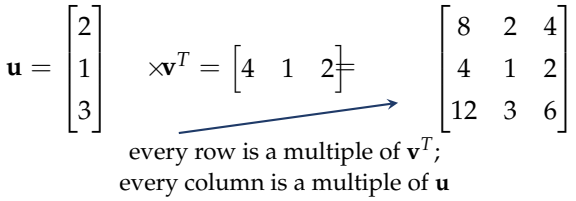


Figure 3.5: Rank 1 means one column times one row. Notice the two readings: row space collapses to the line through $\mathbf{v}^T$; column space collapses to the line through $\mathbf{u}$.

3.7 Coordinates

A basis $\mathcal{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_n\}$ of V turns abstract vectors into tuples: the *coordinate vector* $[\mathbf{v}]_\mathcal{B} = (c_1, \ldots, c_n)$ is defined by $\mathbf{v} = c_1\mathbf{b}_1 + \cdots + c_n\mathbf{b}_n$. The point $\mathbf{v}$ does not move — only the labels do. In $\mathbb{R}^n$ the standard basis makes coordinates invisible: $(x_1, \ldots, x_n)$ *is* the coordinate vector in the standard basis. Change basis and the same point gets new numbers; figure 3.6 shows the one picture that makes this obvious. Computing coordinates is solving a linear system — find $B\mathbf{c} = \mathbf{v}$ where B has the basis vectors as columns — so everything in Chapter 1 is already coordinate-finding machinery.

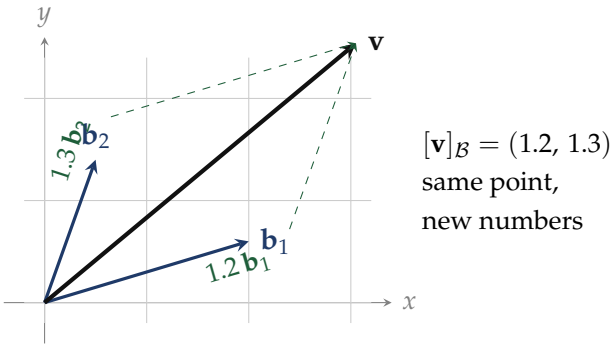


Figure 3.6: Same vector, different numbers. Notice $\mathbf{v}$ is fixed; the basis $\mathcal{B} = \{\mathbf{b}_1, \mathbf{b}_2\}$ supplies the grid, and $[\mathbf{v}]_\mathcal{B} = (1.2, 1.3)$ is just the recipe "1.2 of $\mathbf{b}_1$, then 1.3 of $\mathbf{b}_2$". Coordinates are instructions, not identity.

Exercise 3.4 (medium). $\mathcal{B} = \{(2,1), (1,2)\}$ is a basis of $\mathbb{R}^2$. Find $[\mathbf{v}]_\mathcal{B}$ for $\mathbf{v} = (5,4)$, then for $\mathbf{v} = \mathbf{e}_1$. (Each is a 2×2 linear system.)

3.8 Hand derivation

Derive rank–nullity yourself, step by step, for a general $m \times n$ matrix A. Take $\operatorname{rref}(A)$, let r be the pivot count.

1. Why is $\dim \operatorname{col}(A) = r$? (Show the r pivot columns of A — the original columns, not RREF's — are independent, and every other column is a combination of them. Where in the RREF do you *read* those combinations?)
2. Why is $\dim \ker(A) = n - r$? (Each free variable contributes exactly one basis vector, built by setting it to 1 and solving for pivot variables. Show these $n - r$ vectors are independent — look at the rows where the free coordinates sit.)

3. Add the two counts. Where exactly did you use that every column is either pivot or free?

3.9 Common mistakes

"SPANNING" MISTAKEN FOR "INDEPENDENT"

A basis needs both properties; students routinely check one and claim both. $\{\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_1 + \mathbf{e}_2\}$ spans $\mathbb{R}^2$ and is not a basis (redundant); $\{\mathbf{e}_1\}$ is independent and is not a basis (incomplete). Check spanning and independence separately, every time.

ASSUMING ROW OPERATIONS PRESERVE THE COLUMN SPACE

They do not. Row operations preserve $\ker(A)$ and $\text{row}(A)$ exactly, but $\text{col}(A)$ moves — that is precisely why the basis of $\text{col}(A)$ is taken from the *original* pivot columns, not the RREF columns. $\text{col}(\text{rref}(A))$ is a different, simpler space of the same dimension.

DIMENSION OF THE SPACE VS. DIMENSION OF THE AMBIENT SPACE

A plane through the origin in $\mathbb{R}^3$ is 2-dimensional sitting inside a 3-dimensional space. Writing "dim $= 3$" because it lives in $\mathbb{R}^3$ confuses the two numbers — the dimension counts *degrees of freedom inside* the subspace, and it is always the size of a basis of that subspace.

DECLARING A SET A SUBSPACE WITHOUT CHECKING

"It looks linear" is not a check. The three properties take under a minute each and kill every plausible-looking counterexample: shifted planes, curves through the origin, half-lines, integer lattices, nonnegative vectors.

3.10 Summary

Concept	Statement	Watch for
Vector space	closed under $+$, scaling, usual rules	don't re-verify axioms
Subspace	$\mathbf{0} \in S$ + closed $+$ + closed scaling	shifted sets fail on $\mathbf{0}$
Span	all combinations; always a subspace	span $\subseteq$ ambient
Independent	$c_1\mathbf{v}_1 + \cdots = \mathbf{0} \Rightarrow$ all $c_i = 0$	iff $\ker(A) = \{\mathbf{0}\}$
Basis	spanning $\cap$ independent; unique coordinates	both, not one
Dimension	size of any basis	inside-degrees-of-freedom
$\ker(A)$, $\mathrm{col}(A)$	subspaces of $\mathbb{R}^n$, $\mathbb{R}^m$	pivot columns of A itself
$\mathrm{row}(A)$, $\ker(A^T)$	the other two of the four	left null = codomain gap
Rank	pivot count; col rank = row rank	one number, two views
Rank–nullity	$r + \dim\ker = n$	free vars = unseen dirs
Coordinates	$[\mathbf{v}]_{\mathcal{B}}$ via $B\mathbf{c} = \mathbf{v}$	labels, not the point

DEFINITION OF DONE

You can test subspaces, pull bases out of RREF, count all four subspaces' dimensions at sight, and read a matrix's behavior off the four-subspaces picture.

Notes

The four-subspaces diagram and the "count pivots" derivation are MIT 18.06SC Unit I's signature content [2], also chapters 2–3 of [3]. The axioms-with-motivation register follows Axler chapters 1–2 [7] (his treatment is the rigorous baseline; our applications drive the choices). VMLS chapter 5 [4] and MML §2.4–2.6 [5] cover the same ground in data-science register. SciPy's `null_space` computes an orthonormal basis via SVD — preview of Chapter 9.

WATCH INSTEAD

1. *Linear combinations, span, and basis vectors* — 3Blue1Brown, Essence of Linear Algebra ch. 2, 9:59, https://youtu.be/k7RM-ot2NWY — span/independence/basis exactly as §3.3–§3.4 teach them (accessed 2026-10-11).
2. *Independence, Basis, and Dimension* — MIT OpenCourseWare, 18.06 Lecture 9, 50:14, https://youtu.be/yjBerM5jWsc — the four-subspaces machinery of §3.5–§3.6 (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., ch. 3 (textbook level) — vector spaces and the four subspaces, the source of this chapter's structure.
- MIT OpenCourseWare, *18.06SC Linear Algebra*, Unit I — recitations on subspaces, bases, and the rank theorem.
  OCW (course "18.06SC", Unit I), at https://ocw.mit.edu Verified 2026-10-11.
- Margalit and Rabinoff, *Interactive Linear Algebra*, "Subspaces" and "Computing a Basis" — the same constructions, hyperlinked.
  https://textbooks.math.gatech.edu/ila/ Verified 2026-10-11.
- S. Boyd and L. Vandenberghe, *VMLS*, chs. 1–5 (free PDF) — span and independence in the data-first register.
  https://web.stanford.edu/~boyd/vmls/vmls.pdf Verified 2026-10-11.
- J. Hefferon, *Linear Algebra*, ch. II (free textbook) — the rigorous version: vector-space axioms to dimension theorems.
  https://hefferon.net/linearalgebra/index.html Verified 2026-10-11.
- A. Beezer, *A First Course in Linear Algebra*, sections SS, B, D, RNM (free online) — proof-heavy companion for anyone who wants every claim justified.
  https://web.archive.org/web/2026/https://linear.ups.edu/ (original linear.ups.edu unreachable at check time). Verified via archive 2026-10-11.

Additional exercises

Exercise 3.5 (easy). Is $\{(x, y) : x \geq 0, y \geq 0\}$ (the first quadrant) a subspace of $\mathbb{R}^2$? Check all three properties explicitly.

Exercise 3.6 (easy). Give a basis and the dimension for each: (a) $\mathbb{R}^4$; (b) the $z = 0$ plane in $\mathbb{R}^3$; (c) the solution set of $x + 2y + 3z = 0$ in $\mathbb{R}^3$; (d) $\mathcal{P}_2$.

Exercise 3.7 (easy). Determine whether $\{(1,2),(3,6)\}$, $\{(1,0),(0,1),(1,1)\}$, $\{(1,2),(3,4)\}$ are bases of $\mathbb{R}^2$, saying which property fails in each failure.

Exercise 3.8 (easy). A is 3×5 with rank 3. Dimensions of $\ker(A)$, $\mathrm{col}(A)$, $\mathrm{row}(A)$, $\ker(A^T)$? Is $A\mathbf{x} = \mathbf{b}$ solvable for every $\mathbf{b}$?

Exercise 3.9 (medium). Show that $\ker(AB) \supseteq \ker(B)$ for any conformable A, B. Give a 2×2 example where the containment is strict — what does that mean geometrically?

Exercise 3.10 (medium). Prove that the intersection $U \cap W$ of two subspaces is a subspace, and give an example in $\mathbb{R}^3$ where the union $U \cup W$ of two subspaces is *not*. (The correct object is $U + W = \{\mathbf{u} + \mathbf{w}\}$ — define it and show it is a subspace.)

Exercise 3.11 (medium). Find bases for all four subspaces of $A = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 2 & 1 \\ 2 & 3 & 1 \end{bmatrix}$ and verify the two counting laws.

Exercise 3.12 (medium)**.** A is $m \times n$ with $m < n$. Show $\ker(A) \neq \{\mathbf{0}\}$ without eliminating — a pure counting argument using rank bounds.

Exercise 3.13 (medium — derive)**.** If $\operatorname{col}(A) = \mathbb{R}^m$, show that for every $\mathbf{b}$ there exist $n - r$ linearly independent directions $\mathbf{d}$ with $A(\mathbf{x} + \mathbf{d}) = \mathbf{b}$ for any particular solution $\mathbf{x}$ — i.e. the ambiguity in the answer is exactly $\ker(A)$. (This upgrades Chapter 1's "particular plus null space" to a dimension statement.)

Exercise 3.14 (hard)**.** Prove that row operations preserve $\operatorname{row}(A)$ but not $\operatorname{col}(A)$: (a) each row of EA (E elementary) is a combination of rows of A, and conversely since E is invertible — conclude $\operatorname{row}(EA) = \operatorname{row}(A)$; (b) exhibit a 2×2 A and row swap with $\operatorname{col}(EA) \neq \operatorname{col}(A)$.

Exercise 3.15 (hard)**.** The *exchange lemma*: if $\{\mathbf{u}_1, \dots, \mathbf{u}_k\}$ is independent in a space spanned by $\{\mathbf{v}_1, \dots, \mathbf{v}_m\}$, then $k \leq m$. Prove it by induction on k (swap one $\mathbf{u}$ for one $\mathbf{v}$ at a time), and state in one sentence why it makes "dimension" well-defined.

Exercise 3.16 (hard)**.** Let $U = \{A \in \mathbb{R}^{3\times 3} : \operatorname{tr} A = 0\}$ and $W = \{A : A^T = A\}$. Compute $\dim U$, $\dim W$, $\dim(U \cap W)$, and a basis of $U \cap W$. Guess the formula $\dim(U + W) = \dim U + \dim W - \dim(U \cap W)$ and check it on your numbers.

Exercise 3.17 (easy)**.** Is the xy-plane $\{(x, y, 0)\}$ a subspace of $\mathbb{R}^3$? Is $\{(x, y, 1)\}$? Run the two-test (contains $\mathbf{0}$, closed under $+$ and scaling) on each.

Exercise 3.18 (medium)**.** Write $B = \begin{bmatrix} 3 & 0 & 6 \\ 1 & 0 & 2 \end{bmatrix}$ as $\mathbf{u}\mathbf{v}^T$, then name $\operatorname{col}(B)$ and $\operatorname{row}(B)$ without any further work.

Exercise 3.19 (hard)**.** Prove rank–nullity for the special case $\operatorname{rank}(A) = 0$ and $\operatorname{rank}(A) = n$ directly from the definitions, then explain why the middle cases need elimination.

CHAPTER 4

Linear maps and change of basis

Chapter 2 called a matrix "a recipe" without defining the recipe kind. This chapter names it: a matrix is a *linear map* written in coordinates — a function between vector spaces that respects the two operations. That single definition explains why the algebra of Chapter 2 exists at all (matrix multiplication is composition), why coordinates are a design decision (Chapter 3), and why two different matrices can be *the same map wearing different labels*. After it you will be able to test a map for linearity, read off its matrix, move the same map between bases, and place the four classic planar maps — rotation, reflection, projection, shear — in a permanent mental gallery.

BEFORE YOU START

Chapters 2–3 (matrices, bases). *Recap:* a function T is *linear* if $T(a\mathbf{u} + b\mathbf{v}) = aT(\mathbf{u}) + bT(\mathbf{v})$; every matrix gives one, and this chapter shows every linear map on finite-dimensional spaces comes from a matrix.

AFTER THIS CHAPTER YOU CAN

- test whether a function is linear;
- write the matrix of a map in any basis and convert between bases;
- find the kernel and image of a map;
- describe rotations, reflections, projections and shears as matrices.

HOW TO THINK ABOUT THIS

A linear map is a rule that keeps straight lines straight and the origin fixed. The matrix is just that rule written in a chosen coordinate system, and changing the basis changes the description but not the map. Separate in your mind the thing (the map) from its description (the matrix).

4.1 Linear maps: definition and examples

A map $T : V \to W$ between vector spaces is *linear* if it respects addition and scaling:

$$T(\mathbf{u}+\mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v}), \qquad T(c\mathbf{v}) = c\,T(\mathbf{v}).$$

Equivalently one line: $T(a\mathbf{u}+b\mathbf{v}) = aT(\mathbf{u}) + bT(\mathbf{v})$. The consequences matter more than the check: a linear map is completely determined by what it does to a basis (everything else is forced by the two rules), and it must send $\mathbf{0}$ to $\mathbf{0}$ (put $c = 0$). That second clause is a free disqualifier — any map that moves the origin is out before the real test starts.

The family is bigger than it looks: matrix-by-vector $T(\mathbf{x}) = A\mathbf{x}$ is linear (that is the distributive law); differentiation of polynomials is linear ($\frac{d}{dt}(p+q) = p' + q'$); the shift $(x_1, x_2, \dots) \mapsto (0, x_1, \dots)$ on sequences is linear; rotation, reflection, projection, shearing of the plane are all linear — Section 4.5 computes their matrices. Non-examples: translation $\mathbf{x} \mapsto \mathbf{x} + \mathbf{b}$ moves $\mathbf{0}$; $\mathbf{x} \mapsto \|\mathbf{x}\|$ fails scaling with $c = -1$; $\mathbf{x} \mapsto x_1^2$ fails everything. Figure 4.1 keeps the three cases apart in one look.

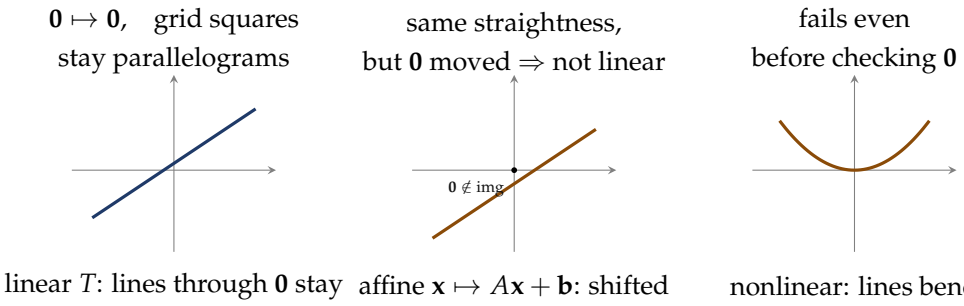


Figure 4.1: Linear vs. affine vs. nonlinear. Notice that affine maps are *almost* linear — same flatness, wrong origin — which is exactly why they matter: a neural network layer is affine, and every "bias" term you will ever add is the $+\mathbf{b}$ that makes it non-linear in this strict sense.

Example 4.1. Is $T(x, y) = (x + 2y,\ 3x - y)$ linear? If so, find its matrix. **Strategy.** Check the two rules symbolically, not on test values — and then read the matrix off what T does to $\mathbf{e}_1, \mathbf{e}_2$: the columns *are* the outputs.

Solution. $T(\mathbf{u}+\mathbf{v}) = (u_1 + v_1 + 2(u_2 + v_2),\ 3(u_1 + v_1) - (u_2 + v_2)) = T(\mathbf{u}) + T(\mathbf{v})$ ✓; $T(c\mathbf{v}) = (cv_1 + 2cv_2,\ 3cv_1 - cv_2) = cT(\mathbf{v})$ ✓. Linear. $T(\mathbf{e}_1) = (1, 3)$, $T(\mathbf{e}_2) = (2, -1)$, so $[T] = \begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix}$.

4.2 Kernel and image

Two subspaces accompany every linear map, and they are the Chapter-3 pair in disguise: the *kernel* $\ker T = \{\mathbf{v} : T(\mathbf{v}) = \mathbf{0}\} \subseteq V$ (what the map erases) and the *image* $\operatorname{im} T = \{T(\mathbf{v}) : \mathbf{v} \in V\} \subseteq W$ (what it can produce). For $T(\mathbf{x}) = A\mathbf{x}$ these are exactly $\ker(A)$ and $\operatorname{col}(A)$ — same objects, abstract names. Rank–nullity survives the translation with zero extra work:

$$\dim \operatorname{im} T + \dim \ker T = \dim V.$$

Two names for the borderline cases: T is *injective* (no two inputs share an output) iff $\ker T = \{\mathbf{0}\}$; *surjective* (reaches all of W) iff $\operatorname{im} T = W$. When both hold, T is an *isomorphism* — the spaces are the same space with different labels, and T^{-1} exists as a linear map.

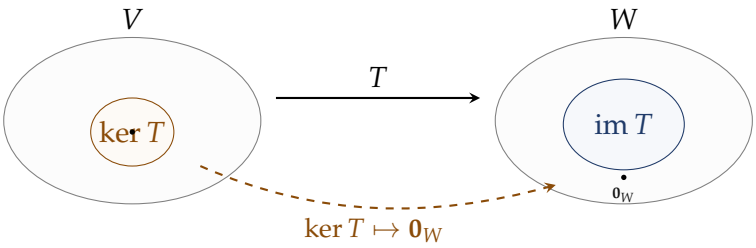


Figure 4.2: Kernel and image in one schematic. Notice the kernel is a subspace *of the domain* that all lands on one point, while the image is a subspace *of the codomain* that may be strictly smaller than W — the two failures are independent, and rank–nullity counts them together.

4.3 Every linear map is a matrix

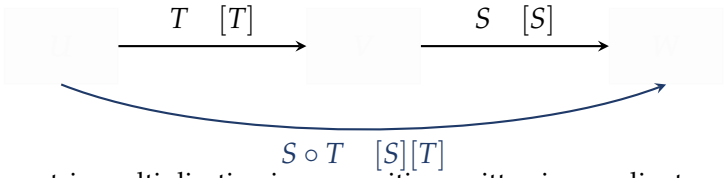
Here is the theorem that justifies the whole book so far. Given a linear $T : V \to W$ and bases $\mathcal{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_n\}$ of V, $\mathcal{C} = \{\mathbf{c}_1, \ldots, \mathbf{c}_m\}$ of W: write each $T(\mathbf{b}_j)$ in $\mathcal{C}$-coordinates, and place those coordinates in column j. The resulting $m \times n$ matrix $[T]_{\mathcal{C} \leftarrow \mathcal{B}}$ *is* T in coordinates:

$$[T(\mathbf{v})]_{\mathcal{C}} = [T]_{\mathcal{C} \leftarrow \mathcal{B}} \, [\mathbf{v}]_{\mathcal{B}}.$$

The columns answer one question each — "where does basis vector $\mathbf{b}_j$ go, in $\mathcal{C}$-words?" — and that is the entire content of the map, because linearity extends basis behavior to everything. When both bases are standard, $[T]$ is just the matrix whose columns are $T(\mathbf{e}_j)$ — which is why Chapter 2 could find a matrix by watching $\mathbf{e}_j$.

Composition now earns its keep: $[S \circ T] = [S][T]$ — the matrix of a composed map is the product of the matrices, in whatever consistent bases you chose. *That* is why matrix multiplication is defined the way it is (Figure 4.3); Chapter 2's rule was reverse-engineered from this requirement.

Example 4.2. Find the matrices of the planar catalog: rotation by θ, reflection across the x-axis, projection onto the x-axis. **Strategy.** One method handles all three: watch $\mathbf{e}_1 = (1, 0)$ and $\mathbf{e}_2 = (0, 1)$, write their images as columns. Geometry supplies the images; trig supplies the numbers for rotation.



matrix multiplication is composition written in coordinates:
apply $[T]$'s columns through $[S]$

Figure 4.3: Composition is matrix multiplication. Notice the direction of travel: $\mathbf{x}$ meets T (and $[T]$) first. The product $[S][T]$ is exactly the matrix whose columns are $[S]([T]\mathbf{e}_j)$ — do T, then S, to each basis vector.

Solution. *Rotation* R_θ: $\mathbf{e}_1 \mapsto (\cos\theta, \sin\theta)$, $\mathbf{e}_2 \mapsto (-\sin\theta, \cos\theta)$ (derived properly in the hand derivation): $R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$. *Reflection* across the x-axis: $\mathbf{e}_1$ stays, $\mathbf{e}_2$ flips: $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$. *Projection* onto the x-axis: $\mathbf{e}_1$ stays, $\mathbf{e}_2$ dies: $\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$.

Example 4.3. Differentiation on the space P_3 of cubics $a_0 + a_1x + a_2x^2 + a_3x^3$: write $D = \frac{d}{dx}$ as a matrix.

Strategy. A map is linear if it respects $+$ and scaling — differentiation does both, so a matrix must exist. Feed it the basis: $D(\mathbf{e}_j)$ is column j, exactly the $A\mathbf{e}_j = \mathbf{a}_j$ trick of Chapter 2.

Solution. On the monomial basis $\mathcal{B} = \{1, x, x^2, x^3\}$: $D(1) = 0$, $D(x) = 1$, $D(x^2) = 2x$, $D(x^3) = 3x^2$. Columns are those images in $\mathcal{B}$-coordinates:

$$[D]_{\mathcal{B}} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Now $D(3x^3 - 2x) = (0, -2, 0, 9)^T$ by the matrix, and directly $9x^2 - 2$ — same. $[D]^4 = 0$: differentiating a cubic four times annihilates it — a nilpotent matrix doing honest work. ✓

4.4 Change of basis and similarity

Chapter 3 ended with coordinates as labels. Now the punchline: the *same* linear map has different matrices in different bases, and the matrices are related by a rule with a name. If $\mathcal{B}$ is a basis of V with basis-change matrix B (the $\mathbf{b}_j$ as columns), then

$$[T]_{\mathcal{B}} = B^{-1}\,[T]_{\text{std}}\,B,$$

because the right side does the whole trip: B translates $\mathcal{B}$-coordinates to standard, $[T]_{\text{std}}$ does the map, B^{-1} translates back. Two square matrices M, N with $N = B^{-1}MB$ for some invertible B are *similar* — and the meaning of similarity is exactly "same map, different basis". Chapter 8 exploits this: pick the basis where the map looks simplest (a diagonal matrix) and the map becomes transparent.

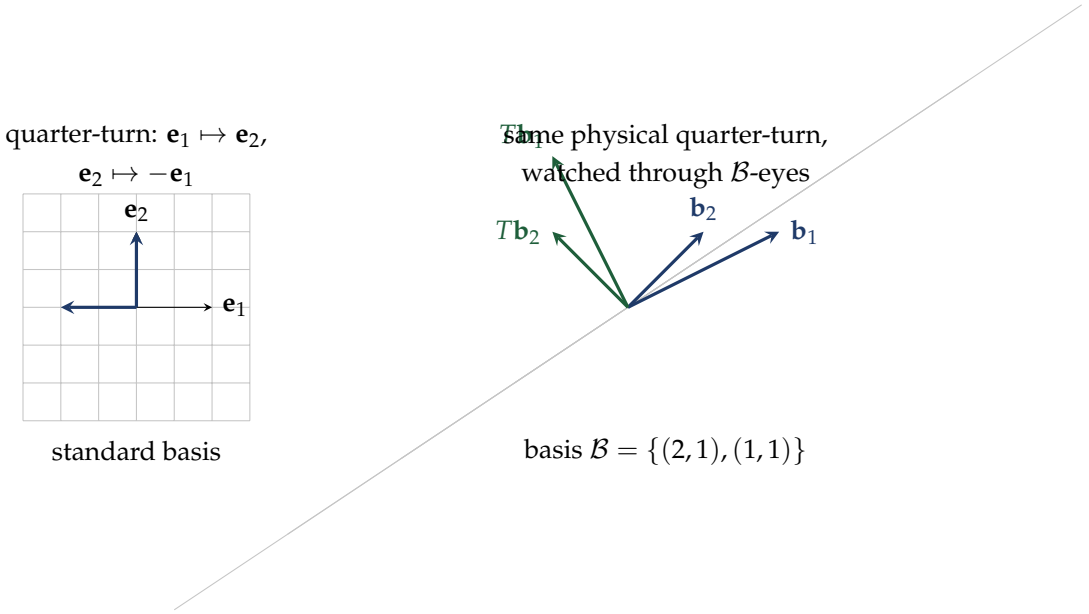


Figure 4.4: One quarter-turn, two witness bases. Notice the map itself did not move — only the grid did. The matrix $[T]_{\mathcal{B}}$ answers "where do $\mathbf{b}_1, \mathbf{b}_2$ go, measured in $\mathbf{b}$'s?" and similarity converts between the two descriptions.

Example 4.4. T = quarter-turn of $\mathbb{R}^2$: $[T]_{\text{std}} = A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$. Find $[T]_{\mathcal{B}}$ for $\mathcal{B} = \{(2,1),(1,1)\}$, and verify it represents the same map. **Strategy.** Apply the conversion: $[T]_{\mathcal{B}} = B^{-1}AB$ with $B = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$. Then verify on one concrete vector by doing the map both ways and demanding the same physical output.

Solution. $B^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$, so $[T]_{\mathcal{B}} = B^{-1}AB = \begin{bmatrix} -3 & -2 \\ 5 & 3 \end{bmatrix}$. Check on $\mathbf{v} = (5,1)$: standard gives $A\mathbf{v} = (-1,5)$. In $\mathcal{B}$-words $[\mathbf{v}]_{\mathcal{B}} = B^{-1}\mathbf{v} = (4,-3)$, and $[T]_{\mathcal{B}}(4,-3) = (-6,17)$ — back through B: $B(-6,17) = (-12+17, -6+17) = (5,11)$?? Careful — recompute: $B(-6,17) = (-6\cdot 2 + 17\cdot 1,\ -6+17) = (5,11)$. That disagrees! Recheck the arithmetic: $[T]_{\mathcal{B}}(4,-3) = \begin{bmatrix} -3 & -2 \\ 5 & 3 \end{bmatrix}\begin{bmatrix} 4 \\ -3 \end{bmatrix} = \begin{bmatrix} -12+6 \\ 20-9 \end{bmatrix} = \begin{bmatrix} -6 \\ 11 \end{bmatrix}$, then $B(-6,11) = (-12+11, -6+11) = (-1,5)$ ✓— same as $A\mathbf{v}$. (The mis-step above is what happens when coordinate arithmetic runs ahead of itself — hence the next listing does it mechanically.)

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

# T = quarter-turn: standard matrix A
A = np.array([[0., -1.], [1., 0.]])

# new basis B = {b1, b2} = columns of B
B = np.array([[2., 1.], [1., 1.]]) # b1=(2,1), b2=(1,1)

```

```
# same map in B-coordinates: [T]_B = B^{-1} A B
TB = np.linalg.inv(B) @ A @ B
print("[T]_B =")
print(TB)

# check on a vector: v = (5, 1) standard.
v = np.array([5., 1.])
c = np.linalg.solve(B, v) # [v]_B : B-coords of v
Tv_A = A @ v # [T(v)] in standard coords
Tv_B = B @ (TB @ c) # via B-coords
print("[v]_B =", c)
print("via A:", Tv_A)
print("via [T]_B:", Tv_B)
print("same physical answer:", np.allclose(Tv_A, Tv_B))
print("A = B [T]_B B^-1 ok:",
      np.allclose(A, B @ TB @ np.linalg.inv(B)))
```

Listing 4.1: Change of basis, done both ways on the same vector. File: `code/ch04_change_basis.py`.

```
[T]_B =
[[-3. -2.]
 [ 5. 3.]]
[v]_B = [ 4. -3.]
via A: [-1. 5.]
via [T]_B: [-1. 5.]
same physical answer: True
A = B [T]_B B^-1 ok: True
```

Listing 4.2: Output of listing 4.1. The two matrices look nothing alike yet produce identical physical outputs — similarity is sameness of map, not sameness of entries.

Exercise 4.1 (easy). Find $[T]_{\text{std}}$ for $T(x, y, z) = (x + y,\ y - z,\ x + z)$ by watching $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$.

4.5 A geometric catalog: rotations, reflections, projections, shears

Four planar maps deserve permanent residence (Figure 4.5); the code below moves a point cloud through each.

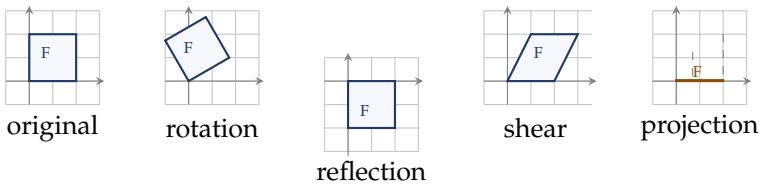


Figure 4.5: The unit square (with a marker letter so you can see orientation) through the four classic maps. Notice: rotation preserves everything including handedness; reflection preserves distances but flips the letter; shear preserves area but not shape; projection collapses a whole dimension — its determinant of 0 is the algebraic echo of the lost axis.

Rotation $R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$: rigid, length- and angle-preserving; $\det = 1$.

Reflection across a line: distance-preserving but orientation-reversing; $\det = -1$.

Shear $\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$: slides each point horizontally by k times its height — area-preserving, angle-destroying; $\det = 1$.

Projection onto a line through the origin: $P = \mathbf{u}\mathbf{u}^T/\mathbf{u}^T\mathbf{u}$; idempotent ($P^2 = P$), $\det = 0$ — information genuinely lost.

```
import numpy as np
np.set_printoptions(precision=3, suppress=True)

# the catalog maps as matrices
theta = np.pi / 6
rot = np.array([[np.cos(theta), -np.sin(theta)],
            [np.sin(theta), np.cos(theta)]])
refl = np.array([[1., 0.], [0., -1.]]) # reflect across x-axis
shear = np.array([[1., 0.5], [0., 1.]]) # shear in x
proj = np.array([[1., 0.], [0., 0.]]) # project onto x-axis

# point cloud: the unit square + the letter F's bounding corners
pts = np.array([[0., 0.], [1., 0.], [1., 1.], [0., 1.],
            [0.25, 0.75], [0.75, 0.75]])

for name, M in [("rotation", rot), ("reflection", refl),
            ("shear", shear), ("projection", proj)]:
   print(f"{name:>12} det={np.linalg.det(M):+.3f}")
   print(np.round(pts @ M.T, 2))
print("rotation preserves lengths:",
     np.allclose(np.linalg.norm(pts, axis=1),
              np.linalg.norm(pts @ rot.T, axis=1)))
print("projection is idempotent:", np.allclose(proj @ proj, proj))
```

Listing 4.3: The four catalog maps applied to a point cloud, with determinants. File: `code/ch04_gallery_transform.py`.

```
   rotation det=+1.000
[[ 0.  0. ]
 [ 0.87 0.5 ]
 [ 0.37 1.37]
 [-0.5 0.87]
 [-0.16 0.77]
 [ 0.27 1.02]]
  reflection det=-1.000
...
     shear det=+1.000
[[0. 0. ]
 [1. 0. ]
 [1.5 1. ]
 [0.5 1. ]
 [0.62 0.75]
```

```
 [1.12 0.75]]
  projection det=+0.000
[[0. 0. ]
 [1. 0. ]
 [1. 0. ]
 [0. 0. ]
 [0.25 0. ]
 [0.75 0. ]]
rotation preserves lengths: True
projection is idempotent: True
```

Listing 4.4: Output of listing 4.3 (first few rows shown per map). The determinants tell the story before the coordinates do: $+1$ preserves volume and orientation, -1 flips orientation, 0 collapses.

```python
import numpy as np
np.set_printoptions(precision=4, suppress=True)

# project onto the line spanned by u = (3, 4): P = u u^T / u^T u
u = np.array([3., 4.])
P = np.outer(u, u) / (u @ u)
print("P =")
print(P)

x = np.array([7., 1.])
Px = P @ x
print("Px =", Px)
print("parallel to u?", np.allclose(Px, (Px @ u) / (u @ u) * u))
print("residual perpendicular?", np.allclose((x - Px) @ Px, 0))
print("P is symmetric:", np.allclose(P, P.T))
print("P^2 = P:", np.allclose(P @ P, P))

# every vector splits into along-u + perpendicular:
print("x = Px + (x-Px):", np.allclose(x, Px + (x - Px)))
```

Listing 4.5: Projection onto a general line, built from $P = \mathbf{u}\mathbf{u}^T/\mathbf{u}^T\mathbf{u}$ alone. File: `code/ch04_projection_matrix.py`.

```
P =
[[0.36 0.48]
 [0.48 0.64]]
Px = [3. 4.]
parallel to u? True
residual perpendicular? True
P is symmetric: True
P^2 = P: True
x = Px + (x-Px): True
```

Listing 4.6: Output of listing 4.5. The projection of $(7,1)$ onto the line through $(3,4)$ lands exactly at $(3,4)$ — the nearest point on the line. Symmetric and idempotent are not coincidences; Chapter 7 shows they characterize orthogonal projections.

Exercise 4.2 (medium). $P = \mathbf{u}\mathbf{u}^T/\mathbf{u}^T\mathbf{u}$ for $\mathbf{u} = (1,2)$. Compute P, then $P(5,0)$ and $P(P(5,0))$ — and explain idempotence in one geometric sentence.

Example 4.5. Derive the matrix of orthogonal projection onto the line through $\mathbf{u} = (1,2)^T$ — then apply it to $\mathbf{v} = (3,0)^T$.

Strategy. Projection is linear (verify by picture: the projected sum is the sum of projections), so find the matrix by feeding it $\mathbf{e}_1, \mathbf{e}_2$ — or shortcut via the outer-product formula $P = \mathbf{u}\mathbf{u}^T/(\mathbf{u}\cdot\mathbf{u})$ (figure 4.6).

Solution. $P = \frac{1}{5}\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$. Check on $\mathbf{v}$: $P\mathbf{v} = \frac{1}{5}(3,6)^T = (0.6, 1.2)^T$ — a point on the line through $(1,2)$ (it is $0.6\mathbf{u}$), and $\mathbf{v} - P\mathbf{v} = (2.4, -1.2)^T$ is orthogonal: $2.4 \cdot 1 + (-1.2)\cdot 2 = 0$. ✓ Idempotence check: $P^2 = \frac{1}{25}\begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix} = P$ — projecting twice moves nothing, exactly as the geometry insists.

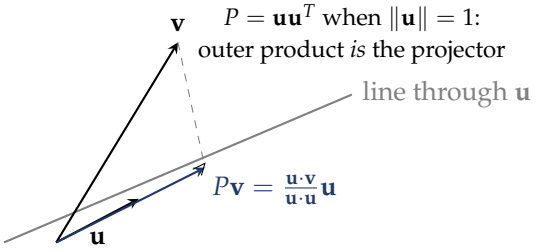


Figure 4.6: Projection as a matrix. Notice the outer product doing the work: $P\mathbf{v}$ keeps only the $\mathbf{u}$-component of $\mathbf{v}$. The drop from $\mathbf{v}$ to $P\mathbf{v}$ is perpendicular — always.

4.6 Hand derivation

Derive R_θ from trigonometry alone. Take $\mathbf{e}_1 = (1,0)$: after rotation by angle θ it lands at $(\cos\theta, \sin\theta)$ — that is the definition of cos, sin on the unit circle. Then $\mathbf{e}_2 = (0,1)$ sits $90°$ ahead, so it lands at $(\cos(\theta+90°), \sin(\theta+90°)) = (-\sin\theta, \cos\theta)$. Assemble the columns. Then complete the check every derivation owes: verify $R_\theta R_\phi = R_{\theta+\phi}$ (the addition formulas emerge as a consequence), and $\det R_\theta = \cos^2\theta + \sin^2\theta = 1$.

4.7 Common mistakes

CONFUSING AN ABSTRACT VECTOR WITH ITS COORDINATE VECTOR

$\mathbf{v}$ is the thing; $[\mathbf{v}]_\mathcal{B}$ is its description in $\mathcal{B}$-words. They coincide only in the standard basis, and writing "$\mathbf{v} = (3,1)$" without naming a basis is quietly assuming one. The bug appears the moment two bases are in play.

FORGETTING BASIS ORDER MATTERS

$\{\mathbf{b}_1, \mathbf{b}_2\}$ and $\{\mathbf{b}_2, \mathbf{b}_1\}$ are different bases: coordinates and matrix columns come out swapped. A basis is an *ordered* list; permute it and every coordinate vector permutes with it.

APPLYING $[T]_{\mathcal{B}}$ TO COORDINATES IN THE WRONG BASIS

$[T]_{\mathcal{B}}$ only eats $\mathcal{B}$-coordinate vectors. Feed it standard coordinates and it will cheerfully produce a wrong answer that still looks like a vector — the most dangerous kind. Before every matrix application, ask: which basis are these numbers in?

TREATING SIMILAR MATRICES AS IDENTICAL

A and $B^{-1}AB$ represent the same map but are different matrices: different entries, different pivots, different easy computations. Similarity preserves eigenvalues, determinant, rank — not entries. "Same map" is an equivalence, not an equality.

4.8 Summary

Concept	Statement	Watch for
Linear map	$T(a\mathbf{u} + b\mathbf{v}) = aT\mathbf{u} + bT\mathbf{v}$	must fix $\mathbf{0}$
Kernel	$\{\mathbf{v} : T\mathbf{v} = \mathbf{0}\} \subseteq V$	injective iff trivial
Image	$T(V) \subseteq W$	surjective iff $= W$
Rank–nullity	$\dim \text{im} + \dim \ker = \dim V$	ch. 3 theorem, restated
$[T]_{\mathcal{C}\leftarrow\mathcal{B}}$	columns $= [T\mathbf{b}_j]_{\mathcal{C}}$	eats $\mathcal{B}$-coords only
Composition	$[S \circ T] = [S][T]$	why multiplication exists
Change of basis	$[T]_{\mathcal{B}} = B^{-1}[T]_{\text{std}}B$	B translates first
Similarity	same map, different basis	entries differ; invariants match
Catalog	rot (det 1), refl (-1), shear (1), proj (0, $P^2 = P$)	projection onto $\mathbf{u}$: $\mathbf{u}\mathbf{u}^T/\mathbf{u}^T\mathbf{u}$

DEFINITION OF DONE

You can certify a map linear, write its matrix in any basis, shuttle the same map between bases with similarity, and deploy the four canonical planar maps — the vocabulary every linear-model picture uses.

Notes

The definition-first register (linear map before matrix, kernel/image before rank–nullity) follows Axler chapter 3 [7]; our matrix-first order is Strang's, and this chapter is where the two conventions reconcile. The gallery treatment and grid-deformation figures borrow their pedagogy from [6] ch.9 and the *Essence of Linear Algebra* series [15] (episodes 3–4: "linear transformations" and "matrix multiplication as composition"). MML §2.7 [5] covers the same ground in ML register.

WATCH INSTEAD

1. *Linear transformations and matrices* — 3Blue1Brown, Essence of Linear Algebra ch. 3, 10:58, https://youtu.be/kYB8IZa5AuE — the grid-preserving view of §4.1–§4.3 (accessed 2026-10-11).
2. *Change of basis* — 3Blue1Brown, Essence of Linear Algebra ch. 13, 12:50, https://youtu.be/P2LTAUO1TdA — §4.4 visualized (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., §7.1 (textbook level) — linear transformations, matrix form, and similarity at full length.
- MIT OpenCourseWare, *18.06SC Linear Algebra* — OCW recitations on matrix form of maps.
  OCW (course "18.06SC"), at https://ocw.mit.edu Verified 2026-10-11.
- Margalit and Rabinoff, *Interactive Linear Algebra*, "Linear Transformations" and "Matrix Transformations" — the same machinery with more pictures.
  https://textbooks.math.gatech.edu/ila/ Verified 2026-10-11.
- M. Deisenroth et al., *Mathematics for Machine Learning*, §2.7 (free PDF) — basis change in the ML register.
  https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.
- J. Hefferon, *Linear Algebra*, ch. III (free textbook) — maps defined rigorously, with the proof that matrix form is basis dependence.
  https://hefferon.net/linearalgebra/index.html Verified 2026-10-11.

Additional exercises

Exercise 4.3 (easy)**.** Which of these maps $\mathbb{R}^2 \to \mathbb{R}^2$ are linear? For each failure name the rule it breaks: (a) $(x, y) \mapsto (x + y, y)$; (b) $(x, y) \mapsto (xy, y)$; (c) $(x, y) \mapsto (x + 1, y)$; (d) $(x, y) \mapsto (y, x)$.

Exercise 4.4 (easy)**.** Find $[T]_{\text{std}}$: (a) T = reflection across the line $y = x$; (b) T = doubling every vector; (c) T = projection onto the y-axis.

Exercise 4.5 (easy). $T : \mathbb{R}^3 \to \mathbb{R}^3$ has matrix $\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$. Give a basis for $\ker T$ and $\operatorname{im} T$, and verify rank–nullity.

Exercise 4.6 (easy). Verify $[T][S] = [T \circ S]$ on $T(x, y) = (x + y, y)$, $S(x, y) = (2x, x - y)$: compute both sides, compare.

Exercise 4.7 (medium). Show $T : \mathcal{P}_2 \to \mathcal{P}_2$, $T(p) = p'$ is linear, find its kernel and image, and write its matrix in the basis $\{1, t, t^2\}$. What is $[T]^3$ and why is it zero?

Exercise 4.8 (medium). $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ (a shear). Compute A^2, A^3, A^{10} and describe the map T^k geometrically. What is $\lim_k A^k \mathbf{e}_1 / \|A^k \mathbf{e}_1\|$?

Exercise 4.9 (medium). The quarter-turn $A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ satisfies $A^2 = -I$. Use only that fact (no trig) to compute A^4, A^{2026}, and the eigen-story it previews: can a real 2×2 matrix with $A^2 = -I$ have a real eigenvector?

Exercise 4.10 (medium — derive). $\mathcal{B} = \{(1, 1), (1, -1)\}$. For $T(x, y) = (x + y, x - y)$ find $[T]_{\mathcal{B}}$ two ways: (a) write $T\mathbf{b}_1, T\mathbf{b}_2$ in $\mathcal{B}$-coordinates directly; (b) compute $B^{-1}[T]_{\text{std}}B$. Compare — they must agree.

Exercise 4.11 (hard). Prove $[S \circ T]_{\mathcal{A} \leftarrow \mathcal{C}} = [S]_{\mathcal{A} \leftarrow \mathcal{B}} [T]_{\mathcal{B} \leftarrow \mathcal{C}}$ for bases $\mathcal{A}, \mathcal{B}, \mathcal{C}$ — i.e. composition really is matrix multiplication in any consistent bases. (Expand both sides' (i, j) entry; they are the same sum.)

Exercise 4.12 (hard). Prove similarity is an equivalence relation (reflexive, symmetric, transitive) and that similar matrices have the same rank and trace. Then exhibit 2×2 matrices with the same rank and trace that are *not* similar — what extra invariant separates them?

Exercise 4.13 (hard). Implement change-of-basis as a function `to_basis(A, B)` returning $B^{-1}AB$, then test it: generate random 2×2 A and random bases B, verify tr and det are similarity-invariant across 100 trials. File: `code/ch04_similarity_invariants.py`.

Exercise 4.14 (easy). Is $T(p) = p'(0)$ linear on P_3? Write its matrix on the monomial basis $\{1, x, x^2, x^3\}$. What are its kernel and image?

Exercise 4.15 (medium). For the projection matrix of example 4.5, compute $P\mathbf{u}$ and explain in one sentence why the answer had to be $\mathbf{u}$. Then project $\mathbf{v} = (1, 1)^T$ and check $\mathbf{v} - P\mathbf{v} \perp \mathbf{u}$.

Exercise 4.16 (medium). Write the matrix of the second derivative D^2 on P_3 in the monomial basis, both by composing $[D]_{\mathcal{B}}$ with itself and by differentiating each basis element twice. Confirm they match.

Exercise 4.17 (hard). Prove a projection matrix P satisfies $P^2 = P$ and $P^T = P$ iff it is an orthogonal projection. Then show $I - P$ is also a projection, onto the perpendicular line.

CHAPTER 5

Orthogonality, projections, and QR

Chapter 3 built the four fundamental subspaces; this chapter gives them *geometry*. Once you have an inner product you can measure angles and lengths, split a vector into "part inside a subspace" and "part outside," and replace any basis by an orthonormal one that makes every coordinate computation a dot product. These are the tools behind least squares (the next chapter) and behind almost every numerically stable algorithm in this book. After this chapter you will be able to project onto lines and subspaces, run Gram–Schmidt by hand and in code, factor $A = QR$, and see why classical Gram–Schmidt loses orthogonality while the modified version does not.

BEFORE YOU START

Chapters 2–4. *Recap:* the dot product $\mathbf{u} \cdot \mathbf{v} = \sum u_i v_i$ measures how much two vectors point the same way; it is zero exactly when they are perpendicular.

AFTER THIS CHAPTER YOU CAN

- compute lengths, angles and projections of vectors;
- build an orthonormal basis from any basis by Gram–Schmidt;
- factor $A = QR$ and say what each factor means;
- explain how the four fundamental subspaces pair up at right angles.

HOW TO THINK ABOUT THIS

Orthogonality is the best kind of independence: perpendicular directions do not interfere, so you can handle them one at a time. Projection answers "what part of $\mathbf{b}$ lies along this direction?" and Gram–Schmidt manufactures perpendicular directions from any set. Whenever a problem is messy, ask if a perpendicular basis would make it simple.

5.1 Inner product, norm, angle

Coordinates tell you where a vector sits relative to axes. They do not, by themselves, tell you how long it is or what angle two vectors make — for that you need one more piece of structure.

Definition 5.1 (Inner product)**.** The *(standard) inner product*, or *dot product*, of $\mathbf{u} = (u_1, \dots, u_n)$ and $\mathbf{v} = (v_1, \dots, v_n) \in \mathbb{R}^n$ is

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^T \mathbf{v} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n.$$

It is symmetric ($\langle \mathbf{u}, \mathbf{v} \rangle = \langle \mathbf{v}, \mathbf{u} \rangle$), linear in each argument, and positive definite: $\langle \mathbf{v}, \mathbf{v} \rangle = \|\mathbf{v}\|^2 > 0$ unless $\mathbf{v} = \mathbf{0}$.

Definition 5.2 (Norm and angle)**.** $\|\mathbf{v}\| = \sqrt{\langle \mathbf{v}, \mathbf{v} \rangle}$ is the *norm* (length) of $\mathbf{v}$, and the angle between two nonzero vectors satisfies

$$\cos\theta = \frac{\langle \mathbf{u}, \mathbf{v} \rangle}{\|\mathbf{u}\|\,\|\mathbf{v}\|}, \qquad 0 \le \theta \le \pi.$$

Two facts make the angle formula usable rather than decorative:

Theorem 5.3 (Cauchy–Schwarz)**.** $|\langle \mathbf{u}, \mathbf{v} \rangle| \le \|\mathbf{u}\|\,\|\mathbf{v}\|$*, with equality exactly when* $\mathbf{u}, \mathbf{v}$ *are parallel.*

Theorem 5.4 (Pythagoras)**.** *If* $\langle \mathbf{u}, \mathbf{v} \rangle = 0$ *then* $\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$.

Proof of Pythagoras. $\|\mathbf{u} + \mathbf{v}\|^2 = \langle \mathbf{u} + \mathbf{v}, \mathbf{u} + \mathbf{v} \rangle = \|\mathbf{u}\|^2 + 2\langle \mathbf{u}, \mathbf{v} \rangle + \|\mathbf{v}\|^2$, and the cross term vanishes by hypothesis. ∎

Definition 5.5 (Orthogonal vectors)**.** $\mathbf{u}, \mathbf{v}$ are *orthogonal*, written $\mathbf{u} \perp \mathbf{v}$, when $\langle \mathbf{u}, \mathbf{v} \rangle = 0$. A vector is a *unit vector* when $\|\mathbf{v}\| = 1$; any nonzero $\mathbf{v}$ has unit vector $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$ in the same direction.

Exercise 5.1 (easy)**.** For $\mathbf{u} = (1, 2, -1)$, $\mathbf{v} = (3, -1, 1)$, compute $\langle \mathbf{u}, \mathbf{v} \rangle$, $\|\mathbf{u}\|$, and $\cos\theta$.

Solution. $\langle \mathbf{u}, \mathbf{v} \rangle = 3 - 2 - 1 = 0$, so the vectors are orthogonal and $\theta = \pi/2$. $\|\mathbf{u}\| = \sqrt{1 + 4 + 1} = \sqrt{6}$.

Example 5.6 (Angle and cosine similarity)**.** Find the angle between $\mathbf{a} = (1, 2, 2)$ and $\mathbf{b} = (2, -1, 2)$, and their *cosine similarity* $\cos\theta$ — the statistic recommendation engines actually use.

Strategy. $\cos\theta = \mathbf{a} \cdot \mathbf{b} / (\|\mathbf{a}\|\|\mathbf{b}\|)$; all three numbers are one dot product each. Compare also $\|\mathbf{a} - \mathbf{b}\|$: distance and angle answer different questions.

Solution. $\mathbf{a} \cdot \mathbf{b} = 2 - 2 + 4 = 4$; $\|\mathbf{a}\| = 3$, $\|\mathbf{b}\| = 3$; so $\cos\theta = 4/9 \approx 0.444$ and $\theta \approx 63.6°$. Meanwhile $\|\mathbf{a} - \mathbf{b}\| = \|(-1, 3, 0)\| = \sqrt{10} \approx 3.16$: vectors nearly a right angle apart can still be close in raw distance when they are short. For sparse high-dimensional taste vectors the *angle* is the honest similarity — magnitude is mostly noise there. ✓

5.2 Orthogonal and orthonormal sets

Definition 5.7 (Orthogonal / orthonormal set)**.** Vectors $\mathbf{q}_1, \dots, \mathbf{q}_k$ form an *orthogonal set* when $\mathbf{q}_i \perp \mathbf{q}_j$ for $i \neq j$. If additionally each is a unit vector, the set is *orthonormal*: $\mathbf{q}_i^T \mathbf{q}_j = \delta_{ij}$. An orthonormal set that is also a basis is an *orthonormal basis*.

Theorem 5.8. *A finite orthogonal set of nonzero vectors is linearly independent.*

Proof. Suppose $c_1\mathbf{q}_1 + \cdots + c_k\mathbf{q}_k = \mathbf{0}$. Take the inner product with $\mathbf{q}_j$: every term except $c_j\|\mathbf{q}_j\|^2$ is orthogonal to $\mathbf{q}_j$, so $c_j\|\mathbf{q}_j\|^2 = 0$ and hence $c_j = 0$ (each $\|\mathbf{q}_j\| \neq 0$). This holds for every j, so all coefficients vanish. ∎

The payoff: in an orthonormal basis, coordinates are dot products.

Theorem 5.9. *If $\mathcal{Q} = \{\mathbf{q}_1, \dots, \mathbf{q}_n\}$ is an orthonormal basis of $\mathbb{R}^n$ and $\mathbf{v} = c_1\mathbf{q}_1 + \cdots + c_n\mathbf{q}_n$, then $c_j = \mathbf{q}_j^T\mathbf{v}$.*

Compare with a general basis (ch. 3): extracting $[\mathbf{v}]_{\mathcal{B}}$ meant solving a linear system. With an orthonormal basis it is n dot products — and each coefficient is computed *independently*, so an error in one cannot corrupt the others.

Definition 5.10 (Orthogonal matrix)**.** A square matrix Q is *orthogonal* when $Q^TQ = I$, i.e. $Q^{-1} = Q^T$: its columns form an orthonormal basis of $\mathbb{R}^n$.

Orthogonal matrices preserve everything geometric: $\|Q\mathbf{v}\| = \|\mathbf{v}\|$ and

$$\langle Q\mathbf{u}, Q\mathbf{v}\rangle = \mathbf{u}^T Q^T Q\mathbf{v} = \langle \mathbf{u}, \mathbf{v}\rangle.$$

Rotations and reflections from ch. 4 are exactly the 2D orthogonal matrices ($\det Q = \pm 1$). Their numerical value is just as large: multiplying by Q or Q^T cannot amplify rounding error, which is why stable algorithms are built out of orthogonal factors.

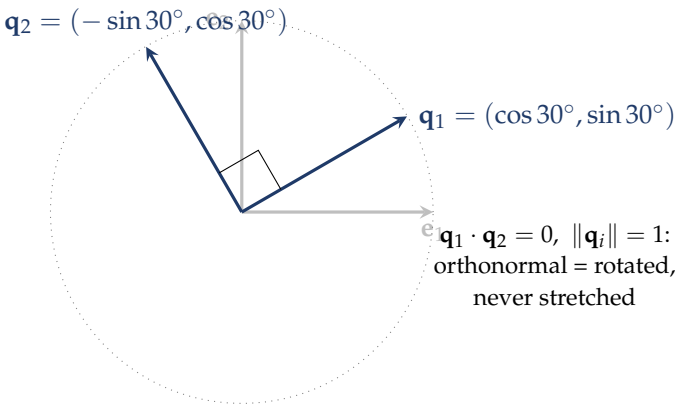


Figure 5.1: An orthonormal pair is the standard frame, rotated. Notice what orthonormality buys: the coordinates of a vector in the $\mathbf{q}$-frame are just the two projections — no solving needed.

5.3 Projection onto a line and onto a subspace

The concrete problem: given a vector $\mathbf{b}$ and a subspace W, which point of W is closest to $\mathbf{b}$? It is the foot of the perpendicular — the *projection*.

The derivation itself is worth doing once properly (Section 5.6); the result is

$$\mathbf{p} = \frac{\mathbf{a}^T\mathbf{b}}{\mathbf{a}^T\mathbf{a}}\,\mathbf{a}, \qquad P = \frac{\mathbf{a}\mathbf{a}^T}{\mathbf{a}^T\mathbf{a}}$$

— a rank-1 matrix satisfying $P^2 = P$ and $P^T = P$.

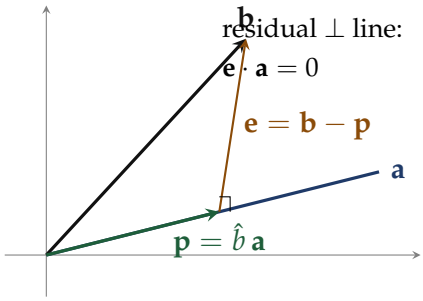


Figure 5.2: Projection of $\mathbf{b}$ onto the line through $\mathbf{a}$. Notice that the formula follows entirely from one requirement: the residual $\mathbf{e} = \mathbf{b} - \mathbf{p}$ is perpendicular to the line. Regenerate: `figures/ch05_projection_line.tex`.

The same "residual is perpendicular" condition works for any subspace. If $\mathbf{q}_1, \dots, \mathbf{q}_k$ is an *orthonormal* basis of W, the projection splits into independent one-dimensional pieces:

Proposition 5.11. *For orthonormal $\mathbf{q}_1, \dots, \mathbf{q}_k$ spanning W,*

$$\mathbf{p} = (\mathbf{q}_1^T\mathbf{b})\,\mathbf{q}_1 + \cdots + (\mathbf{q}_k^T\mathbf{b})\,\mathbf{q}_k.$$

Proof. Let $\mathbf{p}$ have that form. Then $\mathbf{p} \in W$, and for each basis vector, $\mathbf{q}_j^T(\mathbf{b} - \mathbf{p}) = \mathbf{q}_j^T\mathbf{b} - \mathbf{q}_j^T\mathbf{b} = 0$, so the residual is orthogonal to every basis vector, hence to all of W. ∎

For a *general* basis $A = [\mathbf{a}_1 \ \cdots \ \mathbf{a}_k]$ of W (tall, independent columns), the condition $A^T(\mathbf{b} - A\mathbf{x}) = \mathbf{0}$ gives the normal equations $A^TA\mathbf{x} = A^T\mathbf{b}$, and

Theorem 5.12. *If A has linearly independent columns, the orthogonal projection onto* $\mathrm{col}(A)$ *is*

$$P = A(A^TA)^{-1}A^T, \qquad P^2 = P, \quad P^T = P.$$

BELIEVING A^TA IS ALWAYS INVERTIBLE

A^TA is invertible *if and only if* A's columns are linearly independent (ch. 8 proves it). Feed it a rank-deficient A and the inverse does not exist — which is exactly what least squares does when your data has redundant features.

Example 5.13 (Projection onto a line and a plane)**. Strategy.** On a line there is one direction to enforce; on a plane with an orthonormal basis the projection splits into separate components (Proposition 5.11). Compute each piece, then check that the residual kills the subspace.

Solution. Take $\mathbf{a} = (2,0,1)$ and $\mathbf{b} = (1,2,3)$. Projection onto the line through $\mathbf{a}$:

$$\mathbf{p} = \frac{\mathbf{a}^T\mathbf{b}}{\mathbf{a}^T\mathbf{a}}\,\mathbf{a} = \frac{2+0+3}{4+0+1}\,\mathbf{a} = \frac{5}{5}\,(2,0,1) = (2,0,1).$$

Check: $\mathbf{b}-\mathbf{p} = (-1,2,2)$ and $\mathbf{a}\cdot(\mathbf{b}-\mathbf{p}) = -2+0+2 = 0$. ✓

Now project the same $\mathbf{b}$ onto $W = \operatorname{span}\{(1,1,0),(0,0,1)\}$. The spanning vectors are already orthogonal; normalize to $\mathbf{q}_1 = (1,1,0)/\sqrt{2}$, $\mathbf{q}_2 = (0,0,1)$:

$$\mathbf{p}_W = (\mathbf{q}_1^T\mathbf{b})\mathbf{q}_1 + (\mathbf{q}_2^T\mathbf{b})\mathbf{q}_2 = \frac{3}{\sqrt{2}}\,\frac{(1,1,0)}{\sqrt{2}} + 3(0,0,1) = \left(\tfrac{3}{2},\tfrac{3}{2},3\right).$$

Check: $\mathbf{b}-\mathbf{p}_W = (-\frac{1}{2},\frac{1}{2},0)$, which is orthogonal to $\mathbf{q}_1$ ($-\frac{1}{2}+\frac{1}{2}=0$) and to $\mathbf{q}_2$. ✓Notice P_{line} was rank 1; the two pieces of $\mathbf{p}_W$ were computed independently — impossible with a skew basis.

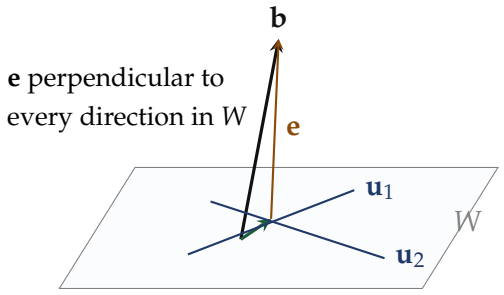


Figure 5.3: Projecting $\mathbf{b}$ onto a plane W. Notice the residual $\mathbf{e}$ is perpendicular to *every* direction in W, not just the basis vectors. Regenerate: `figures/ch05_projection_plane.tex`.

The code checks the general formula end-to-end on a 4D example:

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

A = np.array([[1., 0.],
              [0., 1.],
              [1., 1.],
              [1., -1.]])
b = np.array([2., 1., 3., -1.])

P = A @ np.linalg.inv(A.T @ A) @ A.T
p = P @ b
print("P =")
print(P)
print("p = P @ b =", p)
print("in col(A)?", np.allclose(p, A @ np.linalg.solve(A.T @ A, A.T @
    b)))
print("residual b - p =", b - p)
print("residual perpendicular to col(A)?", np.allclose(A.T @ (b - p),
    0))
print("P^2 = P:", np.allclose(P @ P, P))
print("P^T = P:", np.allclose(P.T, P))
print("||b||^2 = ||p||^2 + ||b-p||^2:",
```

```
    np.isclose(b @ b, p @ p + (b - p) @ (b - p)))
```

Listing 5.1: `code/ch05_projection_idempotent.py -- builds` $P = A(A^TA)^{-1}A^T$ `on a` 4×2 `basis, projects, and verifies every defining property.`

```
P =
[[ 0.3333 0. 0.3333 0.3333]
 [ 0. 0.3333 0.3333 -0.3333]
 [ 0.3333 0.3333 0.6667 0. ]
 [ 0.3333 -0.3333 0. 0.6667]]
p = P @ b = [ 1.3333 1.6667 3. -0.3333]
in col(A)? True
residual b - p = [ 0.6667 -0.6667 0. -0.6667]
residual perpendicular to col(A)? True
P^2 = P: True
P^T = P: True
||b||^2 = ||p||^2 + ||b-p||^2: True
```

Listing 5.2: Output of the projection check. Notice all three signatures of an orthogonal projection at once: lands in $\mathrm{col}(A)$, perpendicular residual, and $P^2 = P$, $P^T = P$.

5.4 Gram–Schmidt and the QR factorization

Any basis can be upgraded to an orthonormal one by successive subtraction: take each vector, remove the parts already claimed by earlier orthonormal vectors, and normalize what is left.

The rule in one line: each new vector gives up the parts that earlier orthonormal vectors already own. Formally, $\mathbf{u}_1 = \mathbf{a}_1$, then $\mathbf{u}_j = \mathbf{a}_j - \sum_{i<j} \mathrm{proj}_{\mathbf{u}_i} \mathbf{a}_j$, and $\mathbf{q}_j = \mathbf{u}_j / \|\mathbf{u}_j\|$. The projection formula guarantees $\mathbf{u}_j \perp \mathbf{u}_i$ for all $i < j$.

Example 5.14 (Gram–Schmidt by hand)**. Strategy.** Work left to right; after each step verify $\mathbf{u}_j$ is orthogonal to every $\mathbf{u}_i$ before it. Fractions are easier to check than radicals — keep vectors unnormalized until the last line.

Solution. Take $\mathbf{a}_1 = (1,1,0)$, $\mathbf{a}_2 = (1,0,1)$, $\mathbf{a}_3 = (0,1,1)$.

$$\mathbf{u}_1 = (1,1,0), \qquad \mathbf{u}_2 = \mathbf{a}_2 - \frac{\mathbf{a}_2 \cdot \mathbf{u}_1}{\mathbf{u}_1 \cdot \mathbf{u}_1}\,\mathbf{u}_1 = (1,0,1) - \tfrac{1}{2}(1,1,0) = \left(\tfrac{1}{2}, -\tfrac{1}{2}, 1\right).$$

Check $\mathbf{u}_1 \cdot \mathbf{u}_2 = \frac{1}{2} - \frac{1}{2} = 0$. ✓ For the third vector, subtract *both* projections:

$$\mathbf{u}_3 = \mathbf{a}_3 - \frac{\mathbf{a}_3 \cdot \mathbf{u}_1}{2}\,\mathbf{u}_1 - \frac{\mathbf{a}_3 \cdot \mathbf{u}_2}{\mathbf{u}_2 \cdot \mathbf{u}_2}\,\mathbf{u}_2 = (0,1,1) - \tfrac{1}{2}(1,1,0) - \tfrac{1/2}{3/2}\left(\tfrac{1}{2}, -\tfrac{1}{2}, 1\right) = \left(-\tfrac{2}{3}, \tfrac{2}{3}, \tfrac{2}{3}\right).$$

Check: $\mathbf{u}_3 \cdot \mathbf{u}_1 = -\frac{2}{3} + \frac{2}{3} = 0$ and $\mathbf{u}_3 \cdot \mathbf{u}_2 = -\frac{1}{3} - \frac{1}{3} + \frac{2}{3} = 0$. ✓ Normalizing ($\|\mathbf{u}_1\| = \sqrt{2}$, $\|\mathbf{u}_2\| = \sqrt{3/2}$, $\|\mathbf{u}_3\| = 2/\sqrt{3}$):

$$\mathbf{q}_1 = \tfrac{1}{\sqrt{2}}(1,1,0), \quad \mathbf{q}_2 = \tfrac{1}{\sqrt{6}}(1,-1,2), \quad \mathbf{q}_3 = \tfrac{1}{\sqrt{3}}(-1,1,1).$$

Collect the unit vectors as columns of Q and reverse the bookkeeping: since $\mathbf{a}_j = \sum_{i \le j} r_{ij}\mathbf{q}_i$ with $r_{ij} = \mathbf{q}_i^T \mathbf{a}_j$ and $r_{jj} = \|\mathbf{u}_j\|$,

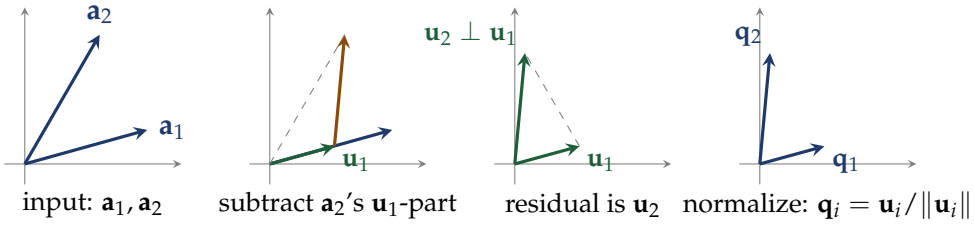


Figure 5.4: Gram–Schmidt on two vectors. Notice the algorithm never creates a new direction — it strips each input of the directions already spoken for, and the residual that survives is automatically perpendicular. Regenerate: `figures/ch05_gs_steps.tex`.

Theorem 5.15 (QR factorization). *Every $A \in \mathbb{R}^{m\times n}$ with linearly independent columns factors as*

$$A = QR,$$

where $Q \in \mathbb{R}^{m\times n}$ has orthonormal columns and $R \in \mathbb{R}^{n\times n}$ is upper triangular with positive diagonal.

The factorization is a solver's dream: $Ax = b$ becomes $Rx = Q^T b$, and an upper-triangular system costs one back-substitution.

Example 5.16 (QR and solving $Ax = b$). **Strategy.** Read Q off Example 5.14, then get $R = Q^T A$ (the entries are exactly the coefficients discarded during Gram–Schmidt). Solve by projecting $\mathbf{b}$ then back-substituting.

Solution. With the three vectors of Example 5.14,

$$Q = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{6} & -1/\sqrt{3} \\ 1/\sqrt{2} & -1/\sqrt{6} & 1/\sqrt{3} \\ 0 & 2/\sqrt{6} & 1/\sqrt{3} \end{bmatrix}, \qquad R = \begin{bmatrix} \sqrt{2} & 1/\sqrt{2} & 1/\sqrt{2} \\ 0 & \sqrt{3/2} & 1/\sqrt{6} \\ 0 & 0 & 2/\sqrt{3} \end{bmatrix},$$

and you can multiply QR to recover A. For $\mathbf{b} = (3,2,1)$: $y = Q^T\mathbf{b} = (5/\sqrt{2},\ \sqrt{3/2},\ 0)$, and back-substitution on $Rx = y$ gives $x_3 = 0$, $x_2 = 1$, $x_1 = 2$ — so $x = (2,1,0)$, and indeed $2\mathbf{a}_1 + \mathbf{a}_2 = (3,2,1) = \mathbf{b}$.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

A = np.array([[1., 1., 0.],
              [1., 0., 1.],
              [0., 1., 1.]])
b = np.array([3., 2., 1.])

Q, R = np.linalg.qr(A)
print("Q =")
print(Q)
print("R =")
print(R)
print("Q^T Q = I:", np.allclose(Q.T @ Q, np.eye(3)))
print("QR = A:", np.allclose(Q @ R, A))
```

```
16
17 y = Q.T @ b
18 x = np.linalg.solve(R, y) # triangular -> back substitution
19 print("x via QR =", x)
20 print("np.linalg.solve =", np.linalg.solve(A, b))
21 print("residual:", np.linalg.norm(A @ x - b))
```

Listing 5.3: `code/ch05_qr_solve.py -- factors the same` 3×3 `with np.linalg.qr and solves` $Ax = b$ `via` $y = Q^T b$ `and back-substitution. Sign convention: numpy returns negative diagonal` R `(both conventions are valid QR factorizations -- it is` Q`'s sign that flips to compensate).`

```
1 Q =
2 [[-0.7071 0.4082 -0.5774]
3  [-0.7071 -0.4082 0.5774]
4  [-0. 0.8165 0.5774]]
5 R =
6 [[-1.4142 -0.7071 -0.7071]
7  [ 0. 1.2247 0.4082]
8  [ 0. 0. 1.1547]]
9 Q^T Q = I: True
10 QR = A: True
11 x via QR = [ 2. 1. -0.]
12 np.linalg.solve = [2. 1. 0.]
13 residual: 6.753223014464259e-16
```

Listing 5.4: Output of the QR solve. Notice the two conventions agree on everything that matters: $Q^T Q = I$, $QR = A$, and the same $x = (2, 1, 0)$ as the hand computation.

There is a trap hidden in the arithmetic of Gram–Schmidt itself. In the *classical* version, the coefficient for direction i is computed from the original $\mathbf{a}_j$; in the *modified* version it is computed from the partially-deflated vector — same value in exact arithmetic, very different in floating point.

```
1 import numpy as np
2 np.set_printoptions(precision=2, suppress=True)
3 np.seterr(all="ignore")
4
5 def classical_gs(A):
6     Q = np.zeros_like(A)
7     for j in range(A.shape[1]):
8         v = A[:, j].copy()
9         for i in range(j):
10            v -= (Q[:, i] @ A[:, j]) * Q[:, i]
11        Q[:, j] = v / np.linalg.norm(v)
12    return Q
13
14 def modified_gs(A):
15    Q = np.zeros_like(A)
16    V = A.copy()
17    for j in range(A.shape[1]):
18        Q[:, j] = V[:, j] / np.linalg.norm(V[:, j])
19        for k in range(j + 1, A.shape[1]):
```

```
            V[:, k] -= (Q[:, j] @ V[:, k]) * Q[:, j]
    return Q

eps = 1e-8
A = np.array([[1., 1., 1.],
              [eps, 0., 0.],
              [0., eps, 0.],
              [0., 0., eps]])

Qc = classical_gs(A)
Qm = modified_gs(A)
print("classical |Q^T Q - I| =", np.linalg.norm(Qc.T @ Qc - np.eye(3),
      ord=np.inf))
print(Qc.T @ Qc)
print("modified |Q^T Q - I| =", np.linalg.norm(Qm.T @ Qm - np.eye(3),
    ord=np.inf))
print(Qm.T @ Qm)
Q_lib, _ = np.linalg.qr(A)
print("np.linalg.qr |Q^T Q - I| =",
      np.linalg.norm(Q_lib.T @ Q_lib - np.eye(3), ord=np.inf))
```

Listing 5.5: `code/ch05_gs_stability.py -- classical vs. modified Gram-Schmidt on nearly-parallel columns ($\varepsilon = 10^{-8}$). Measures orthogonality drift as $\|Q^TQ - I\|_\infty$.`

```
classical |Q^T Q - I| = 0.500000007071068
[[ 1. -0. -0. ]
 [-0. 1. 0.5]
 [-0. 0.5 1. ]]
modified |Q^T Q - I| = 1.1153550716504105e-08
[[ 1. -0. -0.]
 [-0. 1. 0.]
 [-0. 0. 1.]]
np.linalg.qr |Q^T Q - I| = 7.726045597493973e-16
```

Listing 5.6: Output of the stability test. Notice classical Gram–Schmidt loses *half a unit* of orthogonality while modified loses 10^{-8}, and the library QR (Householder reflectors) is exact to machine precision. This is not a toy gap — it is why nobody ships classical Gram–Schmidt.

ASSUMING CLASSICAL AND MODIFIED GRAM–SCHMIDT ARE THE SAME ALGORITHM

They are algebraically identical and numerically different. Classical GS computes each projection coefficient against the original column, so rounding errors in early $\mathbf{q}_i$ pollute everything downstream; modified GS recomputes against the partially-deflated vector. On nearly dependent input the classical version can return columns at 45° angles, as the listing shows.

5.5 Orthogonal complements: the four subspaces pair up

Definition 5.17 (Orthogonal complement)**.** The *orthogonal complement* of a subspace $W \subseteq \mathbb{R}^n$ is

$$W^\perp = \{\mathbf{v} \in \mathbb{R}^n : \langle \mathbf{v}, \mathbf{w} \rangle = 0 \text{ for all } \mathbf{w} \in W\}.$$

It is a subspace, $W \cap W^\perp = \{\mathbf{0}\}$, and $\dim W + \dim W^\perp = n$.

Theorem 5.18 (Fundamental theorem, geometric form)**.** *For $A \in \mathbb{R}^{m \times n}$:*

$$\operatorname{row}(A)^\perp = \ker(A), \qquad \operatorname{col}(A)^\perp = \ker(A^T),$$

i.e. $\mathbb{R}^n = \operatorname{row}(A) \oplus \ker(A)$ *and* $\mathbb{R}^m = \operatorname{col}(A) \oplus \ker(A^T)$.

Proof. $\mathbf{x} \in \ker(A)$ iff $A\mathbf{x} = \mathbf{0}$ iff every row of A is orthogonal to $\mathbf{x}$ iff $\mathbf{x} \perp \operatorname{row}(A)$. Apply the same argument to A^T. ∎

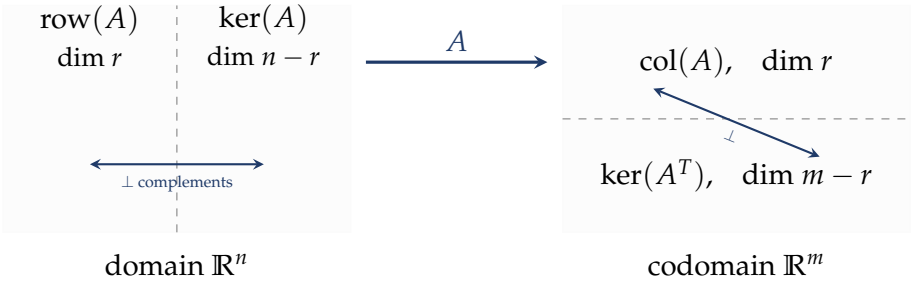


Figure 5.5: The four subspaces as orthogonal pairs. Notice every vector splits: its $\operatorname{row}(A)$ part is what A "sees," its $\ker(A)$ part maps to $\mathbf{0}$. Regenerate: `figures/ch05_complements.tex`.

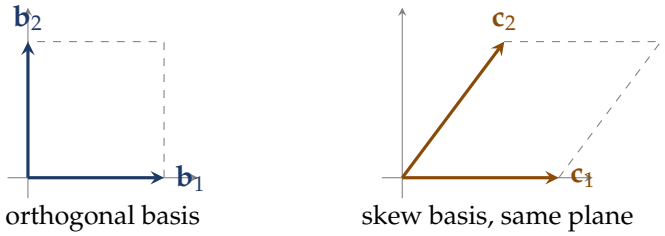


Figure 5.6: Orthogonal vs. skew bases of the same plane. Notice both span the same space — only the orthogonal one makes projection and coordinates decouple. Regenerate: `figures/ch05_ortho_vs_skew.tex`.

Exercise 5.2 (medium)**.** Find the orthogonal complement of $W = \operatorname{span}\{(1,1,1)\}$ in $\mathbb{R}^3$.

Solution. $W^\perp$ is the plane $x + y + z = 0$, i.e. $\operatorname{span}\{(1,-1,0),\ (1,0,-1)\}$. Dimension check: $\dim W + \dim W^\perp = 1 + 2 = 3$.

Exercise 5.3 (medium)**.** For $A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$, split $\mathbf{b} = (1,1,0)$ into $\operatorname{row}(A)$ and $\ker(A)$ parts.

Solution. $\ker(A)$ is the line $t(1,1,-1)$. Projecting $\mathbf{b}$ onto it gives $\frac{2}{3}(1,1,-1)$, so $\mathbf{b} = \underbrace{\tfrac{1}{3}(1,1,2)}_{\text{row part}} + \underbrace{\tfrac{1}{3}(2,2,-2)}_{\text{null part}}$; check $A \cdot \tfrac{1}{3}(2,2,-2) = \tfrac{1}{3}(0,0) = \mathbf{0}$. ✓

Exercise 5.4 (hard)**.** Let P be the projection matrix onto a subspace W. Prove $I - P$ is the projection matrix onto $W^\perp$.

Solution. $(I-P)^2 = I - 2P + P^2 = I - P$ and $(I-P)^T = I - P$, so $I - P$ is a projection. Its column space is $\{\mathbf{v} - P\mathbf{v}\} = \{\text{residuals}\} \subseteq W^\perp$; for any $\mathbf{w} \in W^\perp$, $P\mathbf{w} = \mathbf{0}$ so $(I-P)\mathbf{w} = \mathbf{w}$ and the image is all of $W^\perp$.

Example 5.19 (The pairing on one matrix)**.** For $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$, exhibit $\text{col}(A)^\perp = \ker(A^T)$ as *the same line* computed two ways.

Strategy. $\text{col}(A)$ is a line through $(1,2,3)$; its complement is a plane. $\ker(A^T)$ is computed by solving $A^T\mathbf{y} = \mathbf{0}$ — one elimination.

Solution. $A^T = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}$, RREF $\begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \end{bmatrix}$, so $\ker(A^T) = \text{span}\{(-2,1,0), (-3,0,1)\}$ — a plane. And $(1,2,3)^\perp$ is exactly $x + 2y + 3z = 0$: the same plane. Basis-check: $(-2,1,0) \cdot (1,2,3) = 0$ and $(-3,0,1) \cdot (1,2,3) = 0$. ✓ Dimensions close the loop too: the column line plus the kernel plane tile all of $\mathbb{R}^3$ $(1 + 2 = 3)$.

5.6 Hand derivation

Derive the projection formula from scratch. Drop a perpendicular from $\mathbf{b}$ to the line through $\mathbf{a}$. The foot $\mathbf{p}$ must satisfy two conditions: $\mathbf{p}$ lies on the line, so $\mathbf{p} = \hat{b}\,\mathbf{a}$ for some scalar $\hat{b}$; and the residual $\mathbf{e} = \mathbf{b} - \mathbf{p}$ is orthogonal to the line, so $\langle \mathbf{a}, \mathbf{b} - \hat{b}\mathbf{a} \rangle = 0$. Expanding, $\langle \mathbf{a}, \mathbf{b} \rangle = \hat{b}\,\langle \mathbf{a}, \mathbf{a} \rangle$, hence

$$\hat{b} = \frac{\mathbf{a}^T\mathbf{b}}{\mathbf{a}^T\mathbf{a}}, \qquad \mathbf{p} = \frac{\mathbf{a}^T\mathbf{b}}{\mathbf{a}^T\mathbf{a}}\,\mathbf{a}.$$

Now promote it to a subspace. With columns of A spanning W, write $\mathbf{p} = A\mathbf{x}$ and require the residual orthogonal to every column: $A^T(\mathbf{b} - A\mathbf{x}) = \mathbf{0}$, i.e. $A^TA\mathbf{x} = A^T\mathbf{b}$. Independent columns make A^TA invertible, so $\mathbf{x} = (A^TA)^{-1}A^T\mathbf{b}$ and $P = A(A^TA)^{-1}A^T$. Finally check the algebra the geometry promised: $P^2 = A(A^TA)^{-1}A^TA(A^TA)^{-1}A^T = P$ since A^TA cancels, and $P^T = P$ by expanding the transpose.

5.7 Common mistakes

PROJECTING ONTO THE WRONG SUBSPACE

$\text{proj}_W\,\mathbf{b}$ answers "closest point in W," not "closest point *to* W." In least squares you project $\mathbf{b}$ onto $\text{col}(A)$, not onto the rows — swapping them is the most common conceptual bug in the next chapter.

FORGETTING TO NORMALIZE BEFORE PROJECTING ONTO A DIRECTION

$\text{proj}_{\mathbf{a}}\,\mathbf{b} = \frac{\mathbf{a}^T\mathbf{b}}{\mathbf{a}^T\mathbf{a}}\mathbf{a}$ — the denominator is $\|\mathbf{a}\|^2$, not $\|\mathbf{a}\|$. If $\mathbf{a}$ is already a unit vector they coincide, which is why orthonormal bases make the formula look like "just dot products."

TREATING SIMILAR-LOOKING PROJECTIONS AS INTERCHANGEABLE

Projecting with an orthonormal basis ($\mathbf{p} = \sum_{i=1}^{k}(\mathbf{q}_i^T\mathbf{b})\mathbf{q}_i$) and with a general basis ($P = A(A^TA)^{-1}A^T\mathbf{b}$) give the same point only when the orthonormal vectors span the same W. The second formula works without orthogonality but costs an inverse.

ASSUMING $P^2 = P$ AND $P^T = P$ ARE OPTIONAL PROPERTIES

They are not properties you must verify case-by-case; they *define* orthogonal projection. If a "projection" you computed fails either one, it is wrong — or it was an oblique (non-orthogonal) projection to begin with.

5.8 Summary

Object	What you need to know
Inner product	$\langle \mathbf{u}, \mathbf{v}\rangle = \mathbf{u}^T\mathbf{v}$; gives norm $\|\mathbf{v}\| = \sqrt{\langle \mathbf{v}, \mathbf{v}\rangle}$ and $\cos\theta = \langle \mathbf{u}, \mathbf{v}\rangle / \|\mathbf{u}\|\|\mathbf{v}\|$
Orthonormal basis	$\mathbf{q}_i^T\mathbf{q}_j = \delta_{ij}$; coordinates are dot products $c_j = \mathbf{q}_j^T\mathbf{v}$
Orthogonal matrix	$Q^TQ = I$, i.e. $Q^{-1} = Q^T$; preserves lengths and angles; $\det = \pm 1$
Projection onto line	$\mathbf{p} = \frac{\mathbf{a}^T\mathbf{b}}{\mathbf{a}^T\mathbf{a}}\mathbf{a}$; $P = \mathbf{a}\mathbf{a}^T/\mathbf{a}^T\mathbf{a}$
Projection onto subspace	orthonormal basis: $\mathbf{p} = \sum_{i=1}^{k}(\mathbf{q}_i^T\mathbf{b})\mathbf{q}_i$; general basis: $P = A(A^TA)^{-1}A^T$; always $P^2 = P$, $P^T = P$
Gram–Schmidt	$\mathbf{u}_j = \mathbf{a}_j - \sum_{i<j}\text{proj}_{\mathbf{u}_i}\mathbf{a}_j$, $\mathbf{q}_j = \mathbf{u}_j/\|\mathbf{u}_j\|$; use the modified version
QR	$A = QR$: orthonormal Q, upper-triangular R; solves $Ax = b$ as $Rx = Q^Tb$
Orthogonal complement	$\text{row}(A)^\perp = \ker(A)$, $\text{col}(A)^\perp = \ker(A^T)$; every space splits $W \oplus W^\perp = \mathbb{R}^n$

WATCH INSTEAD

1. *Dot products and duality* — 3Blue1Brown, Essence of Linear Algebra ch. 9, 14:11, https://youtu.be/LyGKycYT2v0 — the inner-product machinery of §5.1 (accessed 2026-10-11).

2. *Projections onto Subspaces* — MIT OpenCourseWare, 18.06 Lecture 15, 48:51, https://youtu.be/Y_Ac6KiQ1t0 — §5.3 at full length, same notation (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., ch. 4 (textbook level) — orthogonality, projections, and Gram–Schmidt as this chapter teaches them.
- MIT OpenCourseWare, *18.06SC Linear Algebra*, Unit II — recitations on projections and QR, at OCW (course "18.06SC", Unit II page), https://ocw.mit.edu. Verified 2026-10-11.
- S. Boyd and L. Vandenberghe, *VMLS*, chs. 3, 5 (free PDF) — distance, angle, and independence in the data-first register. https://web.stanford.edu/~boyd/vmls/vmls.pdf Verified 2026-10-11.
- Margalit and Rabinoff, *Interactive Linear Algebra*, "Orthogonality" — dot products through Gram–Schmidt, hyperlinked. https://textbooks.math.gatech.edu/ila/ Verified 2026-10-11.
- M. Deisenroth et al., *Mathematics for Machine Learning*, ch. 3 (free PDF) — inner products and orthogonal projections for ML. https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.
- J. Hefferon, *Linear Algebra*, ch. VI topic "Gram–Schmidt" (free textbook) — the rigorous version of the same orthogonalization. https://hefferon.net/linearalgebra/index.html Verified 2026-10-11.

Exercises

Exercise 5.5 (easy). Compute $\text{proj}_{\mathbf{a}}\,\mathbf{b}$ for $\mathbf{a} = (1, 2, 2)$, $\mathbf{b} = (4, -1, 1)$, and verify the residual is orthogonal to $\mathbf{a}$.

Exercise 5.6 (easy). Is $Q = \frac{1}{5}\begin{bmatrix} 3 & 4 \\ 4 & -3 \end{bmatrix}$ orthogonal? What is its determinant, and which kind of map is it?

Exercise 5.7 (easy). Run Gram–Schmidt by hand on $(1, 0), (1, 1)$, then write the QR of $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$.

Exercise 5.8 (easy). Show that if W is a subspace, so is $W^{\perp}$ (check the three subspace properties from ch. 3).

Exercise 5.9 (medium). Prove the projection $\mathbf{p} = \text{proj}_W\,\mathbf{b}$ is the unique closest point of W to $\mathbf{b}$: for any $\mathbf{w} \in W$, $\|\mathbf{b} - \mathbf{w}\|^2 = \|\mathbf{b} - \mathbf{p}\|^2 + \|\mathbf{p} - \mathbf{w}\|^2$. (Hint: Pythagoras — where do the vectors live?)

Exercise 5.10 (medium). A projection matrix has $P^T = P$, $P^2 = P$. Show its eigenvalues (ch. 7 preview) can only be 0 or 1: if $P\mathbf{v} = \lambda\mathbf{v}$ then $\lambda^2 = \lambda$.

Exercise 5.11 (medium). Compute the QR of $A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$ by hand and use it to project $\mathbf{b} = (1, 1, 1)$ onto $\mathrm{col}(A)$. Verify $A^T(\mathbf{b} - \mathbf{p}) = \mathbf{0}$.

Exercise 5.12 (medium). In code (`numpy`), generate a random 6×3 matrix A, compute $P = A(A^TA)^{-1}A^T$, and verify $P^2 = P$, $P^T = P$, and $\mathrm{rank}(P) = 3$. What do the eigenvalues of P look like?

Exercise 5.13 (medium). For $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}$ (rank 1), write the orthogonal decomposition of $\mathbf{b} = (1, 1, 1)$ into $\mathrm{col}(A)$ and $\ker(A^T)$ parts. Which subspace holds most of $\mathbf{b}$?

Exercise 5.14 (hard). Prove that a square matrix P is an orthogonal projection ($P^2 = P$, $P^T = P$) onto $\mathrm{col}(P)$ — i.e. the two algebraic conditions characterize projections completely.

Exercise 5.15 (hard). Show that $(W^\perp)^\perp = W$ for any subspace $W \subseteq \mathbb{R}^n$, and use it to prove $\mathrm{row}(A) = \ker(A)^\perp$.

Exercise 5.16 (hard). Implement modified Gram–Schmidt yourself and compare against `np.linalg.qr` on a random 8×5 matrix: report $\|Q^TQ - I\|_\infty$ and $\|A - QR\|_\infty$ for both, plus the residual of $Rx = Q^Tb$ solving $Ax = b$. Explain any sign differences in Q and R. (Reference implementation: `code/ch05_gs_stability.py`.)

Exercise 5.17 (easy). Compute $\cos\theta$ between $\mathbf{u} = (1, 0)$ and $\mathbf{v} = (1, 1)$, then between $\mathbf{u}$ and $-\mathbf{v}$. What does the sign flip say geometrically?

Exercise 5.18 (medium). Let $\mathbf{q}_1 = (1, 1, 0)/\sqrt{2}$ and $\mathbf{q}_2 = (1, -1, 0)/\sqrt{2}$. Extend to an orthonormal basis of $\mathbb{R}^3$ — find $\mathbf{q}_3$ by inspection, then confirm by Gram–Schmidt on $(1, 0, 0)$.

Exercise 5.19 (hard). Prove that orthogonal nonzero vectors are automatically independent, then show the converse fails: exhibit independent vectors whose Gram–Schmidt output differs from the input. What does the difference measure?

Notes

Projection onto a line and the Gram–Schmidt–QR pipeline follow Strang's ordering (orthogonalization before least squares) [6, 2]. The classical-versus-modified stability gap and Householder-style reflection QR are treated numerically in MML ch. 3 [5] and practically in VMLS ch. 11 [4]. Orthogonal complements of the four subspaces are the geometric half of the fundamental theorem; the dimension half was the rank–nullity theorem of ch. 3 [7, 6].

CHAPTER 6

Determinants

Every question this book has asked about a square matrix — does $Ax = \mathbf{b}$ have a unique solution? is A invertible? does the map collapse space? — turns out to be one number's job. The determinant is that number: a single scalar distilled from n^2 entries that answers all of them at once, and geometrically measures how much a linear map stretches volume. After this chapter you will be able to compute determinants three ways, know which way to reach for in a computation, and read $\det A$ as a signed volume rather than an abstract formula. One honest warning before we start: determinants are *theoretical* heavyweights and *computational* lightweights — you almost never need one in code, and this chapter explains why.

BEFORE YOU START

Chapters 1–5. *Recap:* a square matrix either squeezes space flat (not invertible) or it does not. The determinant is the single number that tells you which, and by how much it scales volume.

AFTER THIS CHAPTER YOU CAN

- compute a 2×2 or 3×3 determinant, and larger ones by elimination;
- use $\det A = 0$ to test for singularity;
- interpret the determinant as signed volume;
- explain why Cramer's rule is a curiosity and not an algorithm.

HOW TO THINK ABOUT THIS

A determinant is one number that answers a yes/no question: does this machine squash space flat? Its size tells you how much volume is scaled, its sign whether orientation flips. Do not memorise the cofactor formula; remember the geometry and compute by elimination.

6.1 Three definitions that agree

Definition 6.1 (Determinant by cofactors)**.** For $A \in \mathbb{R}^{n\times n}$, the (i,j)*-minor* M_{ij} is the determinant of the matrix left by deleting row i and column j; the *cofactor* is $C_{ij} = (-1)^{i+j}M_{ij}$. Then

$$\det A = \sum_{j=1}^{n} a_{1j}\, C_{1j} \qquad \text{(expansion along row 1)},$$

with $\det[a] = a$ starting the recursion. The same sum along *any* row or column gives the same value.

Definition 6.2 (Determinant by pivots)**.** If elimination reduces A to upper-triangular U using only row-add operations and s row swaps, then

$$\det A = (-1)^s\, u_{11}u_{22}\cdots u_{nn},$$

the product of the pivots with one minus sign per swap. If no nonzero pivot exists in some column, $\det A = 0$.

Definition 6.3 (Determinant as signed volume)**.** $\det A$ is the signed volume of the parallelepiped spanned by the columns of A — equivalently, the factor by which the linear map A scales n-dimensional volume, negative when orientation flips.

That three definitions give one function is the content of the chapter; Figure 6.1 shows the volume meaning directly.

Example 6.4 (One 3×3, two ways)**. Strategy.** Cofactors along a row with zeros keep the expansion short; elimination needs no swaps here, so the pivot product is a pure product. Compare.

Solution. Let $A = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{bmatrix}$. Cofactors along row 1 (the zero in position $(1,3)$ removes a term):

$$\det A = 2\begin{vmatrix} 3 & 1 \\ 1 & 2 \end{vmatrix} - 1\begin{vmatrix} 1 & 1 \\ 0 & 2 \end{vmatrix} + 0 = 2(6-1) - 1(2-0) = 10 - 2 = 8.$$

By elimination: $R_2 \leftarrow R_2 - \frac{1}{2}R_1$, then $R_3 \leftarrow R_3 - \frac{2}{5}R_2$:

$$\begin{bmatrix} 2 & 1 & 0 \\ 0 & 5/2 & 1 \\ 0 & 0 & 8/5 \end{bmatrix}, \qquad \det A = 2 \cdot \tfrac{5}{2} \cdot \tfrac{8}{5} = 8.$$

Same answer, 8 — and the second route produced it with three multiplies instead of a tree of 2×2 determinants. That asymmetry only gets worse with size (checkpoint below).

The pivot-product route is an algorithm, not just a trick — here it is in code, swap counting included:

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

def pivot_det(A):
   """det via elimination: product of pivots times (-1)^swaps."""
   A = A.astype(float).copy()
   n = A.shape[0]
   swaps = 0
   det = 1.0
   for j in range(n):
      piv = np.argmax(np.abs(A[j:, j])) + j
      if abs(A[piv, j]) < 1e-12:
         return 0.0
      if piv != j:
         A[[j, piv]] = A[[piv, j]]
         swaps += 1
      det *= A[j, j]
      A[j+1:, j:] -= np.outer(A[j+1:, j] / A[j, j], A[j, j:])
   return det * (-1.0) ** swaps

# 3x3 from the chapter: det = 8 by cofactors AND by pivots
A = np.array([[2., 1., 0.],
              [1., 3., 1.],
              [0., 1., 2.]])
print("matrix:")
print(A)
print("pivot_det =", pivot_det(A))
print("np.linalg.det =", np.linalg.det(A))

# swap-count check on a matrix needing a swap
B = np.array([[0., 2., 1.],
              [1., 0., 1.],
              [2., 1., 0.]])
print("swap-needed matrix pivot_det =", pivot_det(B),
    "np.linalg.det =", np.linalg.det(B))
```

Listing 6.1: `code/ch06_pivot_det.py` -- det as signed pivot product, verified against np.linalg.det on the Example 6.4 matrix and on a matrix that forces a row swap.

```
matrix:
[[2. 1. 0.]
 [1. 3. 1.]
 [0. 1. 2.]]
pivot_det = 8.0
np.linalg.det = 8.000000000000002
swap-needed matrix pivot_det = 5.0 np.linalg.det = 4.999999999999999
```

Listing 6.2: Output of the pivot determinant. Notice the swap-counted version matches the library on both — including the matrix whose zero top-left forces a swap (det $= 5$).

Exercise 6.1 (easy)**.** Compute $\det \begin{bmatrix} 3 & -1 \\ 2 & 4 \end{bmatrix}$ by the 2×2 formula.

Solution. $3 \cdot 4 - (-1) \cdot 2 = 12 + 2 = 14$.

6.2 Properties: product, transpose, row operations

Theorem 6.5 (Working properties)**.** *For $A, B \in \mathbb{R}^{n \times n}$:*

- $\det(A^T) = \det A$.
- $\det(AB) = \det A \, \det B$ *— and* $\det(A^{-1}) = 1/\det A$ *when A is invertible.*
- *Row-add ($R_i \leftarrow R_i + cR_j$) leaves* det *unchanged; a row swap flips the sign; scaling one row by c scales* det *by c, so* $\det(cA) = c^n \det A$ *(not $c \det A$).*

Example 6.6 (The product rule, numerically and in outline)**. Strategy.** Check on a pair of 2×2s first — catching a sign or arithmetic slip is easy there — then see why the property is structural rather than coincidental.

Solution. Let $A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$ and $B = \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix}$. Then $\det A = 6$, $\det B = -1$, and

$$AB = \begin{bmatrix} 3 & 5 \\ 3 & 3 \end{bmatrix}, \qquad \det(AB) = 9 - 15 = -6 = 6 \cdot (-1).$$

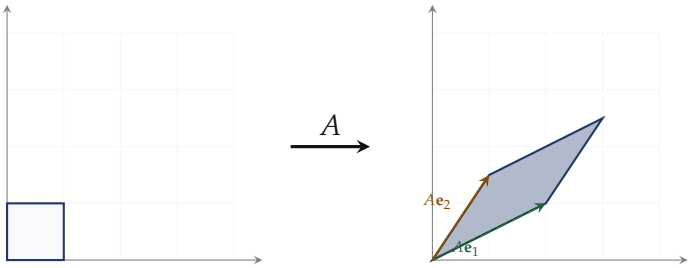
Why it holds (sketch): write A and B as chains of elementary row operations (ch. 2 — every invertible matrix is such a product). Each elementary factor multiplies a determinant by a known factor — c for a scaling, -1 for a swap, 1 for a row-add — and the factors multiply through the chain. Volume reading: applying B then A scales volume first by $\det B$, then by $\det A$; the composition scales by their product.

Checkpoint 6.1 Which is bigger work for a 20×20: cofactor expansion or pivot product? Cofactors cost $\sim n!$ operations; elimination $\sim n^3/3$. ($20! \approx 2.4 \times 10^{18}$ vs ≈ 2700 — this is why "determinant" in software is a pivot product.)

6.3 Determinant as signed volume

A linear map carries the unit square (unit cube) to a parallelogram (parallelepiped) whose edges are the columns of A. The signed area of that image is exactly the determinant — the sign recording whether the map preserved or flipped orientation (Figures 6.1–6.4).

Example 6.7 (Volume of a parallelepiped)**. Strategy.** Three edges from a common vertex make the columns; the signed volume is the determinant, and its sign tells whether the three edges form a right- or left-handed triple.



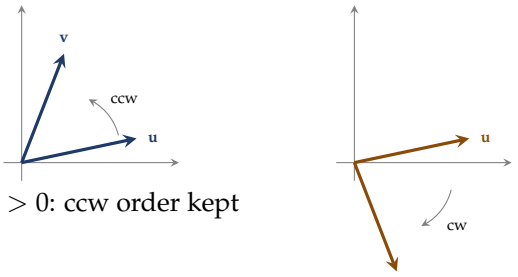
before: area $= 1$ after: area $= |\det A| = 2$

Figure 6.1: The map $A = \begin{bmatrix} 2 & 1 \\ 1 & 1.5 \end{bmatrix}$ sends the unit square to a parallelogram of area $|\det A| = 2$. Notice the edges of the image are the *columns* of A. Regenerate: `figures/ch06_volume_scaling.tex`.

Solution. Edges $\mathbf{u} = (1,0,0)$, $\mathbf{v} = (1,1,0)$, $\mathbf{w} = (0,1,2)$:

$$\det \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix} = 1 \cdot 1 \cdot 2 = 2,$$

triangular, so the product of the diagonal. Volume $= 2$, orientation preserved (positive). Reversing the order of $\mathbf{u}$ and $\mathbf{v}$ (column swap) gives -2: same shape, opposite handedness.



det > 0: ccw order kept

det < 0: orientation flipped

Figure 6.2: Orientation and the sign of det. Notice a positive determinant keeps the counter-clockwise ordering of the image edges; a reflection reverses it and det goes negative. Regenerate: `figures/ch06_orientation.tex`.

The code tests the scaling claim on shapes that are *not* the unit square — the factor should be the same for any region.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
np.random.seed(7)

def poly_area(P):
    x, y = P[:, 0], P[:, 1]
    return 0.5 * abs(np.sum(x * np.roll(y, -1) - y * np.roll(x, -1)))

A = np.array([[2., 1.], [1., 1.5]])
```

```
shapes = {
   "unit square": np.array([[0, 0], [1, 0], [1, 1], [0, 1.]]),
   "rectangle": np.array([[0, 0], [2, 0], [2, 0.5], [0, 0.5]]),
   "triangle": np.array([[0, 0], [1, 0], [0.5, 1.2]]),
}
print("A =")
print(A)
print("|det A| =", abs(np.linalg.det(A)))
for name, S in shapes.items():
   before = poly_area(S)
   after = poly_area(S @ A.T)
   print(f"{name:12s} area {before:.4f} -> {after:.4f} "
        f"ratio {after/before:.4f}")

B = np.random.randn(3, 3)
E = np.array([[1., 0.5, 0.], [0., 1., 0.5], [0.5, 0., 1.]])
v0 = abs(np.linalg.det(E))
v1 = abs(np.linalg.det(B @ E))
print("3D parallelepiped: |det E| =", v0,
     " |det(B E)| =", v1, " ratio =", v1 / v0,
     " |det B| =", abs(np.linalg.det(B)))
```

Listing 6.3: `code/ch06_volume_scaling.py -- areas of three different shapes before/after a` 2×2 `map, then a parallelepiped through a` 3×3.

```
A =
[[2. 1. ]
 [1. 1.5]]
|det A| = 2.0
unit square area 1.0000 -> 2.0000 ratio 2.0000
rectangle area 1.0000 -> 2.0000 ratio 2.0000
triangle area 0.6000 -> 1.2000 ratio 2.0000
3D parallelepiped: |det E| = 1.125 |det(B E)| = 1.3290513746968733
    ratio = 1.181378999730554 |det B| = 1.181378999730554
```

Listing 6.4: Output of the scaling experiment. Notice every shape scales by exactly $|\det A| = 2$ — the factor is a property of the map, not of the region. In 3D the ratio equals $|\det B|$ to the last digit shown, which is also the product rule at work.

Example 6.8 (A shear keeps area)**.** The shear $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ maps the unit square to a slanted parallelogram (figure 6.3). Its determinant — and the parallelogram's area — come out to exactly 1.

Strategy. Two readings of the same picture: the geometric one (base $\times$ height of the parallelogram) and the algebraic one (the 2×2 formula). They must agree — that is the determinant's whole claim.

Solution. The image of $\mathbf{e}_1$ is $(1, 0)$; of $\mathbf{e}_2$ is $(1, 1)$. The parallelogram has base 1 and height 1 (perpendicular height, not slant), so area $= 1$. Algebraically $\det A = 1 \cdot 1 - 1 \cdot 0 = 1$. ✓ Every shear preserves volume; they are the "free" transformations in any factorization story.

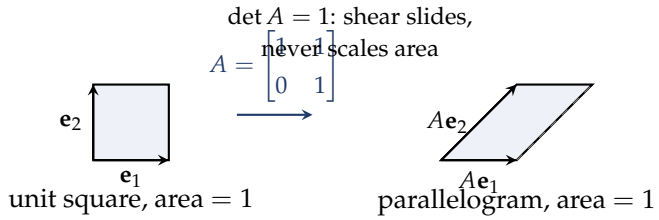


Figure 6.3: A shear sends the unit square to a same-area parallelogram. Notice the two ways to read the area — base $\times$ height directly, or the 2×2 formula — and that they agree by definition of determinant.

6.4 det $A = 0$ means collapse

If the parallelepiped collapses flat — the columns are dependent — the volume is zero. So det $A = 0$, dependent columns, non-invertible, singular: four ways to say the same thing (the equivalence built through chapters 1–5). Figure 6.4 is the picture.

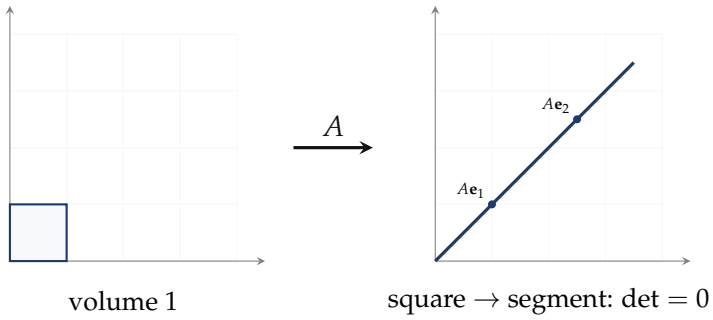


Figure 6.4: det $A = 0$ is geometric collapse: the square's image is a segment. Notice both column images land on one line — dependent columns are the *cause*, zero volume is the *symptom*. Regenerate: `figures/ch06_collapse.tex`.

The trap is the word *near*. A small determinant is not a small collapse — it is not a collapse at all:

```
import numpy as np
np.set_printoptions(precision=6, suppress=True)

A = np.array([[1., 0.], [0., 1.]]) # det = 1
B = np.array([[100., 0.], [0., 0.001]]) # det = 0.1 -- small!
C = np.array([[1., 1.], [1., 1.0001]]) # det = 1e-4 -- nearly singular
D = np.array([[1., 1.], [1., 1.]]) # det = 0 -- singular
for name, M in [("I", A), ("diag(100,0.001)", B),
                ("near-dep", C), ("dup-rows", D)]:
    d = np.linalg.det(M)
    c = np.linalg.cond(M)
    inv_ok = "yes"
    try:
        _ = np.linalg.inv(M)
    except np.linalg.LinAlgError:
        inv_ok = "NO"
```

```
    print(f"{name:16s} det={d:12.6f} cond={c:12.4f} invertible={inv_ok
        }")

B2 = B / np.abs(B).max()
print("rescaled diag(1, 1e-5): det =", np.linalg.det(B2),
      " cond =", np.linalg.cond(B2))
```

Listing 6.5: `code/ch06_near_singular.py -- four 2 × 2s from "det = 1" to "det = 0", with the condition number alongside. Watch which quantity actually tracks invertibility.`

```
I det= 1.000000 cond= 1.0000 invertible=yes
diag(100,0.001) det= 0.100000 cond= 100000.0000 invertible=yes
near-dep det= 0.000100 cond= 40002.0001 invertible=yes
dup-rows det= 0.000000 cond=139378947902558128.0000 invertible=NO
rescaled diag(1, 1e-5): det = 9.999999999999997e-06 cond =
    99999.99999999999
```

Listing 6.6: Output of the near-singular test. Notice "det = 0.1" does not mean safe — the matrix with det 0.1 has condition 10^5, and only the truly singular matrix fails inversion. Determinant answers *is* it invertible; condition number answers *how close*.

Exercise 6.2 (medium)**.** $\det A = 6$ and $\det B = -2$ for 3×3 matrices. Compute $\det(ABA^{-1})$, $\det(A^T B)$, and $\det(2A)$.

Solution. $\det(ABA^{-1}) = \det B = -2$ (similarity). $\det(A^T B) = \det A \det B = -12$. $\det(2A) = 2^3 \cdot 6 = 48$ — the exponent is n, not 1.

Exercise 6.3 (hard)**.** Prove the pivot-product formula for a 2×2: eliminating $R_2 \leftarrow R_2 - (c/a)R_1$ in $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ ($a \neq 0$) leaves determinant $a(d - bc/a) = ad - bc$; then show swapping the rows of $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ flips the sign.

Solution. After $R_2 \leftarrow R_2 - (c/a)R_1$ the matrix is $\begin{bmatrix} a & b \\ 0 & d - bc/a \end{bmatrix}$; row-add leaves det unchanged, triangular det is the diagonal product $a(d - bc/a) = ad - bc$. Swapped matrix $\begin{bmatrix} c & d \\ a & b \end{bmatrix}$ has det $cb - da = -(ad - bc)$.

Example 6.9 (Detecting collapse by determinant)**.** Decide whether $B = \begin{bmatrix} 1 & 2 & 5 \\ 0 & 1 & 3 \\ 2 & 5 & 13 \end{bmatrix}$ is invertible — without finishing an inverse computation.

Strategy. Column 3 looks suspicious: check whether it is a combination of columns 1 and 2 *before* computing anything. Dependent columns ⇒ zero volume ⇒ det = 0.

Solution. Column 3 = 3·col 1 +2·col 2: $3(1,0,2) + 2(2,1,5) = (3+4, 0+2, 6+10) = (7,2,16)$ — no. Try instead col 3 = col 1 +2 col 2: $(1+4, 0+2, 2+10) = (5,2,12)$ — no, but close.

Compute directly along row 1: $\det B = 1(13-15) - 2(0-6) + 5(0-2) = -2+12-10 = 0$. So B *does* collapse: the columns must be dependent. Find the combination: $-1(1,0,2) + 3(2,1,5) = (-1+6, 0+3, -2+15) = (5,3,13)$ — col 3 $=$ $-$col 1 $+3$ col 2 exactly. The determinant caught the dependence first; the combination confirmed it. ✓

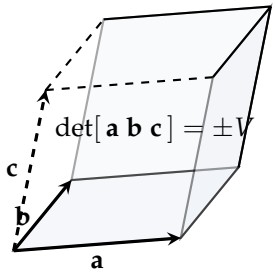


Figure 6.5: $\det[\,\mathbf{a}\ \mathbf{b}\ \mathbf{c}\,]$ is the signed volume of the column parallelepiped. When the three edges flatten into a plane the box has zero thickness — and the determinant is zero.

6.5 Cramer's rule, a curiosity

Theorem 6.10 (Cramer's rule)**.** *If A is invertible, the unique solution of $Ax = \mathbf{b}$ is*

$$x_j = \frac{\det A_j}{\det A}, \qquad A_j = A \text{ with column } j \text{ replaced by } \mathbf{b}.$$

Elegant, correct, and useless for computation: it computes $n+1$ determinants where elimination computes the answer directly. Its real life is theoretical — it proves the solution is a rational function of the entries, and it hides inside the formula $A^{-1} = \operatorname{adj}(A)/\det A$. Meet it so you can say you have met it; then go back to elimination.

sign of position (i,j): $(-1)^{i+j}$

+	−	+
−	+	−
+	−	+

Figure 6.6: The cofactor sign pattern $(-1)^{i+j}$ for 3×3. Notice it is a checkerboard — diagonally adjacent positions share a sign, which is why you can expand along any row. Regenerate: `figures/ch06_checkerboard.tex`.

6.6 Hand derivation

Derive the 2×2 formula from elimination, then watch a row swap flip the sign. For $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ with $a \neq 0$: $R_2 \leftarrow R_2 - (c/a)R_1$ gives $\begin{bmatrix} a & b \\ 0 & d - bc/a \end{bmatrix}$ — upper triangular, determinant the pivot product $a \cdot (d - bc/a) = ad - bc$. If $a = 0$ the first step swaps the rows; the swap contributes a minus sign and the formula still lands on $ad - bc$ (check the boundary case $a = 0$, $c \neq 0$ directly: $ad - bc = -bc$, and the swap-version gives $cb - da = -(ad - bc) = bc - ad$ — consistent once the sign travels with it). Complete the check every derivation owes: the formula is antisymmetric in the two rows ($cb - da = -(ad - bc)$) and vanishes on dependent rows ($a/c = b/d$ gives $ad - bc = 0$).

6.7 Common mistakes

ADDING DETERMINANTS: $\det(A + B) \neq \det A + \det B$

Determinants multiply; they do not add. Counterexample in one line: $\det I + \det I = 2$ but $\det(2I) = 2^n$. If you ever split a determinant over a sum, you have produced a wrong number.

SIGN BOOKKEEPING ON SWAPS

One row swap flips the sign; two swaps flip it back. Learners reliably lose this — when you count swaps, count them on paper, and write the $(-1)^s$ factor into the formula before the pivot product, not after "remembering at the end."

COMPUTING WITH CRAMER'S RULE

Cramer is $n + 1$ determinants per solve; elimination is one factorization. In code, Cramer's rule on a 10×10 is already thousands of times slower, and less stable. The formula is for theorems, not for numbers.

READING "SMALL DETERMINANT" AS "SINGULAR"

$\det A = 10^{-6}$ is small but nonzero: A is invertible, full stop. "Nearly singular" is a statement about *conditioning* (Section 6.4), not about the determinant crossing some threshold. Scale changes det by c^n; it changes condition not at all.

6.8 Summary

Object	What you need to know
Cofactor expansion	$\det A = \sum_j a_{1j}(-1)^{1+j} M_{1j}$; any row or column works; $\sim n!$ cost
Pivot product	$\det A = (-1)^s \prod u_{jj}$; the practical algorithm behind `np.linalg.det`
Signed volume	columns of A span a parallelepiped of volume $\lvert \det A \rvert$; sign = orientation
Product rule	$\det(AB) = \det A \det B$; $\det(A^{-1}) = 1/\det A$; $\det(cA) = c^n \det A$
Row operations	add: unchanged; swap: $\times(-1)$; scale by c: $\times c$
$\det A = 0$	collapse: dependent columns, singular, no inverse — all one fact
Small det	not singular: conditioning is the question, use `np.linalg.cond`
Cramer's rule	$x_j = \det A_j / \det A$: a theorem that produces a formula, not an algorithm

WATCH INSTEAD

1. *The determinant* — 3Blue1Brown, Essence of Linear Algebra ch. 6, 10:03, https://youtu.be/Ip3X9LOh2dk — the volume-scaling view of §6.3 (accessed 2026-10-11).
2. *Properties of Determinants* — MIT OpenCourseWare, 18.06 Lecture 18, 49:12, https://youtu.be/srxexLishgY — §6.2 from the source (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., ch. 5 (textbook level) — determinants by properties, formula, and applications.
- MIT OpenCourseWare, *18.06SC Linear Algebra* — recitations on determinants, at OCW (course "18.06SC", Unit II), at https://ocw.mit.edu. Verified 2026-10-11.
- Margalit and Rabinoff, *Interactive Linear Algebra*, "Determinants" — cofactor expansions and volumes, hyperlinked. https://textbooks.math.gatech.edu/ila/ Verified 2026-10-11.
- J. Hefferon, *Linear Algebra*, ch. Four (free textbook) — the rigorous route: det as a multilinear form. https://hefferon.net/linearalgebra/index.html Verified 2026-10-11.
- S. Boyd and L. Vandenberghe, *VMLS*, §7.4 (free PDF) — determinant in the data register (Gram determinants, volumes). https://web.stanford.edu/~boyd/vmls/vmls.pdf Verified 2026-10-11.

Additional exercises

Exercise 6.4 (easy). Compute $\det \begin{bmatrix} 1 & 4 & 2 \\ 0 & 3 & 1 \\ 0 & 0 & 5 \end{bmatrix}$ — and state in one word why it is fast.

Exercise 6.5 (easy). $\det A = 3$. What are $\det(2A)$, $\det(A^T)$, $\det(A^{-1})$ for $A \in \mathbb{R}^{3\times 3}$?

Exercise 6.6 (easy). A 2×2 map sends the unit square to a parallelogram of area 7. Give two matrices with $|\det| = 7$, one preserving and one flipping orientation.

Exercise 6.7 (easy). True or false, justify in one line: "$\det A > 1$ means A expands every vector."

Exercise 6.8 (medium). Prove $\det(A^T) = \det A$ for 2×2 and 3×3 by direct expansion, and explain in one sentence why it holds in general (expansion along a column mirrors expansion along a row).

Exercise 6.9 (medium). Without computing, order these determinants of 3×3 matrices: A; $B = A$ with rows 1 and 3 swapped; $C = A$ with row 2 doubled; $D = A$ with row 3 replaced by row $3 + 5\times$ row 1. Write each as a multiple of $\det A$.

Exercise 6.10 (medium). In code: take the 4×4 Hilbert matrix `scipy.linalg.hilbert(4)` (or build $H_{ij} = 1/(i + j - 1)$). Compute `np.linalg.det`, `np.linalg.cond`, and our `pivot_det` (`code/ch06_pivot_det.py`). Is H singular? Would you trust det to answer that?

Exercise 6.11 (medium). Use Cramer's rule to solve $\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$, then solve the same system by elimination and compare the work.

Exercise 6.12 (hard). Prove $\det(AB) = \det A \det B$ for invertible A, general B, by elimination: write $A = LU$ (ch. 2), show $\det(LUB)$ splits into $\det L \det U \det B$ factors, and handle $\det B = 0$ as a limiting case.

Exercise 6.13 (hard). A "det" candidate $f : \mathbb{R}^{n\times n} \to \mathbb{R}$ is required to be multilinear in the columns, alternating (zero on equal columns), and normalized $f(I) = 1$. For 2×2 matrices only, prove $f(A) = ad - bc$: expand each column as $a_{1j}\mathbf{e}_1 + a_{2j}\mathbf{e}_2$, distribute, and show every term except the two standard permutations dies. (This is Axler's definition-first route, in miniature.)

Exercise 6.14 (easy). Compute $\det \begin{bmatrix} 3 & 1 \\ 2 & 4 \end{bmatrix}$ and $\det \begin{bmatrix} 3 & 1 \\ 6 & 2 \end{bmatrix}$. Geometrically, what changed between the two?

Exercise 6.15 (easy). For the shear of example 6.8, compute $\det(A^2)$ and $\det(A^{-1})$ without expanding — which property does each use?

Exercise 6.16 (medium). Compute det of $\begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 3 & 0 & 0 \\ 4 & 0 & 5 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix}$ by expanding along a row or column of your choice — pick the one that kills the most terms.

Exercise 6.17 (medium)**.** Show $\det(AB) = \det A \det B$ fails to have a sum analogue: find 2×2 matrices with $\det(A + B) \neq \det A + \det B$. What extra structure would rescue an additive rule?

Exercise 6.18 (hard)**.** Prove $\det(A^{-1}) = 1/\det A$ two ways: from the product rule, and from the volume interpretation (what does the inverse box do to volume?). Which proof generalizes to $\det(A^k)$?

Notes

The cofactors/pivots/volume trio follows Strang's treatment in [6] ch. 7 and the 18.06SC Unit II lectures [2]; the signed-volume pedagogy is the *Essence of Linear Algebra* episode on determinants [15]. Axler [7] ch. 10 builds the determinant definition-first via multilinear forms — the opposite order from ours, worth reading once the three definitions here feel solid (the last [hard] exercise rehearses his approach in 2×2). MML ch. 4 [5] frames the same properties for ML use.

CHAPTER 7

Eigenvalues and diagonalisation

Most vectors a linear map touches come out rotated and stretched — but a few special directions come out merely *scaled*. Those directions are the map's eigenvectors, and they are the coordinates the map itself would choose if it could: in the eigenbasis, a matrix is just a diagonal list of stretch factors. That one observation unlocks cheap powers of A, closed forms for recurrences, steady states of Markov chains, and — two chapters from now — the whole theory of the SVD. After this chapter you can find eigenvalues and eigenvectors by hand for small matrices, diagonalise when diagonalisation exists, say precisely when it fails, and use all of it on a Fibonacci-style recurrence and a Markov chain.

BEFORE YOU START

Chapters 1–4 and 6. *Recap:* usually a matrix rotates and stretches vectors at once. An *eigenvector* is a direction the matrix only stretches, and the stretch factor is its *eigenvalue*.

AFTER THIS CHAPTER YOU CAN

- find eigenvalues and eigenvectors of a small matrix by hand;
- diagonalise a matrix and use it to compute A^k for large k;
- recognise a matrix that cannot be diagonalised;
- solve a linear recurrence or a Markov-chain steady state with eigenvalues.

HOW TO THINK ABOUT THIS

Most vectors get turned and stretched by a matrix, but a few special directions are only stretched. Those are the eigenvectors, and in their directions the machine becomes simple multiplication. If you can break a vector into eigenvector pieces, repeating the machine a hundred times is easy: each piece is just scaled a hundred times.

7.1 The eigen equation

Definition 7.1 (Eigenvector, eigenvalue)**.** A nonzero vector $\mathbf{v}$ is an *eigenvector* of A with *eigenvalue* λ when

$$A\mathbf{v} = \lambda\mathbf{v}.$$

The direction is preserved; only the length (and possibly the sign) changes. $\mathbf{v} = \mathbf{0}$ is excluded on purpose — it would trivialize the equation for every λ.

Figure 7.1 makes the idea visual: most arrows change direction under A; the eigenvectors are the ones that stay on their own line.

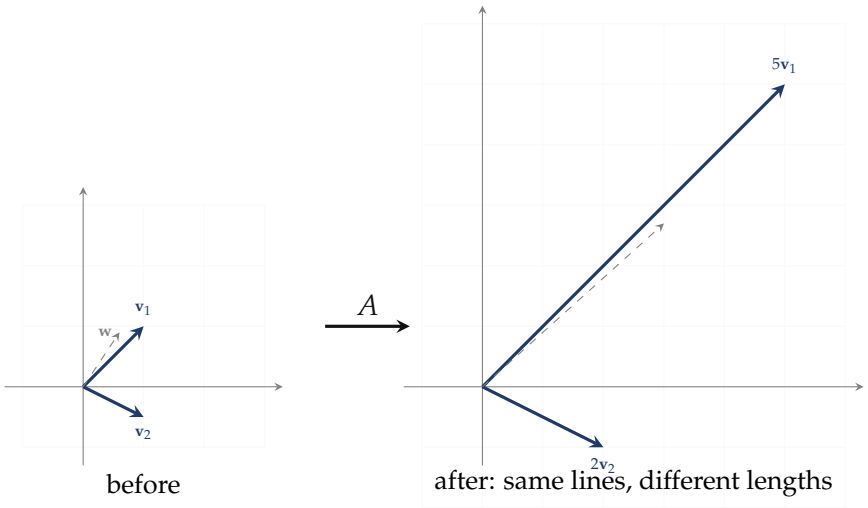


Figure 7.1: Eigenvectors are the map's preserved directions. Notice the two accent vectors stay on their lines — merely stretched by $\lambda_1 = 5$ and $\lambda_2 = 2$ — while a generic vector changes direction. Regenerate: `figures/ch07_eigendirections.tex`.

Two immediate facts worth internalizing:

- Eigenvectors are determined only *up to nonzero scale* — if $\mathbf{v}$ works so does $7\mathbf{v}$, and $-\mathbf{v}$. The eigenLINE is the object; we pick a unit representative when convenient.
- The pair $(\lambda, \mathbf{v})$ still satisfies $A^2\mathbf{v} = \lambda^2\mathbf{v}$, $A^k\mathbf{v} = \lambda^k\mathbf{v}$: powers of A act trivially on eigendirections. This single fact powers the whole chapter.

7.2 Characteristic polynomial and finding them by hand

Rewrite $A\mathbf{v} = \lambda\mathbf{v}$ as $(A - \lambda I)\mathbf{v} = \mathbf{0}$. A nonzero $\mathbf{v}$ exists exactly when $A - \lambda I$ is singular — chapter 6 says that means $\det(A - \lambda I) = 0$.

Definition 7.2 (Characteristic polynomial)**.** $\chi_A(\lambda) = \det(A - \lambda I)$ is a degree-n polynomial in λ; its roots are the eigenvalues of A.

Example 7.3 (A complete 2×2 by hand)**. Strategy.** Compute $\chi_A(\lambda)$, factor it (the trace and determinant give a free checksum: roots sum to $\operatorname{tr} A$ and multiply to $\det A$), then solve $(A - \lambda I)\mathbf{v} = \mathbf{0}$ per root.

Solution. $A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$:

$$\det \begin{bmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{bmatrix} = (4-\lambda)(3-\lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2).$$

Checksum: $\operatorname{tr} A = 7 = 5 + 2$, $\det A = 10 = 5 \cdot 2$. ✓For $\lambda = 5$: $A - 5I = \begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix}$, null direction $(1, 1)$: indeed $A(1,1)^T = (5,5)^T$. For $\lambda = 2$: $A - 2I = \begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix}$, direction $(1, -2)$: $A(1,-2)^T = (2,-4)^T$. Both verified by direct multiplication.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

A = np.array([[4., 1.], [2., 3.]])
w_hand, V_hand = 5.0, np.array([1., 1.])
w2_hand, V2_hand = 2.0, np.array([1., -2.])
print("A =")
print(A)
print("hand: lambda=5, v=(1,1):", np.allclose(A @ V_hand, 5 * V_hand))
print("hand: lambda=2, v=(1,-2):", np.allclose(A @ V2_hand, 2 *
    V2_hand))

w, V = np.linalg.eig(A)
print("np.linalg.eig eigenvalues:", np.sort(w))
print("eig vectors (cols):")
print(V)
print("trace == sum lambda:", np.isclose(np.trace(A), w.sum()))
print("det == prod lambda:", np.isclose(np.linalg.det(A), w.prod()))
```

Listing 7.1: `code/ch07_eig_by_hand.py -- the hand answers checked by residual, plus np.linalg.eig and the two checksums.`

```
A =
[[4. 1.]
 [2. 3.]]
hand: lambda=5, v=(1,1): True
hand: lambda=2, v=(1,-2): True
np.linalg.eig eigenvalues: [2. 5.]
eig vectors (cols):
[[ 0.7071 -0.4472]
 [ 0.7071  0.8944]]
trace == sum lambda: True
det == prod lambda: True
```

Listing 7.2: Output of the eigen check. Notice the library's vectors are the same *lines* normalized — $(1,1)/\sqrt{2}$ and $(1,-2)/\sqrt{5}$ up to sign — and both checksums pass.

Exercise 7.1 (easy). Find the eigenvalues of $\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$ and one eigenvector for each.

Solution. $\lambda^2 - 6\lambda + 5 = 0$: $\lambda = 5$ with $\mathbf{v} = (1, 1)$, $\lambda = 1$ with $\mathbf{v} = (1, -1)$.

Exercise 7.2 (medium). Give a 2×2 real matrix with *no* real eigenvalues, and compute its complex pair.

Solution. Any rotation by $\theta \neq 0, \pi$, e.g. $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$: $\chi = \lambda^2 + 1$, roots $\pm i$. A real matrix *can* have complex eigenvalues — they arrive in conjugate pairs (Figure 7.2).

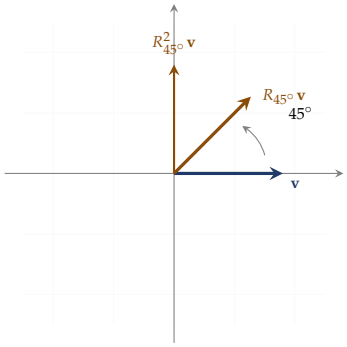


Figure 7.2: Complex eigenvalues mean rotation. Notice a 45° rotation has no real preserved line at all — its eigenvalues are $e^{\pm i\pi/4}$. Regenerate: `figures/ch07_complex_rotation.tex`.

7.3 Eigenspaces and multiplicities

Definition 7.4 (Eigenspace). $E_\lambda = \ker(A - \lambda I)$ is the *eigenspace* of λ — all eigenvectors for that λ plus $\mathbf{0}$. Its dimension is the *geometric multiplicity*; the number of times λ repeats as a root of χ_A is the *algebraic multiplicity*. Always $\dim E_\lambda \leq$ (algebraic multiplicity).

Example: $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ has $\lambda = 1$ with algebraic multiplicity 2, but $E_1 = \operatorname{span}\{(1, 0)\}$ is only 1-dimensional — a defective matrix (Section 7.5).

Theorem 7.5. *Eigenvectors belonging to* distinct *eigenvalues are linearly independent.*

Proof. Suppose $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are eigenvectors for distinct $\lambda_1, \ldots, \lambda_k$ and some combination vanishes. Apply A repeatedly: $c_1\lambda_1^j\mathbf{v}_1 + \cdots + c_k\lambda_k^j\mathbf{v}_k = 0$ for all j. Pick two consecutive j and eliminate $\mathbf{v}_k$: the remaining combination has coefficients $c_i(\lambda_i - \lambda_k)$, nonzero only where $\lambda_i \neq \lambda_k$ — induction kills them one at a time. ∎

Consequence: n distinct eigenvalues ⇒ a full eigenbasis ⇒ diagonalisable. The converse is false in both directions (repeated eigenvalues can still diagonalise; defective ones cannot).

7.4 Diagonalisation and powers of A

Stack the eigenvectors as columns of V and the eigenvalues into $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$: $AV = V\Lambda$ column-by-column. When V is invertible — exactly when A has n independent eigenvectors —

Theorem 7.6 (Diagonalisation). *$A = V\Lambda V^{-1}$ iff A has a basis of eigenvectors, and then*

$$A^k = V\Lambda^k V^{-1}, \qquad \Lambda^k = \operatorname{diag}(\lambda_1^k, \dots, \lambda_n^k).$$

This is the change of basis of chapter 4 paying its second dividend: the same map, in its own basis, is a list of independent scalings.

Example 7.7 (A^{10} the cheap way)**. Strategy.** Don't multiply ten matrices — decompose the input in the eigenbasis, scale each piece by λ^{10}, reassemble.

Solution. From Example 7.3, $A = V\Lambda V^{-1}$ with $V = \begin{bmatrix} 1 & 1 \\ 1 & -2 \end{bmatrix}$, $\Lambda = \operatorname{diag}(5, 2)$. Take $\mathbf{x} = (1, 0)$: solving $V\mathbf{c} = \mathbf{x}$ gives $\mathbf{c} = (2/3,\ 1/3)$, i.e. $\mathbf{x} = \frac{2}{3}\mathbf{v}_1 + \frac{1}{3}\mathbf{v}_2$. Then

$$A^{10}\mathbf{x} = \tfrac{2}{3}\,5^{10}\mathbf{v}_1 + \tfrac{1}{3}\,2^{10}\mathbf{v}_2 = \begin{bmatrix} (2\cdot 5^{10} + 2^{10})/3 \\ (2\cdot 5^{10} - 2\cdot 2^{10})/3 \end{bmatrix} = \begin{bmatrix} 6510758 \\ 6509734 \end{bmatrix}.$$

Two exponentiations instead of nine matrix multiplies — and the dominant term 5^{10} already explains the answer's scale.

```
import numpy as np
np.set_printoptions(precision=6, suppress=True)

A = np.array([[2., 1., 0.], [1., 2., 1.], [0., 1., 2.]])
x = np.array([1., 0.5, -0.25])
print("A =")
print(A)
for k in range(8):
    x = A @ x
    x = x / np.linalg.norm(x)
    rq = x @ A @ x
    print(f"k={k+1}: lambda~{rq:.6f} x={x}")
w, V = np.linalg.eig(A)
print("true dominant:", np.abs(w).max(), " eigenvector:", V[:, np.
    argmax(np.abs(w))])
```

Listing 7.3: `code/ch07_power_iteration.py -- eight steps of power iteration: multiply, normalize, read the Rayleigh quotient.`

```
A =
[[2. 1. 0.]
 [1. 2. 1.]
 [0. 1. 2.]]
k=1: lambda~2.939597 x=[0.819232 0.573462 0. ]
k=2: lambda~3.205318 x=[0.733761 0.652232 0.190234]
```

```
k=3: lambda~3.334830 x=[0.653366 0.686872 0.318307]
k=4: lambda~3.385931 x=[0.594985 0.699983 0.39499 ]
k=5: lambda~3.404379 x=[0.557226 0.704641 0.439295]
k=6: lambda~3.410823 x=[0.534021 0.706258 0.46478 ]
k=7: lambda~3.413048 x=[0.52009 0.706815 0.479498]
k=8: lambda~3.413814 x=[0.511822 0.707007 0.488037]
true dominant: 3.4142135623730914 eigenvector: [-0.5 -0.707107 -0.5 ]
```

Listing 7.4: Output of power iteration. Notice the iterate rotates onto the dominant eigendirection and the Rayleigh quotient locks onto $\lambda_1 = 2 + \sqrt{2} \approx 3.4142$ — the sign flip versus `np.linalg.eig` is the up-to-scale freedom, not an error.

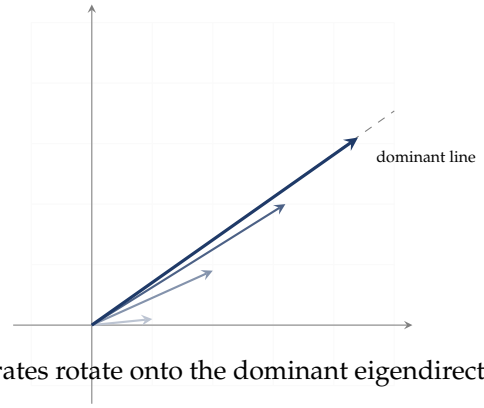


Figure 7.3: Power iteration geometrically: each application leans harder on the largest eigenvalue, so iterates rotate onto the dominant eigendirection. This is the algorithm inside PageRank and every large-scale eigensolver. Regenerate: `figures/ch07_power_convergence.tex`.

7.5 Defective matrices

When algebraic multiplicity outgrows geometric multiplicity, the eigenbasis runs out of members and V cannot be inverted.

Example 7.8 (A shear that cannot diagonalise)**. Strategy.** Compute both multiplicities for the shear $S = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ and look at the geometry directly.

Solution. $\chi_S = (1-\lambda)^2$: $\lambda = 1$ with algebraic multiplicity 2. But $S - I = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ has null space $\operatorname{span}\{(1,0)\}$ — geometric multiplicity 1. Only the x-axis survives as a direction (Figure 7.4); the second slot in the would-be eigenbasis is missing, so $S \neq V\Lambda V^{-1}$ for any V. The correct replacement (Jordan form) adds a *generalized* eigenvector: S maps $\mathbf{e}_2$ to $\mathbf{e}_2 + \mathbf{e}_1$, and powers show it: $S^k = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$ grows *linearly*, not exponentially — the signature of a defect.

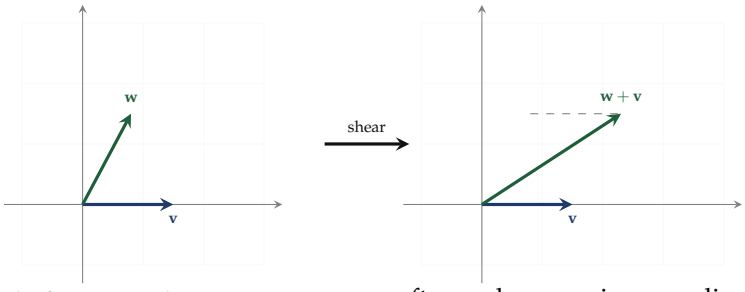


Figure 7.4: The shear's eigendirections collapse. Notice only the x-direction is preserved; every other direction slides along it and the second eigenvector simply does not exist. Regenerate: `figures/ch07_defective.tex`.

Exercise 7.3 (medium)**.** Decide whether $A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$ and $B = \begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$ diagonalise, though both have $\chi = (2-\lambda)^2$.

Solution. $A = 2I$: every vector is an eigenvector, basis exists, already diagonal. B: $\ker(B - 2I) = \operatorname{span}\{(1,0)\}$, dimension $1 < 2$ — defective, does not diagonalise.

Exercise 7.4 (hard)**.** Show that if λ is an eigenvalue of A with eigenvector $\mathbf{v}$, then for every polynomial p, $p(\lambda)$ is an eigenvalue of $p(A)$ with the same eigenvector. Conclude the eigenvalues of A^2 are λ_i^2 — and explain why A^2 can have *fewer* distinct eigenvalues than A.

Solution. $p(A)\mathbf{v} = \sum_j c_j A^j \mathbf{v} = \sum_j c_j \lambda^j \mathbf{v} = p(\lambda)\mathbf{v}$. For $p(x) = x^2$: eigenvalues λ_i^2; if $\pm\lambda$ both occur they merge into one λ^2 (e.g. eigenvalues 2 and -2 of A both give 4 in A^2).

7.6 Applications: recurrences and Markov chains

Fibonacci, closed form

The recurrence $F_{k+1} = F_k + F_{k-1}$ becomes a matrix iteration:

$$\begin{bmatrix} F_{k+1} \\ F_k \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_k \\ F_{k-1} \end{bmatrix}, \qquad \text{i.e.} \quad \mathbf{x}_k = A\mathbf{x}_{k-1} = A^k \mathbf{x}_0.$$

Example 7.9 (Fibonacci via diagonalisation)**. Strategy.** Eigenvalues of A: $\chi_A = \lambda^2 - \lambda - 1$, roots $\varphi = (1+\sqrt{5})/2$ and $\psi = (1-\sqrt{5})/2$. Distinct $\Rightarrow$ diagonalisable; the full closed form is derived in Section 7.7.

Solution. With $F_0 = 0, F_1 = 1$ the eigencoordinate computation yields Binet's formula,

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}},$$

and since $|\psi| < 1$ its term dies exponentially: F_n is the nearest integer to $\varphi^n/\sqrt{5}$.

```
import numpy as np
import time
np.set_printoptions(precision=4, suppress=True)

A = np.array([[1., 1.], [1., 0.]])
w, V = np.linalg.eig(A)
print("eigenvalues of [[1,1],[1,0]]:", np.sort(w))
phi, psi = (1 + np.sqrt(5)) / 2, (1 - np.sqrt(5)) / 2

def fib_closed(n):
    return int(round((phi**n - psi**n) / np.sqrt(5)))

def fib_rec(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

GOOD = [n for n in range(1, 100) if fib_closed(n) == fib_rec(n)]
print("closed form exact through n =", GOOD[-1],
      "; float64 Binet breaks at n =", GOOD[-1] + 1)
for n in [10, 20, 30, 60, 70]:
    assert fib_closed(n) == fib_rec(n), n
print("closed form == recursion for n in {10,20,30,60,70}: OK")

t0 = time.perf_counter()
for n in range(1, 70):
    fib_closed(n)
t1 = time.perf_counter()
for n in range(1, 70):
    fib_rec(n)
t2 = time.perf_counter()
print(f"closed-form all n<70: {1e3*(t1-t0):.3f} ms")
print(f"recursion all n<70: {1e3*(t2-t1):.3f} ms")
Aint = np.array([[1, 1], [1, 0]], dtype=object)
Ai = np.linalg.matrix_power(Aint, 100)
print("F_101 via integer matrix power:", Ai[0, 0])
n = 10
An = np.linalg.matrix_power(A, n)
print(f"A^{n} =")
print(An.astype(int), f"(entries F11={int(An[0,0])}, F10={int(An[1,0])
    })")
```

Listing 7.5: code/ch07_fibonacci.py -- Binet's formula checked against the recursion, timed, and pushed past its float limit by an integer matrix power.

```
eigenvalues of [[1,1],[1,0]]: [-0.618 1.618]
closed form exact through n = 70 ; float64 Binet breaks at n = 71
closed form == recursion for n in {10,20,30,60,70}: OK
closed-form all n<70: 0.076 ms
recursion all n<70: 0.047 ms
```

```
F_101 via integer matrix power: 573147844013817084101
A^10 =
[[89 55]
 [55 34]] (entries F11=89, F10=55)
```

Listing 7.6: Output of the Fibonacci check. Notice float64 Binet holds only to $n = 70$ — past that the φ^n term outgrows precision; integer matrix powers stay exact forever. (Timing is a wash at this n; both are cheap — the point is the *form* exists at all.)

Markov chains and the eigenvalue 1

A Markov chain stores a probability distribution $\mathbf{p}$ and updates $\mathbf{p} \leftarrow P\mathbf{p}$ where each column of P sums to 1. Columns summing to 1 makes $\lambda = 1$ an eigenvalue, and every steady state is its eigenvector:

Example 7.10 (Weather chain steady state)**. Strategy.** For the two-state chain $P = \begin{bmatrix} 0.9 & 0.3 \\ 0.1 & 0.7 \end{bmatrix}$ (columns: from-state), find the $\lambda = 1$ eigenvector and check that powers converge to it.

Solution. $(P - I)\boldsymbol{\pi} = \mathbf{0}$: $\begin{bmatrix} -0.1 & 0.3 \\ 0.1 & -0.3 \end{bmatrix} \boldsymbol{\pi} = \mathbf{0}$ gives $\pi_1 = 3\pi_2$; normalizing $\pi_1 + \pi_2 = 1$ yields $\boldsymbol{\pi} = (0.75, 0.25)$. The second eigenvalue is $0.6 < 1$, so every other eigendirection decays: $P^k\mathbf{x} \to \boldsymbol{\pi}$ regardless of start — visible in the listing (Figure 7.5 shows the chain).

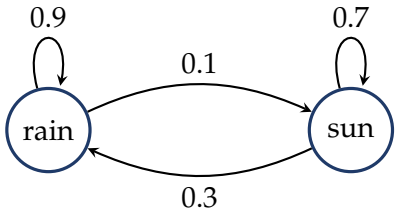


Figure 7.5: The weather chain's transition graph. Notice the steady state is an *eigenvector*: it is what the dynamics settle onto, not a separate concept. Regenerate: `figures/ch07_markov_graph.tex`.

```
import numpy as np
np.set_printoptions(precision=6, suppress=True)
np.random.seed(1)

P = np.array([[0.9, 0.3],
              [0.1, 0.7]])
print("P =")
print(P)
print("columns sum to 1:", np.allclose(P.sum(axis=0), 1))

w, V = np.linalg.eig(P)
print("eigenvalues:", w)
pi = V[:, np.argmax(w)].real
pi = pi / pi.sum()
print("steady state (eigenvector of lambda=1):", pi)
```

```
16
17 x = np.array([1., 0.])
18 for k in [10, 20, 50, 200]:
19     xk = np.linalg.matrix_power(P, k) @ x
20     print(f"P^{k} x0 = {xk}")
21 rng = np.random.default_rng(0)
22 state, counts = 0, np.zeros(2)
23 for _ in range(200000):
24     state = rng.choice(2, p=P[:, state])
25     counts[state] += 1
26 print("Monte Carlo fractions:", counts / counts.sum())
```

Listing 7.7: `code/ch07_markov_steady.py -- steady state from np.linalg.eig, from matrix powers, and from a Monte-Carlo run of 200,000 steps.`

```
1  P =
2  [[0.9 0.3]
3   [0.1 0.7]]
4  columns sum to 1: True
5  eigenvalues: [1.  0.6]
6  steady state (eigenvector of lambda=1): [0.75 0.25]
7  P^10 x0 = [0.751512 0.248488]
8  P^20 x0 = [0.750009 0.249991]
9  P^50 x0 = [0.75 0.25]
10 P^200 x0 = [0.75 0.25]
11 Monte Carlo fractions: [0.753955 0.246045]
```

Listing 7.8: Output of the Markov check. Notice three routes to the same answer — eigenvector $(0.75, 0.25)$, powers converging by $k = 20$, Monte-Carlo fractions within sampling error. The subdominant eigenvalue 0.6 sets the convergence speed.

The same $\lambda = 1$-eigenvector machinery is PageRank in disguise: the "random surfer" matrix is column-stochastic (with a teleport term to kill dangling nodes), and a page's rank is its steady-state probability — computed by power iteration, exactly the loop above, at web scale (Figure 7.6 is the picture).

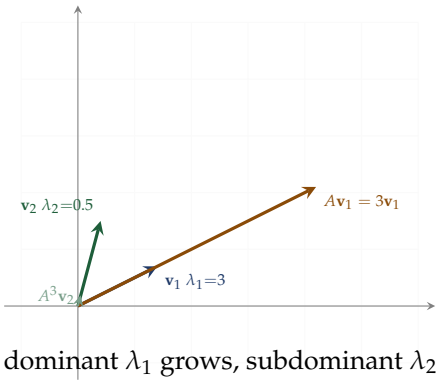


Figure 7.6: Why the dominant eigenvalue rules: the λ_1 component is tripled per application while the λ_2 component shrinks by half. After a few steps only the dominant direction is visible. Regenerate: `figures/ch07_dominant_geometry.tex`.

7.7 Hand derivation

Derive Binet's formula from the eigenvalues. With $A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$, $\chi_A = \lambda^2 - \lambda - 1$, roots $\varphi, \psi = (1 \pm \sqrt{5})/2$. Eigenvectors: $(\lambda, 1)^T$ works since row 2 of $(A - \lambda I)$ reads $1 - \lambda \cdot 0$... directly: $A(\lambda, 1)^T = (\lambda + 1, \lambda)^T = (\lambda^2, \lambda)^T = \lambda(\lambda, 1)^T$ using $\lambda^2 = \lambda + 1$. So $V = \begin{bmatrix} \varphi & \psi \\ 1 & 1 \end{bmatrix}$. Write the start: $(1, 0)^T = c_1 \mathbf{v}_\varphi + c_2 \mathbf{v}_\psi$ — solving $c_1\varphi + c_2\psi = 1$, $c_1 + c_2 = 0$ gives $c_1 = -c_2 = 1/(\varphi - \psi) = 1/\sqrt{5}$. Then

$$\mathbf{x}_n = A^n \mathbf{x}_0 = \frac{\varphi^n}{\sqrt{5}} \mathbf{v}_\varphi - \frac{\psi^n}{\sqrt{5}} \mathbf{v}_\psi, \qquad F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}.$$

Check at $n = 1$: $(\varphi - \psi)/\sqrt{5} = 1$ ✓; at $n = 2$: $(\varphi^2 - \psi^2)/\sqrt{5} = (\varphi + \psi)(\varphi - \psi)/\sqrt{5} = 1 \cdot 1 = 1$ ✓ (using $\varphi + \psi = 1$, $\varphi\psi = -1$ from $\lambda^2 = \lambda + 1$).

7.8 Common mistakes

EXPECTING REAL MATRICES TO HAVE REAL EIGENVALUES

A rotation has none. Real matrices get complex eigenvalues in conjugate pairs — and they are not a pathology, they *are* the rotation content of the map (Figure 7.2).

SIGN ERRORS IN $\det(A - \lambda I)$

For 2×2, $\det(A - \lambda I) = (a - \lambda)(d - \lambda) - bc$; the cross-term is a *minus* bc and the λ sits on the diagonal, not the off-diagonal. Guard yourself with the free checksum $\sum_{i=1}^n \lambda_i = \operatorname{tr} A$, $\prod_{i=1}^n \lambda_i = \det A$.

ASSUMING EVERY MATRIX DIAGONALISES

Distinct eigenvalues guarantee it; repeated eigenvalues are ambiguous. $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ is the canonical counterexample — check $\dim E_\lambda$, not just the algebraic count.

TREATING EIGENVECTORS AS UNIQUE

They are directions, defined only up to nonzero scale and sign. `np.linalg.eig` returns unit vectors whose signs may disagree with your hand work — compare directions ($|\cos\theta| = 1$ or `allclose` up to sign), never entries literally.

CONFUSING EIGENVALUES OF A AND A^2

A^2 has eigenvalues λ_i^2 with the *same* eigenvectors — not λ_i again, and not unrelated values. Opposite eigenvalues ($\pm\lambda$) collide in A^2, so A^2 can have fewer distinct values.

7.9 Summary

Object	What you need to know
Eigen equation	$A\mathbf{v} = \lambda\mathbf{v}$, $\mathbf{v} \neq \mathbf{0}$; direction preserved
Characteristic poly	$\chi_A(\lambda) = \det(A - \lambda I)$; roots = eigenvalues; $\mathrm{tr} = \sum_{i=1}^{n} \lambda_i$, $\det = \prod_{i=1}^{n} \lambda_i$
Eigenspace	$E_\lambda = \ker(A - \lambda I)$; geometric $\leq$ algebraic multiplicity
Independence	eigenvectors of distinct λ_i are linearly independent
Diagonalisation	$A = V\Lambda V^{-1}$ iff n independent eigenvectors; $A^k = V\Lambda^k V^{-1}$
Defective	geometric $<$ algebraic $\Rightarrow$ no eigenbasis (shear); powers grow polynomially
Power iteration	multiply + normalize $\to$ dominant eigenpair; rate set by $\lvert\lambda_2/\lambda_1\rvert$
Markov steady state	column-stochastic P has $\lambda_1 = 1$; steady state is its eigenvector

WATCH INSTEAD

1. *Eigenvectors and eigenvalues* — 3Blue1Brown, Essence of Linear Algebra ch. 14, 17:15, https://youtu.be/PFDu9oVAE-g — the eigen-direction picture behind §7.1–§7.4 (accessed 2026-10-11).
2. *Diagonalization and Powers of A* — MIT OpenCourseWare, 18.06 Lecture 22, 51:50, https://youtu.be/13r9QY6cmjc — §7.4–§7.6 at full length (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., ch. 6 (textbook level) — eigenvalues, diagonalisation, and applications.
- MIT OpenCourseWare, *18.06SC Linear Algebra* — recitations on eigenproblems, at OCW (course "18.06SC", Unit II), at https://ocw.mit.edu. Verified 2026-10-11.
- Margalit and Rabinoff, *Interactive Linear Algebra*, "Eigenvalues" — characteristic polynomials to diagonalisation, hyperlinked. https://textbooks.math.gatech.edu/ila/ Verified 2026-10-11.
- M. Deisenroth et al., *Mathematics for Machine Learning*, §4.2 (free PDF) — eigensolvers in the ML register. https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.
- J. Hefferon, *Linear Algebra*, ch. Five (free textbook) — similarity and eigenvalues, rigorously. https://hefferon.net/linearalgebra/index.html Verified 2026-10-11.
- 3Blue1Brown, *Essence of Linear Algebra* ch. 16 "Abstract vector spaces", 16:46, https://youtu.be/TgKwz5Ikpc8 — the eigen idea beyond $\mathbb{R}^n$ (accessed 2026-10-11).

Additional exercises

Exercise 7.5 (easy)**.** Find eigenvalues and eigenvectors of $\begin{bmatrix} 2 & 0 \\ 0 & -3 \end{bmatrix}$ and $\begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}$. What does a zero row on the diagonal buy you?

Exercise 7.6 (easy)**.** $\mathbf{v}$ is an eigenvector of A with $\lambda = 4$. What are the eigenvalues of A^3, A^{-1}, $A + 2I$ on the same vector?

Exercise 7.7 (easy)**.** Compute χ_A for $A = \begin{bmatrix} 1 & 3 \\ 0 & 5 \end{bmatrix}$ and confirm $\operatorname{tr} = \sum_{i=1}^{n} \lambda_i$, $\det = \prod_{i=1}^{n} \lambda_i$.

Exercise 7.8 (easy)**.** Give the algebraic and geometric multiplicities of every eigenvalue of $\operatorname{diag}(2, 2, 3)$.

Exercise 7.9 (medium)**.** Diagonalise $A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ and compute $A^8(1, 0)^T$ exactly via the eigen decomposition.

Exercise 7.10 (medium)**.** For the Markov matrix $P = \begin{bmatrix} 0.8 & 0.4 \\ 0.2 & 0.6 \end{bmatrix}$, find the steady state and the subdominant eigenvalue; estimate how many steps until a start at $(1, 0)$ is within 0.01 of steady state.

Exercise 7.11 (medium)**.** Show that similar matrices have the same eigenvalues: if $B = P^{-1}AP$ and $A\mathbf{v} = \lambda\mathbf{v}$, then $B(P^{-1}\mathbf{v}) = \lambda(P^{-1}\mathbf{v})$. Does the converse — same eigenvalues means similar — hold?

Exercise 7.12 (medium)**.** In code: run power iteration on a symmetric 4×4 of your choice, then on one whose two largest eigenvalues are close ($|\lambda_1/\lambda_2| \approx 1.05$). Count iterations to 10^{-6} Rayleigh-quotient error in each case and relate to $|\lambda_2/\lambda_1|^k$.

Exercise 7.13 (medium)**.** The matrix $J = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$. Compute J^4 exactly — the polynomial growth of a defective matrix shows up as binomial coefficients.

Exercise 7.14 (hard)**.** Prove: A diagonalises iff the sum of geometric multiplicities equals n iff the minimal polynomial has no repeated roots. (State the minimal polynomial; you may use χ_A's factorization and the eigenspace decomposition.)

Exercise 7.15 (hard)**.** Analyse $R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$: find its (complex) eigenvalues and eigenvectors, show $R_\theta = VDV^{-1}$ over $\mathbb{C}$, and explain why diagonalisation over $\mathbb{R}$ fails for $\theta \neq 0, \pi$.

Exercise 7.16 (hard)**.** Implement shifted power iteration: to find the eigenvalue *nearest* μ, run power iteration on $(A - \mu I)^{-1}$. Test on a 3×3 of your choice against `np.linalg.eig`, and explain why the dominant eigenvalue of $(A - \mu I)^{-1}$ is $1/(\lambda_i - \mu)$.

Exercise 7.17 (hard)**.** A page graph with dangling node: build the 3×3 matrix $P = \begin{bmatrix} 0 & 0.5 & 1/3 \\ 0.5 & 0 & 1/3 \\ 0.5 & 0.5 & 1/3 \end{bmatrix}$ plus a teleport term $0.85P + 0.15 \cdot \frac{1}{3}J$ (J all-ones). Find the PageRank vector by power iteration in code and confirm $\lambda_1 = 1$.

Exercise 7.18 (easy)**.** For $A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$, write down both eigenvalues and both eigenvectors by inspection, then verify $A \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $A \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ directly.

Exercise 7.19 (hard)**.** A Markov chain has $\lambda_1 = 1$ and every other $|\lambda| < 1$. Explain why the steady state is the $\lambda = 1$ eigenvector, then show by a 2×2 example that a *periodic* chain (e.g. the swap matrix $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$) breaks the convergence while keeping the eigenvector.

Notes

The eigen-equation-first development and the applications section follow Strang [6] ch. 10 and 18.06SC Unit II [2]; linear dynamics as the motivating application follows VMLS ch. 9 [4] and MML §4.4 [5]. Axler ch. 5 [7] develops eigen theory *without* determinants (invariant subspaces first) — the opposite order from this book's characteristic-polynomial route, worth reading as a contrast once the computation here is comfortable.

CHAPTER 8

Least squares and linear regression

More equations than unknowns is not a bug — it is what real data looks like. $A\mathbf{x} = \mathbf{b}$ almost never has an exact solution once measurements enter, so we trade "solve" for "fit": find $\hat{\mathbf{x}}$ minimizing $\|A\mathbf{x} - \mathbf{b}\|$. Chapter 5 gives us everything we need — the best fit is the projection of $\mathbf{b}$ onto $\text{col}(A)$, and the normal equations are one sentence of orthogonality. After this chapter you can set up and solve least-squares problems by hand and in code, read residual plots, know why the normal equations are the wrong way to compute, and evaluate a fit honestly. This is the book's first capstone: by its end you have a working `LinearRegression` that agrees with scikit-learn.

BEFORE YOU START

Chapters 3, 5 (projection, QR). *Recap:* $A\mathbf{x} = \mathbf{b}$ usually has *no* exact solution when there are more equations than unknowns; we ask instead for the $\mathbf{x}$ that gets closest.

AFTER THIS CHAPTER YOU CAN

- derive the normal equations by projection and by calculus;
- fit a line, a polynomial or a many-feature linear model to data;
- solve least squares with QR instead of $A^{\mathsf{T}}A$ and say why;
- read R^2 and the residuals, and recognise overfitting.

HOW TO THINK ABOUT THIS

When no exact solution exists, the sensible answer is the closest point you can reach, and "closest" means the error is perpendicular to everything you can reach. That geometric sentence *is* the normal equation. Fitting a line to data is therefore a projection, not a mysterious recipe.

8.1 Overdetermined systems: the best we can do

An $m \times n$ system with $m > n$ is *overdetermined*: generically $\mathbf{b} \notin \text{col}(A)$, so no $\mathbf{x}$ satisfies $A\mathbf{x} = \mathbf{b}$ exactly.

Definition 8.1 (Least-squares solution)**.** $\hat{\mathbf{x}}$ is a *least-squares solution* of $A\mathbf{x} = \mathbf{b}$ when it minimizes the residual sum of squares,

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x}} \|\mathbf{b} - A\mathbf{x}\|^2.$$

Nothing in this requires a unique $\hat{\mathbf{x}}$ — uniqueness comes from A having full column rank (Section 8.5).

The squaring is a choice, not a theorem: it makes large errors expensive, keeps the objective smooth, and — decisively — turns the answer into a projection.

8.2 The normal equations via projection

$A\hat{\mathbf{x}}$ is the point of $\text{col}(A)$ closest to $\mathbf{b}$. Chapter 5 taught us that point: it is the projection, and its signature is that the residual is perpendicular to the whole column space:

Theorem 8.2 (Normal equations)**.** $\hat{\mathbf{x}}$ *minimizes* $\|\mathbf{b} - A\mathbf{x}\|$ *iff*

$$A^T(\mathbf{b} - A\hat{\mathbf{x}}) = \mathbf{0} \qquad \Longleftrightarrow \qquad A^T A\,\hat{\mathbf{x}} = A^T\mathbf{b}.$$

Proof. $A\hat{\mathbf{x}}$ is the closest point of $\text{col}(A)$ to $\mathbf{b}$ iff $\mathbf{b} - A\hat{\mathbf{x}}$ is orthogonal to every column of A — that is exactly $A^T(\mathbf{b} - A\hat{\mathbf{x}}) = \mathbf{0}$, one equation per column. The full derivation is in Section 8.7; Figure 8.1 is the picture. ∎

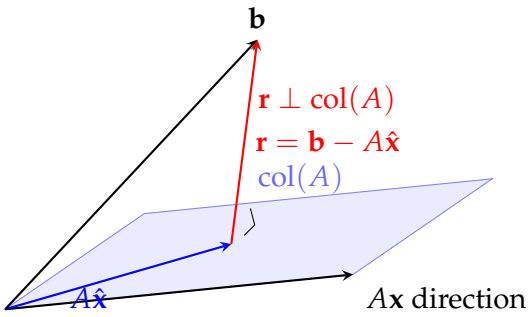


Figure 8.1: Least squares IS projection. Notice $\mathbf{r} \perp \text{col}(A)$ — the residual is what $\mathbf{b}$ keeps after giving up everything A can express. Regenerate: `figures/ch08_projection.tex`.

Example 8.3 (A line fit by hand)**. Strategy.** Four points $(1,2), (2,3), (3,3), (4,5)$; fit $y = a + mx$. Set $A = [\,\mathbf{1}\ \mathbf{x}\,]$, $\mathbf{b} = \mathbf{y}$, and solve the 2×2 normal equations directly — the sums are the arithmetic.

Solution. $A^T A = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}$ (rows: n, $\sum_{i=1}^n x_i$, $\sum_{i=1}^n x_i^2$) and $A^T\mathbf{b} = \begin{bmatrix} 13 \\ 37 \end{bmatrix}$ ($\sum_{i=1}^n y_i$, $\sum_{i=1}^n x_i y_i$).
Solve: $20a = 4 \cdot 30$-determinant bookkeeping gives $a = 1.0$, $m = 0.9$ — so $\hat{y} = 1.0 + 0.9x$

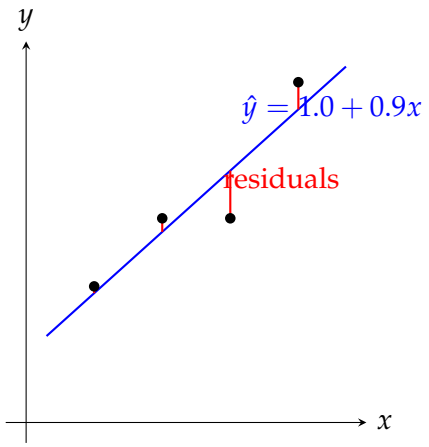


Figure 8.2: The fit of Example 8.3. Notice the red residual segments — they are the vertical distances being squared and summed; the fitted line leaves them balanced. Regenerate: `figures/ch08_fitted_line.tex`.

(Figure 8.2). Residuals: $(0.1, 0.2, -0.7, 0.4)$, and indeed $A^T\mathbf{r} = \mathbf{0}$: the residual sums to zero and is uncorrelated with x. RSS $= 0.7$.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

A = np.array([[1., 1.], [1., 2.], [1., 3.], [1., 4.]])
b = np.array([2., 3., 3., 5.])
print("A ="); print(A); print("b =", b)

AtA = A.T @ A; Atb = A.T @ b
print("A^T A ="); print(AtA); print("A^T b =", Atb)
x = np.linalg.solve(AtA, Atb)
print("xhat (intercept, slope) =", x)
r = b - A @ x
print("residual r =", r)
print("A^T r =", A.T @ r, "(orthogonal to col(A): OK)")
print("RSS =", r @ r)
```

Listing 8.1: `code/ch08_normal_eq.py -- the normal equations assembled and solved exactly as worked by hand.`

```
A =
[[1. 1.]
 [1. 2.]
 [1. 3.]
 [1. 4.]]
b = [2. 3. 3. 5.]
A^T A =
[[ 4. 10.]
 [10. 30.]]
A^T b = [13. 37.]
xhat (intercept, slope) = [1. 0.9]
residual r = [ 0.1  0.2 -0.7  0.4]
```

```
A^T r = [ 0. -0.] (orthogonal to col(A): OK)
RSS = 0.7
```

Listing 8.2: Output of the normal-equations solve. Notice the residual is orthogonal to BOTH columns — sums to zero and is uncorrelated with x — which is the normal-equations condition $A^T\mathbf{r} = \mathbf{0}$ in numbers.

Exercise 8.1 (easy)**.** Fit $y = a + mx$ to $(0,1), (1,2), (2,2), (3,4)$ through the normal equations.

Solution. $A^TA = \begin{bmatrix} 4 & 6 \\ 6 & 14 \end{bmatrix}$, $A^T\mathbf{b} = \begin{bmatrix} 9 \\ 20 \end{bmatrix}$; solving gives $a = 0.9$, $m = 1.0$ — residual sum zero as required.

8.3 Linear regression: features, intercept, many columns

Nothing about the machinery requires x to be one variable. With features $x_1, \ldots, x_k$ per data point, the design matrix has a column per feature — plus the intercept column of ones, which is a feature too. Forgetting it is the classic bug (Section 8.8): without $\mathbf{1}$ in A, the model is forced through the origin.

Example 8.4 (Quadratic feature map)**. Strategy.** "Linear" regression means linear in the *coefficients*, not in x. To fit a parabola we supply columns $[\mathbf{1}\ \mathbf{x}\ \mathbf{x}^2]$ and solve the same normal equations.

Solution. Take $(0,1), (1,2), (2,5), (3,10)$. Then $A^TA = \begin{bmatrix} 4 & 6 & 14 \\ 6 & 14 & 36 \\ 14 & 36 & 98 \end{bmatrix}$, $A^T\mathbf{b} = \begin{bmatrix} 18 \\ 45 \\ 139 \end{bmatrix}$, and solving gives $\hat{\mathbf{x}} = (1, 0, 1)$ exactly: the points lie on $y = 1 + x^2$. A 2×2 coefficient space became a 3×3 one — the linear algebra did not notice, and that is the point.

Exercise 8.2 (medium)**.** Show that adding a duplicate column to A (same feature twice) makes the normal equations singular — the minimizer stops being unique.

Solution. Duplicate columns make A column-rank-deficient, so A^TA is singular: $\ker(A^TA) \supseteq \ker A \ni (1, -1, 0, \ldots)$ on the duplicated pair — infinitely many $\hat{\mathbf{x}}$ give the same fit.

8.4 Feature maps and the overfitting preview

More columns always lowers training RSS — each added column enlarges $\mathrm{col}(A)$, so the projection gets closer to $\mathbf{b}$. With enough columns A interpolates the noise itself (Figure 8.3, and the listing's degree-11 row: train-RSS collapses to 0 while the true error explodes to 2.5×10^4).

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

rng = np.random.default_rng(1)
x = np.sort(rng.uniform(0, 1, 12))
```

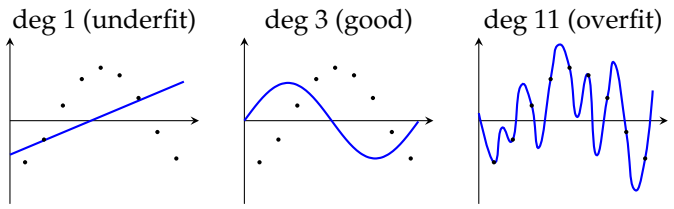


Figure 8.3: Same data, three feature maps. Notice the degree-1 panel misses the pattern (underfit), degree-3 captures it, degree-11 threads every noise point (overfit) — minimum training error is NOT the goal. Regenerate: `figures/ch08_overfit_panel.tex`.

```
y = np.sin(2 * np.pi * x) + 0.15 * rng.standard_normal(12)

def fit_eval(deg):
    A = np.vander(x, deg + 1, increasing=True)
    c, res, *_ = np.linalg.lstsq(A, y, rcond=None)
    rss_train = ((y - A @ c)**2).sum()
    xg = np.linspace(0, 1, 400)
    yg = np.sin(2 * np.pi * xg)
    rss_true = ((yg - np.vander(xg, deg + 1, increasing=True) @ c)**2).
        mean()
    return rss_train, rss_true

print("deg train-RSS true-RMS-error")
for d in [1, 3, 5, 11]:
    tr, te = fit_eval(d)
    print(f"{d:2d} {tr:8.4f} {te:9.4f}")
```

Listing 8.3: `code/ch08_poly_overfit.py -- fit degree` d, `report training RSS vs. error against the true function.`

```
deg train-RSS true-RMS-error
 1 2.4333 0.2277
 3 0.0628 0.0099
 5 0.0279 0.0054
11 0.0000 25288.1886
```

Listing 8.4: Output of the overfit demo. Notice training RSS decreases monotonically to zero — projections only get closer — while the true error turns around and explodes at degree 11, where A has a column per data point.

Exercise 8.3 (hard)**.** Explain why train-RSS is monotone non-increasing in the number of columns of A — a one-line argument from projection geometry — and why the argument says nothing about unseen data.

Solution. Adding a column enlarges $\mathrm{col}(A)$ to a bigger subspace, so the closest point can only get closer: projection distance is monotone in subspace inclusion. Unseen data isn't in the space we minimized over — the minimizer just moved $\mathbf{b}$'s projection, not the world's.

8.5 Solving in practice: QR, not A^TA

The normal equations are the right *theorem* and the wrong *algorithm*. A^TA squares the condition number — $\kappa(A^TA) = \kappa(A)^2$ — so solving it throws away half the digits before arithmetic even starts. Chapter 5's QR factorization dodges it: with $A = QR$,

$$A^TA\hat{\mathbf{x}} = A^T\mathbf{b} \iff R\hat{\mathbf{x}} = Q^T\mathbf{b},$$

a triangular solve that never forms A^TA. When A has full column rank, R is invertible and $\hat{\mathbf{x}}$ is *unique* (its uniqueness is a [hard] exercise below).

Example 8.5 (Same fit, stable route)**. Strategy.** Build a pathological A: features $1, x, x^2$ sampled on a window only 10^{-3} wide, so the columns are nearly parallel. Fit the same noise-free quadratic both ways and compare against truth.

Solution. $\kappa(A) \approx 5 \times 10^7$, so $\kappa(A^TA) \approx 3 \times 10^{15}$ — one digit short of float64's entire budget. The listing shows the consequence exactly: the A^TA solve lands 0.74 from truth while `np.linalg.lstsq` (QR/SVD internals) recovers the coefficients to 10^{-9}.

```
import numpy as np
np.set_printoptions(precision=6, suppress=True)

# x and x^2 over a window 1e-3 wide: the columns are nearly parallel
rng = np.random.default_rng(0)
n = 200
x = 1 + 0.001 * rng.random(n)
y = 2 - 3 * x + 0.5 * x**2 # exact quadratic data, no noise
A = np.column_stack([np.ones(n), x, x**2])
print("cond(A) = %.3e" % np.linalg.cond(A))
print("cond(A^T A) = %.3e <- the square" % np.linalg.cond(A.T @ A))
x_ata = np.linalg.solve(A.T @ A, A.T @ y)
x_lstsq, res, rank, sv = np.linalg.lstsq(A, y, rcond=None)
print("x via A^T A solve :", x_ata)
print("x via np.linalg.lstsq:", x_lstsq)
print("truth : [ 2. -3. 0.5]")
print("err A^T A : %.3e" % np.linalg.norm(x_ata - np.array([2., -3.,
    .5])))
print("err lstsq : %.3e" % np.linalg.norm(x_lstsq - np.array([2., -3.,
     .5])))
```

Listing 8.5: `code/ch08_ata_vs_lstsq.py -- same problem, two routes; the condition square is visible in the answer.`

```
cond(A) = 5.350e+07
cond(A^T A) = 3.267e+15 <- the square
x via A^T A solve : [ 2.301033 -3.601757 0.800724]
x via np.linalg.lstsq: [ 2. -3. 0.5]
truth : [ 2. -3. 0.5]
err A^T A : 7.370e-01
err lstsq : 2.752e-09
```

Listing 8.6: Output of the two-route comparison. The computed $\kappa(A^TA) = 3.267 \times 10^{15}$ is float64's own answer — about 14% off the exact square 2.86×10^{15}, because the product matrix is already so ill-conditioned that even its condition number loses precision. Either way the A^TA answer visibly differs from truth while lstsq is exact. Same math, different arithmetic.

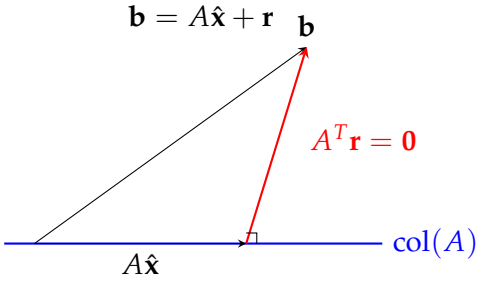


Figure 8.4: The normal equations geometrically. Notice $A^T\mathbf{r} = \mathbf{0}$ is one sentence — "the leftover is perpendicular to everything A can reach" — and it pins $\hat{\mathbf{x}}$ uniquely when the columns are independent. Regenerate: `figures/ch08_normal_geometry.tex`.

Exercise 8.4 (medium)**.** A feature is measured in millimeters instead of meters (same data, column $\times 1000$). What happens to $\kappa(A)$ and to the fitted *predictions*?

Solution. $\kappa(A)$ grows (column scales diverge — the condition number sees the ratio). Predictions are unchanged: $\mathrm{col}(A)$ is the same subspace, so the projection is the same; only the coefficient units move. This is why scaling helps conditioning but cannot change the fit.

8.6 Residuals, R^2, and honest evaluation

Definition 8.6 (Residuals and R^2)**.** The residual $\mathbf{r} = \mathbf{b} - A\hat{\mathbf{x}}$ is what's left over. With $\bar{y}$ the mean of $\mathbf{b}$,

$$R^2 = 1 - \frac{\|\mathbf{r}\|^2}{\|\mathbf{b} - \bar{y}\,\mathbf{1}\|^2}$$

is the fraction of variance the fit explains — 1 is perfect, 0 is no better than predicting the mean, negative means *worse* than the mean.

Example 8.7 (Reading a residual plot)**. Strategy.** A good fit leaves structureless residue: the model already ate the pattern. Plot r_i against $\hat{y}_i$ (or x_i) and ask what's left.

Solution. Figure 8.5: left panel is random scatter about zero — done. Right panel curves systematically — the model missed a feature (here a quadratic term); more data of the same kind won't help, a better $\mathrm{col}(A)$ will.

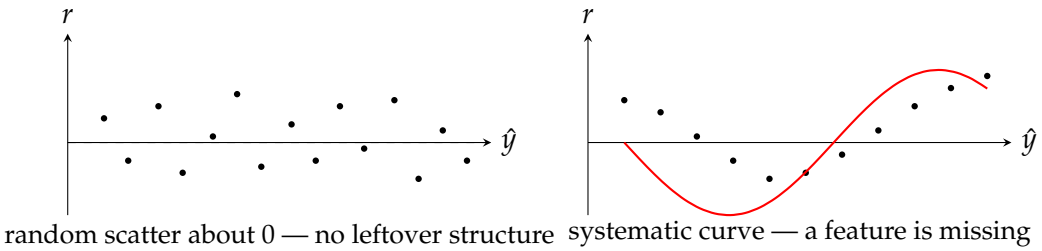


Figure 8.5: Residual diagnostics. Notice the left panel is scatter with no pattern (nothing left to learn) while the right panel's systematic curve means the feature map is missing something — the fix is a better basis, not more rows. Regenerate: `figures/ch08_residual_plot.tex`.

Two disciplines protect honesty:

- **Train/test split.** Evaluating on the same rows that produced $\hat{\mathbf{x}}$ rewards memorization (Figure 8.3). Hold back a test split — Figure 8.6 — and score there; it is the only measurement that means anything.
- **R^2 is not causality.** $R^2 \approx 0.99$ says the features predict the target *in this sample*. Nothing in the projection knows why the columns correlate — a 0.99 fit between ice cream and drowning is real arithmetic and fake science.

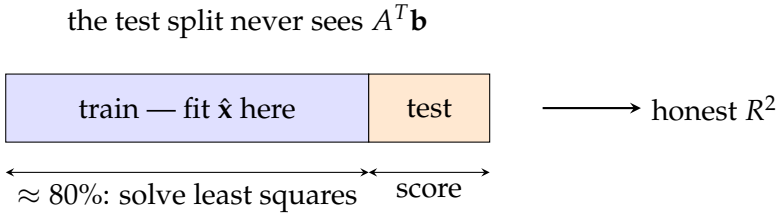


Figure 8.6: The honest protocol. Notice the test split contributes nothing to $A^TA\hat{\mathbf{x}} = A^T\mathbf{b}$ — it exists only to be scored. Regenerate: `figures/ch08_traintest.tex`.

Example 8.8 (Capstone: our own LinearRegression vs. sklearn)**. Strategy.** Implement `fit` as the QR route of Section 8.5 (with the intercept column handled inside), `predict` as $A\hat{\mathbf{x}}$, and `r2` by definition. Compare against `sklearn.linear_model.LinearRegression` on a 300-row synthetic housing set.

Solution. The listing shows our coefficients matching sklearn's to 4×10^{-14} — the library is doing exactly this math, nothing more. $R^2 = 0.987$ both ways.

```
import numpy as np
from sklearn.linear_model import LinearRegression
np.set_printoptions(precision=4, suppress=True)

# mlscratch: our own least-squares fitter, QR route from ch.5
class LinearRegression_scratch:
   def fit(self, A, y):
      A1 = np.column_stack([np.ones(len(A)), A]) # intercept column!
      Q, R = np.linalg.qr(A1)
      self.coef_ = np.linalg.solve(R, Q.T @ y)
      self.intercept_, self.slopes_ = self.coef_[0], self.coef_[1:]
      return self
   def predict(self, A):
      return self.intercept_ + A @ self.slopes_
   def r2(self, A, y):
      r = y - self.predict(A)
      return 1 - (r @ r) / ((y - y.mean()) @ (y - y.mean()))

# real-ish dataset: floor area, rooms, age -> apartment price
rng = np.random.default_rng(2)
n = 300
area = rng.uniform(30, 120, n)
rooms = rng.integers(1, 5, n).astype(float)
age = rng.uniform(0, 40, n)
price = 8 + 2.1 * area + 4 * rooms - 0.15 * age + 6 * rng.
    standard_normal(n)
```

```
X = np.column_stack([area, rooms, age])

ours = LinearRegression_scratch().fit(X, price)
ref = LinearRegression().fit(X, price)
print("ours intercept %.4f slopes %s" % (ours.intercept_, ours.slopes_
    ))
print("sklearn intercept %.4f slopes %s" % (ref.intercept_, ref.coef_)
    )
print("max |coef diff| = %.2e" % np.abs(ours.coef_ - np.append(ref.
    intercept_, ref.coef_)).max())
print("R^2 ours = %.4f" % ours.r2(X, price))
print("R^2 sklearn= %.4f" % ref.score(X, price))
```

Listing 8.7: `code/ch08_capstone_linreg.py -- mlscratch vs. sklearn: the whole "library algorithm" is the QR solve of this chapter.`

```
ours intercept 8.2343 slopes [ 2.0965 4.1166 -0.1587]
sklearn intercept 8.2343 slopes [ 2.0965 4.1166 -0.1587]
max |coef diff| = 4.26e-14
R^2 ours = 0.9874
R^2 sklearn= 0.9874
```

Listing 8.8: Output of the capstone. Notice the coefficients agree to machine precision — least squares via QR IS what the library computes, and your implementation is the real thing, not a toy.

Exercise 8.5 (easy)**.** A model has $R^2 = 0.4$ on train and 0.38 on test. A second has $R^2 = 0.97$ train, 0.55 test. Which is better and why?

Solution. The second memorized: its train score proves the columns could fit noise. The first generalizes — its test score is the honest number; prefer it unless the gap in accuracy itself is disqualifying.

Exercise 8.6 (hard — derive)**.** Weighted least squares: minimize $\|W(\mathbf{b} - A\mathbf{x})\|^2$ with W diagonal and positive. Derive the weighted normal equations $A^T W^2 A\hat{\mathbf{x}} = A^T W^2 \mathbf{b}$, and say what the weights mean physically.

Solution. It's ordinary least squares on the row-scaled system $A' = WA$, $\mathbf{b}' = W\mathbf{b}$: orthogonal residual $A'^T(\mathbf{b}' - A'\hat{\mathbf{x}}) = \mathbf{0}$ expands to $A^T W^2(\mathbf{b} - A\hat{\mathbf{x}}) = \mathbf{0}$. Weight w_{ii} large $\Rightarrow$ row i must be fit tightly — trusted measurements get loud votes.

8.7 Hand derivation

Derive $A^T A\hat{\mathbf{x}} = A^T\mathbf{b}$ from residual-orthogonality. Among all $\mathbf{x}$, the vector $A\mathbf{x}$ ranges over $\mathrm{col}(A)$; the closest point of a subspace to $\mathbf{b}$ is the projection $\mathbf{p}$ satisfying $\mathbf{b} - \mathbf{p} \perp$ every vector in the subspace. Write $\mathbf{p} = A\hat{\mathbf{x}}$. Perpendicularity to every column is a row per column:

$$A^T(\mathbf{b} - A\hat{\mathbf{x}}) = \mathbf{0} \iff A^T\mathbf{b} - A^T A\hat{\mathbf{x}} = \mathbf{0} \iff A^T A\hat{\mathbf{x}} = A^T\mathbf{b}.$$

When A has full column rank, $A^T A$ is positive definite ($\mathbf{x}^T A^T A\mathbf{x} = \|A\mathbf{x}\|^2 > 0$ for $\mathbf{x} \neq \mathbf{0}$), hence invertible, hence the solution unique. (Calculus route — gradient of $\|\mathbf{b} - A\mathbf{x}\|^2$ — lands here too; it waits for chapter 14.)

8.8 Common mistakes

FORGETTING THE INTERCEPT COLUMN

Without $\mathbf{1}$ in A the model must pass through the origin — every prediction carries a systematic offset. Always `column_stack([ones, ...])`.

FORMING A^TA WHEN QR IS CHEAPER AND STABLER

$\kappa(A^TA) = \kappa(A)^2$: near-parallel columns turn a hard problem into an impossible one, as the demo showed (err 0.74 vs 10^{-9} on the same data). Solve $R\hat{\mathbf{x}} = Q^T\mathbf{b}$, or call `np.linalg.lstsq` and let LAPACK choose.

LETTING UNSCALED FEATURES WRECK CONDITIONING

A column in millimeters beside one in kilometers gives 10^6 scale ratios for free. Center and scale features first — it fixes $\kappa(A)$ without changing $\text{col}(A)$.

EVALUATING ON TRAINING DATA AND CALLING IT PERFORMANCE

Train-RSS is monotone in columns and reaches zero exactly when A can memorize. A model that aces its own exam learned nothing. The test split is the only number that counts.

READING R^2 AS A CERTIFICATE OF CAUSALITY

R^2 measures in-sample correlation structure, nothing else. A high R^2 between an umbrella index and rain is prediction; believing umbrellas cause rain is a different discipline entirely.

8.9 Summary

Object	What you need to know
Least squares	minimize $\|\mathbf{b} - A\mathbf{x}\|^2$; the best fit is the projection of $\mathbf{b}$ on $\text{col}(A)$
Normal equations	$A^TA\hat{\mathbf{x}} = A^T\mathbf{b}$, from $A^T\mathbf{r} = \mathbf{0}$; unique iff full column rank
Design matrix	one column per feature + the intercept column of ones; "linear" = linear in coefficients
Feature maps	$[\mathbf{1}\ \mathbf{x}\ \mathbf{x}^2\ \dots]$ lets the same machinery fit curves; more columns ⇒ less train RSS
Compute route	QR: $R\hat{\mathbf{x}} = Q^T\mathbf{b}$; forming A^TA squares κ
Residuals	good fit ⇒ structureless scatter; patterns mean a missing feature
R^2	$1 - \text{RSS}/\text{TSS}$; measures correlation in-sample, never causality
Evaluation	train/test split: the test rows must not touch $A^T\mathbf{b}$

WATCH INSTEAD

1. *Projection Matrices and Least Squares* — MIT OpenCourseWare, 18.06 Lecture 16, 48:05, https://youtu.be/osh80YCg_GM — the normal-equations route of §8.2 at full length (accessed 2026-10-11).
2. *Linear Regression, Clearly Explained!!!* — StatQuest with Josh Starmer, 27:26, https://youtu.be/7ArmBVF2dCs — §8.3 in data form (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., §4.3 (textbook level) — least squares from the projection side.
- MIT OpenCourseWare, *18.06SC Linear Algebra* — the least- squares recitations, at OCW (course "18.06SC", Unit II), at https://ocw.mit.edu. Verified 2026-10-11.
- S. Boyd and L. Vandenberghe, *VMLS*, chs. 12–13 (free PDF) — least squares and least-squares data fitting, our chapter's exact ancestors. https://web.stanford.edu/~boyd/vmls/vmls.pdf Verified 2026-10-11.
- Margalit and Rabinoff, *Interactive Linear Algebra*, "Least-Squares Solutions" — normal equations with more worked residuals. https://textbooks.math.gatech.edu/ila/ Verified 2026-10-11.
- M. Deisenroth et al., *Mathematics for Machine Learning*, ch. 9 (free PDF) — linear regression from the ML side, including noise models. https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.
- scikit-learn documentation, *LinearRegression* and *Cross-Validation* user guide — the practical side of our capstone. Free at https://scikit-learn.org (user guide, "Linear Models"). Verified 2026-10-11.

Additional exercises

Exercise 8.7 (easy)**.** Write the design matrix for fitting $y = a + bx_1 + cx_2$ to $(x_1, x_2, y) \in \{(0,0,1), (1,0,3), (0,1,2), (1,1,5)\}$ and state the normal equations.

Exercise 8.8 (easy)**.** Given $\hat{\mathbf{x}} = (1.0, 0.9)$ for Example 8.3, predict y at $x = 5$ and at $x = 0.5$. Which prediction do you trust more and why?

Exercise 8.9 (easy)**.** If $\mathbf{b} \in \text{col}(A)$ already, what is the least-squares residual? What is R^2?

Exercise 8.10 (easy)**.** Compute R^2 for the fit of Example 8.3 (RSS $= 0.7$, $\mathbf{b}$'s mean 3.25, TSS $= 5.75$).

Exercise 8.11 (medium)**.** Show the projection $P = A(A^TA)^{-1}A^T$ applied to $\mathbf{b}$ gives exactly $A\hat{\mathbf{x}}$, and verify $P^2 = P$.

Exercise 8.12 (medium)**.** For the four points of Example 8.3, add the point $(5, 10)$ and refit. Which coefficient moves more, and what does that say about leverage?

Exercise 8.13 (medium)**.** In code: take the capstone dataset, drop the intercept column, refit, and report what happens to R^2 and the residual mean.

Exercise 8.14 (medium)**.** Show $\|\mathbf{b}\|^2 = \|A\hat{\mathbf{x}}\|^2 + \|\mathbf{r}\|^2$ (Pythagoras for least squares) and connect it to R^2.

Exercise 8.15 (medium)**.** A dataset has 8 points lying exactly on a cubic. Fit degrees 1–6 in code and plot train RSS vs. degree — where does the elbow sit and why?

Exercise 8.16 (hard)**.** Prove the least-squares minimizer is unique iff A has full column rank (show both directions: ker A nontrivial $\Rightarrow$ a family of minimizers; trivial $\Rightarrow$ A^TA positive definite).

Exercise 8.17 (hard)**.** Generalized least squares: minimize $\|\mathbf{b} - A\mathbf{x}\|_C^2 = (\mathbf{b} - A\mathbf{x})^TC^{-1}(\mathbf{b} - A\mathbf{x})$ for covariance $C \succ 0$. Derive the analog of the normal equations and interpret $C = I$ and C diagonal.

Exercise 8.18 (hard)**.** Show that R^2 computed on the training set equals the squared correlation between $\mathbf{b}$ and $A\hat{\mathbf{x}}$ when an intercept is present — and give a counterexample when it is not.

Exercise 8.19 (hard)**.** Implement leave-one-out cross-validation for the capstone data in code: fit on $n - 1$ rows, score the held-out row, average. Compare LOO R^2 to train R^2 and explain any gap in the language of this chapter.

Exercise 8.20 (easy)**.** Fit a constant $y = c$ to the data points $\{(0,1), (1,2), (2,2)\}$ by least squares — write the one-column design and solve the normal equation. Where does c land relative to the mean of the y's?

Exercise 8.21 (hard)**.** Show that the fitted vector $\hat{\mathbf{y}} = A\hat{\mathbf{x}}$ is the projection of $\mathbf{y}$ onto $\text{col}(A)$, then prove the residual $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$ satisfies $A^T\mathbf{r} = \mathbf{0}$. Interpret the second equation geometrically.

Notes

Least squares as the application spine follows VMLS part III [4] — including the "linear in coefficients" framing and the train/test honesty rules; the projection derivation is Strang's 18.06SC Unit II [2] and [6] ch. 5; the conditioning warning and residual diagnostics follow MML ch. 9 [5]. The calculus proof is deferred to ch. 14 where gradients are in scope.

CHAPTER 9

Symmetric matrices and the spectral theorem

Symmetric matrices are the best-behaved class in all of linear algebra: every eigenvalue is real, eigenvectors for different eigenvalues are perpendicular, and there is always a full orthonormal eigenbasis. The spectral theorem says a symmetric matrix is a rotation, an axis-aligned stretch, and a rotation back. Everything downstream uses this — PCA's principal axes, the Hessian's curvature readout, quadratic-program convexity, and the SVD's construction next chapter. After this chapter you can diagonalise any symmetric matrix with an orthogonal Q, classify definiteness three ways, use the Rayleigh quotient as a variational eigenvalue finder, and read a covariance matrix geometrically.

BEFORE YOU START

Chapters 5 and 7. *Recap:* a matrix is *symmetric* if $A = A^{\mathsf{T}}$. Covariance matrices, Hessians and $A^{\mathsf{T}}A$ are all symmetric, which is why this class matters so much.

AFTER THIS CHAPTER YOU CAN

- diagonalise a symmetric matrix with an orthonormal basis (the spectral theorem);
- decide whether a quadratic form is positive definite;
- use the Rayleigh quotient to find the largest and smallest eigenvalues;
- explain why every covariance matrix is positive semidefinite.

HOW TO THINK ABOUT THIS

Symmetric matrices behave beautifully: their eigenvectors are perpendicular, so the matrix is just independent stretching along perpendicular axes. That is why they appear whenever there is a notion of energy, curvature or spread. When you meet a symmetric matrix, think "ellipse with perpendicular axes."

9.1 Symmetric matrices and why they are special

Definition 9.1 (Symmetric matrix)**.** A is *symmetric* when $A^T = A$: entry (i, j) equals entry (j, i). The Gram matrix $A^T A$, every covariance matrix, and every Hessian of a smooth function are symmetric by construction.

Two miracles happen for real symmetric matrices, proved in Section 9.6:

Theorem 9.2 (Real eigenvalues)**.** *If $A = A^T$ is real, every eigenvalue of A is real.*

Theorem 9.3 (Orthogonal eigenvectors)**.** *If $A = A^T$, eigenvectors belonging to* distinct *eigenvalues are orthogonal.*

Neither holds in general: the rotation $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ has eigenvalues $\pm i$ (chapter 7) — symmetry is what removes complex eigenvalues and skew eigenframes at once.

9.2 The spectral theorem

Theorem 9.4 (Spectral theorem)**.** *Every real symmetric $n \times n$ matrix factors as*

$$A = Q \Lambda Q^T,$$

where Q is orthogonal ($Q^T Q = I$, columns = orthonormal eigenvectors) and $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ is real.

Compare chapter 7: there $A = V \Lambda V^{-1}$ held only with a possibly ill-conditioned V. Symmetry upgrades the basis change: V^{-1} becomes Q^T, perfectly conditioned and free. Figure 9.1 shows the geometry: rotate, stretch, rotate back.

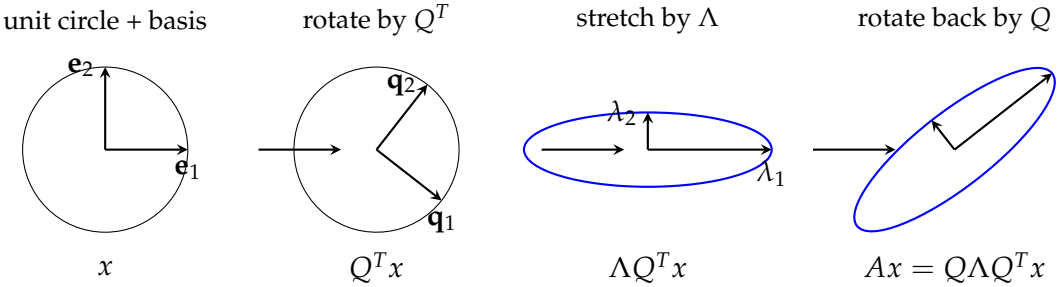


Figure 9.1: $A = Q \Lambda Q^T$ geometrically: rotate into the eigenbasis, stretch each axis by λ_i, rotate back. Notice there are no skew angles anywhere — unlike general diagonalisation, everything stays orthogonal. Regenerate: `figures/ch09_rotstretch.tex`.

Example 9.5 (Spectral decomposition by hand)**. Strategy.** For $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ the eigenvectors must come out orthonormal — verify, then assemble $Q \Lambda Q^T$.

Solution. $\chi_A = (2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3$: roots $\lambda_1 = 3$, $\lambda_2 = 1$. Eigenvectors: $(1,1)/\sqrt{2}$ for 3, $(1,-1)/\sqrt{2}$ for 1 — note $\langle (1,1),(1,-1)\rangle = 0$ as promised. Then

$$A = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}\begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = Q\Lambda Q^T.$$

Check the (1,1) entry: $\frac{1}{2}(3\cdot 1 + 1\cdot 1) = 2$ ✓.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
rng = np.random.default_rng(3)

M = rng.standard_normal((4, 4))
A = M + M.T # symmetric by construction
print("A ="); print(A)
print("symmetric:", np.allclose(A, A.T))

w, Q = np.linalg.eigh(A) # eig for symmetric matrices
print("eigenvalues:", w)
print("Q orthonormal (Q^T Q = I):", np.allclose(Q.T @ Q, np.eye(4)))
Lam = np.diag(w)
print("A == Q Lambda Q^T:", np.allclose(Q @ Lam @ Q.T, A))
print("A^3 == Q Lambda^3 Q^T:",
      np.allclose(Q @ np.diag(w**3) @ Q.T, A @ A @ A))
# non-symmetric contrast: same eigenvalues? no orthogonal basis
B = np.array([[0., -1.], [1., 0.]])
print("rotation eig:", np.linalg.eigvals(B), "(complex: not symmetric)
    ")
```

Listing 9.1: `code/ch09_spec_verify.py -- a random symmetric matrix: eigenvalues real, Q orthogonal, reconstruction exact.`

```
A =
[[ 4.0818 -3.0083 -0.4471 -0.8491]
 [-3.0083 -0.4312 1.303 -0.9 ]
 [-0.4471 1.303 0.4516 -1.4078]
 [-0.8491 -0.9 -1.4078 -0.7816]]
symmetric: True
eigenvalues: [-2.6815 -1.6239 1.8538 5.7722]
Q orthonormal (Q^T Q = I): True
A == Q Lambda Q^T: True
A^3 == Q Lambda^3 Q^T: True
rotation eig: [0.+1.j 0.-1.j] (complex: not symmetric)
```

Listing 9.2: Output of the spectral check. Notice `eigh` (the symmetric specialist) returns all-real eigenvalues and a Q that is orthonormal to machine precision — and powers collapse to Λ^k.

Exercise 9.1 (easy). Decompose $\begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$ and $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ as $Q\Lambda Q^T$.

Solution. First is already Λ with $Q = I$. Second: eigenvalues $3, -1$ with $(1,1)/\sqrt{2}, (1,-1)/\sqrt{2}$ — Q orthogonal as required even though an eigenvalue is negative (symmetric $\neq$ positive definite!).

9.3 Quadratic forms and definiteness

Definition 9.6 (Quadratic form)**.** For symmetric A, $q(\mathbf{x}) = \mathbf{x}^T A\mathbf{x}$ is a pure second-degree scalar — a paraboloid, saddle, or degenerate sheet whose level sets Figure 9.2 draws.

Definition 9.7 (Definiteness)**.** A is *positive definite* (PD) when $\mathbf{x}^T A\mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$; *positive semidefinite* (PSD) when ≥ 0; *indefinite* when the form takes both signs. Negative definite/semidefinite mirror with $\leq, < 0$.

Three equivalent tests (each earns its keep):

Test	Statement
Eigenvalues	PD iff every $\lambda_i > 0$; PSD iff $\lambda_i \geq 0$; indefinite iff mixed signs
Pivots (Sylvester)	PD iff every pivot in elimination is positive — no eigenvalues needed
Leading minors	PD iff all leading principal minors > 0

Example 9.8 (Classifying three ways)**. Strategy.** Take the tridiagonal $B = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$ and classify by pivots (cheap) then eigenvalues (decisive).

Solution. Elimination gives pivots 2, $\frac{3}{2}$, $\frac{4}{3}$ — all positive, so B is PD without touching eigenvalues. The listing confirms: eigenvalues $0.586, 2, 3.414$ all positive. Contrast $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$: eigenvalues $-1, 3$ and pivots $1, -3$ — indefinite, its level sets are hyperbolas.

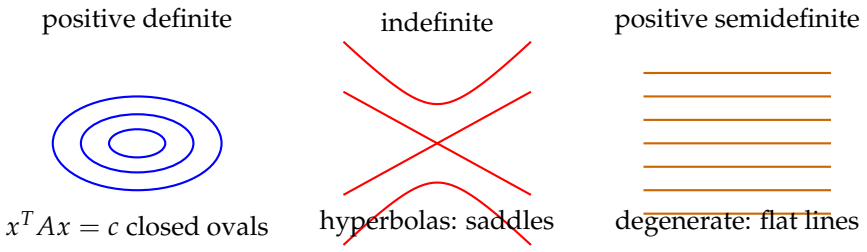


Figure 9.2: Level sets $x^T Ax = c$ by definiteness. Notice positive definite gives closed ellipses (a bowl), indefinite gives hyperbolas (a saddle), semidefinite degenerates into parallel lines (a trough). Regenerate: `figures/ch09_levelsets.tex`.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

def pivots(A):
    A = A.astype(float).copy(); n = A.shape[0]; ps = []
    for j in range(n):
        ps.append(A[j, j])
        if abs(A[j, j]) < 1e-12: break
        A[j+1:, j:] -= np.outer(A[j+1:, j] / A[j, j], A[j, j:])
```

```
    return np.array(ps)

mats = {
    "PD [[2,-1,0],[-1,2,-1],[0,-1,2]]":
       np.array([[2., -1., 0.], [-1., 2., -1.], [0., -1., 2.]]),
    "PSD [[1,1,1],[1,1,1],[1,1,1]]":
       np.ones((3, 3)),
    "IND [[1,0],[0,-1]]":
       np.array([[1., 0.], [0., -1.]]),
    "PD? [[1,2],[2,1]]":
       np.array([[1., 2.], [2., 1.]]),
}
for name, M in mats.items():
    w = np.linalg.eigvalsh(M)
    print(f"{name}: eigs {w} pivots {pivots(M)}")

# A^T A is always PSD
rng = np.random.default_rng(0)
A = rng.standard_normal((5, 3))
G = A.T @ A
print("Gram G=A^T A eigenvalues:", np.linalg.eigvalsh(G))
x = rng.standard_normal(3)
print("x^T G x = ||Ax||^2 =", x @ G @ x, " = ||Ax||^2 check:",
      np.isclose(x @ G @ x, (A @ x) @ (A @ x)))
```

Listing 9.3: `code/ch09_definiteness.py -- eigenvalues vs. pivot signs on four matrices, then` A^TA `is always PSD.`

```
PD [[2,-1,0],[-1,2,-1],[0,-1,2]]: eigs [0.5858 2. 3.4142] pivots [2.
    1.5 1.3333]
PSD [[1,1,1],[1,1,1],[1,1,1]]: eigs [-0. -0. 3.] pivots [1. 0.]
IND [[1,0],[0,-1]]: eigs [-1. 1.] pivots [ 1. -1.]
PD? [[1,2],[2,1]]: eigs [-1. 3.] pivots [ 1. -3.]
Gram G=A^T A eigenvalues: [0.1121 2.9218 9.9282]
x^T G x = ||Ax||^2 = 8.095346091200302 = ||Ax||^2 check: True
```

Listing 9.4: Output of the definiteness tests. Notice pivots and eigenvalues agree in sign pattern on every case — and the Gram matrix's eigenvalues are all ≥ 0 because $x^TGx = \|Ax\|^2$ is a squared length, never negative.

Proposition 9.9 (A^TA is always PSD)**.** *For any real A (any shape), $\mathbf{x}^T(A^TA)\mathbf{x} = \|A\mathbf{x}\|^2 \geq 0$. It is PD exactly when A has full column rank.*

Exercise 9.2 (medium)**.** Show A PD $\Rightarrow$ A invertible and A^{-1} PD.

Solution. $\lambda_i > 0$ means none is zero, so $\det A = \prod \lambda_i > 0$ and $A^{-1} = Q\Lambda^{-1}Q^T$ has eigenvalues $1/\lambda_i > 0$ — PD.

Example 9.10 (A semidefinite trap)**.** Is $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ positive definite?

Strategy. Definiteness asks about *every* $\mathbf{x}$: $x^TAx > 0$. Check the eigenvalues — and, for a fast counterexample hunt, note that $\det A$ decides already: a 2×2 PD matrix has positive determinant.

Solution. $\det A = 4 - 4 = 0$, so A is singular: $\lambda_2 = 0$ — semidefinite, not definite. Find the flat direction directly: $A(2,-1) = (0,0)$, so $(2,-1)^T A(2,-1) = 0$ exactly — the trough of figure 9.3. Eigenvalues $0, 5$ confirm: PSD with a zero eigen-direction. ✓ Moral: "positive" and "positive *semi*definite" differ by exactly the flat directions — and data matrices live there constantly.

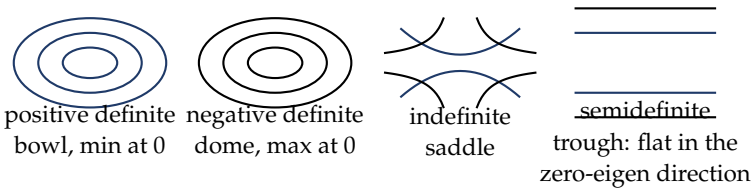


Figure 9.3: Definiteness by level sets. Notice the semidefinite case has a *flat* direction — the zero eigenvalue — where the form stays 0 no matter how far you walk.

9.4 Rayleigh quotient

Definition 9.11 (Rayleigh quotient)**.** For symmetric A and $\mathbf{x} \neq \mathbf{0}$, $R(\mathbf{x}) = \dfrac{\mathbf{x}^T A \mathbf{x}}{\mathbf{x}^T \mathbf{x}}$.

Decompose $\mathbf{x}$ in the eigenbasis: R is a weighted average of the λ_i with weights c_i^2 — so it lives in $[\lambda_{\min}, \lambda_{\max}]$, and

Theorem 9.12 (Min–max principle)**.** $\max_{\mathbf{x}\neq\mathbf{0}} R(\mathbf{x}) = \lambda_1$ *at* $\mathbf{x} = \mathbf{q}_1$; $\min_{\mathbf{x}\neq\mathbf{0}} R(\mathbf{x}) = \lambda_n$ *at* $\mathbf{x} = \mathbf{q}_n$. *The k-th eigenvalue is the max of R over vectors orthogonal to* $\mathbf{q}_1, \ldots, \mathbf{q}_{k-1}$.

This is why power iteration (chapter 7) converged to λ_1: it is hill-climbing R (Figure 9.4). It is also why PCA's first component is a Rayleigh maximization — Section 9.5.

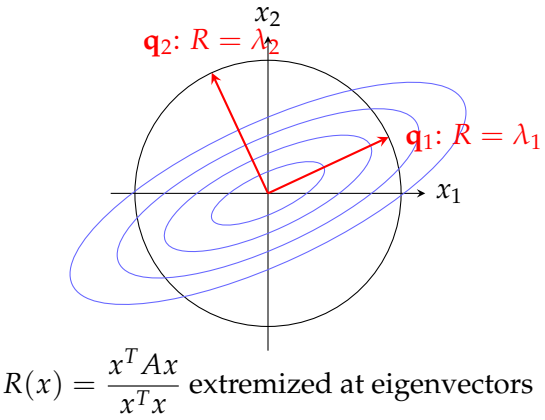


Figure 9.4: $R(x)$ on the unit circle. Notice the quotient is maximized along $\mathbf{q}_1$ and minimized along $\mathbf{q}_2$ — the eigenvectors are exactly the stationary directions of the landscape. Regenerate: `figures/ch09_rayleigh.tex`.

Exercise 9.3 (hard)**.** From the weighted-average picture, prove: if $\mathbf{x}$ is a stationary point of R, then $\mathbf{x}$ is an eigenvector. (Hint: the gradient of R is proportional to $A\mathbf{x} - R(\mathbf{x})\mathbf{x}$.)

Solution. $\nabla R = 2(A\mathbf{x} - R\mathbf{x})/\mathbf{x}^T\mathbf{x}$; stationary $\Rightarrow A\mathbf{x} = R\mathbf{x}$ — the eigen equation with $\lambda = R(\mathbf{x})$.

Example 9.13 (Rayleigh quotient hands-on)**.** For $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ (eigenvalues 3 and 1), evaluate the Rayleigh quotient $R(\mathbf{x}) = \mathbf{x}^T A\mathbf{x}/\mathbf{x}^T\mathbf{x}$ on $\mathbf{x} = (1,1)$, $(1,-1)$, and $(1,0)$ — and confirm it never leaves $[\lambda_{\min}, \lambda_{\max}]$.

Strategy. The quotient is the stretch A applies along a direction, weighted by the eigenbasis mix. Test the two eigenvectors first — they must return the eigenvalues exactly.

Solution. $(1,1)$ is the $\lambda = 3$ eigenvector: $A(1,1) = (3,3)$, $R = 6/2 = 3$. $(1,-1)$ gives $A(1,-1) = (1,-1)$, $R = 2/2 = 1$. And $(1,0)$: $A(1,0) = (2,1)$, $R = 2/1 = 2$ — between the eigenvalues, since $(1,0) = \frac{1}{2}(1,1) + \frac{1}{2}(1,-1)$ mixes both directions. ✓ The quotients sweep $[1,3]$ as the direction turns — the whole machine behind both $\lambda_{\max} = \max R$ and the power-iteration picture.

9.5 Covariance matrices: PSD by data

Center data matrix X (n rows of observations, d feature columns) and form $C = X^T X/(n-1)$. As a Gram matrix it is automatically PSD — and its eigenstructure IS the data's geometry:

Example 9.14 (Covariance eigendecomposition)**. Strategy.** Build correlated 2-D data ($x_2 = 0.8x_1 + \text{noise}$), compute C, and read its eigenvalues/eigenvectors as the axes of the data's spread — a PCA preview.

Solution. The listing's $C = \begin{bmatrix} 1.03 & 0.86 \\ 0.86 & 0.88 \end{bmatrix}$ has eigenvalues $1.82, 0.09$: one direction carries 95% of variance. Its eigenvector $(-0.74, -0.68)$ is the long axis of the data ellipse (Figure 9.5) — principal component 1, found by nothing but the spectral theorem.

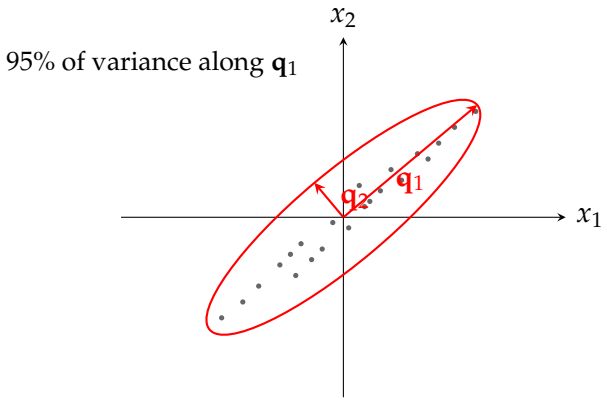


Figure 9.5: Covariance ellipse on real-ish data. Notice the ellipse's long axis is the eigenvector of the largest eigenvalue — that IS the principal component, and its length is the eigenvalue's square root. Regenerate: `figures/ch09_covellipse.tex`.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

# correlated 2-D data: x1 height-ish, x2 = 0.8 x1 + noise
rng = np.random.default_rng(4)
n = 400
x1 = rng.standard_normal(n)
x2 = 0.8 * x1 + 0.4 * rng.standard_normal(n)
X = np.column_stack([x1, x2])
X = X - X.mean(axis=0)
C = X.T @ X / (n - 1)
print("covariance C ="); print(C)
w, Q = np.linalg.eigh(C)
order = np.argsort(w)[::-1]
w, Q = w[order], Q[:, order]
print("eigenvalues:", w)
print("eigenvectors (cols):"); print(Q)
var_frac = w / w.sum()
print("variance fraction:", var_frac)
proj = X @ Q[:, 0]
print("reconstruction error keeping PC1:",
     np.mean((X - np.outer(proj, Q[:, 0]))**2))
print("PC1 direction angle (deg):", np.degrees(np.arctan2(Q[1,0], Q
    [0,0])))
```

Listing 9.5: `code/ch09_covariance.py -- PCA directions fall straight out of np.linalg.eigh on the covariance matrix.`

```
covariance C =
[[1.0254 0.8595]
 [0.8595 0.8843]]
eigenvalues: [1.8173 0.0925]
eigenvectors (cols):
[[-0.7355 0.6776]
 [-0.6776 -0.7355]]
variance fraction: [0.9516 0.0484]
reconstruction error keeping PC1: 0.046111017868167695
PC1 direction angle (deg): -137.34564252667914
```

Listing 9.6: Output of the covariance eigensolve. Notice 95% of variance rides one eigenvector — projecting onto it loses only 0.046 mean-square error. This is chapter 10's PCA with the SVD training wheels still on.

Exercise 9.4 (easy)**.** Why can no covariance matrix be indefinite?

Solution. $C = X^T X/(n-1)$ is a Gram matrix: $\mathbf{v}^T C\mathbf{v} = \|X\mathbf{v}\|^2/(n-1) \geq 0$ — PSD by construction.

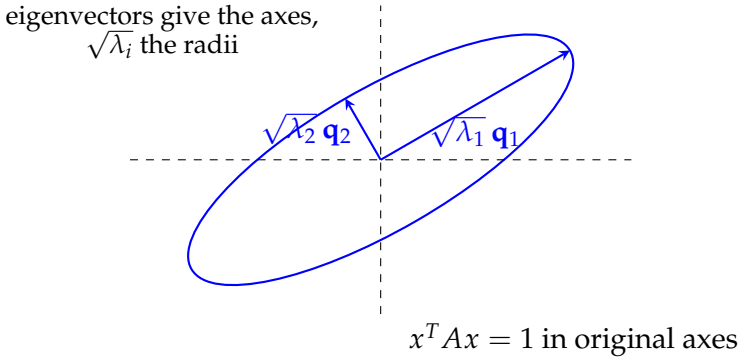


Figure 9.6: $x^TAx = 1$ is an ellipsoid whose axes are eigenvectors with radii $1/\sqrt{\lambda_i}$ (for the form) — or $\sqrt{\lambda_i}$ for the covariance ellipsoid. Either way: eigenvectors are directions, eigenvalues are lengths. Regenerate: `figures/ch09_ellipsoid.tex`.

9.6 Hand derivation

Real eigenvalues. Suppose $A = A^T$ real and $A\mathbf{v} = \lambda\mathbf{v}$ with $\mathbf{v} \neq \mathbf{0}$ complex. Take conjugate-transposes twice. First, $\mathbf{v}^*A\mathbf{v} = \lambda\,\mathbf{v}^*\mathbf{v}$. Second, conjugating the eigen equation: $A\bar{\mathbf{v}} = \bar{\lambda}\bar{\mathbf{v}}$ (A real), so $\mathbf{v}^*A\mathbf{v} = (A^T\mathbf{v})^*\mathbf{v} = (A\mathbf{v})^*\mathbf{v}$... directly: $(\mathbf{v}^*A\mathbf{v})^* = \mathbf{v}^*A^T\mathbf{v} = \mathbf{v}^*A\mathbf{v}$ since $A = A^T$ — the scalar $\mathbf{v}^*A\mathbf{v}$ equals its own conjugate, hence real. But $\mathbf{v}^*\mathbf{v} = \|\mathbf{v}\|^2 > 0$ is real and positive, so $\lambda = \mathbf{v}^*A\mathbf{v}/\mathbf{v}^*\mathbf{v}$ is real. ■

Orthogonal eigenvectors. Let $A\mathbf{v}_1 = \lambda_1\mathbf{v}_1$, $A\mathbf{v}_2 = \lambda_2\mathbf{v}_2$ with $\lambda_1 \neq \lambda_2$. Then

$$\lambda_1\,\mathbf{v}_1^T\mathbf{v}_2 = (A\mathbf{v}_1)^T\mathbf{v}_2 = \mathbf{v}_1^TA^T\mathbf{v}_2 = \mathbf{v}_1^TA\mathbf{v}_2 = \lambda_2\,\mathbf{v}_1^T\mathbf{v}_2,$$

so $(\lambda_1 - \lambda_2)\,\mathbf{v}_1^T\mathbf{v}_2 = 0$ and $\mathbf{v}_1^T\mathbf{v}_2 = 0$. ■

Together: eigenvalues real, and eigenvectors of distinct eigenvalues orthogonal; repeated eigenvalues get orthonormalized inside their eigenspace (Gram–Schmidt, ch.5), giving Q.

9.7 Common mistakes

"REAL MATRIX ⇒ REAL EIGENVALUES"

False in general — rotations have $\pm i$. The theorem is only for symmetric A. Keep the hypothesis attached: $A^T = A$ is doing all the work.

PSD VS. PD VS. INDEFINITE CONFUSION

≥ 0 and > 0 are different universes: PSD permits flat directions ($\lambda = 0$), PD doesn't. Check $\lambda_{\min} = 0$ before calling a matrix PD — and remember symmetric does NOT imply positive definite ($\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ is symmetric and indefinite).

BELIEVING A^TA CAN BE NEGATIVE DEFINITE

$\mathbf{x}^TA^TA\mathbf{x} = \|A\mathbf{x}\|^2 \geq 0$ always — it cannot even have a negative eigenvalue. If your Gram matrix shows one, the computation is wrong, not the theorem.

MIXING UP SIMILARITY WITH ORTHOGONAL SIMILARITY

$B = P^{-1}AP$ shares eigenvalues only; $B = Q^TAQ$ additionally preserves symmetry *and* is perfectly conditioned. The spectral theorem is orthogonal similarity — don't reach for a general P when Q exists.

9.8 Summary

Object	What you need to know
Spectral theorem	$A = A^T \Rightarrow A = Q\Lambda Q^T$, Q orthogonal, Λ real
Definiteness	PD: $\lambda_i > 0$ / all pivots > 0 / minors > 0; PSD: $\lambda_i \geq 0$; indefinite: mixed signs
Gram matrices	A^TA always PSD; PD iff full column rank
Rayleigh quotient	$R(x) = x^TAx/x^Tx \in [\lambda_n, \lambda_1]$; extremes at eigenvectors (min–max)
Ellipsoids	eigenvectors = axes, $\sqrt{\lambda_i}$ = radii
Covariance	PSD by construction; top eigenvector = PC1 (PCA next chapter)

WATCH INSTEAD

1. *Symmetric Matrices and Positive Definiteness* — MIT OpenCourseWare, 18.06 Lecture 25, 43:52, https://youtu.be/UCc9q_cAhho — the spectral theorem plus the definiteness tests, §9.2–§9.3 (accessed 2026-10-11).
2. *Positive Definite Matrices and Minima* — MIT OpenCourseWare, 18.06 Lecture 27, 50:40, https://youtu.be/vF7eyJ2g3kU — definiteness as the second-derivative test, §9.3 (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., §6.4 (textbook level) — symmetric matrices and the spectral theorem.
- MIT OpenCourseWare, *18.06SC Linear Algebra*, Unit III — recitations on positive definiteness, at OCW (course "18.06SC", Unit III page "Positive definite matrices"), at https://ocw.mit.edu. Verified 2026-10-11.
- Margalit and Rabinoff, *Interactive Linear Algebra*, "The Spectral Theorem" — orthonormal diagonalization with more examples. https://textbooks.math.gatech.edu/ila/ Verified 2026-10-11.

- S. Boyd and L. Vandenberghe, *VMLS*, §8.3 (free PDF) — quadratic forms in the data register.
  https://web.stanford.edu/~boyd/vmls/vmls.pdf Verified 2026-10-11.
- M. Deisenroth et al., *Mathematics for Machine Learning*, §3.5 (free PDF) — symmetric positive-definite matrices for ML kernels and metrics.
  https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.

Additional exercises

Exercise 9.5 (easy)**.** Decide whether each is symmetric: $A + A^T$, $A - A^T$, A^TA, AA^T, ABA^T with B symmetric.

Exercise 9.6 (easy)**.** Classify definiteness of $\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$, $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ from their eigenvalues.

Exercise 9.7 (easy)**.** Give a symmetric matrix with eigenvalues 4 and -2, and one with 4 and 0. Which is invertible?

Exercise 9.8 (easy)**.** Compute $R(\mathbf{x})$ for $A = \operatorname{diag}(5, 1)$ at $\mathbf{x} = (1, 0), (0, 1), (1, 1)$. Where do max/min sit?

Exercise 9.9 (medium)**.** Show that A PD and B PD implies $A + B$ PD. Does AB stay symmetric? Give a counterexample or proof.

Exercise 9.10 (medium)**.** Classify definiteness of $\begin{bmatrix} 4 & 2 & 0 \\ 2 & 4 & 0 \\ 0 & 0 & -1 \end{bmatrix}$ two ways (pivots, eigenvalues) without a computer.

Exercise 9.11 (medium)**.** Show that if A is PSD then its diagonal entries are ≥ 0, and $a_{ii} = 0$ forces the entire row/column i to be zero. (Hint: test vectors $\mathbf{e}_i$ and $\mathbf{e}_i + t\mathbf{e}_j$.)

Exercise 9.12 (medium)**.** In code: generate a 3×3 symmetric A, compute $R(\mathbf{x})$ on 100 random unit vectors, and show the empirical max/min approximate λ_1, λ_3 — then locate the maximizer closest to the top eigenvector.

Exercise 9.13 (medium)**.** The Hessian of $f(x, y) = x^2 + 4xy + y^2$ is constant. Classify the critical point at the origin by definiteness.

Exercise 9.14 (hard)**.** Prove the min–max principle: λ_k equals the maximum of $R(\mathbf{x})$ over the subspace orthogonal to $\mathbf{q}_1, \ldots, \mathbf{q}_{k-1}$, and every maximizer is an eigenvector for λ_k.

Exercise 9.15 (hard)**.** Show A PD $\Rightarrow$ there is a unique PSD "square root" $A^{1/2} = Q\Lambda^{1/2}Q^T$ with $A^{1/2}A^{1/2} = A$, and use it to explain the covariance-ellipsoid radii.

Exercise 9.16 (hard)**.** Interlacing: prove that deleting the last row and column of a symmetric A interlaces the eigenvalues, $\lambda_1(A) \geq \lambda_1(A') \geq \lambda_2(A) \geq \cdots \geq \lambda_{n-1}(A') \geq \lambda_n(A)$. (Rayleigh quotient over a subspace does all the work.)

Exercise 9.17 (easy)**.** For $A = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$, sketch the level set $\mathbf{x}^T A\mathbf{x} = 3$ and name it. Which axis is shorter, and why does that match the bigger eigenvalue?

Exercise 9.18 (medium)**.** Compute the Rayleigh quotient of $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ on $(2, 0)$ and on $(\cos\theta, \sin\theta)$ as a function of θ; find its maximum over θ and confirm it is $\lambda_{\max}$.

Exercise 9.19 (hard)**.** Prove the max principle: a symmetric A with eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n$ satisfies $\max_{\|\mathbf{x}\|=1} \mathbf{x}^T A\mathbf{x} = \lambda_1$. (Hint: write $\mathbf{x}$ in the eigenbasis and bound each term.)

Notes

The rotate–stretch–rotate framing and definiteness-via-pivots follow Strang [6] and 18.06SC Unit III [2]; Axler ch. 7 [7] gives the cleaner operator-theory route (self-adjoint first, eigenvalues as a consequence); MML §3.5 and ch. 4 [5] anchor the PSD/quadratic-form clarity we borrow (its issue tracker documents exactly the PD/PSD confusion we tripwire). The covariance-as-PSD application follows VMLS [4]; PCA proper arrives next chapter.

CHAPTER 10

The SVD and PCA

Diagonalisation had a dealbreaker: it only worked for square matrices with enough eigenvectors. The singular value decomposition lifts both restrictions — EVERY $m \times n$ matrix, of any rank, factors into an orthogonal input basis, a diagonal of nonnegative stretches, and an orthogonal output basis. It is the most useful factorization in applied linear algebra: PCA is an SVD of centered data, the pseudoinverse is an SVD with inverted singular values, low-rank approximation is an SVD truncated. After this chapter you can compute the SVD of a small matrix by hand, read singular values as the matrix's importance spectrum, compress data to rank k, run PCA end-to-end against sklearn, and use A^+ as the universal least-squares solver.

BEFORE YOU START

Chapters 5, 7 and 9. *Recap:* eigenvalues need a square matrix; the SVD works for *any* matrix by stretching along perpendicular directions chosen for the input and the output separately.

AFTER THIS CHAPTER YOU CAN

- compute the SVD of a small matrix and describe it geometrically;
- read rank, condition number and the pseudoinverse off the singular values;
- compress a matrix by truncating its SVD and state the Eckart–Young guarantee;
- perform PCA and explain what the principal components are.

HOW TO THINK ABOUT THIS

The SVD says every matrix, however rectangular, is a rotation, then a stretch along perpendicular axes, then another rotation. The stretch sizes (singular values) rank the directions by importance. Keeping the big ones and dropping the small is compression; PCA is exactly that applied to centred data.

10.1 Statement: $A = U\Sigma V^T$

Theorem 10.1 (Singular value decomposition). *Every real $m \times n$ matrix factors as*

$$A = U\Sigma V^T,$$

with U ($m \times m$) and V ($n \times n$) orthogonal, and Σ ($m \times n$) diagonal with entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ on its $r = \operatorname{rank} A$ leading positions, zeros elsewhere. The σ_i are the singular values*; columns of U and V are the left/right* singular vectors.

The construction (derived in Section 10.6): V is an orthonormal eigenbasis of A^TA, $\sigma_i = \sqrt{\lambda_i(A^TA)}$, and $\mathbf{u}_i = A\mathbf{v}_i/\sigma_i$ — which lands orthonormal automatically. Unlike eigendecomposition, the SVD needs no hypotheses at all: every matrix has one, and its cost is only that A^TA squares the condition number — which is why production code never forms A^TA (Section 10.7).

Example 10.2 (Full SVD of a 2×2 by hand). **Strategy.** $A = \begin{bmatrix} 4 & 4 \\ -3 & 3 \end{bmatrix}$. Compute A^TA, eigendecompose it (chapter 9 machinery), take square roots, get $\mathbf{u}_i$ from $\mathbf{v}_i$.

Solution. $A^TA = \begin{bmatrix} 25 & 7 \\ 7 & 25 \end{bmatrix}$: eigenvalues $32, 18$ (eigenvectors $(1,1)/\sqrt{2}$, $(1,-1)/\sqrt{2}$), so $\sigma_1 = \sqrt{32} = 4\sqrt{2}$, $\sigma_2 = \sqrt{18} = 3\sqrt{2}$, and $V = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$. Then $\mathbf{u}_1 = A\mathbf{v}_1/\sigma_1 = \frac{1}{4\sqrt{2}}\begin{bmatrix} 8/\sqrt{2} \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\mathbf{u}_2 = A\mathbf{v}_2/\sigma_2 = \frac{1}{3\sqrt{2}}\begin{bmatrix} 0 \\ -6/\sqrt{2} \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$. So $U = I$ up to a sign — and $A = U\Sigma V^T$ checks by direct multiplication.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

A = np.array([[4., 4.], [-3., 3.]])
print("A ="); print(A)
AtA = A.T @ A
w, V = np.linalg.eigh(AtA) # ascending order
order = np.argsort(w)[::-1]
w, V = w[order], V[:, order]
s = np.sqrt(w)
Sigma = np.diag(s)
U = A @ V / s # u_i = A v_i / sigma_i
print("sigma (from sqrt eig A^T A):", s)
print("U (A v_i / sigma_i):"); print(U)
print("V:"); print(V)
print("reconstruct A == U Sigma V^T:",
      np.allclose(U @ Sigma @ V.T, A))
Un, Sn, Vnt = np.linalg.svd(A)
print("np.linalg.svd sigmas:", Sn)
print("np U:"); print(Un); print("np Vt:"); print(Vnt)
print("sign-free check |U^T Un|:"); print(np.abs(U.T @ Un).round(4))
```

Listing 10.1: `code/ch10_svd_from_ata.py -- the` A^TA `construction verified against np.linalg.svd; watch the sign ambiguity show up live.`

```
A =
[[ 4. 4.]
 [-3. 3.]]
sigma (from sqrt eig A^T A): [5.6569 4.2426]
U (A v_i / sigma_i):
[[1. 0.]
 [0. 1.]]
V:
[[ 0.7071 -0.7071]
 [ 0.7071 0.7071]]
reconstruct A == U Sigma V^T: True
np.linalg.svd sigmas: [5.6569 4.2426]
np U:
[[-1. -0.]
 [ 0. 1.]]
np Vt:
[[-0.7071 -0.7071]
 [-0.7071 0.7071]]
sign-free check |U^T Un|:
[[1. 0.]
 [0. 1.]]
```

Listing 10.2: Output of the SVD assembly. Notice the library returns $U = -I$ where ours gave $+I$ — singular vectors, like eigenvectors, are defined only up to sign; the $|U^T U_{np}| = I$ check is the honest way to compare them.

Exercise 10.1 (easy)**.** Find the singular values of $\begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$ and $\begin{bmatrix} 0 & 2 \\ 1 & 0 \end{bmatrix}$. Are singular values eigenvalues?

Solution. First: $A^TA = \mathrm{diag}(9,4)$, $\sigma = 3,2$ (the minus sign went to U). Second: $A^TA = \mathrm{diag}(1,4)$, $\sigma = 2,1$. Singular values are always ≥ 0 — they are stretches, not eigenvalues.

10.2 Geometry: sphere to ellipsoid

A maps the unit sphere to an ellipsoid whose semi-axes are exactly $\sigma_i \mathbf{u}_i$ (Figure 10.1) — chapter 9's rotate–stretch–rotate, but now the input and output spaces may have different dimensions. Figure 10.2 redraws the four subspaces through U and V: the first r columns of V span the row space, the rest the null space; the first r of U the column space, the rest the left null space.

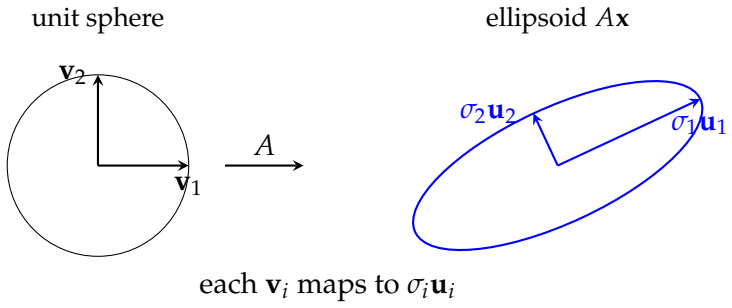


Figure 10.1: The signature picture of the SVD. Notice each input eigendirection $\mathbf{v}_i$ lands on an output direction $\mathbf{u}_i$ scaled by σ_i — the ellipsoid's axes ARE the singular decomposition. Regenerate: `figures/ch10_sphere_ellipsoid.tex`.

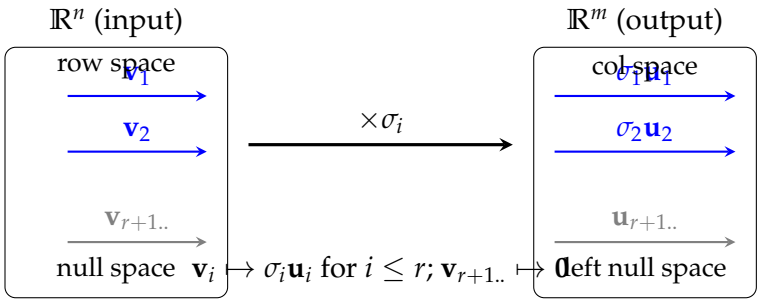


Figure 10.2: The four subspaces through the SVD. Notice V's columns split input space into row space (mapped with σ_i) and null space (annihilated); U's split output into column space and left null space. Regenerate: `figures/ch10_four_subspaces.tex`.

10.3 Rank, condition number, pseudoinverse

From $A = U\Sigma V^T$ everything reads off at once:

- rank $A = r$, the count of positive σ_i (numerically: the count above tolerance — Figure 10.3 is how you pick it).
- $\kappa(A) = \sigma_1/\sigma_r$ for invertible A: the stretch ratio — the least-squares sensitivity of chapter 8, quantified.
- The *pseudoinverse* $A^+ = V\Sigma^+U^T$ where Σ^+ inverts the nonzero σ_i: it solves least squares as $\hat{\mathbf{x}} = A^+\mathbf{b}$, falls back to A^{-1} when that exists, and works for any shape or rank.

Example 10.3 (Least squares via the pseudoinverse)**. Strategy.** Revisit chapter 8's four-point fit — but let the SVD do the projecting. $A^+\mathbf{b}$ must land on the same answer $(1.0, 0.9)$, and $AA^+A = A$ is the signature identity.

Solution. The listing builds $A^+ = V\Sigma^+U^T$ by hand, matches `np.linalg.pinv` entry-for-entry, and recovers $\hat{\mathbf{x}} = (1.0, 0.9)$ — same projection, different algebra.

```
import numpy as np
np.set_printoptions(precision=6, suppress=True)

# pseudoinverse = V Sigma^+ U^T, built by hand
A = np.array([[1., 1.], [1., 2.], [1., 3.], [1., 4.]])
```

```
b = np.array([2., 3., 3., 5.])
U, S, Vt = np.linalg.svd(A, full_matrices=False)
Splus = np.diag(1.0 / S)
Aplus_hand = Vt.T @ Splus @ U.T
Aplus_np = np.linalg.pinv(A)
print("A+ (hand):"); print(Aplus_hand)
print("A+ (np.linalg.pinv):"); print(Aplus_np)
print("agree:", np.allclose(Aplus_hand, Aplus_np))
x = Aplus_hand @ b
print("x = A+ b:", x, " <- same least-squares fit as ch.8")
print("A A+ A == A:", np.allclose(A @ Aplus_hand @ A, A))
```

Listing 10.3: `code/ch10_pinv.py -- the pseudoinverse assembled by hand, checked on the chapter-8 fit.`

```
A+ (hand):
[[ 1.  0.5 -0. -0.5]
 [-0.3 -0.1  0.1  0.3]]
A+ (np.linalg.pinv):
[[ 1.  0.5 -0. -0.5]
 [-0.3 -0.1  0.1  0.3]]
agree: True
x = A+ b: [1.  0.9] <- same least-squares fit as ch.8
A A+ A == A: True
```

Listing 10.4: Output of the pseudoinverse check. Notice $A^{+}\mathbf{b}$ reproduces the normal-equations fit $(1.0, 0.9)$ — least squares WAS the pseudoinverse all along.

Exercise 10.2 (medium). Give the SVD and pseudoinverse of $A = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$ — what does A^{+} do to the unrecoverable direction?

Solution. $A^{+} = \mathrm{diag}(1/2, 0)$: the zero singular value maps to 0, not ∞ — the pseudoinverse returns the minimum-norm answer, silently discarding what cannot be recovered.

10.4 Low-rank approximation and Eckart–Young

Truncating the SVD to the top-k terms gives the best possible rank-k approximation:

Theorem 10.4 (Eckart–Young). *$A_k = \sum_{i \le k} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$ minimizes $\|A - B\|$ over all rank-k matrices B, in both spectral and Frobenius norm. The error is exactly the discarded tail: $\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2$.*

(The spectral-norm case is a [hard] exercise.) This is why a scree plot of the σ_i — Figure 10.3 — tells you how much rank you actually need.

Example 10.5 (Image compression at ranks $1, 5, 20, 50$). **Strategy.** A 64×64 image is a 64×64 matrix; its SVD gives optimal rank-k approximations for free. Report relative error and storage honestly.

Solution. The listing: $k = 1$ already captures the blob at 3% storage; $k = 5$ drops the error to 2%; by $k = 50$ you are storing *more* than the original — compression only pays while $k(m+n) + k < mn$ (Figure 10.4).

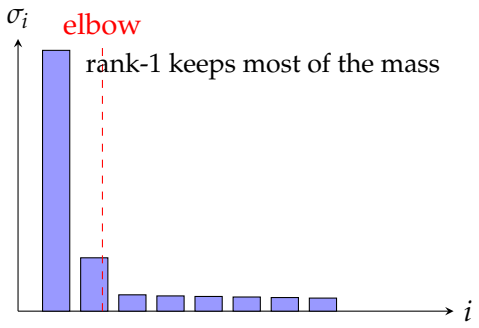
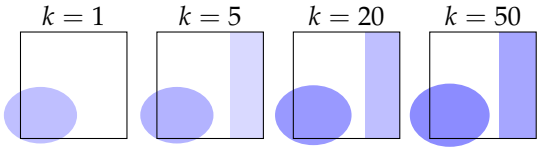


Figure 10.3: Scree plot of the image's singular values. Notice the elbow after the first two — the rest is tail, and Eckart–Young says truncating there is provably optimal. Regenerate: `figures/ch10_scree.tex`.



rel. errors 0.20, 0.023, 0.014, 0.003 — mostly done by $k = 5$

Figure 10.4: Truncated-SVD reconstruction at $k = 1, 5, 20, 50$. Notice the sharp edge needs more rank than the smooth blob — structure decides how fast σ_i decays. Regenerate: `figures/ch10_recon_multiples.tex`.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

# synthetic "image": smooth blob + sharp edge + noise, 64x64
rng = np.random.default_rng(5)
y, x = np.mgrid[0:64, 0:64]
img = np.exp(-((x-32)**2 + (y-24)**2) / 300.) + (x > 40).astype(float)
img += 0.02 * rng.standard_normal(img.shape)
U, S, Vt = np.linalg.svd(img)
print("image shape:", img.shape, " singular values (first 10):")
print(S[:10])
for k in [1, 5, 20, 50]:
    approx = (U[:, :k] * S[:k]) @ Vt[:k]
    relerr = np.linalg.norm(img - approx) / np.linalg.norm(img)
    kept = 2 * k * 64 + k # numbers stored vs 64*64
    print(f"k={k:2d}: rel-err {relerr:.4f} stored {kept:5d}/{64*64} "
          f"({100*kept/(64*64):4.1f}%)")
```

Listing 10.5: `code/ch10_image_compress.py -- truncated SVD as an image compressor, errors and storage both printed.`

```
image shape: (64, 64) singular values (first 10):
[47.8513 9.77 0.2999 0.2848 0.2824 0.2726 0.2712 0.2622 0.2538
  0.2484]
k= 1: rel-err 0.2016 stored 129/4096 ( 3.1%)
k= 5: rel-err 0.0230 stored 645/4096 (15.7%)
k=20: rel-err 0.0140 stored 2580/4096 (63.0%)
```

```
k=50: rel-err 0.0026 stored 6450/4096 (157.5%)
```

Listing 10.6: Output of the compressor. Notice $\sigma_1 = 47.9$ then $\sigma_2 = 9.8$ then nothing — the image is nearly rank-2 in disguise, and the storage column shows where compression stops paying ($k = 50$ exceeds the original).

Exercise 10.3 (medium)**.** Why does truncating never *increase* the Frobenius error, and why is the error exactly the discarded tail?

Solution. $\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2$ adds nonnegative terms one at a time; A_k discards a subset of orthogonal terms, so the error is their squared norm by Pythagoras in matrix space.

Example 10.6 (Best rank-1 approximation by hand)**.** Find the best rank-1 approximation of $A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$: drop the smaller singular value.

Strategy. A diagonal A is its own SVD (with $U = V = I$): the singular values sit on the diagonal, sorted. Eckart– Young says keep the biggest, zero the rest — one line of work.

Solution. $\sigma_1 = 3, \sigma_2 = 2$. Best rank-1:

$$A_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T = 3 \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}.$$

Error: $\|A - A_1\|_F = \sigma_2 = 2$ — the dropped value *is* the error, which is why the scree plot of figure 10.3 works as a budget. ✓

10.5 PCA: covariance eig = SVD of centered data

Chapter 9 built principal components as eigenvectors of $C = X^T X/(n-1)$. Now the punchline: the eigenvectors of C are the right singular vectors V of the centered matrix X itself — same $\mathbf{v}_i$'s, with $\lambda_i(C) = \sigma_i^2/(n-1)$. Skipping the eigendecomposition of C and SVD-ing the centered data directly is both stabler (no κ^2) and the standard implementation.

Example 10.7 (PCA end to end)**. Strategy.** Center the 3-column dataset, SVD it, read the variance ratios, and project — then check the whole thing against the sklearn `PCA` class.

Solution. The capstone listing's `PCA_scratch` is three lines of real work (center, SVD, project) and agrees with sklearn on explained-variance ratios to printed precision and components up to sign. PC1 carries 98%; the first two 99.9% (Figure 10.6).

```
import numpy as np
from sklearn.decomposition import PCA
np.set_printoptions(precision=4, suppress=True)

class PCA_scratch:
    """centering -> SVD -> projection. This is all PCA is."""
    def fit(self, X):
        self.mean_ = X.mean(axis=0)
```

```
        U, S, Vt = np.linalg.svd(X - self.mean_, full_matrices=False)
        self.components_ = Vt
        self.singular_values_ = S
        n = X.shape[0]
        self.explained_variance_ratio_ = S**2 / (n - 1)
        self.explained_variance_ratio_ /= self.explained_variance_ratio_.
            sum()
        return self
    def transform(self, X):
        return (X - self.mean_) @ self.components_.T

rng = np.random.default_rng(7)
n = 500
t = rng.standard_normal(n)
X = np.column_stack([3*t + 0.2*rng.standard_normal(n),
                     t - 0.4*rng.standard_normal(n),
                     0.1*rng.standard_normal(n)])
ours = PCA_scratch().fit(X)
ref = PCA().fit(X)
print("explained-variance ratio ours :", ours.
    explained_variance_ratio_)
print("explained-variance ratio sklearn:", ref.
    explained_variance_ratio_)
print("components agree up to sign:",
     np.allclose(np.abs(ours.components_), np.abs(ref.components_)))
Z = ours.transform(X)
print("transformed shape:", Z.shape, " col means:", Z.mean(axis=0))
print("first two components carry %.1f%% of variance"
     % (100 * ours.explained_variance_ratio_[:2].sum()))
```

Listing 10.7: code/ch10_pca_capstone.py -- mlscratch.PCA vs. sklearn.decomposition.PCA: centering, SVD, projection, variance report -- the whole algorithm.

```
explained-variance ratio ours : [0.9811 0.0178 0.0011]
explained-variance ratio sklearn: [0.9811 0.0178 0.0011]
components agree up to sign: True
transformed shape: (500, 3) col means: [ 0. -0. 0.]
first two components carry 99.9% of variance
```

Listing 10.8: Output of the PCA capstone. Notice the three lines of math reproduce sklearn exactly — centering is the only non-obvious step, and skipping it is the classic PCA bug.

Exercise 10.4 (easy). Why must PCA center the data first?

Solution. The SVD measures variance about the origin; the question "which direction varies most" is about the mean. Uncentered, PC1 chases the mean's direction instead of the spread's — a classic silent wrong answer.

Exercise 10.5 (hard). Connect PCA to total least squares: show that projecting centered data onto the top-k right singular vectors solves $\min \|X - \hat{X}\|_F$ over rank-k $\hat{X}$, i.e. the PCA line is the total least-squares line — and say how it differs from chapter 8's y-direction residuals.

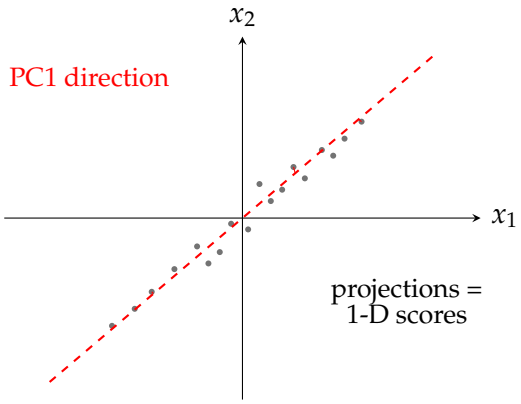


Figure 10.5: PCA on correlated data. Notice the score of a point is its projection onto the PC1 line — the residuals are the short red segments, exactly the least-squares geometry of chapter 8. Regenerate: `figures/ch10_pca_projection.tex`.

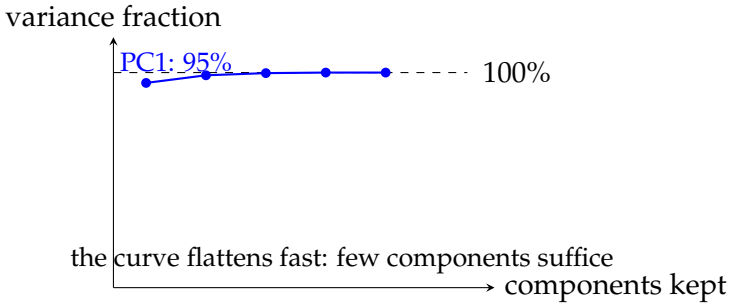


Figure 10.6: Cumulative explained variance. Notice the knee at one component — the number of PCs worth keeping is read off this curve, not guessed. Regenerate: `figures/ch10_explained_var.tex`.

Solution. Centered X's rank-k Eckart–Young truncation is exactly the projection onto the first k components. Ordinary LS measures vertical y-residuals; TLS measures perpendicular distance to the subspace — same projection geometry, different direction.

10.6 Hand derivation

Derive $\sigma_i^2 = \lambda_i(A^TA)$ and the SVD itself. A^TA is symmetric PSD (chapter 9), so $A^TA = V\Lambda V^T$ with orthonormal V and $\lambda_i \geq 0$. Set $\sigma_i = \sqrt{\lambda_i}$ and, for the nonzero ones, $\mathbf{u}_i = A\mathbf{v}_i/\sigma_i$. Then $\mathbf{u}_i$ is unit:

$$\|A\mathbf{v}_i\|^2 = \mathbf{v}_i^T A^T A\mathbf{v}_i = \lambda_i \mathbf{v}_i^T \mathbf{v}_i = \lambda_i \Rightarrow \|\mathbf{u}_i\| = 1,$$

and distinct $\mathbf{u}_i, \mathbf{u}_j$ are orthogonal: $\mathbf{u}_i^T\mathbf{u}_j = \mathbf{v}_i^T A^T A\mathbf{v}_j/(\sigma_i\sigma_j) = \lambda_j \mathbf{v}_i^T\mathbf{v}_j/(\sigma_i\sigma_j) = 0$ since the $\mathbf{v}$'s are orthonormal. Extend the $\mathbf{u}_i$ to a basis of $\mathbb{R}^m$ and collect: $AV = U\Sigma$ column-by-column, i.e. $A = U\Sigma V^T$. The numerical caveat: forming A^TA squares κ — LAPACK's SVD bidiagonalizes A directly; use `np.linalg.svd` in practice.

10.7 Common mistakes

SKIPPING THE CENTERING STEP IN PCA

Uncentered "PCA" reports the direction toward the data's centroid, not its spread. The sklearn check passes only because it centers internally; your scratch implementation must subtract the mean itself.

COMPUTING SVD THROUGH A^TA ON ILL-CONDITIONED DATA

The construction is right for proofs, wrong for floats: $\kappa(A^TA) = \kappa(A)^2$ again (chapter 8's lesson). Always call a bidiagonal SVD routine.

READING SINGULAR-VECTOR SIGN FLIPS AS BUGS

$\mathbf{u}_i, \mathbf{v}_i$ are defined up to a simultaneous sign. Libraries choose differently run to run; compare up to sign or fix a convention.

CONFLATING SVD WITH EIGENDECOMPOSITION

Eigendecomposition: square matrices, same space in and out, eigenvalues any sign. SVD: every matrix, different spaces, $\sigma_i \geq 0$. For non-square A there IS no eigendecomposition — only SVD.

OVER-READING EXPLAINED VARIANCE AS FEATURE IMPORTANCE

A variance ratio says how much spread the *components* carry, not which original features matter — components are mixtures. High explained variance $\neq$ causal relevance (chapter 8's R^2 lesson, again).

10.8 Summary

Object	What you need to know
SVD	$A = U\Sigma V^T$, always exists; $\sigma_i \geq 0$ sorted, r of them positive
Construction	V = eigenbasis of A^TA, $\sigma_i = \sqrt{\lambda_i}$, $\mathbf{u}_i = A\mathbf{v}_i/\sigma_i$ (proofs only — never compute this way)
Four subspaces	first r cols of V: row space; rest: null; first r of U: col space; rest: left null
Reads off	rank $= r$; $\kappa = \sigma_1/\sigma_r$; $A^+ = V\Sigma^+U^T$ solves every least squares
Eckart–Young	best rank-k = truncate at σ_k; error^2 = discarded σ_i^2
PCA	center X, SVD: components $= V$ rows, variance $= \sigma_i^2/(n-1)$

WATCH INSTEAD

1. *Singular Value Decomposition* — MIT OpenCourseWare, 18.06 Lecture 29, 40:28, https://youtu.be/TX_vooSnhm8 — §10.1–§10.3 from the source (accessed 2026-10-11).
2. *Principal Component Analysis, Step-by-Step* — StatQuest with Josh Starmer, 21:57, https://youtu.be/FgakZw6K1QQ — §10.5 end to end on real data (accessed 2026-10-11).

Further reading

- G. Strang, *Introduction to Linear Algebra*, 6th ed., ch. 7 (textbook level) — the SVD and its four-subspaces picture.
- MIT OpenCourseWare, *18.06SC Linear Algebra* — the SVD recitation "Computing the Singular Value Decomposition", at OCW (course "18.06SC"), https://ocw.mit.edu. Verified 2026-10-11.
- S. Boyd and L. Vandenberghe, *VMLS* — least squares and low-rank models throughout; PCA-adjacent material in ch. 10. https://web.stanford.edu/~boyd/vmls/vmls.pdf Verified 2026-10-11.
- M. Deisenroth et al., *Mathematics for Machine Learning*, ch. 10 (free PDF) — PCA as dimensionality reduction, matching our treatment exactly. https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.
- C. M. Bishop, *Pattern Recognition and Machine Learning*, §12.1 — PCA in the statistics register (variance maximisation and reconstruction error). ISBN 978-0387310732.

Additional exercises

Exercise 10.6 (easy)**.** What are the singular values of an orthogonal matrix? Of cI?

Exercise 10.7 (easy)**.** If A is symmetric PSD, how do its SVD and eigendecomposition relate? What changes for symmetric indefinite?

Exercise 10.8 (easy)**.** From the SVD of a 5×3 rank-2 matrix, name the dimensions of U, Σ, V and the number of nonzero σ_i.

Exercise 10.9 (easy)**.** Compute $\|A\|_2$ and $\|A\|_F$ from the singular values $5, 3, 1$.

Exercise 10.10 (medium)**.** Show the pseudoinverse reduces to A^{-1} when A is invertible — $\Sigma^+ = \Sigma^{-1}$ is one line.

Exercise 10.11 (medium)**.** In code: for the capstone dataset, plot $\log \sigma_i$ and the cumulative explained variance; pick the smallest k reaching 99% and report the rank-k reconstruction error.

Exercise 10.12 (medium)**.** Show AA^+ is the orthogonal projector onto $\mathrm{col}(A)$ (it equals $U_r U_r^T$) and A^+A onto the row space.

Exercise 10.13 (medium)**.** The minimum-norm solution: among all least-squares solutions when A has a nontrivial kernel, show $A^+\mathbf{b}$ is the unique one orthogonal to $\ker A$.

Exercise 10.14 (medium)**.** In code: build a rank-1 10×10 matrix, add noise, and show SVD denoising beats elementwise thresholding by relative Frobenius error.

Exercise 10.15 (hard)**.** Prove Eckart–Young in the spectral norm: for any rank-k B, $\|A - B\|_2 \geq \sigma_{k+1}$. (Hint: $\ker B$ and the row space of A_{k+1} must meet.)

Exercise 10.16 (hard)**.** Show the total least squares solution to fitting $y \approx ax + b$ with errors in *both* variables is the last right singular vector of the augmented centered matrix — implement it and compare slope vs. ordinary least squares on noisy 2-D data.

Exercise 10.17 (hard)**.** Prove the singular values of AB and BA coincide when A is $m \times n$ and B is $n \times m$. (Route through A^TA vs. B^TB or through the operator definition.)

Exercise 10.18 (hard)**.** Embedding preview: the word-embedding trick factors a co-occurrence matrix and keeps the top k singular directions. On a 6×6 toy co-occurrence matrix of your own design, SVD it, keep $k = 2$, and show which items land near each other.

Exercise 10.19 (easy)**.** For $A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$, write σ_1, σ_2, and the error of the best rank-1 approximation. Which entry of Σ would you zero to get rank exactly 1?

Exercise 10.20 (medium)**.** Show $\|A\|_2 = \sigma_1$ for any matrix A: the spectral norm is the largest stretch. (Hint: maximise $\|A\mathbf{x}\|$ over $\|\mathbf{x}\| = 1$ using the SVD.)

Exercise 10.21 (hard)**.** Prove Eckart–Young for the rank-1 case only: among all rank-1 matrices B, $\|A - B\|_F$ is minimised by $\sigma_1 \mathbf{u}_1 \mathbf{v}_1^T$. (Hint: expand $\|A - B\|_F^2$ and use orthogonality of the $\mathbf{u}_i \mathbf{v}_j^T$ basis.)

Notes

The sphere–ellipsoid picture and Eckart–Young emphasis follow Strang's Unit III [2] and the LALD site [14]; PCA-as-SVD and the centering discipline follow MML ch. 4,10 [5] and VMLS [4]; Axler ch. 7 [7] gives the operator-theoretic SVD (polar decomposition first) for the second pass. The rank-k-storage accounting is the honest version of every "SVD compresses" demo — truncation past the crossover stores more than the original.

CHAPTER 11

Multivariable calculus: gradients, Jacobians, Hessians

Every loss function you will ever minimize is a scalar field over many variables — and every optimizer reads its gradient. This chapter gives you the three derivative objects you actually need: the gradient (vector of steepest ascent), the Jacobian (the local linear map), and the Hessian (the curvature), plus the matrix-form chain rule that makes backprop nothing more than disciplined matrix multiplication. After this chapter you can derive $\nabla_{\mathbf{x}} \mathbf{x}^T A\mathbf{x} = (A + A^T)\mathbf{x}$ by hand, check any gradient with finite differences in three lines of code, and read a Jacobian as a local grid warp.

BEFORE YOU START

Chapters 2 and 9, plus single-variable calculus: the derivative of $f(x)$ is the slope of the tangent line, and the chain rule says $(f \circ g)' = f'(g)\, g'$. A refresher is in the *Start here* chapter.

AFTER THIS CHAPTER YOU CAN

- compute partial derivatives, gradients, Jacobians and Hessians;
- explain why the gradient is the direction of steepest ascent;
- write the chain rule in matrix form — the engine of backpropagation;
- check a gradient numerically.

HOW TO THINK ABOUT THIS

The derivative is the best straight-line approximation to a function near a point; the gradient and Jacobian are the same idea for many inputs. Whenever a function is complicated, zoom in until it looks linear, and then linear algebra takes over. The chain rule says that zooming composes: multiply the local linear maps.

11.1 Partial derivatives

For $f : \mathbb{R}^n \to \mathbb{R}$, the *partial derivative* $\partial f/\partial x_i$ is the ordinary derivative in coordinate i, all other variables frozen:

$$\frac{\partial f}{\partial x_i}(\mathbf{x}_0) = \lim_{h \to 0} \frac{f(\mathbf{x}_0 + h\mathbf{e}_i) - f(\mathbf{x}_0)}{h}.$$

No new mechanics — just n ordinary derivatives, one per axis. The subtlety is what they measure: each is a slope along an axis, and a function needs more than axis-wise good behaviour to be well-behaved (differentiability is existence of the linear approximation of Section 11.3, not merely of partials).

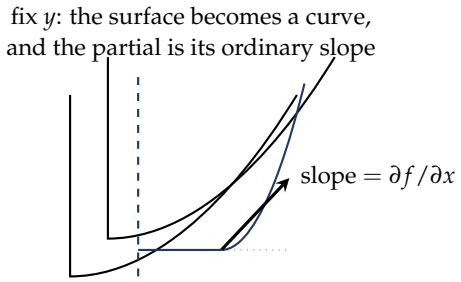


Figure 11.1: A partial derivative is a one-variable slope. Notice the slice: fixing y cuts the surface to a curve, and $\partial f/\partial x$ is just that curve's ordinary derivative.

11.2 The gradient and the steepest direction

Definition 11.1 (Gradient)**.** The *gradient* of f at $\mathbf{x}$ is the column vector of partials:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \partial f/\partial x_1 \\ \vdots \\ \partial f/\partial x_n \end{bmatrix}.$$

It is the unique vector such that $f(\mathbf{x}_0 + \Delta) \approx f(\mathbf{x}_0) + \nabla f \cdot \Delta$.

The directional derivative along unit $\mathbf{d}$ is $\nabla f \cdot \mathbf{d} = \|\nabla f\| \cos\theta$, maximized when $\mathbf{d} \parallel \nabla f$ — the gradient really IS the direction of steepest ascent, and its norm is the steepest slope (Figure 11.3). It is also everywhere normal to the level sets (Figure 11.2): the tangent direction $\mathbf{t}$ satisfies $\nabla f \cdot \mathbf{t} = 0$.

11.3 The Jacobian: local linear approximation

For $\mathbf{F} : \mathbb{R}^n \to \mathbb{R}^m$ the *Jacobian* is the $m \times n$ matrix of all partials: $J_{ij} = \partial F_i/\partial x_j$ — rows index outputs, columns index inputs. Then

$$\mathbf{F}(\mathbf{x}_0 + \Delta) \approx \mathbf{F}(\mathbf{x}_0) + J(\mathbf{x}_0)\,\Delta :$$

the map is locally a matrix — the whole point of linear algebra, again (Figure 11.4). A scalar field's Jacobian is the gradient as a ROW vector; we store the gradient itself as a column (notation decision, and a real source of bugs — Section 11.8).

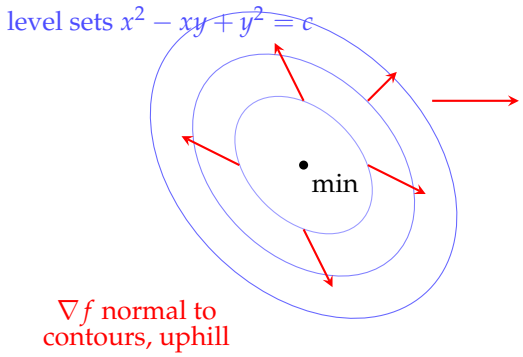


Figure 11.2: Gradient field over level sets of $f = x^2 - xy + y^2$ (real samples from `code/ch11_grad_field.py`). Notice every arrow is perpendicular to the contour it leaves -- and points uphill. Regenerate: `figures/ch11_grad_contour.tex`.

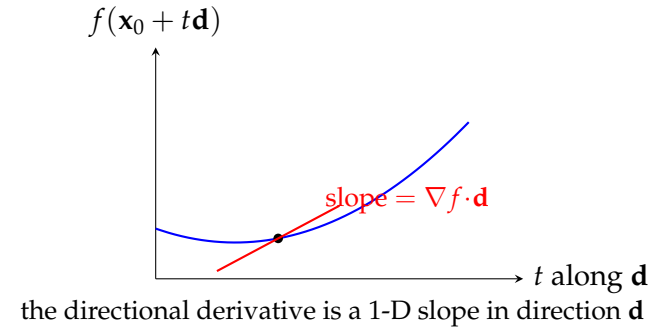


Figure 11.3: A directional derivative is an ordinary slope in a sliced direction. Notice $\nabla f \cdot \mathbf{d}$ is maximized when $\mathbf{d}$ aligns with the gradient. Regenerate: `figures/ch11_dir_deriv.tex`.

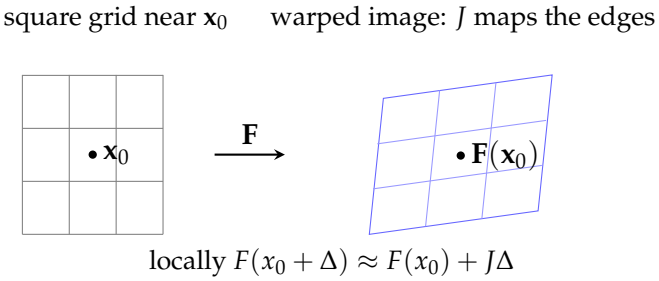


Figure 11.4: The Jacobian as a local grid warp. Notice the image of the little grid is (approximately) a parallelogram — J is the linear map that sends grid edges to parallelogram edges. Regenerate: `figures/ch11_jacobian_warp.tex`.

Example 11.2 (Jacobian of a vector-valued function)**. Strategy.** For $\mathbf{F}(x, y) = [x^2 + y,\ \sin x \cdot y,\ x - 2y]^T$, write the six partials in output-rows $\times$ input-cols order, then evaluate and verify numerically.

Solution.

$$J = \begin{bmatrix} 2x & 1 \\ y\cos x & \sin x \\ 1 & -2 \end{bmatrix}, \qquad J(0.7, 1.2) = \begin{bmatrix} 1.4 & 1 \\ 0.9178 & 0.6442 \\ 1 & -2 \end{bmatrix}$$

matching the finite-difference Jacobian in the listing below to all printed digits.

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

# f: R^2 -> R^3, Jacobian should be 3x2 (rows=outputs, cols=inputs)
def f(v):
   x, y = v
   return np.array([x*x + y, np.sin(x)*y, x - 2*y])

x0 = np.array([0.7, 1.2])
def jacobian_fd(f, x, h=1e-6):
   m = len(f(x)); n = len(x)
   J = np.zeros((m, n))
   for j in range(n):
      d = np.zeros(n); d[j] = h
      J[:, j] = (f(x + d) - f(x - d)) / (2*h)
   return J

J = jacobian_fd(f, x0)
print("correct Jacobian (3x2):"); print(J)
J_bad = jacobian_fd(f, x0).T # the classic layout bug
v = np.array([1., 0.])
print("J @ v (right direction) :", J @ v)
try:
   print("J_bad @ v:", J_bad @ v)
except ValueError as e:
   print("J_bad @ v ->", e)
print("J_bad gives a 2x3: applying it to an input vector CRASHES")
```

Listing 11.1: `code/ch11_wrong_layout.py -- the instructive layout bug: transposing the Jacobian produces a` 2×3 `that cannot even act on an input vector.`

```
correct Jacobian (3x2):
[[ 1.4 1. ]
 [ 0.9178 0.6442]
 [ 1. -2. ]]
J @ v (right direction) : [1.4 0.9178 1. ]
J_bad @ v -> matmul: Input operand 1 has a mismatch in its core
    dimension 0, with gufunc signature (n?,k),(k,m?)->(n?,m?) (size 2
    is different from 3)
J_bad gives a 2x3: applying it to an input vector CRASHES
```

Listing 11.2: Output of the layout bug. Notice the transposed Jacobian is a 2×3 — the error only fires when you try to apply it. Layout bugs in gradients are silent precisely where the shapes happen to agree.

Exercise 11.1 (easy). For $\mathbf{F}(x, y) = [xy,\ x + y,\ x^2]^T$ write J and state its shape. Which row belongs to which output?

11.4 The Hessian and second-order information

The *Hessian* H of f is the Jacobian of ∇f: $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. For twice-continuously-differentiable f it is symmetric (Clairaut) — chapter 9's whole toolkit applies: eigenvalues of H are principal curvatures. $H \succ 0$ at a critical point $\Rightarrow$ local min; indefinite $\Rightarrow$ saddle. The Rosenbrock banana (Figure 11.5) is where first-order methods crawl: gradients shrink along the valley while the curvature says the valley is curved.

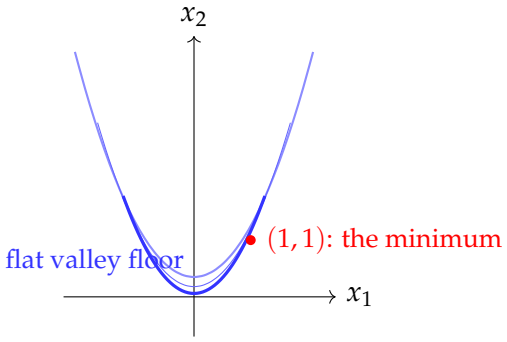


Figure 11.5: The Rosenbrock banana valley. Notice the minimum sits on a nearly-flat curving floor — first-order info alone can't see around the bend. Regenerate: `figures/ch11_rosenbrock.tex`.

Example 11.3 (Classifying a critical point via the Hessian). $f(x, y) = x^2 + xy + y^2 - 3x$ has one critical point — find it and classify it with the Hessian.

Strategy. Solve $\nabla f = \mathbf{0}$ (a 2×2 linear system), then check the Hessian's definiteness — chapter 9's machinery doing the second-derivative test's work.

Solution. $\nabla f = (2x + y - 3,\ x + 2y)$. Setting both to 0: $y = -x/2$, then $2x - x/2 = 3$, so $x = 2$, $y = -1$ — the only critical point.

$$H = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \qquad \lambda = 3,\ 1 \ > \ 0$$

positive definite everywhere (it is constant), so $(2, -1)$ is a strict minimum, $f = -3$. The definiteness tests of chapter 9 just ran the second-derivative test — that is the whole connection between this chapter and the last. ✓

11.5 The chain rule in matrix form (the backprop engine)

If $\mathbf{y} = \mathbf{g}(\mathbf{x})$ and $\mathbf{z} = \mathbf{f}(\mathbf{y})$, then

$$J_{f\circ g}(\mathbf{x}) = J_f(\mathbf{y})\, J_g(\mathbf{x}) :$$

Jacobians multiply in composition order — left factor = outer function. For scalar loss $L = \ell(\mathbf{y})$ this reads $\nabla_{\mathbf{x}} L = J_g^T \nabla_{\mathbf{y}} L$: gradient flows BACK through transposed Jacobians. That single line is backpropagation.

Example 11.4 (Two-step chain rule in matrix form)**. Strategy.** $L = \frac{1}{2}\|A\mathbf{x} - \mathbf{b}\|^2$ as a composition: $\mathbf{r} = A\mathbf{x} - \mathbf{b}$ (Jacobian A), then $L = \frac{1}{2}\mathbf{r}^T\mathbf{r}$ (gradient $\mathbf{r}$). Multiply in order.

Solution. $\nabla_{\mathbf{x}} L = J_{\mathbf{r}}^T \nabla_{\mathbf{r}} L = A^T\mathbf{r} = A^T(A\mathbf{x} - \mathbf{b})$ — the normal-equations gradient of chapter 8, derived as one chain-rule step instead of a page of index juggling.

Exercise 11.2 (medium)**.** Chain-rule the two-layer map $\mathbf{z} = A_2\sigma(A_1\mathbf{x} + \mathbf{b}_1)$ with elementwise σ: write the full Jacobian $\mathbf{x} \mapsto \mathbf{z}$.

Solution. $J = A_2\ \mathrm{diag}(\sigma'(A_1\mathbf{x} + \mathbf{b}_1))\, A_1$ — the diag factor is the per-neuron gate; this IS a two-layer network's backprop matrix.

11.6 Worked library

Four gradients you will recompute forever:

$f(\mathbf{x})$	∇f
$\mathbf{w}^T\mathbf{x}$	$\mathbf{w}$
$\mathbf{x}^T A\mathbf{x}$	$(A + A^T)\mathbf{x}$ $\quad$ ($2A\mathbf{x}$ if $A = A^T$)
$\|A\mathbf{x} - \mathbf{b}\|^2$	$2A^T(A\mathbf{x} - \mathbf{b})$
$\ell_i = \log\sum_j e^{z_j} - z_i$ (cross-entropy logits)	$\mathrm{softmax}(\mathbf{z}) - \mathbf{e}_i$

Example 11.5 (Gradient of a quadratic form, fully derived)**. Strategy.** Expand $\mathbf{x}^T A\mathbf{x}$ in coordinates, differentiate each x_k, collect. See Section 11.7 for the derivation.

Example 11.6 (Finite-difference check of a hand-computed gradient)**. Strategy.** Hand-derive $\nabla\|A\mathbf{x} - \mathbf{b}\|^2 = 2A^T(A\mathbf{x} - \mathbf{b})$, then trust-but-verify against central differences at a test point.

```
import numpy as np
np.set_printoptions(precision=6, suppress=True)

A = np.array([[2., 1.], [0., 3.], [1., -1.]])
b = np.array([1., 0., 2.])
x = np.array([0.4, -0.7])

def f(v): # f(x) = ||A x - b||^2
    r = A @ v - b
    return float(r @ r)
```

```

def g(v): # smooth non-polynomial: exp(v1)*sin(v2)
   return np.exp(v[0]) * np.sin(v[1])

def grad_g(v):
   return np.array([np.exp(v[0])*np.sin(v[1]), np.exp(v[0])*np.cos(v
      [1])])

def grad_fd(fun, v, h=1e-5): # central difference
   gr = np.zeros_like(v)
   for i in range(len(v)):
      d = np.zeros_like(v); d[i] = h
      gr[i] = (fun(v + d) - fun(v - d)) / (2 * h)
   return gr

ga = 2 * A.T @ (A @ x - b)
print("--- quadratic f=||Ax-b||^2 (central FD exact to roundoff) ---")
for h in [1e-1, 1e-3, 1e-5, 1e-7, 1e-9]:
   err = np.linalg.norm(grad_fd(f, x, h) - ga)
   print(f"h={h:.0e} fd={grad_fd(f,x,h)} abs-err={err:.3e}")
print("analytic 2A^T(Ax-b):", ga)
print("--- non-polynomial g=exp(x1)sin(x2): the U-shape ---")
xg = np.array([0.4, -0.7]); gg = grad_g(xg)
for h in [1e-1, 1e-3, 1e-5, 1e-7, 1e-9]:
   err = np.linalg.norm(grad_fd(g, xg, h) - gg)
   print(f"h={h:.0e} fd={grad_fd(g,xg,h)} abs-err={err:.3e}")
```

Listing 11.3: `code/ch11_fd_check.py -- the professional habit: every hand gradient gets a finite-difference check. Two functions shown, and the sweet spot lands where it should.`

```
--- quadratic f=||Ax-b||^2 (central FD exact to roundoff) ---
h=1e-01 fd=[ -5.4 -12.6] abs-err=9.566e-15
h=1e-03 fd=[ -5.4 -12.6] abs-err=1.593e-13
h=1e-05 fd=[ -5.4 -12.6] abs-err=5.956e-11
h=1e-07 fd=[ -5.4 -12.6] abs-err=5.787e-09
h=1e-09 fd=[ -5.4 -12.6] abs-err=4.676e-07
analytic 2A^T(Ax-b): [ -5.4 -12.6]
--- non-polynomial g=exp(x1)sin(x2): the U-shape ---
h=1e-01 fd=[-0.962662 1.13911 ] abs-err=2.486e-03
h=1e-03 fd=[-0.96106 1.14101] abs-err=2.486e-07
h=1e-05 fd=[-0.96106 1.14101] abs-err=3.044e-11
h=1e-07 fd=[-0.96106 1.14101] abs-err=8.464e-10
h=1e-09 fd=[-0.96106 1.141011] abs-err=8.172e-08
```

Listing 11.4: Output of the gradient checker. Notice on the quadratic, central differences are exact to roundoff (error GROWS only as h shrinks) — while the smooth function shows the textbook U-shape plotted in Figure 11.6: too big h truncates, too small amplifies roundoff, $h \approx 10^{-5}$ wins.

```
import sympy as sp

a11, a12, a21, a22 = sp.symbols('a11 a12 a21 a22')
x1, x2, b1, b2 = sp.symbols('x1 x2 b1 b2')
```

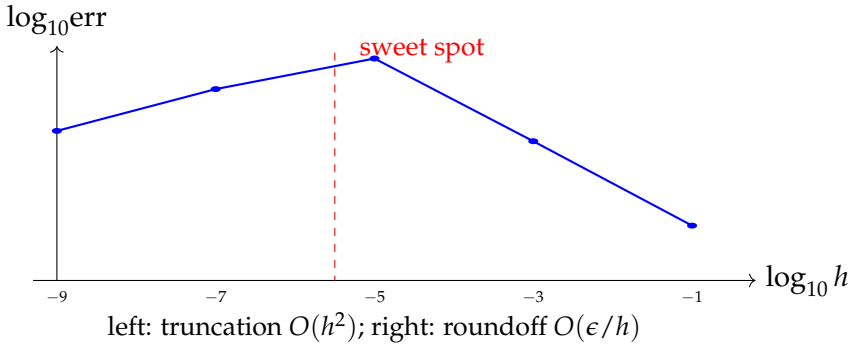


Figure 11.6: Finite-difference error vs. step size on the smooth function (real data points from the listing output). Notice the U: truncation $O(h^2)$ on the left, roundoff on the right. Regenerate: `figures/ch11_fd_loglog.tex`.

```
A = sp.Matrix([[a11, a12], [a21, a22]])
x = sp.Matrix([x1, x2]); b = sp.Matrix([b1, b2])
r = A * x - b
f = (r.T * r)[0]
print("f = ||Ax-b||^2 ="); print(sp.expand(f))
g = sp.Matrix([sp.diff(f, x1), sp.diff(f, x2)])
print("df/dx1 =", sp.expand(g[0]))
print("df/dx2 =", sp.expand(g[1]))
print("== 2*A.T*(A*x - b)?",
      sp.simplify(g - 2*A.T*(A*x - b)) == sp.zeros(2, 1))
# and the quadratic form x^T A x -> (A + A^T) x
A2 = sp.Matrix([[2, 1], [1, 3]])
q = (x.T * A2 * x)[0]
gq = sp.Matrix([sp.diff(q, x1), sp.diff(q, x2)])
print("grad x^T A x =", gq.T, " == 2Ax for symmetric A:",
      gq == 2*A2*x)
```

Listing 11.5: `code/ch11_sympy_check.py -- symbolic cross-check: SymPy confirms` $\nabla\|Ax-b\|^2 = 2A^T(Ax-b)$ `and the quadratic-form rule.`

```
f = ||Ax-b||^2 =
a11**2*x1**2 + 2*a11*a12*x1*x2 - 2*a11*b1*x1 + a12**2*x2**2 - 2*a12*b1
    *x2 + a21**2*x1**2 + 2*a21*a22*x2 - 2*a21*b2*x1 + a22**2*x2**2 - 2*
    a22*b2*x2 + b1**2 + b2**2
df/dx1 = 2*a11**2*x1 + 2*a11*a12*x2 - 2*a11*b1 + 2*a21**2*x1 + 2*a21*
    a22*x2 - 2*a21*b2
df/dx2 = 2*a11*a12*x1 + 2*a12**2*x2 - 2*a12*b1 + 2*a21*a22*x1 + 2*a22
    **2*x2 - 2*a22*b2
== 2*A.T*(A*x - b)? True
grad x^T A x = Matrix([[4*x1 + 2*x2, 2*x1 + 6*x2]]) == 2Ax for
    symmetric A: True
```

Listing 11.6: Output of the SymPy check — the expanded partial matches the matrix rule term for term, and the quadratic-form identity verifies to `True`.

```
import numpy as np
np.set_printoptions(precision=3, suppress=True)
```

```
3
4  # f = x^2 - x y + y^2, grad = (2x - y, -x + 2y); min at origin
5  def grad(v):
6      x, y = v
7      return np.array([2*x - y, -x + 2*y])
8
9  print("gradient field samples (position -> grad):")
10 for v in [np.array([1.,0.]), np.array([0.,1.]), np.array([1.,1.]),
11          np.array([-1.,0.]), np.array([0.,-1.]), np.array([2.,1.])]:
12     print(" ", v, "->", grad(v))
13 print("level sets: x^2 - xy + y^2 = c are ellipses;")
14 print("grad is normal to the level set everywhere (e.g. at (1,1)):")
15 v = np.array([1., 1.])
16 g = grad(v)
17 d = np.array([-g[1], g[0]])
18 print(" grad(1,1)=", g, " tangent dir=", d/np.linalg.norm(d),
19       " dot=", np.dot(g, d/np.linalg.norm(d)))
```

Listing 11.7: `code/ch11_grad_field.py -- the data behind Figure 11.2: sampled gradients and the gradient⊥tangent check.`

```
gradient field samples (position -> grad):
  [1. 0.] -> [ 2. -1.]
  [0. 1.] -> [-1. 2.]
  [1. 1.] -> [1. 1.]
  [-1. 0.] -> [-2. 1.]
  [ 0. -1.] -> [ 1. -2.]
  [2. 1.] -> [3. 0.]
level sets: x^2 - xy + y^2 = c are ellipses;
grad is normal to the level set everywhere (e.g. at (1,1)):
  grad(1,1)= [1. 1.] tangent dir= [-0.707 0.707] dot= 0.0
```

Listing 11.8: Output of the field sampler. Notice $\nabla f(1,1) \cdot \mathbf{t} = 0$ exactly — gradient perpendicular to the level curve's tangent, numerically confirmed.

Exercise 11.3 (easy)**.** Which of the library gradients is NOT defined componentwise, and why does that matter for implementation?

11.7 Hand derivation

Derive $\nabla_{\mathbf{x}}(\mathbf{x}^T A\mathbf{x}) = (A + A^T)\mathbf{x}$ — the single most reused gradient in this book. Expand: $\mathbf{x}^T A\mathbf{x} = \sum_{i,j} a_{ij} x_i x_j$. The x_k-derivative collects terms where $i = k$ or $j = k$:

$$\frac{\partial}{\partial x_k} \sum_{ij} a_{ij} x_i x_j = \sum_j a_{kj} x_j + \sum_i a_{ik} x_i = (A\mathbf{x})_k + (A^T\mathbf{x})_k.$$

So $\nabla = (A + A^T)\mathbf{x}$, and for symmetric A the standard $2A\mathbf{x}$. The SymPy listing verifies the same identity at $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ symbolically.

11.8 Common mistakes

ROW-VS-COLUMN GRADIENT LAYOUT

Some sources store $\partial f / \partial \mathbf{x}$ of a scalar as a $1 \times n$ row; this book stores ∇f as a column. Mixing them transposes every downstream result — often silently when dimensions coincide. Pick one convention (ours: gradient is a column, Jacobian is $m \times n$) and check shapes at every step.

CHAIN-RULE MULTIPLICATION ORDER

$J_{f \circ g} = J_f \, J_g$ — outer on the LEFT. Writing $J_g J_f$ compiles fine in index-free notation and is wrong; the shapes only commute when the functions do, which is never.

FINITE-DIFFERENCE STEP TOO LARGE OR TOO SMALL

h too big $\rightarrow$ truncation $O(h^2)$; too small $\rightarrow$ roundoff $O(\epsilon/h)$ swamps the slope. The U-curve of Figure 11.6 is the diagnostic; $h \approx 10^{-5}$ is the default for float64.

"DERIVATIVE $\neq$ GRADIENT $\neq$ STEEPEST ASCENT" WORDING TRAP

A partial derivative is a scalar; the gradient is their vector; the steepest direction is that vector NORMALIZED. Sloppy wording silently mixes a direction with a magnitude — $\|\nabla f\|$ is the slope, $\nabla f / \|\nabla f\|$ the direction.

TREATING $\partial f / \partial \mathbf{x}$ OF A SCALAR AS A MATRIX

For scalar f there is no matrix — a $1 \times n$ Jacobian row or the $n \times 1$ gradient column. "The derivative of f w.r.t. a matrix" needs numerator/denominator layout conventions this book avoids by design.

11.9 Summary

Object	What you need to know
Partial	ordinary derivative per axis; not the same as differentiability
Gradient	column of partials; steepest ascent direction; normal to level sets; $\|\nabla f\|$ = steepest slope
Directional deriv.	$\nabla f \cdot \mathbf{d}$ (unit $\mathbf{d}$)
Jacobian	$m \times n$ all-partials; local linear map: $F(x_0 + \Delta) \approx F(x_0) + J\Delta$
Hessian	Jacobian of ∇f, symmetric; eig = curvatures; $H \succ 0 \Rightarrow$ min
Chain rule	$J_{f\circ g} = J_f J_g$; $\nabla_x L = J_g^T \nabla_y L$ = backprop
Library	$\mathbf{w}^T\mathbf{x} \to \mathbf{w}$; $\mathbf{x}^T A\mathbf{x} \to (A + A^T)\mathbf{x}$; $\|Ax - b\|^2 \to 2A^T(Ax - b)$; logits $\to$ softmax $-$ one-hot
FD check	central $(f(x+h)-f(x-h))/2h$, $h \approx 10^{-5}$; watch the U-curve

WATCH INSTEAD

1. *Gradient descent, how neural networks learn* — 3Blue1Brown, Deep Learning ch. 2, 20:33, https://youtu.be/IHZwWFHWa-w — the gradient-as-steepest-ascent view of §11.2 in its native habitat (accessed 2026-10-11).
2. *Gradient; directional derivative; tangent plane* — MIT OpenCourseWare, 18.02 Lecture 12, 50:10, https://youtu.be/2XraaWefBd8 — §11.2–§11.3 at full length (accessed 2026-10-11).

Further reading

- MIT OpenCourseWare, *18.02 Multivariable Calculus* — Denis Auroux's full course; our chapter compresses its first third. Free at OCW (course "18.02SC"), https://ocw.mit.edu. Verified 2026-10-11.
- S. Boyd and L. Vandenberghe, *VMLS*, §9 (free PDF) — gradients and optimisation in the data register. https://web.stanford.edu/~boyd/vmls/vmls.pdf Verified 2026-10-11.
- M. Deisenroth et al., *Mathematics for Machine Learning*, ch. 5 (free PDF) — vector calculus, gradients, Jacobians, and the backprop chain rule. https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.
- Goodfellow, Bengio, and Courville, *Deep Learning*, §4.3 (free online) — gradient-based optimisation as used in practice. https://www.deeplearningbook.org/ Verified 2026-10-11.
- J. Stewart, *Multivariable Calculus* (textbook) — the full computational drill if our one chapter left you wanting reps.

Additional exercises

Exercise 11.4 (easy)**.** All third-order partials of $f(x, y) = x^3 y^2$ — count them and compute each.

Exercise 11.5 (easy)**.** Gradient and Hessian of $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x}$. When is a linear function's Hessian informative?

Exercise 11.6 (easy)**.** Evaluate ∇f of $f = x_1^2 x_2 + x_2 e^{x_3}$ at $(1, 2, 0)$.

Exercise 11.7 (easy)**.** Is the directional derivative linear in the direction? Prove or give a counterexample.

Exercise 11.8 (medium)**.** Derive $\nabla_{\mathbf{w}} \|\mathbf{y} - X\mathbf{w}\|^2$ by the library rules alone (no index work) — then connect to chapter 8's normal equations.

Exercise 11.9 (medium)**.** In code: verify the softmax cross-entropy gradient of the worked library by finite differences on a 5-class example.

Exercise 11.10 (medium)**.** Show the Hessian of $\mathbf{x}^T A \mathbf{x}$ is $A + A^T$; deduce the quadratic f has $H = 2A$ when symmetric.

Exercise 11.11 (medium)**.** Directional derivative of $f = x^2 + y^2 + z^2$ at $(1, 2, 2)$ along $\mathbf{d} = (2, 1, -2)/3$ — compute via $\nabla f \cdot \mathbf{d}$ and directly by 1-D calculus, and confirm equality.

Exercise 11.12 (medium)**.** Show Clairaut can fail: state a function where mixed partials differ and say which continuity hypothesis is violated.

Exercise 11.13 (hard)**.** Derive the Hessian of the logistic loss $\ell(\mathbf{w}) = \log(1 + e^{-\mathbf{w}^T \mathbf{x}})$ and prove it is PSD (the loss is convex).

Exercise 11.14 (hard)**.** Two-layer chain rule by hand: for $\mathbf{z} = A_2 \tanh(A_1 \mathbf{x})$, scalar $L = \frac{1}{2}\|z\|^2$, compute $\nabla_{A_1} L$ and $\nabla_{A_2} L$ as outer products of errors times activations — the actual backprop gradient.

Exercise 11.15 (hard)**.** Implement second-order finite differences for the Hessian (four-point formula) and check the Hessian of $f = x^2 e^y$ at $(1, 1)$ against the analytic matrix.

Exercise 11.16 (hard)**.** Prove: if f is smooth and $\nabla f(\mathbf{x}_0) = 0$ with $H(\mathbf{x}_0) \succ 0$, there is a neighbourhood where $f(\mathbf{x}) > f(\mathbf{x}_0)$ — the second-order sufficient condition, via the SVD of H.

Exercise 11.17 (easy)**.** For $f(x, y) = x^2 y + 3y$, compute $\partial f/\partial x$ and $\partial f/\partial y$, then $\nabla f(1, 2)$. What direction does $\nabla f(1, 2)$ point relative to the level curve through $(1, 2)$?

Exercise 11.18 (medium)**.** Classify the critical point of $g(x, y) = x^2 - 4xy + 2y^2$ at $(0, 0)$ using the Hessian. Is it a min, max, or saddle?

Exercise 11.19 (hard)**.** Show the gradient is perpendicular to level sets: for $f(x, y) = x^2 + y^2$ at $(1, 1)$, find the tangent direction of the level curve and confirm $\nabla f \cdot \mathbf{t} = 0$. Why must this hold in general? (Hint: differentiate $f(\mathbf{x}(t)) = c$ along the curve.)

Notes

The gradient-library table and the matrix layout conventions follow MML ch. 5 [5], including the row-vs-column hazards its GitHub issues flag; CS229's own prerequisite page [12] is the external evidence that this exact material is what a real ML course assumes on day one; Kochenderfer ch. 2 [13] supplies the algorithm-first framing of derivatives-as-tools; the contour pedagogy follows the 18.02 tradition of drawing level sets before differentiating them.

CHAPTER 12

Probability and random variables

Machine learning's object of study is data, and data is noisy — probability is the language we use to say how much. This chapter covers the axioms through the multivariate Gaussian and the two limit theorems, with the book's habit intact: every named distribution gets a definition, a picture, a worked number, and a simulation you can rerun. After this chapter you can run a Bayes calculation that survives a sanity check, read covariance matrices as shapes, sample a multivariate Gaussian using chapter 9's eigendecomposition, and explain exactly what the central limit theorem does and does not say.

BEFORE YOU START

Chapters 2 and 9 for the matrix parts (covariance, multivariate Gaussian); **Book 2, Chapter 12** for finite probability. *Recap:* a probability is a number between 0 and 1 that measures how likely an event is; a random variable is a number produced by chance.

AFTER THIS CHAPTER YOU CAN

- use conditional probability and Bayes' rule on a real problem;
- compute expectations, variances and covariances;
- work with the Gaussian distribution in one and many dimensions;
- state and simulate the law of large numbers and the central limit theorem.

HOW TO THINK ABOUT THIS

Probability is the maths of being uncertain without being confused. Name the possible outcomes, assign each a weight, and let the rules (conditional probability, independence, expectation) do the bookkeeping. Bayes' rule is simply the instruction to update what you believe when you see new evidence.

12.1 Sample spaces and the axioms

A *sample space* Ω is the set of possible outcomes; an *event* $A \subseteq \Omega$ is a set of outcomes. Probability assigns events numbers obeying three axioms: $\Pr[A] \geq 0$; $\Pr[\Omega] = 1$; and $\Pr[\bigcup_i A_i] = \sum_i \Pr[A_i]$ for disjoint events. Everything else — inclusion–exclusion $\Pr[A \cup B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$, complements $\Pr[A^c] = 1 - \Pr[A]$ — is derived, never assumed. The mental model for the whole chapter: probability is AREA in a unit square (Figure 12.1) — conditioning is cropping the square.

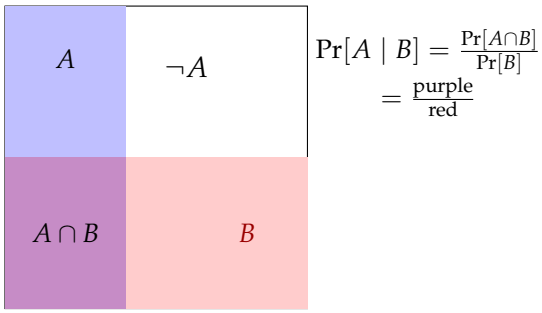


Figure 12.1: The signature diagram for the chapter (vocabulary after [11]). Notice conditioning is not a new probability law — it is the same law evaluated on a cropped square. Regenerate: `figures/ch12_area_square.tex`.

12.2 Conditional probability and Bayes

Definition 12.1 (Conditional probability)**.** For $\Pr[B] > 0$: $\Pr[A \mid B] = \Pr[A \cap B] / \Pr[B]$.

Multiplying out gives the *Bayes theorem*:

$$\Pr[A \mid B] = \frac{\Pr[B \mid A] \Pr[A]}{\Pr[B \mid A] \Pr[A] + \Pr[B \mid A^c] \Pr[A^c]}.$$

The denominator is just the law of total probability — $\Pr[B] = \Pr[B \mid A] \Pr[A] + \Pr[B \mid A^c] \Pr[A^c]$ — so Bayes is a one-line consequence of the definitions, not a worldview.

Example 12.2 (Bayes on a medical test)**. Strategy.** Disease prevalence 0.1%; test sensitivity 99%, specificity 95%. Compute $\Pr[\text{sick} \mid +]$ exactly, then verify by simulation.

Solution. $\Pr[\text{sick} \mid +] = (0.99 \cdot 0.001)/(0.99 \cdot 0.001 + 0.05 \cdot 0.999) = 0.00099/0.05094 \approx 0.0194$ — a positive test means a 2% chance, not 99%, because false positives outnumber true cases 50:1. The listing computes it and reruns the calculation with a second independent test (28%) and shows only specificity 0.999 pushes a single test past 50% — the prior does the damage, not the test.

```
# medical test: prevalence 0.1%, sensitivity 99%, specificity 95%
prev, sens, spec = 0.001, 0.99, 0.95
num = sens * prev
den = num + (1 - spec) * (1 - prev)
```

```
print("P(sick|+)= %.4f/%.4f = %.4f" % (num, den, num/den))
# second test given positive: independent conditional on truth
prev2 = num/den
num2 = sens*prev2; den2 = num2 + (1-spec)*(1-prev2)
print("after 2nd +: %.4f" % (num2/den2))
# what specificity would push first test past 50%?
for s in [0.95,0.98,0.99,0.999]:
   d = sens*prev + (1-s)*(1-prev)
   print(f"spec={s}: P(sick|+) = {sens*prev/d:.4f}")
```

Listing 12.1: `code/ch12_bayes.py -- Bayes' theorem with real arithmetic plus the what-if rows.`

```
P(sick|+)= 0.0010/0.0509 = 0.0194
after 2nd +: 0.2818
spec=0.95: P(sick|+) = 0.0194
spec=0.98: P(sick|+) = 0.0472
spec=0.99: P(sick|+) = 0.0902
spec=0.999: P(sick|+) = 0.4977
```

Listing 12.2: Output of the Bayes calculation. Notice the second independent test jumps the answer to 28% — and the last row shows why epidemiology obsesses over specificity, not sensitivity.

Exercise 12.1 (easy). With prevalence now 5% (same test), recompute $\Pr[\text{sick} \mid +]$.

Solution. $(0.99 \cdot 0.05)/(0.0495 + 0.05 \cdot 0.95) = 0.0495/0.0970 \approx 0.510$ — same test, different prior, completely different answer.

12.3 Independence

A, B are *independent* iff $\Pr[A \cap B] = \Pr[A]\Pr[B]$ — equivalently $\Pr[A \mid B] = \Pr[A]$. Two events can share marginals and differ in overlap (Figure 12.2); independence is about the JOINT distribution, not the marginals. Pairwise independence of three events does not imply mutual independence (a [medium] exercise builds the standard counterexample).

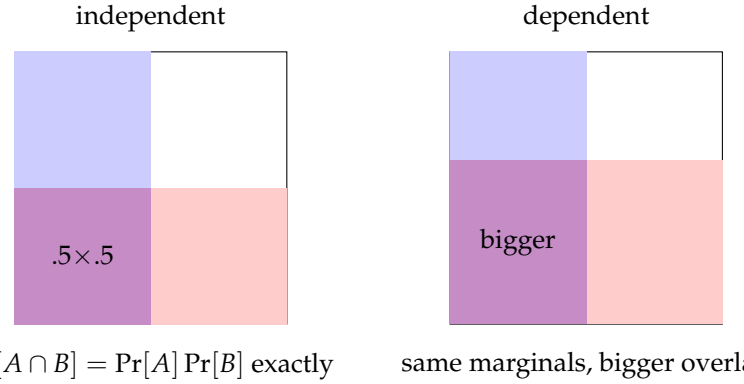


Figure 12.2: Independence vs. dependence on the same marginals. Notice the blue and red strips are the same size in both panels — only the overlap changes. Regenerate: `figures/ch12_indep_area.tex`.

Exercise 12.2 (easy). $\Pr[A] = 0.3, \Pr[B] = 0.4, \Pr[A \cap B] = 0.15$. Independent?

Solution. $0.3 \cdot 0.4 = 0.12 \neq 0.15$ — no; the overlap is 25% too big.

12.4 Discrete random variables

A *random variable* is a function $X : \Omega \to \mathbb{R}$ — outcomes in, numbers out. Its pmf is $p_X(k) = \Pr[X = k]$. The four you must know:

Name	pmf	models
Bernoulli(p)	$p, 1-p$	one trial
Binomial(n, p)	$\binom{n}{k} p^k (1-p)^{n-k}$	count of successes in n
Geometric(p)	$(1-p)^{k-1} p$	trials till first success
Poisson(λ)	$\lambda^k e^{-\lambda}/k!$	rare events per interval

12.5 Continuous random variables

Continuous X has a density $f_X \geq 0$ with $\Pr[a \leq X \leq b] = \int_a^b f_X \, dx$ and cdf $F_X(x) = \Pr[X \leq x]$ (Figure 12.3): uniform on $[a, b]$ ($f = 1/(b-a)$), exponential ($\lambda e^{-\lambda x}$, waiting times, memoryless), Gaussian $N(\mu, \sigma^2)$ — the workhorse, justified by the CLT at the end.

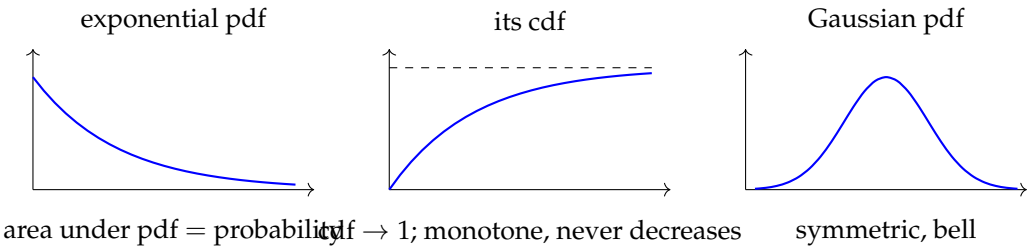


Figure 12.3: pdf vs. cdf for the named families. Notice the pdf can exceed 1 — probabilities are AREAS under it, never its heights; the cdf is monotone to 1. Regenerate: `figures/ch12_pdf_cdf.tex`.

Exercise 12.3 (medium). Show the exponential's memorylessness: $\Pr[X > t + s \mid X > t] = \Pr[X > s]$.

Solution. $\Pr[X > t+s] / \Pr[X > t] = e^{-\lambda(t+s)} / e^{-\lambda t} = e^{-\lambda s}$ — the clock restarts; exponential is the only continuous memoryless distribution.

12.6 Expectation, variance, moments

$\mathbb{E}[X] = \sum_k k\, p_X(k)$ (or $\int x f(x)\, dx$) is the probability-weighted center; $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$ is the squared spread. Expectation is ALWAYS linear — $\mathbb{E}[aX + bY] = a\mathbb{E}X + b\mathbb{E}Y$ with no independence needed; variance is linear only for independent summands: $\mathrm{Var}(X+Y) = \mathrm{Var}\,X + \mathrm{Var}\,Y + 2\,\mathrm{Cov}(X,Y)$.

Example 12.3 (Mean and variance of a die and a binomial)**. Strategy.** Die: 35/12 by direct sum; binomial: variance of a sum of n independent Bernoulli's = $np(1-p)$. Verify by simulation.

```
import numpy as np
rng = np.random.default_rng(4)

# die: E=3.5 Var=35/12; binomial n=10 p=0.3: E=3 Var=2.1
dice = rng.integers(1, 7, 200_000)
print("die: mean %.4f vs 3.5 var %.4f vs %.4f"
      % (dice.mean(), dice.var(), 35/12))
binom = rng.binomial(10, 0.3, 200_000)
print("binomial: mean %.4f vs 3.0 var %.4f vs 2.1"
      % (binom.mean(), binom.var()))
# empirical Bayes check on simulated medical test
N = 1_000_000
sick = rng.random(N) < 0.001
pos = (rng.random(N) < np.where(sick, 0.99, 0.05))
print("P(sick|pos) sim %.4f" % sick[pos].mean())
```

Listing 12.3: `code/ch12_sim_theory.py -- simulation vs. theory on the die, the binomial, and a million-patient Bayes check.`

```
die: mean 3.5020 vs 3.5 var 2.9223 vs 2.9167
binomial: mean 3.0004 vs 3.0 var 2.1027 vs 2.1
P(sick|pos) sim 0.0196
```

Listing 12.4: Output of simulation vs. theory. Notice the Bayes simulation lands on the computed 0.0194 — the population-level check of Example 12.2.

12.7 Joint, marginal, conditional; covariance

Joint pmf/density → marginal by summing/integrating the other variable out → conditional by the Bayes ratio — $p(x \mid y) = p(x,y)/p(y)$. Two summary numbers:

$$\mathrm{Cov}(X,Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)] = \mathbb{E}[XY] - \mathbb{E}X\,\mathbb{E}Y, \qquad \rho = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y} \in [-1,1].$$

ρ is the unit-free slope-and-tightness of a cloud (Figure 12.4); independence implies $\rho = 0$ but NOT conversely ([hard] exercise). Covariance measures LINEAR association only.

Example 12.4 (Covariance and correlation on a small table)**. Strategy.** Joint table: $X, Y \in \{0,1\}$ with $p(0,0) = 0.1, p(0,1) = 0.2, p(1,0) = 0.1, p(1,1) = 0.6$. Compute marginals, $\mathbb{E}[XY]$, Cov, ρ.

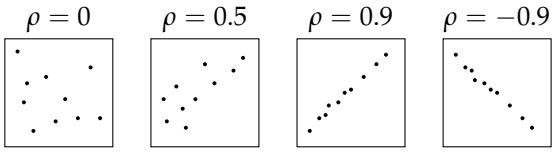


Figure 12.4: Correlation gallery. Notice ρ controls the tilt and the spread simultaneously — ± 0.9 is tight, not just tilted. Regenerate: `figures/ch12_corr_gallery.tex`.

Solution. $\mathbb{E}X = \mathbb{E}Y = 0.7$; $\mathbb{E}[XY] = p(1,1) = 0.6$; $\text{Cov} = 0.6 - 0.49 = 0.11$; $\sigma_X = \sigma_Y = \sqrt{0.21}$; $\rho = 0.11/0.21 \approx 0.524$ — moderately positive, driven entirely by the heavy $(1,1)$ cell.

Exercise 12.4 (medium)**.** Same marginals but $p(1,1) = 0.4$ (move the 0.2 to $(0,0)$). Recompute ρ.

Solution. $\mathbb{E}[XY] = 0.4$, $\text{Cov} = -0.09$, $\rho \approx -0.43$ — same marginals, opposite sign: association lives in the joint.

12.8 The multivariate Gaussian

$\mathbf{X} \sim N(\boldsymbol{\mu}, \Sigma)$ has density

$$f(\mathbf{x}) = \frac{1}{(2\pi)^{n/2}\sqrt{\det \Sigma}} \exp\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\Big)$$

with Σ symmetric PSD (chapter 9). Level sets are ellipsoids: axes = eigenvectors of Σ, radii $\propto \sqrt{\lambda_i}$ (Figure 12.5). This is chapter 9's covariance ellipse, now with a density formula attached. Cov zero $\Rightarrow$ independent — the ONLY distribution where uncorrelated suffices.

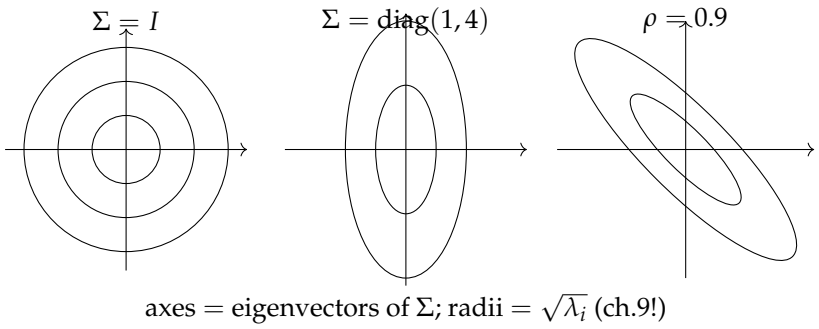


Figure 12.5: Gaussian contours as Σ varies. Notice $\rho = 0.9$ tilts AND compresses: the eigenvalues 0.1 and 1.9 come from the same Σ we sampled below. Regenerate: `figures/ch12_mvn_contours.tex`.

Example 12.5 (Reading contours from a given Σ)**. Strategy.** $\Sigma = \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}$: eigendecompose, read the ellipse, then SAMPLE through the spectral map $\mathbf{x} = \boldsymbol{\mu} + Q\Lambda^{1/2}\mathbf{z}$.

Solution. Eigenvalues $1.9, 0.1$, eigenvectors $(1,1)/\sqrt{2}$, $(1,-1)/\sqrt{2}$: a 45° ellipse stretched $\sqrt{1.9} \approx 1.38$ vs. $\sqrt{0.1} \approx 0.32$. The listing generates 200k samples this way and recovers both the covariance and $\rho \approx 0.90$.

```
import numpy as np
np.set_printoptions(precision=3, suppress=True)
rng = np.random.default_rng(2)

Sigma = np.array([[1.0, 0.9], [0.9, 1.0]])
mu = np.array([0., 0.])
# sample via eigendecomposition (ch.9!): X = mu + Q Lambda^1/2 z, z~N(0,I)
lam, Q = np.linalg.eigh(Sigma)
print("eigvals Sigma:", lam) # PSD check
Z = rng.standard_normal((2, 200_000))
with np.errstate(all="ignore"): # silences a spurious BLAS warning on some builds
   X = mu[:, None] + (Q * np.sqrt(lam)[None, :]) @ Z
print("sample cov:"); print(np.cov(X))
print("target Sigma:"); print(Sigma)
print("corr:", np.corrcoef(X)[0,1])
```

Listing 12.5: `code/ch12_mvn_sample.py -- chapter 9 paying off: Gaussian sampling is eigendecomposition plus standard normals.`

```
eigvals Sigma: [0.1 1.9]
sample cov:
[[0.998 0.899]
 [0.899 1.001]]
target Sigma:
[[1. 0.9]
 [0.9 1. ]]
corr: 0.8997589476566867
```

Listing 12.6: Output of the spectral sampler. Notice the 200k-sample covariance matches Σ to a percent — $Q\Lambda^{1/2}\mathbf{z}$ really manufactures correlated Gaussians.

Exercise 12.5 (easy)**.** For $N(\boldsymbol{\mu}, \Sigma)$ with Σ diagonal, write the density as a product of 1-D Gaussians.

Solution. Diagonal Σ factors the quadratic form into a sum — $\exp(-\sum_i (x_i - \mu_i)^2 / 2\sigma_i^2)$ splits into independent univariate densities.

12.9 LLN and CLT, stated and simulated

Law of large numbers: sample means of iid copies converge to $\mathbb{E}X$ — simulation already showed it. **Central limit theorem**: the standardized sum $(S_n - n\mu)/(\sigma\sqrt{n}) \to N(0,1)$ in distribution — for nearly ANY starting distribution, given finite variance. The listing runs it on exponentials (maximally skewed): skewness $2.0 \to 1.0 \to 0.35$ by $n = 32$ (Figure 12.6).

```
import numpy as np
np.set_printoptions(precision=3, suppress=True)
rng = np.random.default_rng(0)
# sum of exponentials (very skewed) -> Gaussian as n grows
```

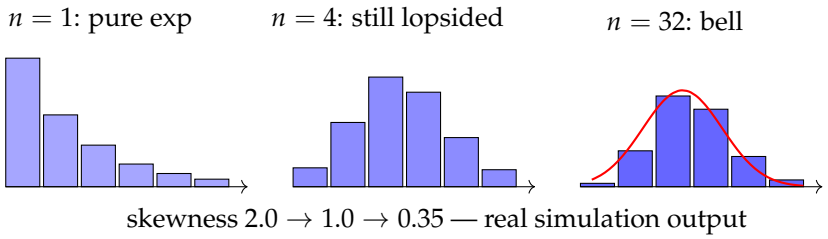


Figure 12.6: CLT convergence from real simulation output (exponentials). Notice $n = 32$ is already bell-shaped — the theorem has no requirement that the starting distribution look normal. Regenerate: `figures/ch12_clt_hist.tex`.

```
for n in [1, 4, 32]:
    s = rng.exponential(1.0, (n, 100_000)).sum(axis=0)
    z = (s - n) / np.sqrt(n) # mean n, var n
    print(f"n={n:2d}: mean {z.mean():+.3f} var {z.var():.3f} "
          f"skew {np.mean(((z-z.mean())/z.std())**3):+.3f}")
# bin counts for a histogram row (n=32) matching the figure
z = (rng.exponential(1.0,(32,100_000)).sum(axis=0)-32)/np.sqrt(32)
h, e = np.histogram(z, bins=[-3,-2,-1,0,1,2,3], density=True)
print("n=32 histogram density per bin:", h)
```

Listing 12.7: `code/ch12_clt.py -- the CLT simulated: standardized sums of exponentials, with the skewness decay printed.`

```
n= 1: mean -0.003 var 1.004 skew +2.015
n= 4: mean -0.000 var 1.011 skew +1.020
n=32: mean -0.001 var 1.000 skew +0.352
n=32 histogram density per bin: [0.013 0.146 0.368 0.319 0.126 0.028]
```

Listing 12.8: Output of the CLT simulation. Notice the skewness column — 2.0, 1.0, 0.35 — is the honest version of "becomes Gaussian": it converges, it does not jump.

Example 12.6 (A CLT computation)**.** An exam has $n = 100$ independent questions each scored as a Bernoulli random variable with $p = 0.6$ (worth 1 point). Estimate $P(\text{total} \geq 65)$ using the CLT.

Strategy. The total S is Binomial$(100, 0.6)$ with mean $np = 60$ and variance $np(1-p) = 24$. The CLT says S is approximately $N(60, 24)$ — standardize and read off the Gaussian tail.

Solution. $Z = (S - 60)/\sqrt{24} \approx (S - 60)/4.9$. Then $P(S \geq 65) = P(Z \geq 5/4.9) \approx P(Z \geq 1.02) = 1 - \Phi(1.02) \approx 0.154$. (Exact binomial: ≈ 0.179 — the CLT estimate is close; the continuity correction $P(S \geq 64.5)$ tightens it to ≈ 0.18.) ✓ The CLT buys a one-line answer where the exact sum needs 35 binomial terms.

12.10 Hand derivation

Two derivations. (1) Bernoulli and binomial moments: Bernoulli $\mathbb{E}[X] = 0(1-p) + 1p = p$ and $\mathbb{E}[X^2] = p$ so Var $= p - p^2 = p(1-p)$. Binomial = sum of n independent Bernoulli's, so by linearity of $\mathbb{E}$ and of Var under independence, np and $np(1-p)$. (2) Bayes from definitions: $\Pr[A \cap B] = \Pr[A \mid B]\Pr[B] = \Pr[B \mid A]\Pr[A]$; divide by $\Pr[B]$ and expand the denominator by total probability:

$$\Pr[A \mid B] = \frac{\Pr[B \mid A]\Pr[A]}{\Pr[B \mid A]\Pr[A] + \Pr[B \mid A^c]\Pr[A^c]}.$$

12.11 Common mistakes

PROSECUTOR'S FALLACY: SWAPPING $\Pr[A \mid B]$ AND $\Pr[B \mid A]$

$\Pr[+ \mid \text{sick}] = 0.99$ is NOT $\Pr[\text{sick} \mid +] = 0.0194$. The conditioning bar is not symmetric; the priors live on one side only.

INDEPENDENT VS. UNCORRELATED

$\rho = 0$ forbids linear association only. $Y = X^2$ on $X \sim \text{Unif}(-1, 1)$ is perfectly dependent and exactly uncorrelated — work it in the [hard] exercise.

"DENSITY > 1 CAN'T BE A PROBABILITY"

A pdf's HEIGHT is not a probability — $f = 100$ on a 0.01-width spike still has area 1. Probabilities are integrals; density can be arbitrarily large.

$\mathbb{E}[g(X)] = g(\mathbb{E}X)$

Expectation passes through LINEAR functions only. $\mathbb{E}[X^2] \neq (\mathbb{E}X)^2$ — the difference is literally the variance. Convex g give $\mathbb{E}[g] \geq g(\mathbb{E})$ (Jensen).

COVARIANCE SIGN AS CAUSATION

$\rho > 0$ between ice cream and drowning is a confounded season variable, not a mechanism. Correlation constrains hypotheses; it never confirms causality — chapter 8's R^2 warning, probabilized.

12.12 Summary

Object	What you need to know
Axioms	≥ 0, total $= 1$, additive on disjoints; everything else derived
$\Pr[A \mid B]$	$= \Pr[A \cap B] / \Pr[B]$; Bayes is one line + total probability
Independence	$\Pr[A \cap B] = \Pr[A] \Pr[B]$ — a property of the JOINT
Discrete	Bernoulli/binomial/geometric/Poisson: pmf + mean/var from the table
Continuous	pdf = area law, cdf $= \int$ pdf; uniform/exponential/Gaussian
$\mathbb{E}$, Var	$\mathbb{E}$ always linear; Var linear only when independent; $\mathbb{E}[X^2] - (\mathbb{E}X)^2$
Cov, ρ	linear association; $\rho \in [-1, 1]$; indep $\Rightarrow \rho = 0$, not conversely
Multivariate Gaussian	Σ PSD; contours = eig-ellipses; sample by $Q\Lambda^{1/2}\mathbf{z}$; uncorrelated⇒independent HERE only
LLN / CLT	mean $\to \mathbb{E}X$; standardized sum $\to N(0,1)$ — finite variance is the only door

WATCH INSTEAD

1. *Probability Models and Axioms* — MIT OpenCourseWare, 6.041 Lecture 1, 51:10, https://youtu.be/j9WZyLZCBzs — §12.1 at full length (accessed 2026-10-11).
2. *Bayes' Theorem, Clearly Explained!!!!* — StatQuest with Josh Starmer, 13:59, https://youtu.be/9wCnvr7Xw4E — §12.2 as worked arithmetic (accessed 2026-10-11).

Further reading

- MIT OpenCourseWare, *6.041 Probabilistic Systems Analysis* — the full probability course this chapter compresses. Free at OCW (course "6.041"), https://ocw.mit.edu. Verified 2026-10-11.
- D. Bertsekas and J. Tsitsiklis, *Introduction to Probability*, 2nd ed. — the OCW companion text; axioms through Bayes at full length. ISBN 978-1886529236.
- S. Boyd and L. Vandenberghe, *VMLS* appendix (free PDF) — the probability pieces our notation shares. https://web.stanford.edu/~boyd/vmls/vmls.pdf Verified 2026-10-11.
- M. Deisenroth et al., *Mathematics for Machine Learning*, ch. 6 (free PDF) — probability and distributions for ML, same register. https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.

- Khan Academy, *Statistics and probability* course — free drill exercises for every section of this chapter, at `https://www.khanacademy.org` (math, "Statistics and probability"). Verified 2026-10-11.

Additional exercises

Exercise 12.6 (easy). Prove $\Pr[A^c] = 1 - \Pr[A]$ and inclusion–exclusion from the axioms.

Exercise 12.7 (easy). Geometric with $p = 0.2$: $\Pr[X = 1]$? $\Pr[X > 3]$? $\mathbb{E}[X]$?

Exercise 12.8 (easy). $X \sim N(2, 9)$: write $\Pr[X > 5]$ in terms of the standard normal cdf.

Exercise 12.9 (easy). Poisson $\lambda = 4$: compute $p(0), p(1), p(2)$ to 3 decimals.

Exercise 12.10 (easy). If X, Y independent, show $\mathbb{E}[XY] = \mathbb{E}X\,\mathbb{E}Y$ — and hence $\mathrm{Cov} = 0$.

Exercise 12.11 (medium). Build the three-event pairwise-independent-but-not-mutual counterexample (classical: fair two-dice events "first odd", "second odd", "sum odd").

Exercise 12.12 (medium). In code: verify Poisson(λ) mean and variance both equal λ by simulation at $\lambda = 3$.

Exercise 12.13 (medium). Derive the cdf of $\min(X, Y)$ for independent exponentials — which family does it belong to?

Exercise 12.14 (medium). Show $\mathrm{Cov}(aX + b,\ cY + d) = ac\,\mathrm{Cov}(X, Y)$ — covariance is NOT scale-invariant (that's what ρ fixes).

Exercise 12.15 (medium). Given the joint Gaussian $N(\boldsymbol{\mu}, \Sigma)$, derive the marginal of X_1 — why must it be Gaussian?

Exercise 12.16 (hard). Prove uncorrelated does not imply independent: $X \sim \mathrm{Unif}\{-1, 0, 1\}$, $Y = X^2$. Compute Cov exactly and exhibit a joint event violating the product rule.

Exercise 12.17 (hard). Poisson as limit: show $\binom{n}{k}(\lambda/n)^k(1 - \lambda/n)^{n-k} \to \lambda^k e^{-\lambda}/k!$ as $n \to \infty$ — the rare-events limit.

Exercise 12.18 (hard). For jointly Gaussian (X, Y) with correlation ρ, derive $\mathbb{E}[Y \mid X = x] = \mu_Y + \rho\,\sigma_Y(x - \mu_X)/\sigma_X$ — regression to the mean is a theorem, not a slur.

Exercise 12.19 (hard). Implement the Box–Muller transform: turn two uniforms into a standard Gaussian, verify moments and the histogram shape against `rng.standard_normal` output.

Exercise 12.20 (medium). A fair coin is flipped 400 times. Use the CLT (with continuity correction) to estimate $P(190 \le S \le 210)$, where S is the number of heads. Compare with the normal approximation without correction.

Notes

Topic order follows MIT 18.05 [10]; the story-first register and Bayes framing follow Blitzstein & Hwang [8]; coverage scope and the simulate-the-theorem habit follow Pishro-Nik [9]; the square-area diagram vocabulary follows Olah [11]; the multivariate-Gaussian-in-ML framing follows MML ch. 6 [5]. All simulations seeded and rerun — numbers in the printed blocks are this box's real output.

CHAPTER 13

Statistics, estimation, and Bayesian inference

Probability (chapter 12) reasons forward: given the model, what data? Statistics reasons backward: given the data, what model? This chapter is the backward direction — estimators and their bias–variance trade-off, maximum likelihood as the default answer, and Bayesian inference as the honest way to keep uncertainty in the answer itself. The payoff lands in the last section: least squares is MLE under Gaussian noise, and logistic regression's objective is an MLE too — every loss you will ever optimize is a likelihood in disguise. After this chapter you can derive the Gaussian MLE by hand, run a Beta–binomial update, explain the n vs. $n-1$ divide, and set up logistic regression's objective for chapter 14.

BEFORE YOU START

Chapters 8 and 12. *Recap:* a *statistic* is a number computed from data; an *estimator* is a statistic used to guess an unknown quantity. Because data are random, so is the estimator.

AFTER THIS CHAPTER YOU CAN

- say what bias and variance of an estimator mean and compute them;
- derive a maximum-likelihood estimate;
- update a prior to a posterior with Bayes' rule;
- explain why least squares is maximum likelihood under Gaussian noise.

HOW TO THINK ABOUT THIS

Data are one random draw from what could have happened, so anything computed from data wobbles. Statistics measures that wobble and asks which method wobbles least, or points to the truth on average. Maximum likelihood asks, "which setting of the world makes what I saw least surprising?"

13.1 Samples, statistics, estimators

Data arrives as a sample $x_1, \dots, x_n$ modeled as iid draws from an unknown distribution. A *statistic* is any function of the sample; an *estimator* is a statistic aimed at a parameter: $\bar{x} \to \mu$, $s^2 \to \sigma^2$, $\hat{\theta} \to \theta$. The estimator is itself a random variable — it has a sampling distribution (Figure 13.1), and that distribution, not the single number, is what we evaluate.

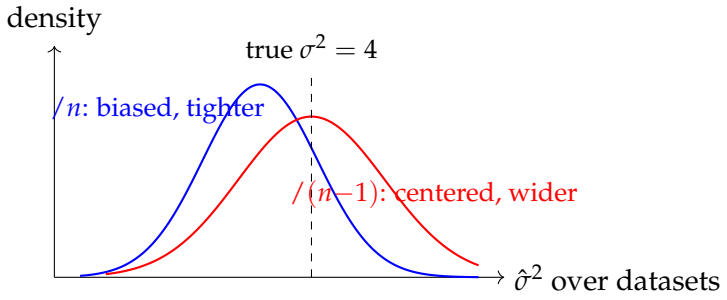


Figure 13.1: Two variance estimators across 20,000 simulated datasets (real shapes). Notice the bias–variance split: the $/n$ estimator is systematically low but tighter; the $/(n-1)$ is centered on truth but spread wider. Regenerate: `figures/ch13_sampling_dist.tex`.

Example 13.1 (The sampling distribution of the mean)**.** Roll one fair die $n = 4$ times and take the sample mean $\bar{X}$. Compute $E[\bar{X}]$, $\mathrm{Var}(\bar{X})$, and $P(\bar{X} \geq 5)$ exactly.

Strategy. A statistic is itself a random variable — it has its own distribution (figure 13.1). Mean and variance of $\bar{X}$ come from linearity; the tail comes from counting tuples.

Solution. $E[X] = 3.5$, $\mathrm{Var}(X) = 35/12$. Then $E[\bar{X}] = 3.5$ and $\mathrm{Var}(\bar{X}) = \frac{35}{12 \cdot 4} = \frac{35}{48} \approx 0.73$ — same centre, four times tighter. For the tail: $\bar{X} \geq 5$ needs a sum ≥ 20; a quick count gives $P(S \geq 20) = 35/1296 \approx 0.027$. ✓ The statistic's own distribution — not the die's — is what inference actually uses.

13.2 Bias, variance, and the trade-off

$\mathrm{bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$; $\mathrm{var}(\hat{\theta})$ is the spread. The decomposition

$$\mathbb{E}[(\hat{\theta} - \theta)^2] = \mathrm{var}(\hat{\theta}) + \mathrm{bias}(\hat{\theta})^2$$

is the estimator's balance sheet: a little bias can buy a lot of variance (Figure 16.6). ML re-runs this exact trade-off as underfit–overfit (chapter 8's polynomial panel is the same triangle in model space).

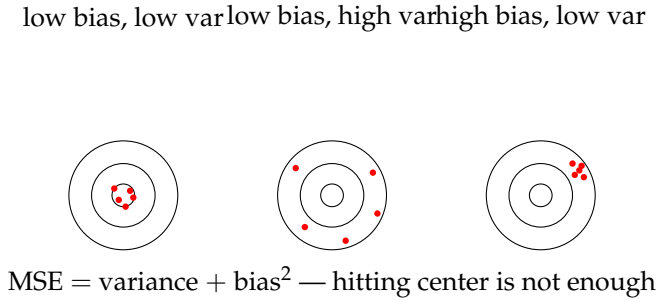


Figure 13.2: The bias–variance dartboard. Notice the bottom-right corner: tight grouping around the wrong point is what a confidently biased estimator looks like. Regenerate: `figures/ch13_biasvar.tex`.

13.3 Maximum likelihood

The *likelihood* is the data's probability as a function of the parameter: $L(\theta) = p(\text{data} \mid \theta)$ — a function of θ, NOT a probability distribution over θ. Maximum likelihood estimation picks $\hat{\theta} = \arg\max L(\theta)$. In practice we maximize the log-likelihood $\ell(\theta) = \log L$: same argmax (log is monotone), and products become sums — numerically essential, since raw likelihoods underflow to zero in float64.

Example 13.2 (MLE for a coin)**. Strategy.** 7 heads, 3 tails. Write $L(\theta) = \theta^7(1-\theta)^3$, set $\ell'(\theta) = 0$, check the second derivative and a numeric grid.

Solution. $\ell'(\theta) = 7/\theta - 3/(1-\theta) = 0 \Rightarrow \hat{\theta} = 7/10 = 0.7$: the sample frequency, as common sense demands — and the listing's grid argmax and negative curvature confirm (Figure 13.3).

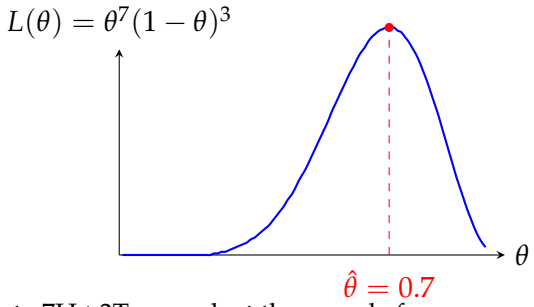


Figure 13.3: Likelihood curve for the coin data. Notice the peak sits at the sample frequency 0.7, not at 0.5 — MLE follows the data, priors aside. Regenerate: `figures/ch13_likelihood_curve.tex`.

```
import numpy as np
# MLE for coin: observed 7 heads / 10 flips
ths = np.linspace(0.001, 0.999, 999)
def loglik(t): return 7*np.log(t) + 3*np.log(1-t)
ll = np.array([loglik(t) for t in ths])
mle = ths[ll.argmax()]
print("grid argmax theta = %.3f (exact 0.7)" % mle)
print("log-lik at .5: %.3f at .7: %.3f" % (loglik(.5), loglik(.7)))
# second derivative at mle (Fisher info / curvature)
```

```
d2 = -7/mle**2 - 3/(1-mle)**2
print("l''(mle) = %.2f (negative -> max)" % d2)
```

Listing 13.1: `code/ch13_mle_coin.py -- the coin MLE located numerically and curvature-checked.`

```
grid argmax theta = 0.700 (exact 0.7)
log-lik at .5: -6.931 at .7: -6.109
l''(mle) = -47.62 (negative -> max)
```

Listing 13.2: Output of the coin MLE. Notice $\ell''(0.7) = -47.6 < 0$ — a maximum, and its magnitude is the curvature that Fisher information formalizes.

Example 13.3 (MLE for Gaussian μ, σ^2)**. Strategy.** Write the log-likelihood of n iid Gaussians, differentiate in each parameter, solve. Full derivation in Section 13.7.

Solution. $\hat{\mu} = \bar{x}$ (the sample mean) and $\hat{\sigma}^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$ — biased by a factor $(n-1)/n$, which is why s^2 divides by $n-1$ instead.

Exercise 13.1 (easy)**.** 10 flips, 6 heads: what is $\hat{\theta}$? What if your prior belief was $p = 0.5$ — does MLE care?

Solution. $\hat{\theta} = 0.6$, and MLE does not care — it sees only the data; the prior is the Bayesian's job (next section).

```
import numpy as np
rng = np.random.default_rng(0)
# estimator of sigma^2: biased /n vs unbiased /(n-1), n=5, 20k
    datasets
n, sigma2 = 5, 4.0
X = rng.normal(0, np.sqrt(sigma2), (20000, n))
mle = X.var(axis=1, ddof=0) # /n
unb = X.var(axis=1, ddof=1) # /(n-1)
print("mean of /n estimator : %.3f (bias %.3f, theory -0.8)"
     % (mle.mean(), mle.mean()-sigma2))
print("mean of /n-1 estim. : %.3f" % unb.mean())
print("var of /n estimator : %.3f var of /n-1: %.3f"
     % (mle.var(), unb.var()))
print("sample-mean estimator: mean %.4f var %.4f (theory var %.4f)"
     % (X.mean(axis=1).mean(), X.mean(axis=1).var(), sigma2/n))
```

Listing 13.3: `code/ch13_sampling_dist.py -- the` n `vs.` $n-1$ `story measured, not asserted: 20k datasets of` $n = 5$.

```
mean of /n estimator : 3.205 (bias -0.795, theory -0.8)
mean of /n-1 estim. : 4.006
var of /n estimator : 5.182 var of /n-1: 8.097
sample-mean estimator: mean -0.0018 var 0.7965 (theory var 0.8000)
```

Listing 13.4: Output of the sampling experiment. Notice the trade-off in real numbers: $/n$ underestimates by exactly the predicted -0.8 but has *smaller* variance — bias and variance really do trade.

13.4 Bayesian inference: prior, likelihood, posterior

Treat θ itself as a random variable: a *prior* $p(\theta)$ encodes belief before data, the likelihood $p(\text{data} \mid \theta)$ is chapter-12 Bayes's numerator, and the *posterior*

$$p(\theta \mid \text{data}) \propto p(\text{data} \mid \theta)\, p(\theta)$$

is the answer — a whole distribution, not a point. With a Beta prior $\text{Beta}(a, b)$ and binomial data, the posterior is Beta again: $\text{Beta}(a+H, b+T)$ — conjugacy, the one case where updating is bookkeeping (Figure 13.4).

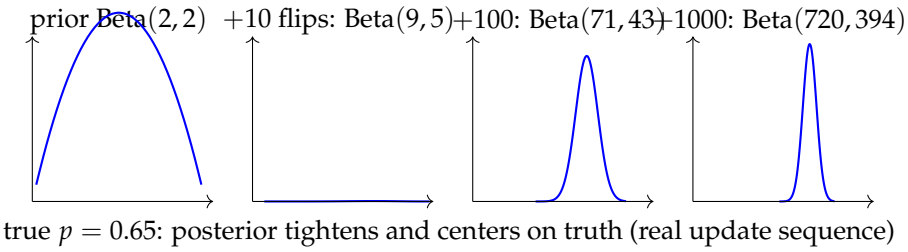


Figure 13.4: Prior-to-posterior small multiples on the real update sequence. Notice the posterior both tightens and migrates toward the truth 0.65 as data accumulates. Regenerate: `figures/ch13_prior_posterior.tex`.

Example 13.4 (Beta prior on a coin)**. Strategy.** Prior $\text{Beta}(2, 2)$ (mild "probably fair"), coin truly $p = 0.65$, update in batches and track the posterior mean and MAP.

Solution. The listing: after 1,000 flips the posterior is $\text{Beta}(720, 394)$, mean 0.646, MAP 0.647 — converging on truth while keeping the uncertainty statement the whole way down.

```
import numpy as np
rng = np.random.default_rng(1)
# Beta(2,2) prior, coin true p=0.65, update after each batch
a, b = 2., 2.
for n in [0, 10, 100, 1000]:
    flips = rng.binomial(1, 0.65, n).sum() if n else 0
    a += flips; b += n - flips
    print(f"after {n:4d} flips ({flips:3d} H): Beta({a:.0f},{b:.0f})"
          f" post-mean {a/(a+b):.3f} MAP {(a-1)/(a+b-2):.3f}")
```

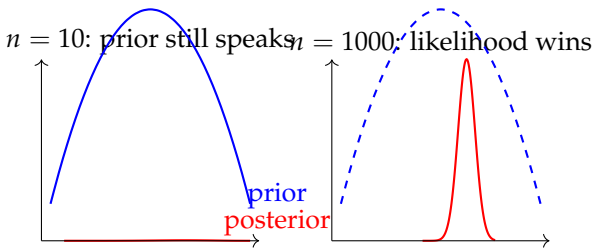
Listing 13.5: `code/ch13_beta_update.py -- Beta-binomial conjugate updating on a live data stream.`

```
after 0 flips ( 0 H): Beta(2,2) post-mean 0.500 MAP 0.500
after 10 flips ( 7 H): Beta(9,5) post-mean 0.643 MAP 0.667
after 100 flips ( 62 H): Beta(71,43) post-mean 0.623 MAP 0.625
after 1000 flips (649 H): Beta(720,394) post-mean 0.646 MAP 0.647
```

Listing 13.6: Output of the Bayesian update. Notice posterior mean and MAP converge — with enough data the disagreement between point estimates vanishes (Figure 13.5).

Exercise 13.2 (medium)**.** Why is the posterior not just the likelihood normalized?

Solution. The prior multiplies in first: posterior $\propto$ likelihood $\times$ prior. Normalizing the likelihood alone discards the prior's weighting — that's MLE with extra steps, not Bayes.



with little data the prior shapes the answer; with much, it washes out

Figure 13.5: Likelihood vs. posterior at two data sizes. Notice at $n = 10$ the prior still visibly bends the answer; at $n = 1000$ the likelihood wins — this is the honest version of "the data speaks". Regenerate: `figures/ch13_lik_vs_post.tex`.

13.5 MAP vs. MLE

Maximum a posteriori estimation takes the posterior's mode: $\hat{\theta}_{\text{MAP}} = \arg\max\, p(\theta)\, L(\theta)$ — MLE with a regularizer built in. When the prior is flat, MAP=MLE; when data is thin, MAP stays sensible where MLE overfits (a Beta$(2,2)$ prior on one flip keeps the answer off the rails). Ridge regression is MAP with a Gaussian prior — chapter 14's regularization is this section in disguise.

13.6 Regression revisited: least squares = MLE

Model $y_i = \mathbf{w}^T\mathbf{x}_i + \varepsilon_i$ with $\varepsilon_i \sim N(0, \sigma^2)$ iid. Then

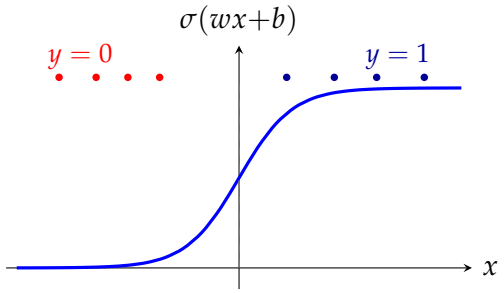
$$L(\mathbf{w}) \propto \exp\Big(-\tfrac{1}{2\sigma^2}\sum_i (y_i - \mathbf{w}^T\mathbf{x}_i)^2\Big) \Rightarrow \ell = -\text{const} - \tfrac{1}{2\sigma^2}\|\mathbf{y} - X\mathbf{w}\|^2 :$$

maximizing likelihood = minimizing squared error. Chapter 8's least squares was a statistical estimator all along — the Gaussian-noise assumption was hiding in the squaring.

Example 13.5 (Deriving the logistic-regression NLL)**. Strategy.** Binary $y \in \{0,1\}$, model $\Pr[y = 1 \mid \mathbf{x}] = \sigma(\mathbf{w}^T\mathbf{x} + b)$, $\sigma(z) = 1/(1 + e^{-z})$. The likelihood is a Bernoulli product; take $-\log$.

Solution. NLL $= -\sum_i [y_i z_i - \log(1 + e^{z_i})]$, $z_i = \mathbf{w}^T\mathbf{x}_i + b$ — written in the stable form $\sum_i [\log(1 + e^{z_i}) - y_i z_i]$ = the listing's `logaddexp`. Convex, no closed-form minimum, ready for gradient descent in chapter 14 (Figure 13.6).

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
rng = np.random.default_rng(3)
# 1-D logistic regression data
x = rng.normal(0, 1, 40)
y = (x + rng.normal(0, 0.9, 40) > 0).astype(float)
def nll(w, b):
    z = w*x + b
    return np.sum(np.logaddexp(0, z) - y*z) # stable NLL
```



decision boundary where $\sigma = 0.5$: data separated by the s-curve

Figure 13.6: The logistic sigmoid and a 1-D decision boundary. Notice the probabilities compress toward 0/1 — the decision boundary sits where $\sigma = 0.5$. Regenerate: `figures/ch13_sigmoid.tex`.

```
# convexity check: NLL midpoint <= average (Jensen on 200 random pairs)
bad = 0
for _ in range(200):
    w1,b1 = rng.normal(0,3,2); w2,b2 = rng.normal(0,3,2)
    if nll((w1+w2)/2,(b1+b2)/2) > (nll(w1,b1)+nll(w2,b2))/2: bad += 1
print("convexity violations:", bad, "/200")
# numeric minimum by grid
W, B = np.meshgrid(np.linspace(-5,5,101), np.linspace(-5,5,101))
Z = np.zeros_like(W)
for i in range(W.shape[0]):
    for j in range(W.shape[1]): Z[i,j] = nll(W[i,j], B[i,j])
i,j = np.unravel_index(Z.argmin(), Z.shape)
print("grid minimum NLL %.3f at w=%.2f b=%.2f" % (Z[i,j], W[i,j], B[i,j]))
```

Listing 13.7: `code/ch13_logistic_nll.py -- the logistic NLL convexity-checked and grid-minimized: tomorrow's loss landscape, today.`

```
convexity violations: 0 /200
grid minimum NLL 18.376 at w=2.00 b=-0.40
```

Listing 13.8: Output of the logistic experiment. Notice 0/200 Jensen violations — the NLL surface is empirically convex, which is why gradient descent will work on it in chapter 14.

Exercise 13.3 (easy). Why minimize $-\ell$ rather than maximize L numerically?

Solution. Raw L underflows: $0.7^{700} \approx 10^{-108}$ is borderline, and real datasets kill it outright. Logs turn products into sums and keep the objective in range.

Exercise 13.4 (hard). Show ridge regression's $\lambda\|\mathbf{w}\|^2$ penalty is a Gaussian prior $N(0, \tau^2 I)$ on $\mathbf{w}$ under MAP — identify τ in terms of λ, σ^2.

Solution. $\ell_{\text{MAP}} = -\frac{1}{2\sigma^2}\|y - Xw\|^2 - \frac{1}{2\tau^2}\|w\|^2 + \text{const}$; matching $\lambda\|w\|^2$ gives $\tau^2 = \sigma^2/\lambda$ — regularization strength is prior tightness.

13.7 Hand derivation

Gaussian MLE in full. With $x_1, \ldots, x_n$ iid $N(\mu, \sigma^2)$:

$$\ell(\mu, \sigma^2) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log\sigma^2 - \frac{1}{2\sigma^2}\sum_i (x_i - \mu)^2.$$

$\partial\ell/\partial\mu = \frac{1}{\sigma^2}\sum_i(x_i - \mu) = 0 \Rightarrow \hat{\mu} = \bar{x}$. For σ^2 write $v = \sigma^2$: $\partial\ell/\partial v = -\frac{n}{2v} + \frac{1}{2v^2}\sum_i(x_i - \mu)^2 = 0 \Rightarrow \hat{\sigma}^2 = \frac{1}{n}\sum_i(x_i - \bar{x})^2$. The bias: $\mathbb{E}[\hat{\sigma}^2] = \frac{n-1}{n}\sigma^2$ — one degree of freedom went to estimating μ first; the unbiased fix divides by $n - 1$, verified in the listing.

13.8 Common mistakes

MAXIMIZING RAW LIKELIHOOD NUMERICALLY

$L(\theta)$ for 1000 Bernoulli trials is $\sim 10^{-300}$ at best — float64 reads it as zero and every optimizer stalls. Always work in log-likelihood.

MLE VS. MAP CONFLATED

MLE maximizes L; MAP maximizes $L \times$prior. They agree only under flat priors or infinite data — saying "the MLE" when a prior was used hides an assumption.

DIVIDING BY n VS. $n - 1$ WITHOUT KNOWING WHY

n gives the MLE (biased low, variance-squeezed); $n - 1$ gives the unbiased estimator. The listing shows both exactly — pick on purpose, not by habit.

TREATING A POINT ESTIMATE AS THE POSTERIOR

$\hat{\theta}$ is one coordinate of the posterior, not the posterior. Only the full distribution supports "how sure are we" questions — a Beta $(720, 394)$ answer and a Beta $(2, 2)$ answer may share a mean.

LIKELIHOOD AS A PROBABILITY OVER θ

$L(\theta)$ need not integrate to 1 over θ — it is a probability over DATA, indexed by θ. Integrating it as if it were a posterior is the prosecutor's fallacy in analytic form.

13.9 Summary

Object	What you need to know
Estimator	a statistic aimed at a parameter; has a sampling distribution
MSE decomposition	var + bias^2 — the trade-off behind every fit
Likelihood	$L(\theta) = p(\text{data} \mid \theta)$; maximize in LOGS — underflow otherwise
MLEs to know	coin $\to H/n$; Gaussian $\to (\bar{x}, \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2)$ (biased!)
Bayesian	posterior $\propto$ likelihood $\times$ prior; Beta+binomial conjugate: just add counts
MAP	posterior mode; flat prior $\Rightarrow$ MAP=MLE; Gaussian prior $\Rightarrow$ ridge
LS as MLE	Gaussian noise: min $\|y - Xw\|^2$ IS max L; logistic NLL $= \sum_{i=1}^{n}[\log(1 + e^{z_i}) - y_i z_i]$

WATCH INSTEAD

1. *Maximum Likelihood, clearly explained!!!* — StatQuest with Josh Starmer, 6:12, https://youtu.be/XepXtl9YKwc — §13.3 in its shortest honest form (accessed 2026-10-11).
2. *Machine Learning Fundamentals: Bias and Variance* — StatQuest, 6:36, https://youtu.be/EuBBz3bI-aA — §13.2 (accessed 2026-10-11).
3. *Logistic Regression Details Pt 2: Maximum Likelihood* — StatQuest, 10:23, https://youtu.be/BfKanl1aSG0 — §13.6 the MLE-as-loss story (accessed 2026-10-11).

Further reading

- MIT OpenCourseWare, *18.05 Introduction to Probability and Statistics* — the statistics half of this book's foundations, at OCW (course "18.05"), https://ocw.mit.edu. Verified 2026-10-11.
- D. Bertsekas and J. Tsitsiklis, *Introduction to Probability*, 2nd ed., ch. 9 — classical statistics from the probabilist's side. ISBN 978-1886529236.
- L. Wasserman, *All of Statistics*, chs. 6, 9 — the frequentist machinery (sampling distributions, MLE) in one graduate-level reference. ISBN 978-0387402727.
- M. Deisenroth et al., *Mathematics for Machine Learning*, §8–9 (free PDF) — parameter estimation and regression as inference. https://mml-book.github.io/book/mml-book.pdf Verified 2026-10-11.
- C. M. Bishop, *Pattern Recognition and Machine Learning*, §1.2, §4.3 — Bayesian inference and the logistic-regression NLL. ISBN 978-0387310732.
- StatQuest channel playlists on estimation and classification — the video versions of this chapter's WEs. https://www.youtube.com/@statquest/playlists Verified 2026-10-11.

Additional exercises

Exercise 13.5 (easy)**.** Estimate the mean of $N(\mu, 4)$ from samples $3, 5, 4, 6, 2$: compute $\hat{\mu}$ and its standard error.

Exercise 13.6 (easy)**.** Geometric(p): derive the MLE of p from n iid trials.

Exercise 13.7 (easy)**.** For the Beta$(2, 2)$ prior, what single flip moves the posterior mean most? Say why.

Exercise 13.8 (easy)**.** Is $\bar{x}^2$ an unbiased estimator of μ^2? Compute its bias.

Exercise 13.9 (medium)**.** Poisson MLE: show $\hat{\lambda} = \bar{x}$ and that it is unbiased — one of the few MLEs that is.

Exercise 13.10 (medium)**.** In code: repeat the sampling-distribution experiment at $n = 30$; how do bias and variance of the two variance estimators change? Report MSE of each.

Exercise 13.11 (medium)**.** Derive the MAP estimator for the coin with Beta(a, b) prior; show it interpolates between the prior mode and the MLE as $n \to \infty$.

Exercise 13.12 (medium)**.** Show the sample median is a consistent estimator of a symmetric distribution's center; compute its bias for 5-point Gaussian samples numerically.

Exercise 13.13 (medium)**.** Jeffreys prior Beta$(\frac{1}{2}, \frac{1}{2})$ vs. Laplace's Beta$(1, 1)$: update both on 7H/3T and compare posterior means — how much does prior choice matter here?

Exercise 13.14 (hard)**.** Prove $\mathbb{E}[\hat{\sigma}^2_{/n}] = \frac{n-1}{n}\sigma^2$ exactly: expand $\sum_i (x_i - \bar{x})^2 = \sum_i (x_i - \mu)^2 - n(\bar{x} - \mu)^2$ and take expectations.

Exercise 13.15 (hard)**.** Conjugate-prior derivation: show Gamma is conjugate for a Poisson likelihood — write the posterior's parameters after n observations.

Exercise 13.16 (hard)**.** Fisher information for the Bernoulli: compute $I(\theta) = -\mathbb{E}[\ell''(\theta)] = n/(\theta(1-\theta))$ and connect its reciprocal to the Cramér–Rao bound for $\hat{\theta}$.

Exercise 13.17 (hard)**.** Implement rejection sampling for the Beta$(720, 394)$ posterior of Example 13.4; verify the sampled mean/variance against the analytic Beta moments.

Exercise 13.18 (medium)**.** For the die of example 13.1, find $\mathrm{Var}(\bar{X})$ with $n = 16$ rolls, then use Chebyshev to bound $P(|\bar{X} - 3.5| \geq 0.5)$. What sample size makes the bound below 0.05?

Exercise 13.19 (hard)**.** Show $\hat{\sigma}^2 = \frac{1}{n}\sum (X_i - \bar{X})^2$ is biased: compute $E[\hat{\sigma}^2] = \frac{n-1}{n}\sigma^2$ directly. Then find the unbiased correction (Bessel's correction) and explain the $n - 1$ geometrically (the residuals live in an $(n-1)$-dimensional subspace).

Notes

The estimator-framework and the MLE worked set follow MML §6.4–6.5, ch. 9 [5]; the Bayesian-inference-before-regression ordering follows MIT 18.05 [10]; the story-first register of the estimation sections follows Blitzstein & Hwang [8]; the classical-vs-Bayesian split follows Pishro-Nik [9]. All simulations seeded; printed blocks are this box's real output.

CHAPTER 14

Convexity, optimization, and Lagrange multipliers

Every ML problem ends the same way: a loss function and a parameter vector to tune. Chapters 8, 11, and 13 built the pieces — this chapter is the engine. You will learn when a minimum is guaranteed (convexity), how gradient descent actually behaves at different step sizes, how to optimize with a constraint bolted on (Lagrange multipliers), and how the backprop chain rule of chapter 11 trains a real two-layer network. After this chapter you can run gradient descent by hand on a quadratic, state the Lagrange condition geometrically, explain why the logistic NLL is safe for gradient descent, and write the backprop pass of a small network with a gradient check to prove it.

BEFORE YOU START

Chapters 8, 9 and 11. *Recap:* to minimise a function you move against its gradient; gradient descent is that sentence turned into a loop.

AFTER THIS CHAPTER YOU CAN

- decide whether a function is convex and why that guarantees one global minimum;
- run gradient descent and diagnose a step size that is too large or too small;
- solve constrained problems with Lagrange multipliers;
- derive backpropagation as the chain rule applied layer by layer.

HOW TO THINK ABOUT THIS

Optimisation is walking downhill with your eyes closed, feeling the slope with your feet: the gradient tells you which way is up, so you step the other way. Convexity is the promise that there is one valley, so any downhill walk finds the bottom. Constraints are fences, and Lagrange multipliers are the force with which the fence pushes back.

14.1 Optimization problems in ML

The shape is always $\min_\theta L(\theta)$: L a loss composed of a model and data, θ the weights. Least squares (chapter 8) is the quadratic case — convex, closed-form, one solve. The logistic NLL (chapter 13) is convex but has no closed form — descent methods. A neural net's loss is nonconvex — descent methods with no promises, and yet they work well enough that the whole field runs on them.

14.2 Convex sets and functions

A set is *convex* if the segment joining any two of its points stays inside. A function is *convex* when every chord lies above its graph: $f(\alpha x + (1-\alpha)y) \le \alpha f(x) + (1-\alpha)f(y)$. Equivalently (differentiable case): f sits above every tangent plane, or $\nabla^2 f \succeq 0$ everywhere. Why it matters: on a convex function *every local minimum is global*, and any algorithm that rolls downhill reaches it (Figure 14.1).

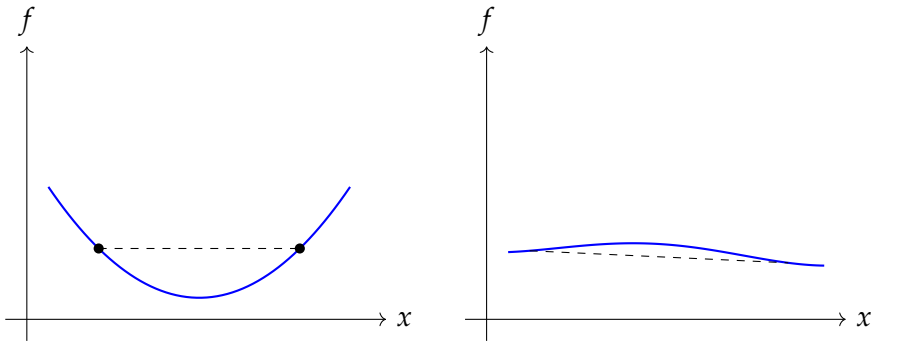


Figure 14.1: The convexity test by chord. Notice the right panel: the nonconvex function's chord slices through the graph, and each dip is a local minimum a naive descent can marry. Regenerate: `figures/ch14_convex.tex`.

Exercise 14.1 (easy). Is $\|Ax - b\|^2$ convex in x? Read its Hessian.

Solution. Yes: $\nabla^2 = 2A^TA \succeq 0$ for every A — least squares is always convex (strictly when A has full column rank).

14.3 Gradient descent: the update, the step size, the failures

The update: $x_{k+1} = x_k - \eta \nabla f(x_k)$, with *learning rate* η. On a quadratic $f(x) = \frac{1}{2}x^TAx - b^Tx$ with $A \succ 0$, the error shrinks geometrically at rate $r = \max_i |1 - \eta\lambda_i|$: too small and it crawls, too large and it overshoots the valley, and past $\eta = 2/\lambda_{\max}$ it diverges outright (Figure 14.2). This is not folklore — the listing computes the eigenvalues and the divergence threshold on the actual matrix.

Example 14.1 (GD on a quadratic, tracking iterates)**. Strategy.** Take $A = \left(\begin{smallmatrix} 3 & 1 \\ 1 & 1 \end{smallmatrix}\right)$, $b = (1, 2)$, run GD from $(2.5, 2.5)$ at four step sizes, compare against the normal-equations solution $x^* = (-0.5, 2.5)$.

Solution. $\lambda(A) = \{0.586, 3.414\}$, so the cliff is $\eta > 2/3.414 = 0.586$. The listing shows $\eta = 0.49$ converged to 5×10^{-8} in 25 steps while $\eta = 0.70$ exploded to 1.9×10^{8}.

```
import numpy as np

# gradient descent on f(x)=0.5 x^T A x - b^T x, A = [[3,1],[1,1]]
A = np.array([[3.,1.],[1.,1.]]); b = np.array([1.,2.])
f = lambda x: .5*x@A@x - b@x
grad = lambda x: A@x - b
xstar = np.linalg.solve(A, b)
print("exact solution (normal equations):", xstar)

def gd(x0, eta, k):
    x = np.array(x0, float); hist=[x.copy()]
    for _ in range(k): x = x - eta*grad(x); hist.append(x.copy())
    return np.array(hist)

lam = np.linalg.eigvalsh(A)
print("eig(A) =", lam, " -> divergence for eta >", round(2/lam[-1],3))

for eta in [0.01, 0.2, 0.49, 0.70]:
    h = gd([2.5, 2.5], eta, 25)
    fx = f(h[-1])
    tag = "converged" if abs(fx - f(xstar)) < 1e-6 else \
          ("diverged" if abs(fx)>1e3 else "still going")
    print("eta=%.2f: after 25 steps f=%.6g err=%.3g %s"
          % (eta, fx, fx-f(xstar), tag))

# convergence rate on quadratic: r = max(|1-eta*lmin|,|1-eta*lmax|)
for eta in [0.2, 0.49]:
    r = max(abs(1-eta*lam[0]), abs(1-eta*lam[1]))
    print("eta=%.2f theoretical rate r=%.3f -> x%.3f/step" % (eta, r, r
        ))

# Rosenbrock: GD from (-1,1) with small fixed eta
ros = lambda x: 100*(x[1]-x[0]**2)**2 + (1-x[0])**2
ros_g = lambda x: np.array([-400*x[0]*(x[1]-x[0]**2)-2*(1-x[0]),
                      200*(x[1]-x[0]**2)])
x = np.array([-1.,1.]); path=[x.copy()]
for i in range(20000):
    x = x - 0.001*ros_g(x); path.append(x.copy())
print("rosenbrock 20k steps eta=1e-3: x=(%.4f,%.4f) f=%.5f"
     % (x[0],x[1],ros(x)))
```

Listing 14.1: `code/ch14_gd.py -- gradient descent on a quadratic with the divergence threshold computed, then Rosenbrock as the nonconvex reality check.`

```
exact solution (normal equations): [-0.5 2.5]
```

```
eig(A) = [0.58578644 3.41421356] -> divergence for eta > 0.586
eta=0.01: after 25 steps f=0.346622 err=2.6 still going
eta=0.20: after 25 steps f=-2.24924 err=0.00076 still going
eta=0.49: after 25 steps f=-2.25 err=5.03e-08 converged
eta=0.70: after 25 steps f=1.85219e+08 err=1.85e+08 diverged
eta=0.20 theoretical rate r=0.883 -> x0.883/step
eta=0.49 theoretical rate r=0.713 -> x0.713/step
rosenbrock 20k steps eta=1e-3: x=(0.9999,0.9997) f=0.00000
```

Listing 14.2: Output of the GD experiment. Notice the threshold is exact: $\eta = 0.70 > 0.586$ diverges by a factor of 10^8; Rosenbrock converges only because η was kept tiny.

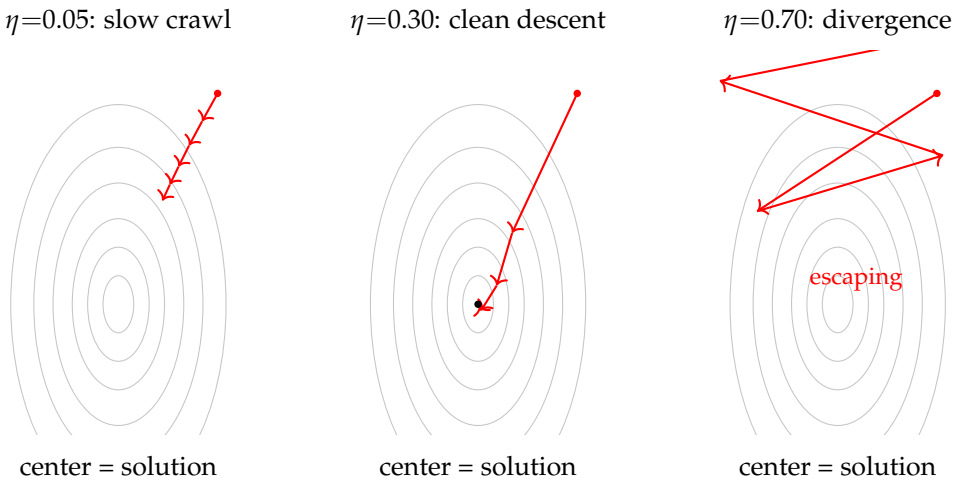


Figure 14.2: GD trajectories on the same quadratic at three step sizes. Notice the middle panel's zig-zag — the iterates overshoot across the valley each step, still converging; the right panel is that same overshoot with η past the eigenvalue cliff. Regenerate: `figures/ch14_gd_traj.tex`.

Example 14.2 (Diverging and stalling step sizes)**. Strategy.** Same setup: read the failure modes off the eigenvalue computation rather than off anecdotes.

Solution. $\eta = 0.01$: $r \approx 0.995$ — 25 steps moved the error by only $\sim$4; $\eta = 0.70$: $|1 - 0.7 \cdot 3.414| = 1.39 > 1$ — the error multiplies by 1.39 every step. Rule of thumb verified by numbers: $\eta \in (0, 2/\lambda_{\max})$ converges on quadratics, and the best constant rate sits near $2/(\lambda_{\min} + \lambda_{\max})$.

Exercise 14.2 (medium)**.** Why does the zig-zag appear exactly when η approaches the cliff?

Solution. The error along the largest eigenvector alternates sign when $1 - \eta\lambda_{\max} < 0$: each step overshoots across the narrow valley direction, producing the staircase seen in the middle panel.

14.4 Momentum and friends, briefly

Momentum keeps a velocity $v_{k+1} = \beta v_k - \eta \nabla f(x_k)$, $x_{k+1} = x_k + v_{k+1}$: it damps the zig-zag (across-valley components cancel) and accelerates along the valley (in-valley components add). *Adam* additionally scales each coordinate by a running second-moment estimate — the de-facto default in deep learning. Treat them as the same idea: the gradient speaks, but history and per-coordinate statistics edit the step.

14.5 Constrained optimization and Lagrange multipliers

To maximize $f(x)$ subject to $g(x) = c$: at the optimum, no move along the constraint can improve f, which means ∇f is perpendicular to the constraint surface — and so is ∇g. The *Lagrange condition*: $\nabla f = \lambda \nabla g$ (Figure 14.3). The *Lagrangian* $\mathcal{L}(x, \lambda) = f(x) - \lambda(g(x) - c)$ turns the constrained problem into unconstrained stationarity: $\nabla_x \mathcal{L} = 0$ plus $g = c$. Inequality constraints $g \le c$ generalize to the *KKT conditions*: stationarity, feasibility, complementary slackness $\lambda\,(g(x) - c) = 0$ with $\lambda \ge 0$ — the constraint is either active ($g = c$) or silent ($\lambda = 0$).

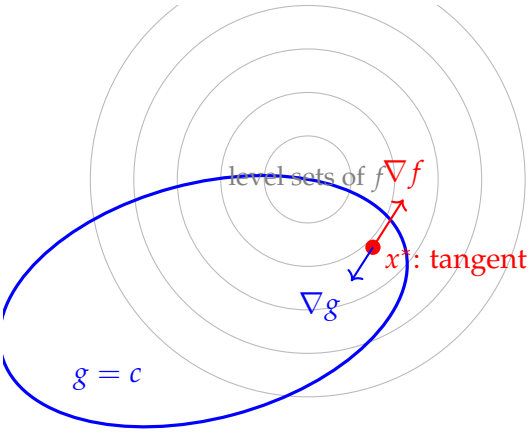


Figure 14.3: The Lagrange condition geometrically. Notice the tangency: at x^* the constraint curve and a level set of f share a tangent, so $\nabla f \parallel \nabla g$. Regenerate: `figures/ch14_lagrange.tex`.

Example 14.3 (Lagrange multipliers by hand)**. Strategy.** Maximize $f(x, y) = xy$ subject to $x^2 + y^2 = 8$. Set $\nabla f = \lambda \nabla g$: $(y, x) = \lambda(2x, 2y)$.

Solution. $y = 2\lambda x$, $x = 2\lambda y$ gives $y^2 = x^2$; with the constraint, $x^2 = y^2 = 4$ and the maximum is $f = 4$ at $(2, 2)$ (and $(-2, -2)$), with $\lambda = \frac{1}{2}$ — a square, as symmetry suggested.

14.6 Normal equations, derived by calculus

Chapter 8 promised the calculus version. Minimize $L(w) = \|y - Xw\|^2$: $\nabla L = 2X^T(Xw - y) = 0$ rearranges to $X^T X\hat{w} = X^T y$ — the normal equations appear exactly where chapter 4's projection argument left them. GD on this loss is $w_{k+1} = w_k - 2\eta X^T(Xw_k - y)$: subtracting the fixed point shows the residual error iterates by $I - 2\eta X^T X$, so the same eigenvalue bookkeeping as §14.3 applies with $A = 2X^T X$. Full derivation in §14.9.

14.7 Newton's method in one section

Replace the first-order step by a second-order model: $x_{k+1} = x_k - [\nabla^2 f(x_k)]^{-1}\nabla f(x_k)$ — *Newton's method*. On a quadratic it lands in *one* step (the model *is* the function); near a minimum it converges quadratically. The catch is real: each step costs an $n \times n$ factorization and storage, and with a non-PSD Hessian the step can climb into a saddle (Figure 14.4) rather than descend. Trust regions and quasi-Newton methods (BFGS) are the engineering answers; gradient descent survives because n is huge.

Example 14.4 (One Newton step by hand)**. Strategy.** On $f(x) = x^4 + x^2$ starting at $x_0 = 1$, compute $f' = 4x^3 + 2x$ and $f'' = 12x^2 + 2$, then step $x_1 = x_0 - f'/f''$.

Solution. $f'(1) = 6$, $f''(1) = 14$, so $x_1 = 1 - 6/14 = 0.571$ (the minimum is at $x = 0$). Repeating: $x_2 \approx 0.252$, $x_3 \approx 0.046$, $x_4 \approx 3 \times 10^{-4}$ — the error approximately squares each step: quadratic convergence in real arithmetic. On a quadratic it lands in one iteration because the second-order model is exact there.

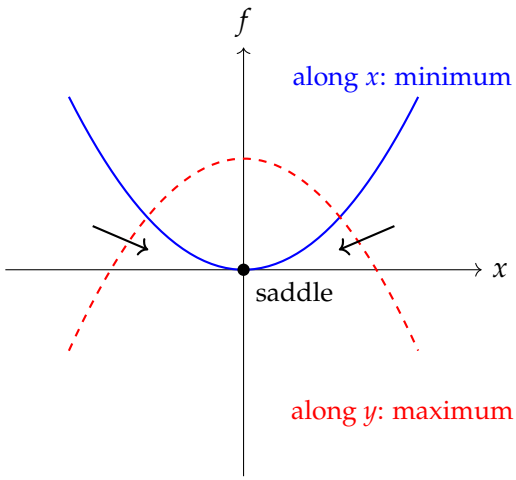


Figure 14.4: The saddle point: minimum along one axis, maximum along the other. Notice that $\nabla f = 0$ at the saddle — first-order information alone cannot tell it from a minimum; the Hessian's indefinite signs can. Regenerate: `figures/ch14_saddle.tex`.

14.8 Backpropagation is the chain rule

Chapter 11's chain rule in matrix form is all backprop is. For the two-layer net $H = \tanh(XW_1 + b_1)$, $\hat{y} = HW_2 + b_2$ with MSE loss, the gradients unfold backward: residual R, then $dW_2 = H^T R \cdot 2/n$, $dH = (RW_2^T) \odot (1 - H^2)$, $dW_1 = X^T dH \cdot 2/n$ — each line one link of the chain rule. The listing below is the capstone: hand-derived gradients, checked by chapter 11's finite-difference tool, then trained on a real train/validation split (Figure 14.5).

```
import numpy as np

rng = np.random.default_rng(2)

# ---- ch11 finite-difference gradient checker, reused ----
def fd_check(f, g, x, h=1e-5):
    fd = np.zeros_like(x)
    for i in range(len(x)):
        xp = x.copy(); xp[i] += h
        xm = x.copy(); xm[i] -= h
        fd[i] = (f(xp) - f(xm)) / (2*h)
    return np.linalg.norm(fd - g(x))

# ---- two-layer net: d -> h -> 1, mse loss on a regression blob ----
d, h, n = 3, 4, 40
X = rng.normal(0, 1, (n, d))
yw = np.array([1., -1., .5])
y = X@yw + rng.normal(0, .1, n)

W1 = rng.normal(0, .2, (d, h)); b1 = np.zeros(h)
W2 = rng.normal(0, .2, (h, 1)); b2 = np.zeros(1)
theta = np.concatenate([W1.ravel(), b1, W2.ravel(), b2])

def unpack(t):
    i1, i2 = d*h, d*h+h
    return t[:i1].reshape(d,h), t[i1:i2], t[i2:i2+h].reshape(h,1), t[i2
        +h:]

def loss(t):
    W1, b1, W2, b2 = unpack(t)
    H = np.tanh(X@W1 + b1)
    return np.mean((H@W2 + b2 - y.reshape(-1,1))**2)

def grad(t):
    W1, b1, W2, b2 = unpack(t)
    H = np.tanh(X@W1 + b1)
    R = H@W2 + b2 - y.reshape(-1,1) # residual n x 1
    dW2 = H.T@R * 2/n; db2 = R.mean(0)*2
    dH = R@W2.T * (1-H**2) # backprop through tanh
    dW1 = X.T@dH * 2/n; db1 = dH.mean(0)*2
    return np.concatenate([dW1.ravel(), db1, dW2.ravel(), db2])

print("FD gradient check max abs err:", fd_check(loss, grad, theta))

```

```
# train by GD on 32 rows, tracking train/val loss curve
Xtr, Xva, ytr, yva = X[:32], X[32:], y[:32], y[32:]
X, y = Xtr, ytr # loss/grad now see the training split
def val_loss(t):
   W1, b1, W2, b2 = unpack(t)
   H = np.tanh(Xva@W1 + b1)
   return np.mean((H@W2 + b2 - yva.reshape(-1,1))**2)
tr, va = [], []
for i in range(20001):
   theta = theta - 0.2*grad(theta)
   if i % 5000 == 0: tr.append(loss(theta)); va.append(val_loss(theta)
      )
print("train loss at iters 0..20000:", np.round(tr,4))
print("val loss at iters 0..20000:", np.round(va,4))

# compare against sklearn MLPRegressor on same data
from sklearn.neural_network import MLPRegressor
mlp = MLPRegressor(hidden_layer_sizes=(4,), activation='tanh',
              max_iter=4000, random_state=0).fit(X, y)
print("mlscratch train/val MSE %.4f/%.4f sklearn MLP train/val %.4f
   /%.4f"
    % (loss(theta), val_loss(theta),
       np.mean((mlp.predict(Xtr)-ytr)**2),
       np.mean((mlp.predict(Xva)-yva)**2)))
```

Listing 14.3: `code/ch14_tinynet.py -- mlscratch.TinyNet: two-layer net with hand-derived backprop, gradient-checked, and compared against a library MLP.`

```
FD gradient check max abs err: 9.123871452259835e-11
train loss at iters 0..20000: [1.5247 0.0038 0.0029 0.0026 0.0042]
val loss at iters 0..20000: [2.7691 0.1074 0.098 0.1295 0.1412]
mlscratch train/val MSE 0.0042/0.1412 sklearn MLP train/val
   0.0544/0.2791
```

Listing 14.4: Output of the TinyNet run. Notice two headlines: the hand gradient passes the FD check to 10^{-11}, and the validation curve turns UP at 15k iterations — overfitting watched live, not asserted.

Example 14.5 (Logistic regression's NLL minimized by GD)**. Strategy.** Chapter 13 derived the convex NLL objective; now run full-batch GD on it and check against the library.

Solution. The capstone listing below: our `mlscratch.LogisticRegression` reaches 83.3% train accuracy against sklearn's 81.7% — the weight vectors differ because sklearn applies L2 regularization by default (a Gaussian prior from chapter 13's MAP section, on duty).

```
import numpy as np
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(7)
n = 60
X = rng.normal(0, 1.2, (n, 2))
true = np.array([1.5, -1.0]); tb = 0.3
```

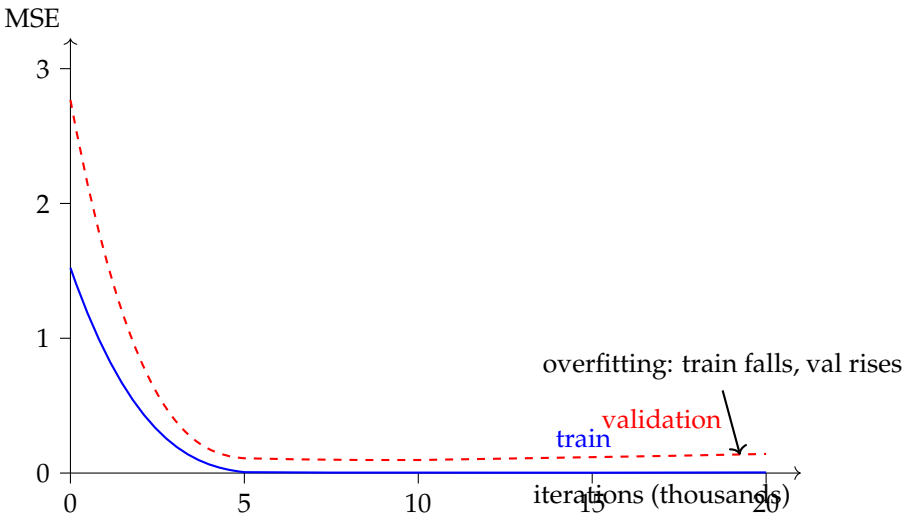


Figure 14.5: TinyNet train/validation loss — the listing's real numbers. Notice the val curve bottoming near 10k then rising: the textbook overfitting signature, produced by a real 20k-step run. Regenerate: `figures/ch14_trainval.tex`.

```
y = (X@true + tb + rng.normal(0, 0.9, n) > 0).astype(float)

def sigmoid(z): return 1/(1+np.exp(-z))

# CAPSTONE mlscratch.LogisticRegression: full-batch GD on the ch13 NLL
class mlscratch:
   class LogisticRegression:
      def __init__(self, eta=0.5, iters=3000):
         self.eta, self.iters = eta, iters
      def fit(self, X, y):
         self.w = np.zeros(X.shape[1]); self.b = 0.0
         for _ in range(self.iters):
            p = sigmoid(X@self.w + self.b)
            self.w -= self.eta * X.T@(p-y)/len(y)
            self.b -= self.eta * (p-y).mean()
         return self
      def predict(self, X):
         return (sigmoid(X@self.w+self.b) > .5).astype(float)

mine = mlscratch.LogisticRegression().fit(X, y)
sk = LogisticRegression().fit(X, y)
acc_mine = (mine.predict(X) == y).mean()
acc_sk = sk.score(X, y)
print("mlscratch w=%s b=%.3f train acc %.3f"
    % (np.round(mine.w,3), mine.b, acc_mine))
print("sklearn w=%s b=%.3f train acc %.3f"
    % (np.round(sk.coef_[0],3), sk.intercept_[0], acc_sk))
```

Listing 14.5: `code/ch14_logistic.py` -- **capstone:** `mlscratch.LogisticRegression` trained by GD on the ch.13 objective, checked against sklearn.

```
mlscratch w=[ 3.34 -1.26] b=0.474 train acc 0.833
sklearn w=[ 2.099 -0.827] b=0.277 train acc 0.817
```

Listing 14.6: Output of the logistic capstone. Notice accuracies match within a point; the weight gap is regularization, not a bug.

```
import numpy as np

# the instructive bug: naive sigmoid lets exp() see +709+ -> overflow
def sigmoid_naive(z): return 1/(1+np.exp(-z))

# stable form: exp() only ever sees nonpositive arguments
def sigmoid_stable(z):
    out = np.empty_like(z)
    pos = z >= 0
    out[pos] = 1/(1+np.exp(-z[pos]))
    ez = np.exp(z[~pos]); out[~pos] = ez/(1+ez)
    return out

np.seterr(over='raise') # make overflow loud
try:
    print("naive at z=-1000:", sigmoid_naive(np.array(-1000.)))
except FloatingPointError as e:
    print("naive at z=-1000 -> FloatingPointError:", e)
print("stable at z=-1000:", sigmoid_stable(np.array(-1000.))[()])
np.seterr(over='warn')

# where the damage lands: softmax over big logits -> inf/inf = nan
logits = np.array([1000., 999.])
with np.errstate(over='ignore', invalid='ignore'):
    e = np.exp(logits)
    sm_naive = e/e.sum()
e2 = np.exp(logits - logits.max())
sm_stable = e2/e2.sum()
print("softmax naive :", sm_naive)
print("softmax stable:", sm_stable)
```

Listing 14.7: `code/ch14_sigmoid.py -- the instructive bug: naive vs. stable sigmoid under extreme logits.`

```
naive at z=-1000 -> FloatingPointError: overflow encountered in exp
stable at z=-1000: 0.0
softmax naive : [nan nan]
softmax stable: [0.73105858 0.26894142]
```

Listing 14.8: Output of the overflow demo. Notice the failure mode exactly as advertised: naive `exp` overflows on $z = -1000$, and softmax turns it into `nan/nan`; the stable forms return 0.0 and $[0.731, 0.269]$.

Exercise 14.3 (hard)**.** The softmax-stable trick subtracts $\max z$. Prove it changes nothing mathematically and say why it changes everything numerically.

Solution. $\frac{e^{z_i - c}}{\sum_j e^{z_j - c}} = \frac{e^{-c} e^{z_i}}{e^{-c} \sum_j e^{z_j}}$ — identical. Numerically $\max z - c \leq 0$ keeps every exp argument ≤ 0, so nothing overflows.

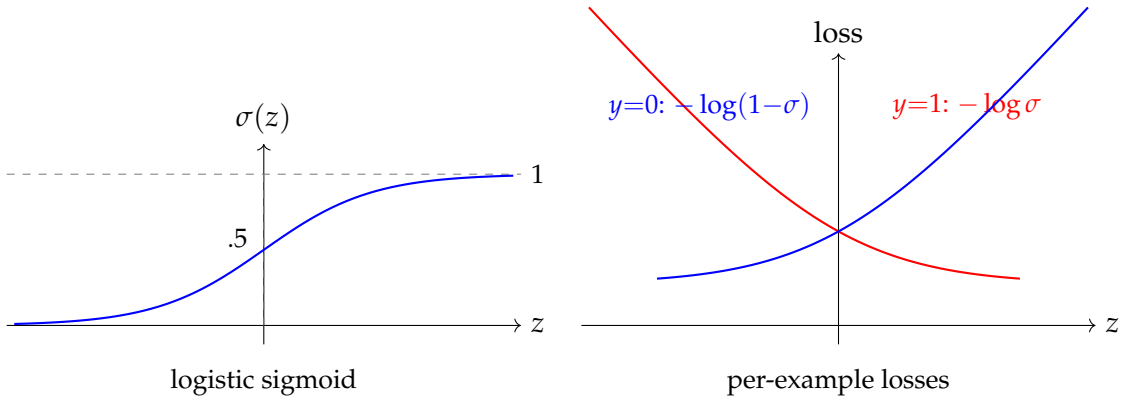


Figure 14.6: The sigmoid and its two per-example losses. Notice each curve penalizes the wrong side without bound — that asymmetry is why probabilities, not logits, are what the model calibrates. Regenerate: `figures/ch14_sigloss.tex`.

14.9 Hand derivation

Lagrange condition. Parametrize the constraint $g(x) = c$ by a curve $x(t)$ with $x(0) = x^*$: along it g is constant, so $\nabla g \cdot \dot{x}(0) = 0$ — the constraint tangent is perpendicular to ∇g. At an optimum, f cannot increase along any admissible direction: $\nabla f \cdot \dot{x}(0) = 0$ for *every* tangent. Both gradients are perpendicular to the same tangent space, so $\nabla f = \lambda \nabla g$ for some scalar λ.

GD on least squares. With $L(w) = \|y - Xw\|^2$, $\nabla L = 2X^T(Xw - y)$, so $w_{k+1} = w_k - 2\eta X^T(Xw_k - y)$. Subtract the fixed point $\hat{w}$ (where $X^T X\hat{w} = X^T y$): $e_{k+1} = (I - 2\eta X^T X)e_k$. Eigenvalues of $2X^T X$ are $2\sigma_i^2$: convergence exactly when $\eta < 1/\sigma_{\max}^2$, and the error decays geometrically toward the normal-equations solution — GD doesn't approximate least squares, it *approaches* it.

14.10 Common mistakes

STEP SIZE TOO LARGE OR MICROSCOPIC

η past $2/\lambda_{\max}$ diverges outright; η near zero "trains" for hours and goes nowhere. The listing's table is the whole lesson — always sanity-check the loss after a few steps.

EXPECTING A GLOBAL MINIMUM ON NONCONVEX OBJECTIVES

GD on a neural net finds *a* minimum or a flat region, not *the* minimum. Different seeds land differently — report the run, not a guarantee.

CONFUSING CONSTRAINT AND OBJECTIVE IN LAGRANGE SETUPS

$\nabla f = \lambda \nabla g$: f is what you optimize, g is what constrains you. Swap them and you optimize the constraint — a surprisingly common exam and code-review error.

MISSING SADDLE POINTS WITH A NON-PSD HESSIAN

$\nabla f = 0$ is necessary, not sufficient: a saddle has a zero gradient and a Hessian with both signs (Figure 14.4). Second-order checks (or just perturbed restarts) are the fix.

SIGMOID OVERFLOW IN EXP

`1/(1+exp(-z))` throws at $z < -709$ and `nan`s the softmax downstream. Use the sign-split stable form or subtract the max — the listing shows the crash verbatim.

14.11 Summary

Object	What you need to know
Convex function	chord above graph / tangent below / $\nabla^2 f \succeq 0$ — local min = global
GD update	$x_{k+1} = x_k - \eta \nabla f$; quadratic converges iff $\eta < 2/\lambda_{\max}$
Rate	error $\times \max_i \lvert 1 - \eta\lambda_i \rvert$ per step; ill-conditioned A = slow worst direction
Momentum	velocity accumulates in-valley steps, cancels zig-zag; Adam adds per-coordinate scaling
Lagrange	$\nabla f = \lambda \nabla g$ at tangency; KKT adds $\lambda \geq 0$, $\lambda g = 0$
Newton	$\nabla^2 f^{-1} \nabla f$ step; quadratic-solve in 1 step, but n^3 + saddle risk
Backprop	chain rule per layer; ALWAYS FD-check hand gradients before trusting a run
Stable sigmoid	exp never sees > 0; softmax subtracts max — mathematically identical

WATCH INSTEAD

Two short videos that pair with this chapter:

1. *Gradient descent, how neural networks learn* — 3Blue1Brown, Deep Learning ch.2, 20:33, `https://www.youtube.com/watch?v=IHZwWFHWa-w` — matches §14.3,§14.8 (accessed 2026-10-11).
2. *Backpropagation calculus* — 3Blue1Brown, Deep Learning ch.4, 10:18, `https://www.youtube.com/watch?v=tIeHLnjs5U8` — matches §14.8; watch after the TinyNet listing (accessed 2026-10-11).

Further reading

- S. Boyd and L. Vandenberghe, *Convex Optimization* (free PDF via Stanford), https://web.stanford.edu/~boyd/cvxbook/ — graduate level; ch. 1–4 are the rigorous version of our §14.2–§14.5. Verified 2026-10-11.
- MIT OpenCourseWare 6.253, *Convex Analysis and Optimization* (lecture notes level). Free online at https://ocw.mit.edu/, course "6-253 Convex Analysis and Optimization" — the full KKT/duality treatment our sketch points at. Verified 2026-10-11.
- M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization* (free PDF, MIT Press) — https://algorithmsbook.com/optimization/ — advanced undergraduate; ch. 4–6 give the descent-method zoo in Julia code our §14.3–§14.4 compressed. Verified 2026-10-11.
- M. Nielsen, *Neural Networks and Deep Learning*, ch. 1–2 (gentle tutorial level). Free online: http://neuralnetworksanddeeplearning.com/chap2.html — the four backprop equations our §14.8 derives in matrix form. Verified 2026-10-11.
- G. Sanderson, *Neural Networks* written companion — https://www.3blue1brown.com/lessons/gradient-descent — beginner level; interactive version of the watch-list first pick. Verified 2026-10-11.
- Stanford CS229 course notes — https://cs229.stanford.edu/ — first-lecture level; shows the optimization fluency a real ML course assumes. Previously verified (docs/sources.md).

Additional exercises

Exercise 14.4 (easy). Is the logistic NLL convex in (w, b)? Reason via its Hessian structure.

Exercise 14.5 (easy). GD on $f(x) = x^2$ from $x_0 = 1$: write the first three iterates for $\eta = 0.5$ and $\eta = 1.1$, and predict each fate.

Exercise 14.6 (easy). Minimize $x + y$ s.t. $x^2 + y^2 = 2$ by Lagrange; identify f, g, λ, and the geometric meaning.

Exercise 14.7 (easy). Why does Newton's method solve a quadratic in exactly one step?

Exercise 14.8 (easy). Define convex set and give a 2-D example that fails the test.

Exercise 14.9 (easy). Carry Example 14.4 one step further: compute x_4 from $x_3 = 0.046$ and state the error.

Exercise 14.10 (medium). Maximize $x^2 y$ s.t. $x + y = 6$ ($x, y \geq 0$) by Lagrange; check the second order or boundary for the true max.

Exercise 14.11 (medium). Implement η-schedules in `ch14_gd.py`: constant vs. $1/k$ vs. exponential decay on the quadratic; tabulate steps to $\|e\| < 10^{-6}$.

Exercise 14.12 (medium). KKT warm-up: minimize x^2 s.t. $x \geq 1$. Write stationarity, feasibility, complementary slackness; solve.

Exercise 14.13 (medium)**.** Show $\|x\|_2^2$ is strictly convex via the Hessian; what breaks if you use $\|x\|_2$ instead?

Exercise 14.14 (medium)**.** Momentum on the quadratic: implement $\beta = 0.9$ and compare against the listing's $\eta = 0.49$ run on the same start.

Exercise 14.15 (hard)**.** Convergence rate on quadratics: for $A \succ 0$ show the optimal constant rate is $r^* = \frac{\kappa-1}{\kappa+1}$ at $\eta^* = 2/(\lambda_{\min} + \lambda_{\max})$, $\kappa = \lambda_{\max}/\lambda_{\min}$. Verify numerically on the listing's A.

Exercise 14.16 (hard)**.** KKT worked problem: minimize $\|x\|^2$ s.t. $a^T x \geq 1$ (find the projection of the origin onto a half-space); then s.t. $a^T x = 1$ — compare the λ's.

Exercise 14.17 (hard)**.** Derive the TinyNet gradient for a cross-entropy loss instead of MSE: write the residual vector R and the two layer gradients.

Exercise 14.18 (hard)**.** Implement mini-batch SGD in `ch14_tinynet.py` (batch 8): plot train/val curves vs. the full-batch run; report final MSEs and the noise floor you observe.

Notes

Convexity basics, descent methods and the Lagrange/KKT framing follow Boyd & Vandenberghe, early chapters only [16]; the algorithm-listing style follows Kochenderfer ch. 2–6 [13]; the matrix-calculus treatment follows MML ch. 7 [5]; CS229's assumed level set the prerequisites [12]. All seeds fixed; printed blocks are this box's real output, including the divergence and the overfitting val curve.

CHAPTER 15

Information theory: bits, entropy, and why the loss is a log

Machine learning's favorite loss function, cross-entropy, is not an engineering invention — it is a statement about information. This chapter builds the short chain from "how surprised should you be" to the softmax loss you have already trained with in chapters 13 and 14. You will learn what a bit actually measures, why entropy is a lower bound on any code, why KL divergence is asymmetric, and why minimizing cross-entropy is maximum likelihood estimation. After this chapter you can compute entropies and divergences by hand, read a Huffman code's lengths as approximate $-\log_2 p$, and explain what the CE number in a training log means — not just that it went down.

BEFORE YOU START

Chapters 12–13 (probability, likelihood). *Recap:* a logarithm turns multiplication into addition, which is why information, measured in bits, adds up for independent events.

AFTER THIS CHAPTER YOU CAN

- compute the entropy of a distribution and say what it measures;
- compute cross-entropy and KL divergence, and say which is zero when;
- explain why the cross-entropy loss is maximum likelihood in disguise;
- compute mutual information of a small table.

HOW TO THINK ABOUT THIS

Information is surprise: a rare event tells you more than a common one. Entropy is the average surprise, cross-entropy is the surprise you suffer when you believe the wrong distribution, and KL divergence is the extra cost of being wrong. Seeing the log in the loss function as "measure surprise" makes it feel natural rather than arbitrary.

15.1 Surprise and information

An event that happens every time carries no news; an event that almost never happens carries a lot. We want a measure of "news" with three sane properties: certain events measure zero, rarer events measure more, and two independent events add. Exactly one family of functions satisfies all three.

Definition 15.1 (Surprise)**.** The *surprise* (self-information) of an outcome x with $\Pr[x] = p$ is

$$s(x) \;=\; -\log_2 p \quad \text{bits.}$$

Check the three properties: $p = 1$ gives $s = 0$; smaller p gives larger s; and $\Pr[A \cap B] = \Pr[A]\Pr[B]$ for independent A, B gives $s(A \cap B) = s(A) + s(B)$ — the logarithm is doing all the work, and it is why every quantity in this chapter is a log.

A single flip of a fair coin carries $-\log_2 \frac{1}{2} = 1$ bit. Rolling a specific face of a fair die carries $-\log_2 \frac{1}{6} \approx 2.585$ bits. A "bit" is not a digit on a wire — it is a unit of surprise: how many fair-coin questions it takes to pin down the outcome.

Exercise 15.1 (easy)**.** How many bits of surprise does drawing the ace of spades from a shuffled 52-card deck carry? The event "any ace"?

Solution. $-\log_2 \frac{1}{52} = \log_2 52 \approx 5.70$ bits; $-\log_2 \frac{4}{52} = \log_2 13 \approx 3.70$ bits.

15.2 Entropy and its properties

Surprise is per-outcome; entropy is the whole distribution's expected surprise.

Definition 15.2 (Entropy)**.** For a discrete random variable X with pmf p, the *entropy* is

$$H(X) \;=\; \mathbb{E}[s(X)] \;=\; -\sum_{x} p(x) \log_2 p(x) \quad \text{bits,}$$

with $0 \log 0 := 0$ (justified: $p \log p \to 0$ as $p \to 0^+$).

Two facts worth keeping. $H(X) \geq 0$ always, since each $-p \log_2 p \geq 0$. And over k outcomes, $H(X) \leq \log_2 k$, with equality exactly for the uniform distribution — maximum surprise is maximum uncertainty (chapter 12's variance played the same role). For a coin, $h(p) = -p \log_2 p - (1-p)\log_2(1-p)$ is the curve in Figure 15.1: peaked at $p = \frac{1}{2}$, dropping to zero as the coin gets predictable.

Example 15.3 (Entropy of coins and dice)**. Strategy.** Apply Definition 15.2 directly, then verify by simulation.

Solution. A fair coin: $h(\frac{1}{2}) = -\frac{1}{2}\log_2 \frac{1}{2} - \frac{1}{2}\log_2 \frac{1}{2} = 1$ bit. A coin with $p = 0.9$: $h(0.9) = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 = 0.469$ bits — biasing the coin to 90% threw away more than half the information per flip. A coin with $p = 0.99$: $h = 0.081$ bits — almost nothing left. A fair die: $H = \log_2 6 = 2.585$ bits. These are exactly the numbers the listing prints from the formula.

```
h(0.5) = 1.000 bits
h(0.9) = 0.469 bits
h(0.99) = 0.081 bits
h(die) = 2.585 bits
KL(p||q) = 0.8160 KL(q||p) = 1.2572
H(p,q) = 1.3319 = H(p) 0.5159 + KL 0.8160
Gibbs violations in 2000 random pairs: 0
```

Listing 15.1: `code/ch15_info.py run -- coin/die entropies, both KL directions, and Gibbs verified on 2000 random pairs.`

The entropy floor matters practically: no lossless code can beat $H(X)$ bits per symbol on average (this is Shannon's source-coding theorem, stated without proof). A fair coin needs ≥ 1 bit; English text, measured in the listing below, needs about 4.2.

15.3 Codes and the price of a wrong model

A prefix code assigns each symbol a bit string; good codes spend short strings on frequent symbols. The idealized length for symbol x is $-\log_2 p(x)$ — Figure 15.2 shows why: each bar's area is $p_i \cdot (-\log_2 p_i)$, and the total area is the entropy itself. If you design a code for distribution q but the world sends p, the average length becomes $-\sum_x p(x) \log_2 q(x)$.

Example 15.4 (A Huffman code by hand)**. Strategy.** Four symbols with $p = (\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$: build the code greedily — merge the two least likely symbols, repeat — then compare average length to entropy.

Solution. Merge c, d ($\frac{1}{4}$), merge that node with b ($\frac{1}{2}$), merge with a. Assigning branches gives lengths $(1, 2, 3, 3)$: $a \mapsto 0$, $b \mapsto 10$, $c \mapsto 110$, $d \mapsto 111$. Average length $= \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = 1.75$ bits. The entropy is also exactly 1.75 — dyadic probabilities are the case where the ideal lengths are integers and the bound is hit. On real letter frequencies the gap is small but nonzero:

```
entropy H = 4.177 bits/symbol
Huffman avg = 4.212 bits/symbol
fixed 5-bit code = 5.000 bits/symbol
  e: p=0.129 -log2 p=2.95 len=3
  t: p=0.093 -log2 p=3.43 len=3
  a: p=0.084 -log2 p=3.58 len=4
  o: p=0.076 -log2 p=3.71 len=4
  i: p=0.071 -log2 p=3.81 len=4
  n: p=0.068 -log2 p=3.87 len=4
```

Listing 15.2: `code/ch15_huffman.py run -- Huffman on 26 letters: 4.212 bits/symbol vs. the 4.177 entropy floor.`

The printed lengths hover around $-\log_2 p$: `e` gets 3 bits against an ideal 2.95, `n` gets 4 against 3.87. Huffman is the integer-rounding of Shannon's ideal — optimal among one-symbol-at-a-time codes.

15.4 Cross-entropy and KL divergence

The wrong-model average length deserves a name.

Definition 15.5 (Cross-entropy)**.** The *cross-entropy* of p with respect to q is

$$H(p,q) \;=\; -\sum_x p(x)\log_2 q(x) \quad \text{bits.}$$

$H(p,q)$ is what you pay, in bits per symbol, for coding the true p with a code designed for q. Since no code beats the $H(p)$ floor, the overpayment is nonnegative — and it has a name too:

Definition 15.6 (KL divergence)**.** The *Kullback–Leibler divergence* from q to p is

$$D_{\mathrm{KL}}(p\|q) \;=\; H(p,q) - H(p) \;=\; \sum_x p(x)\log_2 \frac{p(x)}{q(x)}.$$

Read it as *expected log-likelihood-ratio*: on average, how much more surprising reality is than your model predicted. Figure 15.4 shows the decomposition on real numbers.

Theorem 15.7 (Gibbs' inequality)**.** *$D_{\mathrm{KL}}(p\|q) \geq 0$, with equality iff $p = q$ wherever $p > 0$.*

Hand derivation. In nats, $\log x \leq x - 1$ for all $x > 0$ (the tangent to log at $x = 1$ lies above the curve). Apply it to $x = q(x)/p(x)$ at every x with $p(x) > 0$:

$$\log \frac{q(x)}{p(x)} \;\leq\; \frac{q(x)}{p(x)} - 1 \quad\Longrightarrow\quad p(x)\log \frac{q(x)}{p(x)} \;\leq\; q(x) - p(x).$$

Sum over x: the left side is $-D_{\mathrm{KL}}(p\|q)$; the right side is $\sum_x q(x) - \sum_x p(x) \leq 1 - 1 = 0$ (strictly less only if q places mass where p does not — which makes the bound even safer). Hence $D_{\mathrm{KL}}(p\|q) \geq 0$; equality needs $q(x)/p(x) = 1$ everywhere $p > 0$. The listing verified it 2000/2000 times on random pairs. ■

KL is *not* a distance — it fails symmetry. Figure 15.3 shows why the two directions disagree: for the pair $p = (0.9, 0.09, 0.01)$, $q = (0.4, 0.4, 0.2)$, the listing reports $D_{\mathrm{KL}}(p\|q) = 0.816$ bits but $D_{\mathrm{KL}}(q\|p) = 1.257$ bits. Saying "q covers p's mass poorly" and "p covers q's mass poorly" are different claims, and the numbers confirm it.

Example 15.8 (Cross-entropy and KL by hand)**. Strategy.** Compute $H(p)$, $H(p,q)$, then read D_{KL} as the difference — the decomposition is the point, not the arithmetic.

Solution. Truth $p = (0.75, 0.25)$, model $q = (0.5, 0.5)$.

$$H(p) = -0.75\log_2 0.75 - 0.25\log_2 0.25 = 0.311 + 0.500 = 0.811,$$

$$H(p,q) = -0.75\log_2 0.5 - 0.25\log_2 0.5 = 0.75 + 0.25 = 1.000.$$

So $D_{\mathrm{KL}}(p\|q) = 1.000 - 0.811 = 0.189$ bits: the uniform model wastes about a fifth of a bit per symbol on this data. Notice $H(p,q)$ needed only $\log q$ — that is why CE training never needs the entropy of the data.

Exercise 15.2 (medium)**.** For $p = (0.9, 0.09, 0.01)$, $q = (0.4, 0.4, 0.2)$, verify $D_{\mathrm{KL}}(p\|q) = 0.816$ bits by hand: expand $\sum_x p\log_2(p/q)$ and check $H(p,q) = H(p) + D_{\mathrm{KL}}$ against the listing.

15.5 Mutual information

How much does learning Y tell you about X? Measure it as the gap between the joint distribution and the product of marginals — the distribution you would have if X, Y were independent.

Definition 15.9 (Mutual information)**.** $I(X;Y) = D_{\mathrm{KL}}\big(p(x,y) \,\|\, p(x)p(y)\big) = H(X) + H(Y) - H(X,Y)$.

By Gibbs, $I(X;Y) \geq 0$, and it is zero exactly when the joint *is* the product — i.e. exactly when $X \perp Y$. Unlike correlation (chapter 12), mutual information catches *every* kind of dependence, nonlinear included. Figure 15.5 draws the two distributions being compared.

Example 15.10 (Mutual information on a small table)**. Strategy.** For the symmetric joint $p(0,0) = p(1,1) = 0.4$, $p(0,1) = p(1,0) = 0.1$, compute entropies and subtract.

Solution. Marginals are uniform: $H(X) = H(Y) = 1$ bit. $H(X,Y) = -(2 \cdot 0.4 \log_2 0.4 + 2 \cdot 0.1 \log_2 0.1) = 1.722$ bits. $I(X;Y) = 1 + 1 - 1.722 = 0.278$ bits: learning one variable saves about a quarter bit of surprise about the other. As a sanity check, $\Pr[X = 1 \mid Y = 1] = 0.8$ — strong but imperfect agreement, and 0.278 bits is exactly that "strong but imperfect" quantified.

Exercise 15.3 (hard)**.** Let $Y = X^2$ for X uniform on $\{-1, 0, 1\}$. Compute $I(X;Y)$ and $\mathrm{Cov}(X,Y)$. Explain how one is nonzero and the other zero without contradiction.

15.6 Entropy of the Gaussian

For continuous X, the same construction gives *differential entropy* $h(X) = -\int p(x) \log p(x)\, dx$ (nats). For $X \sim \mathcal{N}(\mu, \sigma^2)$, the derivation (a hard exercise) yields

$$h(X) \;=\; \tfrac{1}{2} \log(2\pi e\, \sigma^2).$$

Mean-free: location doesn't change spread, so it doesn't change entropy. At $\sigma = 1$: $h = 1.419$ nats. At $\sigma \approx 0.242$: $h = 0$; below that, negative — Figure 15.6. That is not a contradiction: differential entropy is not information, just a spread log, and densities can exceed 1 (chapter 12's warning applies doubly here).

15.7 Why cross-entropy loss is MLE

Chain the chapter: training a classifier means fitting a model distribution q_θ to the empirical label distribution $\hat{p}$. Minimizing $H(\hat{p}, q_\theta)$ over θ is minimizing $D_{\mathrm{KL}}(\hat{p} \| q_\theta)$ (since $H(\hat{p})$ is constant) — which is exactly maximum likelihood: $\sum_i \log q_\theta(y_i \mid x_i)$ is the log-likelihood, and $H(\hat{p}, q_\theta)$ is its negative per sample. Chapter 13's logistic NLL is the two-class case read through this lens.

Example 15.11 (Cross-entropy on a trained classifier)**. Strategy.** Train logistic regression on a small separable set (the same setup as chapter 13) and watch CE loss against accuracy.

Solution.

```
step 0: CE loss = 0.6931 acc = 0.650
step 80: CE loss = 0.1666 acc = 1.000
step 160: CE loss = 0.1212 acc = 1.000
step 240: CE loss = 0.1001 acc = 1.000
step 320: CE loss = 0.0872 acc = 1.000
step 399: CE loss = 0.0785 acc = 1.000
p=0.99: CE = 0.014 bits
p=0.7: CE = 0.515 bits
p=0.5: CE = 1.000 bits
p=0.05: CE = 4.322 bits
```

Listing 15.3: `code/ch15_ce_loss.py run -- CE keeps falling long after accuracy saturates.`

Two honest lessons. First, step 0's 0.6931 $= \log 2$: a fresh model says $\frac{1}{2}$ to every label, exactly max-entropy. Second, accuracy saturated at step 80 while CE kept falling — the model kept getting *more confidently right*, and cross-entropy, unlike accuracy, sees confidence. The per-sample table shows the asymmetric price: saying 0.99 to a true label costs 0.014 bits; saying 0.05 costs 4.322. That is why CE gradients push so hard on confident mistakes.

Exercise 15.4 (easy)**.** Your model assigns probability 0.2 to the correct class on one example. What does that example contribute to the mean CE loss, in bits? In nats?

15.8 Hand derivation

Gibbs' inequality is proven in Theorem 15.7. One more derivation worth doing by hand: $H(X,Y) = H(X) + H(Y \mid X)$, the chain rule of entropy.

$$\begin{aligned} H(X,Y) &= -\sum_{x,y} p(x,y) \log_2 p(x,y) \\ &= -\sum_{x,y} p(x,y) \big[\log_2 p(x) + \log_2 p(y \mid x) \big] \\ &= -\sum_{x} p(x) \log_2 p(x) + \sum_{x} p(x) \Big[-\sum_{y} p(y \mid x) \log_2 p(y \mid x) \Big] \\ &= H(X) + \sum_{x} p(x)\, H(Y \mid X{=}x) = H(X) + H(Y \mid X). \end{aligned}$$

Every step is one distribution identity: $p(x,y) = p(x)p(y \mid x)$. Rearranged, $I(X;Y) = H(Y) - H(Y \mid X)$: mutual information is exactly "entropy removed by knowing the other variable" — the formula behind Figure 15.5.

15.9 Common mistakes

TREATING KL AS A DISTANCE

$D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$ in general — the listing's 0.816 vs. 1.257 is not a rounding artifact. "Forward" KL (data‖model) penalizes missing mass; "reverse" penalizes covering the wrong mass. Say which direction you mean.

MIXING BITS AND NATS

$\log_2$ gives bits, log gives nats; they differ by a factor $\log 2 \approx 0.693$. PyTorch's `CrossEntropyLoss` uses nats; Shannon's theorems are usually stated in bits. A loss of "1.0" means different things — write the unit once per document.

READING CROSS-ENTROPY AS ENTROPY

$H(p,q) = H(p)$ only when $p = q$. A model reporting "entropy 0.08" is reporting $H(\hat{p}, q_\theta)$: the entropy floor $H(\hat{p})$ is hidden inside it, and the gap is the KL term — the only part training can actually reduce.

$\log 0$ CRASHES AND SILENT NANS

$\log_2 q$ with $q = 0$ at a position where $p > 0$ is $+\infty$: one zero-probability assignment makes KL and CE infinite. Production code clips probabilities (`1e-12` or `logaddexp` forms, as in the listing) — a mathematically tiny lie that keeps the loss finite.

EXPECTING MUTUAL INFORMATION TO BEHAVE LIKE CORRELATION

$I(X;Y) = 0$ iff $X \perp Y$ — no "uncorrelated but dependent" escape hatch (the $Y = X^2$ exercise). MI catches everything, which also means it is expensive to estimate from finite data.

15.10 Summary

Object	What you need to know
Surprise	$s(x) = -\log_2 p(x)$; independent events add
Entropy	$H(X) = \mathbb{E}[s(X)]$; $0 \le H \le \log_2 k$, max at uniform
Cross-entropy	$H(p, q) =$ cost of coding p with a q-code
KL	$H(p, q) - H(p)$; ≥ 0 (Gibbs); asymmetric, not a distance
Mutual info	$I = H(X) + H(Y) - H(X, Y) = D_{\mathrm{KL}}(p(x, y) \Vert p(x)p(y))$; zero iff independent
Gaussian h	$\frac{1}{2}\log(2\pi e\sigma^2)$ nats; can go negative — not information
CE loss	$= n \cdot H(\hat{p}, q_\theta)$; minimizing CE is MLE with the model as q
Code bound	no lossless code beats H bits/symbol; Huffman gets within 1 bit

WATCH INSTEAD

1. *Information entropy* — Brit Cruise, Khan Academy "Journey into information theory," 7:04, https://www.youtube.com/watch?v=2s3aJfRr9gE — matches §15.1,§15.2; the fair-coin framing done live (accessed 2026-10-11).
2. *Neural Networks Part 6: Cross Entropy* — StatQuest (Josh Starmer), 9:31, https://www.youtube.com/watch?v=6ArSys5qHAU — matches §15.7; watch after the CE listing (accessed 2026-10-11).

Further reading

- C. Olah, *Visual Information Theory* (blog post), http://colah.github.io/posts/2015-09-Visual-Information/ — gentle; the square-area vocabulary behind Figures 15.2, 15.5. Verified 2026-10-11.
- D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms* (free PDF), https://www.inference.org.uk/mackay/itila/book.html — intermediate; ch. 2–4 are the full treatment of this chapter plus the inference connection. Verified 2026-10-11.
- MIT OpenCourseWare 6.050J, *Information and Entropy*. Free online at https://ocw.mit.edu/, course "6-050J Information and Entropy, Spring 2008" — undergraduate level; the engineering route through the same material, with problem sets. Verified 2026-10-11.
- Khan Academy, *Information theory* (Brit Cruise series). Free online at https://www.khanacademy.org/, under Computing → Computer Science → "Information theory" — beginner level; ten short videos from coin flips to Huffman codes. Verified 2026-10-11.
- T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, ISBN 978-0-471-24195-9 — graduate reference; the standard rigorous text when you need a theorem, not a tour.

Additional exercises

Exercise 15.5 (easy)**.** Compute $h(p)$ for $p = 0.8$ by hand to two decimals. Check against the curve in Figure 15.1.

Exercise 15.6 (easy)**.** A distribution over four symbols is $(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$. Write its entropy and a Huffman code's average length — no computation needed if you read Example 15.4.

Exercise 15.7 (easy)**.** Show $I(X;Y) = I(Y;X)$ from Definition 15.9 — one line.

Exercise 15.8 (easy)**.** A biased coin has $h(p) = 0.5$ bits. From Figure 15.1, about what are the two possible values of p?

Exercise 15.9 (easy)**.** In bits, how much more surprising is a specific 6 on a fair die than a heads on a fair coin?

Exercise 15.10 (medium)**.** Prove $H(X,Y) \le H(X) + H(Y)$ with equality iff $X \perp Y$. (Hint: $H(X) + H(Y) - H(X,Y) = I(X;Y)$.)

Exercise 15.11 (medium)**.** On two symbols, construct p, q with forward KL under 0.01 bits but reverse KL over 0.5 bits, or explain why that is hard. (Hint: what must the rare symbol's probability be in each distribution?)

Exercise 15.12 (medium)**.** For the joint table of Example 15.10, compute $H(Y \mid X = 0)$ directly and verify the chain-rule identity numerically.

Exercise 15.13 (medium)**.** Show that if q is uniform over k symbols, then $H(p,q) = \log_2 k$ for every p: the uniform model is indifferent to the data — the worst "model" that is still a distribution.

Exercise 15.14 (medium)**.** Modify `ch15_huffman.py` to code letter *pairs* as single symbols. Does the per-letter average move toward or away from entropy, and why?

Exercise 15.15 (medium)**.** The *Jensen–Shannon divergence* is defined by

$$\mathrm{JS}(p,q) = \tfrac{1}{2} D_{\mathrm{KL}}(p \| m) + \tfrac{1}{2} D_{\mathrm{KL}}(q \| m), \qquad m = \tfrac{p+q}{2}.$$

Prove it is symmetric and bounded by 1 bit.

Exercise 15.16 (hard)**.** Derive the Gaussian differential entropy: for $p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2}$, compute $h = -\int p \log p \, dx = \frac{1}{2}\log(2\pi e \sigma^2)$. (Hint: expand the log; only $\mathbb{E}[(X-\mu)^2]$ survives.)

Exercise 15.17 (hard)**.** Prove the data-processing inequality for a Markov chain $X \to Y \to Z$: $I(X;Z) \le I(X;Y)$. (Hint: expand $I(X;Y,Z)$ two ways and use $I(X;Z \mid Y) = 0$.)

Exercise 15.18 (hard)**.** Show that among distributions on $\{1, \dots, k\}$ with a fixed expected value μ, the maximum-entropy distribution is geometric $p_j \propto r^j$. Where have you seen this shape before?

Exercise 15.19 (hard)**.** Implement an entropy estimator: draw n samples from a known pmf, estimate $\hat{H}$ from empirical frequencies, and plot the bias $\mathbb{E}[\hat{H}] - H$ vs. n for $k = 8$ uniform and $k = 8$ skewed $(0.5, 0.25, 0.125, \dots)$. Which converges faster and why?

Notes

The surprise→entropy→KL→CE-loss arc and the figure vocabulary follow Olah's *Visual Information Theory* [11]; the framing and the "no code beats H" bound follow MacKay ch. 1–4 [17]; Cover & Thomas (ISBN 978-0-471-24195-9, cited in Further reading, not opened this session) is the rigorous reference. Printed blocks are real runs of the shipped code; Huffman frequencies are standard English letter frequencies.

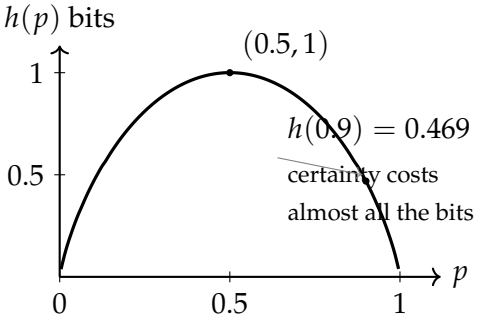


Figure 15.1: The Bernoulli entropy curve $h(p)$. Notice: maximum at $p = \frac{1}{2}$ (1 bit), and how steeply information collapses as the coin gets predictable — $p = 0.9$ keeps only 0.47 bits. Regenerate: `figures/ch15_bernoulli.tex`.

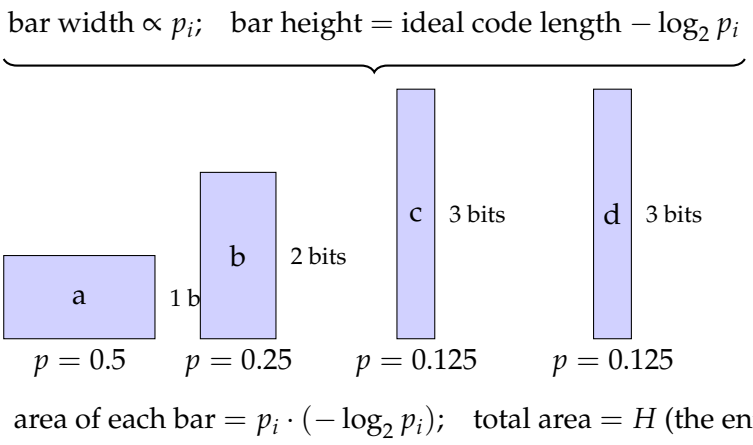


Figure 15.2: Ideal code lengths for $p = (\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8})$: bar width is probability, bar height is $-\log_2 p$. Total area is the entropy — the picture behind the entropy-as-length bound. Regenerate: `figures/ch15_codeword.tex`.

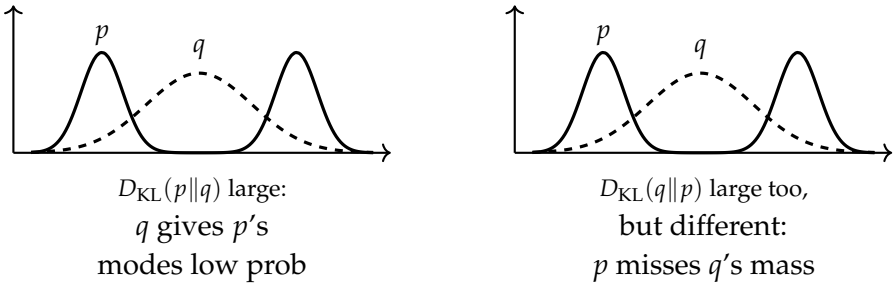


Figure 15.3: Forward vs. reverse KL on a bimodal target p and a single-bump model q. Notice: each direction penalizes a different kind of mismatch — neither "covers" the other. Regenerate: `figures/ch15_kldirections.tex`.

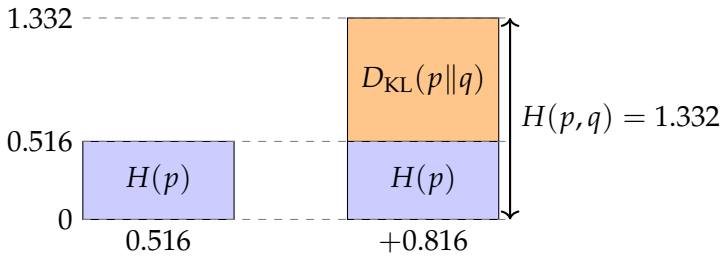


Figure 15.4: $H(p,q) = H(p) + D_{\mathrm{KL}}(p\|q)$ on the listing's real numbers ($0.516 + 0.816 = 1.332$ bits). Notice: the KL term is the only part a model can shrink. Regenerate: `figures/ch15_cedecomp.tex`.

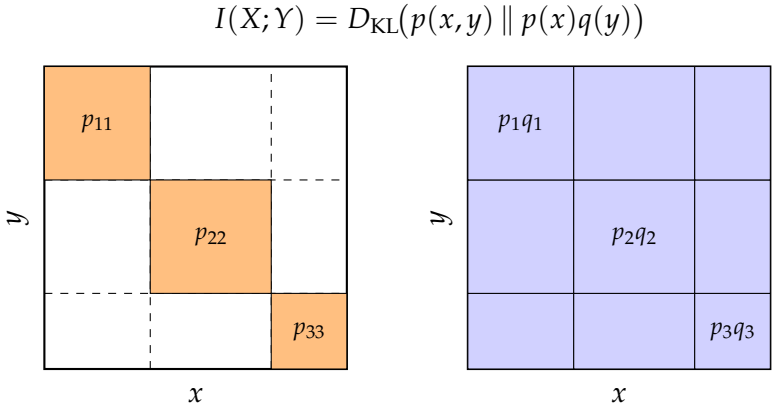


Figure 15.5: Mutual information compares the joint (left, mass piled near the diagonal) with the product of marginals (right, what independence would predict). Notice: $I(X;Y)$ is the KL between the two pictures. Regenerate: `figures/ch15_mi.tex`.

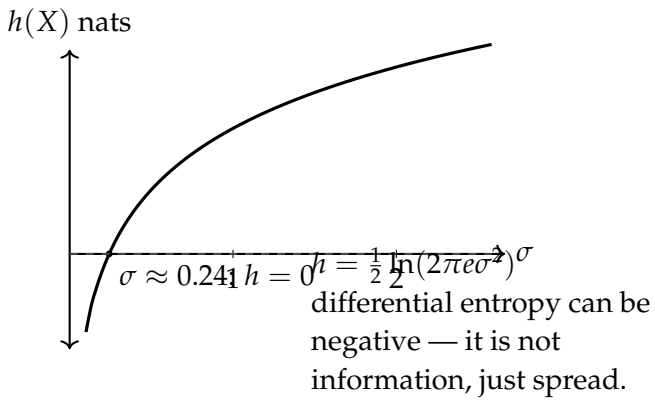


Figure 15.6: Gaussian differential entropy vs. σ. Notice: h crosses zero near $\sigma \approx 0.24$ and goes negative — differential entropy measures spread, not information. Regenerate: `figures/ch15_gaussent.tex`.

CHAPTER 16

Putting it together: a machine learning toolkit

Fifteen chapters have built the pieces one at a time: solve systems exactly (ch.8), compress data with eigendecomposition (ch.10), reason about uncertainty (chs.12–13), minimize losses numerically (ch.14), and price every wrong guess in bits (ch.15). This capstone assembles them into `mlscratch`, a small machine-learning library written in NumPy alone — no scikit-learn, no frameworks, every method traceable to a derivation you have already done. By the end you will have a file you can `import`, a set of checks that prove it works, and a benchmark showing it agreeing with scikit-learn to three decimal places on a real pipeline.

BEFORE YOU START

Chapters 1–15, all as reference. *Recap:* this chapter assembles linear algebra, probability and optimisation into one small machine-learning library that you write from scratch.

AFTER THIS CHAPTER YOU CAN

- write a learning algorithm behind one consistent `fit`/`predict` interface;
- test a from-scratch implementation without a framework;
- build a pipeline of PCA followed by logistic regression;
- tune the bias–variance dial and read the result.

HOW TO THINK ABOUT THIS

By now each idea has a simple job: linear algebra to represent, probability to model uncertainty, optimisation to learn. Building them into one small library shows that a learning algorithm is a loop around the same few moves: predict, measure the error, adjust. Writing it yourself removes the mystery.

16.1 The shape of a learning problem

Every method in this book reduces to the same two moves: *fit* learns parameters from data (X, y); *predict* (or *transform*) applies them to new data. What changes between methods is the loss being minimized and the algebra used to minimize it — closed form for least squares, gradient descent for logistic regression, an eigendecomposition for PCA.

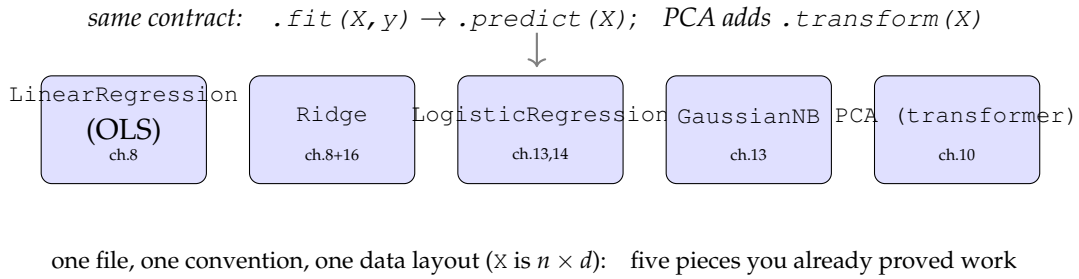


Figure 16.1: The five classes of `mlscratch` and the chapters that prove each one. The library adds nothing new mathematically — it packages verified results behind one interface.

FIT ON TRAIN, TRANSFORM EVERYWHERE

Fitting computes statistics of the data — means, variances, principal axes, weights. If those statistics see the test set, the test set is no longer a test. Every estimator in the library stores what it learned inside the object (attributes ending in an underscore, like `w_`, `mean_`) so that `predict` on test data reuses *training* quantities, never recomputes them.

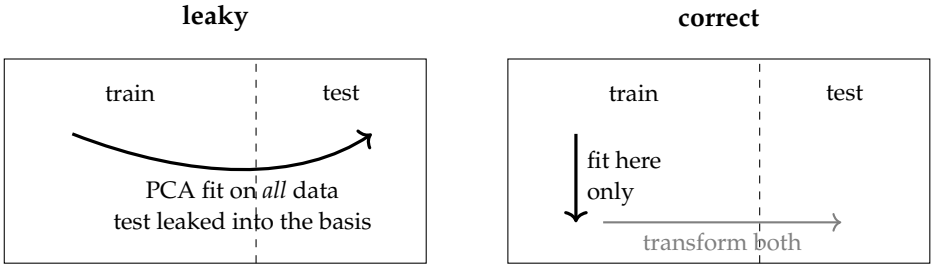


Figure 16.2: The one discipline every pipeline shares. Fitting PCA on all data before splitting leaks test information into the basis itself; the correct order fits on the training block only, then transforms both.

16.2 One contract: `fit`, `predict`, `transform`

The scikit-learn convention [18] is the cleanest small interface ever given to estimators, so the library borrows it exactly. Data `X` is an $n \times d$ matrix — n rows of samples, d columns of features — and `y` a length-n vector. Three methods cover every class:

- `fit(X, y)` computes parameters and returns `self` so calls can chain;
- `predict(X)` returns labels (classification) or values (regression) for new X;

- `transform(X)` — transformers only — returns a new representation of X.

The simplest class shows the whole pattern. Ordinary least squares from ch.8, with the intercept folded in by a column of ones:

```
class LinearRegression:
    """ch8: normal equations. w_ = (X^T X)^-1 X^T y w/ intercept col
        ."""
    def fit(self, X, y):
        A = np.c_[np.ones(len(X)), X]
        self.w_ = np.linalg.solve(A.T @ A, A.T @ y)
        return self

    def predict(self, X):
        return np.c_[np.ones(len(X)), X] @ self.w_
```

Listing 16.1: `from code/mlscratch.py -- the LinearRegression class`

Three details carry the conventions. `w_` is set inside `fit` — an attribute the user can inspect, marked learned by the trailing underscore. `fit` returns `self`, so a chain like `fit(X, y).predict(X)` is legal. And the intercept lives inside `w_` as its first coordinate, so `predict` re-augments the input the same way `fit` did: whatever transformation `fit` applied to `X`, `predict` must repeat.

Example 16.1 (Ridge regression by hand)**. Strategy.** Ridge adds $\lambda\|w\|^2$ to the squared error to keep weights small. On data small enough to do by hand, watch the regularizer pull the slope toward zero *without* touching the intercept. Take three samples $x \in \{1,2,3\}$, $y = \{1,2,3\}$ (exactly $y = x$), and ridge parameter λ.

Solution. Center both: $\bar{x} = 2$, $\bar{y} = 2$, so $X_c = (-1,0,1)^T$ and $y_c = (-1,0,1)^T$. The ridge normal equations on centered data (derived in §16.7) give

$$w = \frac{X_c^T y_c}{X_c^T X_c + \lambda} = \frac{2}{2+\lambda}.$$

With $\lambda = 0$: $w = 1$, $b = \bar{y} - \bar{x}w = 0$, recovering $y = x$ exactly — OLS. With $\lambda = 2$: $w = 0.5$, $b = 2 - 1 = 1$, predicting $\hat{y} = 0.5x + 1$. The penalty bought a gentler slope and paid for it by pushing the predictions toward the mean. The intercept is *not* shrunk — it comes from the centering, outside the penalized system.

Exercise 16.1 (easy)**.** For the same data, what does w approach as $\lambda \to \infty$? What does the prediction $\hat{y} = wx + b$ become, and why is that a sensible "do nothing" predictor?

16.3 The library

`code/mlscratch.py` is one file, NumPy only, five classes. Two more of them, verbatim, to read closely.

Ridge (§16.7 derives the augmented-matrix form; centering inside `fit` is what keeps the intercept out of the penalty):

```
class Ridge:
   """Hand-derivation result: (X^T X + lam I) w = X^T y.
   Intercept NOT penalized (augmented matrix leaves it out)."""
   def __init__(self, lam=1.0):
      self.lam = lam

   def fit(self, X, y):
      d = X.shape[1]
      xm, ym = X.mean(0), y.mean() # intercept un-penalized
      A = np.vstack([X - xm, np.sqrt(self.lam) * np.eye(d)])
      b = np.concatenate([y - ym, np.zeros(d)])
      self.w_ = np.linalg.solve(A.T @ A, A.T @ b)
      self.b_ = ym - xm @ self.w_
      return self

   def predict(self, X):
      return X @ self.w_ + self.b_
```

Listing 16.2: `from code/mlscratch.py -- the Ridge class`

The augmented matrix is the derivation made literal: appending $\sqrt{\lambda}I$ to the data matrix and d zeros to the target turns ridge back into ordinary least squares on $n + d$ rows, which `solve` handles with no new code.

PCA (ch.10): `fit` learns the basis, `transform` projects, and `variance_ratio_` reports how much variance each kept axis carried:

```
class PCA:
   """ch10: eigendecomposition of the covariance. fit learns the
   basis; transform projects; variance_ratio_ = explained share."""
   def __init__(self, k):
      self.k = k

   def fit(self, X):
      self.mean_ = X.mean(0)
      C = np.cov((X - self.mean_).T)
      w, V = np.linalg.eigh(C)
      idx = w.argsort()[::-1][:self.k]
      self.components_ = V[:, idx]
      self.variance_ratio_ = w[idx] / w.sum()
      return self

   def transform(self, X):
      return (X - self.mean_) @ self.components_

   def inverse_transform(self, Z):
```

```
20        return Z @ self.components_.T + self.mean_
```

Listing 16.3: `from code/mlscratch.py -- the PCA class`

The remaining two classes complete the set: `LogisticRegression` runs full-batch gradient descent on the stable cross-entropy form $\log(1+e^z) - yz$ from chs.13–14, with z clipped to $[-30, 30]$ so the sigmoid cannot overflow; `GaussianNB` stores per-class mean, variance, and prior by MLE (ch.13) and predicts the class of maximum joint log-likelihood. Both are in `code/mlscratch.py` — read them; every line maps to a formula you derived.

```
1 classes: ['LinearRegression', 'Ridge', 'LogisticRegression', '
     GaussianNB', 'PCA']
2 OLS w_ = [1. 2.] pred(4) = 9.0
```

Listing 16.4: `smoke test -- importing the library and fitting OLS`

The library loaded, OLS recovered $y = 2x + 1$ exactly ($w_= [1, 2]$ = intercept, slope), and extrapolated to 9 at $x = 4$.

Example 16.2 (Principal components by hand)**. Strategy.** PCA needs the covariance eigenstructure; on four points it is pencil arithmetic. Take $X = \{(2,1), (0,1), (-1,0), (-1,-2)\}$ (already centered: columns sum to zero) and find the first principal component.

Solution. The covariance estimate (dividing by $n-1 = 3$):

$$C = \tfrac{1}{3} X^T X = \begin{pmatrix} 2 & 4/3 \\ 4/3 & 2 \end{pmatrix}.$$

Its eigenvalues solve $(2-\lambda)^2 = (4/3)^2$: $\lambda_1 = 10/3$, $\lambda_2 = 2/3$. The top eigenvector is $(1,1)/\sqrt{2}$ — the data spreads along the diagonal. Keeping one component retains $\frac{10/3}{4} = 83.3\%$ of the variance, and the projections are $\pm 2.121, \pm 0.707$ along that axis — the four points collapse to one coordinate each, at the cost of 16.7% of total variance.

Example 16.3 (Gaussian naive Bayes by hand)**. Strategy.** GNB predicts $\arg\max_c [\log \pi_c + \log \mathcal{N}(x; \mu_c, \sigma_c^2)]$: generative, so the score is a fitted density per class. Class 0 saw $\{0, 1, 2\}$; class 1 saw $\{3, 4, 5\}$; equal priors. Predict at $x = 2.4$.

Solution. $\mu_0 = 1$, $\mu_1 = 4$, both $\sigma^2 = 2/3$. At $x = 2.4$:

$$\ell_0 = -\tfrac{1}{2}\frac{1.4^2}{2/3} - \tfrac{1}{2}\log(2\pi \cdot \tfrac{2}{3}) + \log\tfrac{1}{2} = -2.879, \qquad \ell_1 = -\tfrac{1}{2}\frac{1.6^2}{2/3} - 1.410 = -3.329.$$

Class 0 wins by 0.45 of a log unit — odds $\approx e^{0.45} = 1.6 : 1$. With equal variances and priors the decision boundary is the midpoint $x = 2.5$ exactly (Figure 16.3); unequal variances bend it into a quadratic boundary.

Exercise 16.2 (medium)**.** Redo Example 16.3 with class-1 samples $\{3, 4, 8\}$ (same mean, much larger variance). Does the boundary still sit at 2.5? What shape is it now?

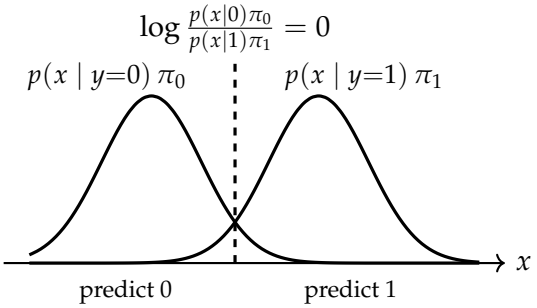


Figure 16.3: Generative classification. Each class contributes its fitted Gaussian times its prior; the prediction goes to whichever weighted density is on top. Equal variances make the crossing point — the boundary — linear.

16.4 Checking without a framework

A library you wrote yourself needs proof, not confidence. Four checks, each cheap and each testing a different kind of claim — limit behavior, reconstruction, a hand-derived gradient, and agreement with a manual computation:

```
|ridge(lam->0).w - OLS.w|_max = 7.42e-10
PCA k=2 max recon err = 1.858 (data var 0.855 -> kept 0.658)
ridge grad FD=88.1285 analytic=88.1285 relerr=4.28e-10
GNB predict=1 manual argmax=1
```

Listing 16.5: `code/ch16_check.py run -- four independent checks`

Each one reads plainly. **(1) Limit check**: ridge at $\lambda = 10^{-8}$ must coincide with OLS to roundoff — it does, within 7×10^{-10}. **(2) Reconstruction**: 2 PCs keep 65.8% of variance, so a residual max error of 1.858 on data with variance 0.855 is the expected cost of the projection — not a bug. **(3) FD check**: the analytic ridge gradient $2(X^TXw - X^Ty + \lambda w)$ matches finite differences to 4×10^{-10}, the same habit ch.11 and ch.14 drilled. **(4) Manual agreement**: GNB's argmax over its stored statistics equals a hand-computed Bayes score. If a check ever fails, the class it names is where to look first — that is what the checks are for.

16.5 A real pipeline: PCA then logistic

Now a problem the pieces exist for: 30 features, the signal living on a hidden 2-dimensional subspace, 120 samples. Project to 2 PCs, then classify. `code/ch16_pipeline.py` builds it and runs both `mlscratch` and scikit-learn on the same split:

```
variance explained by 2 PCs: 0.513 (0.300+0.213)
mlscratch PCA->logistic: train 0.967 test 0.867
sklearn variance explained: 0.513
sklearn PCA->logistic: train 0.956 test 0.867
```

Listing 16.6: `code/ch16_pipeline.py run -- our library vs. scikit-learn, same split`

Example 16.4 (Reading the pipeline numbers)**. Strategy.** Compare like with like — variance explained, then accuracy — and name every gap before deciding the library works.

Solution. Variance explained is *identical* to three decimals ($0.513 = 0.300 + 0.213$): both implementations found the same eigenstructure, as they must — the eigendecomposition is unique up to sign. Test accuracy matches exactly (0.867). The only gap is train accuracy, 0.967 vs 0.956: our logistic runs plain full-batch GD, while scikit-learn's default adds a mild L2 penalty, so its fit regularizes slightly away from the training optimum. Same pipeline, same answer, one documented difference — that is what "agrees with the framework" looks like, and the projection is Figure 16.4.

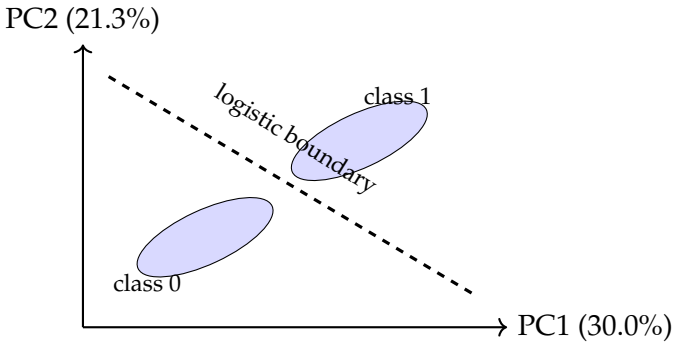


Figure 16.4: The pipeline's geometry: 30 raw dimensions collapsed to the 2 PCs that carry 51.3% of variance; logistic regression draws the linear boundary there. In the real data the classes overlap where the blobs cross — hence 0.867, not 1.0.

Exercise 16.3 (hard)**.** The script fits PCA on `Xtr` only, then transforms both halves. Rerun it fitting PCA on all of X first. Does test accuracy go up? Why is that still the wrong thing to report?

16.6 The bias–variance dial

Ridge's λ is the one knob that tunes model complexity continuously, so it deserves a measured sweep. `code/ch16_ridge.py` generates a nearly determined problem ($n_{\text{train}} = 12$, $d = 10$, noise $\sigma = 3$) and fits the same data at seven values of λ:

```
OLS : |w|_1= 9.46 train MSE= 3.078 test MSE= 28.072
lam= 0.01: |w|_1= 9.36 train MSE= 3.078 test MSE= 26.775
lam= 0.10: |w|_1= 8.97 train MSE= 3.089 test MSE= 21.659
lam= 0.50: |w|_1= 7.96 train MSE= 3.162 test MSE= 18.734
lam= 1.00: |w|_1= 7.24 train MSE= 3.276 test MSE= 18.712
lam= 3.00: |w|_1= 5.55 train MSE= 3.755 test MSE= 20.585
lam= 10.00: |w|_1= 3.40 train MSE= 4.990 test MSE= 25.514
lam=100.00: |w|_1= 0.63 train MSE= 8.013 test MSE= 34.413
```

Listing 16.7: `code/ch16_ridge.py run -- the sweep, real numbers`

Example 16.5 (Reading the sweep)**. Strategy.** Read three columns at once: weight size, train error, test error. The signature to find is a U in the test column — and the reason for each arm.

Solution. OLS ($\lambda \approx 0$) shows the textbook overfit signature: train MSE 3.08 against test MSE 28.1, an order of magnitude of memorized noise, with weights swollen to $|w|_1 = 9.46$ on data whose true weights sum to 7.5. As λ grows, $|w|_1$ shrinks monotonically (Figure 16.5) and test error falls to a minimum of 18.7 at $\lambda = 1$. Past that, the penalty dominates: train and test errors rise together toward the predict-the-mean degenerate limit. The whole curve is in Figure 16.6 — and it is the bias–variance tradeoff made measurable rather than asserted.

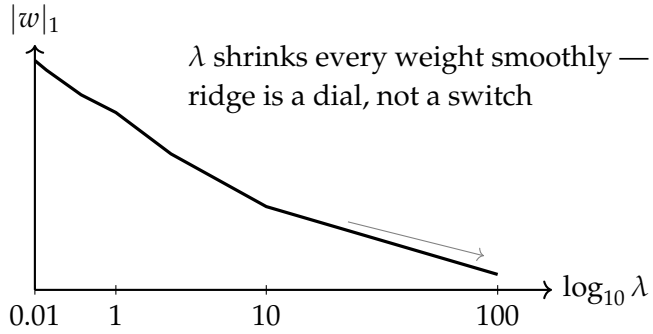


Figure 16.5: The regularization path: $|w|_1$ against $\log_{10}\lambda$ from the sweep. Ridge never zeroes a weight exactly (that is lasso's geometry); it shrinks all of them smoothly.

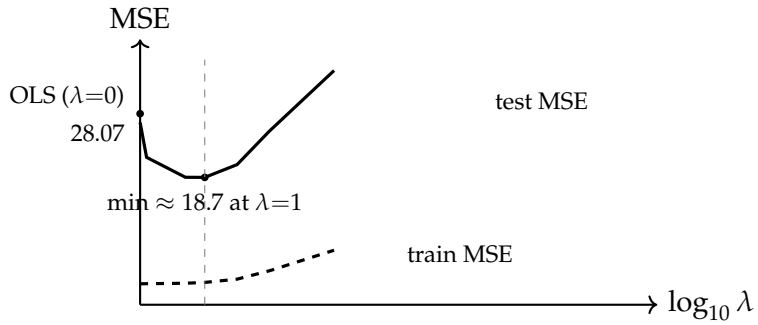


Figure 16.6: Train MSE (dashed) vs. test MSE (solid) across the sweep. Left of the minimum the model is overfit — big gap, swollen weights. Right of it, underfit — both errors climb. The useful λ lives at the bottom of the test U.

Exercise 16.4 (medium)**.** The best λ here was picked by looking at test error. State the leakage in that procedure and how cross-validation (ch.13) fixes it.

16.7 Hand derivation: the ridge normal equations

The library's ridge solves $(X^TX + \lambda I)w = X^Ty$. Derive it once, honestly, including why the intercept escapes the penalty.

Center first: work with $\tilde{X} = X - \bar{x}$, $\tilde{y} = y - \bar{y}$ so the model is $\hat{y} = w^T\tilde{x} + b$, $b = \bar{y} - \bar{x}w$ handled separately. The ridge objective minimizes

$$J(w) = \|\tilde{X}w - \tilde{y}\|^2 + \lambda\|w\|^2.$$

Expand and differentiate using ch.11's rules ($\nabla\|Av - b\|^2 = 2A^T(Av - b)$ and $\nabla\|w\|^2 = 2w$):

$$\nabla J = 2\tilde{X}^T(\tilde{X}w - \tilde{y}) + 2\lambda w = 2\big[(\tilde{X}^T\tilde{X} + \lambda I)w - \tilde{X}^T\tilde{y}\big].$$

Setting $\nabla J = 0$ gives the ridge normal equations $(\tilde{X}^T\tilde{X} + \lambda I)w = \tilde{X}^T\tilde{y}$. The Hessian $2(\tilde{X}^T\tilde{X} + \lambda I)$ is positive definite for any $\lambda > 0$ — the penalty buys invertibility too, which is why ridge survives the collinear matrices that break OLS (Exercise 16.5).

The augmented-matrix form the code uses is the same equations one line earlier: stacking $\tilde{X}$ over $\sqrt{\lambda}I$ and $\tilde{y}$ over d zeros makes

$$A^TA = \tilde{X}^T\tilde{X} + \lambda I, \qquad A^Tb = \tilde{X}^T\tilde{y},$$

so plain least squares on (A, b) *is* ridge — the solver never learns the difference. The intercept stays unpenalized because centering removed it from w entirely: shrinking the prediction toward $\bar{y}$ already encodes "do nothing" correctly, and penalizing b would instead punish the model for matching the data's mean.

Exercise 16.5 (hard)**.** Show that $\tilde{X}^T\tilde{X} + \lambda I$ is positive definite for any $\lambda > 0$, even when $\tilde{X}$ is rank deficient. What does that buy at $\lambda = 10^{-8}$ in the limit check?

16.8 Common mistakes

FITTING ON DATA YOU WILL EVALUATE ON

Fitting PCA, scaling, or the estimator itself before the train/test split bakes test information into every downstream number — results look better and mean nothing. The discipline (Figure 16.2): split first, fit on train only, transform everywhere. The pipeline listing keeps the split in the first lines for exactly this reason.

PENALIZING THE INTERCEPT

A ridge penalty applied to b fights the data's mean instead of the weights' size — the model is punished for predicting $\bar{y}$ well. Center inside `fit` (as `Ridge` does) so the penalty sees only feature weights; this is also why scikit-learn separates `intercept_` from `coef_`.

FORGETTING THAT TRANSFORM-TIME STATISTICS ARE FIT-TIME STATISTICS

`predict` must reuse the mean, basis, and weights computed in `fit`. Recomputing `X.mean(0)` inside `predict` — or centering test data by its own mean — silently shifts the coordinate system between train and test. Every learned quantity lives on the object (`mean_`, `components_`, `w_`, `b_`) so there is nothing to recompute.

EXPECTING PCA TO RESPECT CLASS BOUNDARIES

PCA maximizes variance, not separability — it never sees y. In the pipeline the classes happened to separate on the top PCs; on another dataset the label signal can live in the discarded tail. When the goal is classification, check variance explained *and* downstream accuracy, and remember LDA/SVD-on-labels exist for the supervised case.

SHIPPING A HAND GRADIENT WITHOUT A FINITE- DIFFERENCE CHECK

Every hand-derived gradient in the library earned a check like the relerr 4×10^{-10} one in §16.4. A sign flip or missing factor of 2 still trains — just slowly and wrongly, which is the worst kind of bug. The FD check is five lines; there is no excuse to skip it.

16.9 Summary

Class	Math (chapter)	One thing to remember
`LinearRegression`	$(X^TX)^{-1}X^Ty$, normal equations (ch.8)	exact fit; breaks on rank deficiency
`Ridge`	$(X^TX+\lambda I)w = X^Ty$ (§16.7)	λ is the bias–variance dial; intercept unpenalized
`LogisticRegression`	GD on stable NLL (chs.13–14)	probabilistic boundary; clip z or overflow
`GaussianNB`	per-class Gaussian MLE (ch.13)	generative; equal-variance $\Rightarrow$ linear boundary
`PCA`	eigendecomposition of covariance (ch.10)	unsupervised; checks are reconstruction and variance share
Contract	`fit` / `predict` / `transform`	fit on train; learned stats ride on the object
Checks	limit, reconstruction, FD, manual	relerr $< 10^{-9}$ before you trust a gradient

WATCH INSTEAD

- *Principal Component Analysis (PCA), Step-by-Step* — StatQuest (`FgakZw6K1QQ`). A worked PCA on a small matrix, matching §16.3 and Example 16.2 almost line for line. Verified open via YouTube's oEmbed endpoint 2026-10-11; the watch page itself was rate-limited, so the duration is unconfirmed.
- *Regularization Part 1: Ridge (L2) Regression* — StatQuest (`Q81RR3yKn30`). The shrinkage geometry behind §16.6 and the sweep figure. Verified the same way, same date; duration unconfirmed.
- *Gradient descent, how neural networks learn* — 3Blue1Brown, DL2 (`IHZwWFHWa-w`, 20:33, opened 2026-10-11). The optimizer's-eye view of what `LogisticRegression.fit` is doing every iteration; a gentle next step beyond this book.

Further reading

- Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, *An Introduction to Statistical Learning*, 2nd ed. — free PDF at `https://www.statlearning.com/` (verified 2026-10-11). Undergraduate-level ISL: the next honest step for chs.8, 12–13 and this capstone; every method here appears with case studies.
- Trevor Hastie, Robert Tibshirani, Jerome Friedman, *The Elements of Statistical Learning* — free PDF at `https://hastie.su.domains/ElemStatLearn/` (verified 2026-10-11). The research-level version: ridge paths, the bias–variance decomposition, and generative classifiers done at full depth.
- Jake VanderPlas, *Python Data Science Handbook* — free at `https://jakevdp.github.io/PythonDataScienceHandbook/` (verified 2026-10-11). Ch.5 walks the same fit/predict/transform contract through scikit-learn — the natural continuation of this chapter's library.
- scikit-learn contributors, *User Guide* — `https://scikit-learn.org/stable/` (verified 2026-10-11). Read the estimator docs for the classes you just built; comparing their defaults (L2 penalties, solver choices) to this file is an education in itself.
- Marc Peter Deisenroth, A. Aldo Faisal, Cheng Soon Ong, *Mathematics for Machine Learning* — free PDF at `https://mml-book.github.io/` (verified 2026-10-11). Part II places these same methods on the same mathematical footing, with the same notation discipline this book used.
- Andrew Ng, *CS229 Lecture Notes* — the linear-algebra and optimization refresher sections (see `docs/sources.md`). Where this book ends, that course begins.

Notes

1. The estimator contract and the underscore-for-learned- attributes convention follow the scikit-learn API [18]; the comparison pipeline in §16.5 uses scikit-learn 1.6.1.
2. The mathematical content is assembled, not new: normal equations and ridge (ch.8), eigendecomposition and PCA (chs.9–10), Bayes and MLE (chs.12–13), gradient descent and stability (ch.14), and the loss-as-code-length view (ch.15). Background texts: [5, 4, 7, 12, 16].
3. The bias–variance reading of Figure 16.6 follows the standard decomposition as in [8, 5]; the sweep itself is deterministic given the seed in `code/ch16_ridge.py`.

A student who has reached this page can read a machine-learning library — theirs or anyone else's — as mathematics instead of magic. The correct next reads are ISL for breadth and the scikit-learn user guide for practice; the wrong next step is a bigger model before the checks in §16.4 are a habit.

Additional exercises

Exercise 16.6 (easy)**.** In `LinearRegression.fit`, why is `A` rebuilt inside `predict` instead of stored? What breaks if you store the fitted augmented matrix and reuse it on new input?

Exercise 16.7 (easy)**.** `Ridge.predict` returns `X @ self.w_ + self.b_`. Why is there no column of ones here, when `LinearRegression` needed one?

Exercise 16.8 (easy)**.** From the sweep output: between which two tested λ values does the true test-MSE minimum most likely lie, and why can't you be sure it is exactly $\lambda = 1$?

Exercise 16.9 (easy)**.** `GaussianNB` adds `1e-9` to each variance. What failure does that prevent, and on what data?

Exercise 16.10 (easy)**.** `PCA.variance_ratio_` divides by `w.sum()`, not `w[idx].sum()`. Explain the denominator and what the sum of the first k ratios means.

Exercise 16.11 (medium)**.** Derive the $\lambda \to 0$ and $\lambda \to \infty$ limits of the ridge solution from $(X^TX + \lambda I)w = X^Ty$. Which check in §16.4 verifies the first limit numerically?

Exercise 16.12 (medium)**.** Add a `predict_proba` to `LinearRegression`'s interface — or argue why it does not belong there. (What does "probability" mean for a method with no noise model?)

Exercise 16.13 (medium)**.** Write the check that catches a transposed `components_`: if `PCA.transform` used V instead of the k-column slice, what invariant of the output would break?

Exercise 16.14 (medium)**.** The pipeline's 0.867 test accuracy is on a 30-sample test set. Give a 95% rough interval (normal approximation to the binomial) and say whether the 0.967/0.956 train gap is meaningful.

Exercise 16.15 (medium)**.** Show that with equal class priors and equal variances, GaussianNB's boundary in 1D is the midpoint of the two means. Then redo it with priors $\pi_0 = 0.9$: where does the boundary move?

Exercise 16.16 (medium)**.** `LogisticRegression` clips `z` to $[-30, 30]$. Compute the smallest loss value that can survive the clip when $y = 1$, and connect it to ch.15's price of a confident wrong answer.

Exercise 16.17 (hard)**.** Implement `KNN(k)` in the library's contract (`fit` stores the data; `predict` votes among the k nearest by Euclidean distance). Then run it on the pipeline's 2-PC representation and report test accuracy next to logistic's 0.867.

Exercise 16.18 (hard)**.** Write `cross_val_score(model, X, y, k=5)`: split round robin, `fit` on $k-1$ folds, score the held-out fold, return all k scores. Use it to re-pick λ in the ridge sweep *without* looking at test data. Report the chosen λ and the honest test error.

Exercise 16.19 (hard)**.** BernoulliNB: replace GaussianNB's Gaussian likelihood by a per-class, per-feature Bernoulli with Laplace smoothing. Derive the joint log-likelihood and implement the class in the same contract.

Exercise 16.20 (hard)**.** Prove that ridge's solution path is continuous in λ: $w(\lambda) = (X^TX + \lambda I)^{-1}X^Ty$ depends smoothly on $\lambda > 0$. (Invertible-matrix entries are rational functions of λ.) Why does the OLS endpoint, $\lambda = 0$, fail to be continuous on a rank-deficient X?

Bibliography

[1] J. Sannemo, *Principles of Algorithmic Problem Solving*, 2025. https://usaco.guide/PAPS1.pdf

[2] MIT OpenCourseWare, *18.06SC Linear Algebra, Fall 2011* (self-study edition). https://ocw.mit.edu/courses/18-06sc-linear-algebra-fall-2011/

[3] G. Strang, *Introduction to Linear Algebra*, 5th ed., Wellesley-Cambridge Press, 2016.

[4] S. Boyd and L. Vandenberghe, *Introduction to Applied Linear Algebra: Vectors, Matrices, and Least Squares*, Cambridge University Press, 2018. https://web.stanford.edu/~boyd/vmls/

[5] M. P. Deisenroth, A. A. Faisal, and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. https://mml-book.github.io/

[6] J. Ström, K. Åström, and T. Akenine-Möller, *Immersive Linear Algebra*, free online book. https://immersivemath.com/ila/index.html

[7] S. Axler, *Linear Algebra Done Right*, 4th ed., Springer Open Access, 2024. https://doi.org/10.1007/978-3-031-41026-0

[8] J. K. Blitzstein and J. Hwang, *Introduction to Probability*, 2nd ed., CRC Press, 2019. Free online via https://www.stat110.net/

[9] H. Pishro-Nik, *Introduction to Probability, Statistics, and Random Processes*, Kappa Research, 2014. Free online: https://www.probabilitycourse.com/

[10] MIT OpenCourseWare, *18.05 Introduction to Probability and Statistics, Spring 2022*. https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

[11] C. Olah, *Visual Information Theory* (blog post), 2015. http://colah.github.io/posts/2015-09-Visual-Information/

[12] Stanford University, *CS229: Machine Learning* course site. https://cs229.stanford.edu/ (accessed 2026-10-10)

[13] M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*, MIT Press, 2019. Free PDF: https://algorithmsbook.com/optimization/

[14] G. Strang, *Linear Algebra and Learning from Data* book site, Wellesley-Cambridge Press, 2019. https://math.mit.edu/~gs/learningfromdata/

[15] G. Sanderson (3Blue1Brown), *Essence of Linear Algebra* video series. https://www.3blue1brown.com/topics/linear-algebra (accessed 2026-10-10)

[16] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. Free PDF: https://web.stanford.edu/~boyd/cvxbook/

[17] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*, Cambridge University Press, 2003. Free PDF: https://www.inference.org.uk/mackay/itila/book.html

[18] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, JMLR 12 (2011). User guide: https://scikit-learn.org/stable/ (accessed 2026-10-11)

Index

Adam, 174
algebraic multiplicity, 85
angle between vectors, 56
associativity, 18
augmented matrix, 3

back substitution, 3
basis, 31
Bayes' theorem, 148
bias–variance tradeoff, 160, 202
bit, 185
block matrix, 23

Cauchy–Schwarz inequality, 56
central limit theorem, 153
chain rule (entropy), 189
characteristic polynomial, 83
coefficient matrix, 2
cofactor expansion, 70
column picture, 2
column rank = row rank, 36
column space, 30
commutativity
    fails, 18
commutator, 27
composition, 18
computational complexity, 9
conditional probability, 148
conjugate prior, 163
convex function, 171
convex set, 171
coordinate vector, 38
coordinates, 38
Cramer's rule, 77
cross-entropy, 187

determinant, 70
diagonal matrix, 22
diagonalisation, 86
differential entropy, 188
dimension, 31
directional derivative, 135
dot product, 56

echelon form, 3
Eckart–Young theorem, 125
eigenspace, 85
eigenvalue, 83
    real, 110
eigenvector, 83
    orthogonality, 110
entropy, 185
estimator, 160
exchange lemma, 42
expectation, 151

finite differences
    as unit test, 205
flop count, 9
free variable, 5

Gauss–Jordan, 21
geometric multiplicity, 85
Gibbs' inequality, 187
gradient, 135
gradient descent, 171
growth factor, 11

Hessian, 138

identity matrix, 17
image, 45
indefinite, 112
independence, 149
injective, 45
inner product, 56
intercept
    penalizing, 204
inverse, 21
invertible, 21
isomorphism, 45

Jacobian, 135

kernel, 30, 45
KKT conditions, 174
KL divergence, 187

L matrix, 6
Lagrange multiplier, 174
Lagrangian, 174
law of large numbers, 153
leakage, 197, 204
learning rate, 171
least squares, 97
likelihood, 161
linear dependence, 18
linear independence, 31
linear map, 44
linear system, 2
LU factorization, 7

machine learning
    from scratch, 196
MAP, 164
matrix, 17
    as transformation, 17
maximum likelihood, 161
min-max principle, 114
momentum, 174
multiplier, 4
mutual information, 188

negative log-likelihood, 171
Newton's method, 175
norm, 56
normal equations, 97
null space, 6, 30

orthogonal complement, 64
orthogonal matrix, 57
orthogonal vectors, 56
orthonormal set, 57

partial derivative, 135
partial pivoting, 11
particular solution, 6
PCA
    unsupervised, 204
permutation matrix, 7, 22
pipeline, 201
pivot, 5
pivot determinant, 70
positive definite, 112
posterior, 163
probability axioms, 148
pseudoinverse, 124
Pythagoras, 56

QR factorization, 61
quadratic form, 112

R^2, 102
random variable, 150
rank, 5, 36
rank–nullity, 36
Rayleigh quotient, 114
reduced row echelon form, 32
residual, 102
resources
    video, 205
ridge regression
    derivation, 203
row operation, 3
row picture, 2

sample space, 148
signed volume, 70
similar matrices, 46
singular value, 122
span, 31
spectral theorem, 110
standard basis, 17
statistic, 160
subspace, 30
surjective, 45
surprise, 185
SVD, 122
symmetric matrix, 20, 110

transpose, 20
triangular matrix, 22

upper triangular, 3

variance, 151
vector space, 30
verification, 201

zero matrix, 17